DEVOPS ENGINEERING MASTERY

CI/CD PIPELINE ENGINEERING

Continuous Integration, Continuous Delivery, Deployment Automation & Release Engineering

VOLUME 05 • FIRST EDITION

```
nahid@technova:~$ gh workflow run ci.yml
✓ build  •  ✓ test  •  ✓ lint  •  ✓ docker build
nahid@technova:~$ docker push ghcr.io/technova/app:v1.4.0
✓ image pushed — digest sha256:9f2c…
nahid@technova:~$ kubectl rollout status deployment/technova-app
```

GitHub Actions • GitLab CI/CD • Jenkins • Docker • MERN

First Edition • Published 8 November 2026 • Dhaka, Bangladesh

24 Chapters • 24 Guided Labs • 3 Major Projects • 120 MCQs

CI/CD Pipeline Engineering

DEVOPS ENGINEERING MASTERY — Volume 05

Continuous Integration, Continuous Delivery, Deployment Automation ও Release Engineering

First Edition — Published 8 November 2026.

Hands-on environment: GitHub Actions, GitLab CI/CD and Jenkins, with Node.js 20, Docker, and a MERN (MongoDB, Express, React, Node.js) example application called TechNova. All domains, IP addresses and credentials in examples are fictional (technova.example, 203.0.113.x, CHANGE-ME-...). Sample terminal outputs are illustrative. Cloud-provider comparisons (AWS, Microsoft Azure, Google Cloud, IBM Cloud) are high-level and operational; provider details vary by region and plan, and official provider documentation is the authority — no pricing or service limits are stated in this book.

How to use this book. Read the parts in order — each part builds on the previous one. Keep a Git repository and a Docker-ready machine beside you and do every hands-on exercise and every lab in the companion Lab Workbook before moving on. Do not copy YAML blindly; read the explanation under each configuration field first. Destructive operations are always flagged with a WARNING box and a safe alternative. Never practise on a production pipeline: use a test repository and a disposable environment, and never paste a real secret into an example — use the fictional placeholders shown.

A note on honesty. Reading a book alone does not make anyone an expert. This book teaches the foundations and the practical judgment to progress toward professional engineering — but the judgment itself comes from doing the labs, breaking a test pipeline, fixing it, and repeating. There is no shortcut for that practice.

Conventions. Bangla is the explanatory language; technical terms, command names, APIs and configuration keys stay in English. `monospace` marks commands, file paths and code. **CONCEPT** explains an idea, **REAL WORLD** shows an industry scenario, **WARNING** marks risk, **PRO TIP** gives professional advice, and **LAB** marks hands-on work.

© 2026 DevOps Engineering Mastery series. All rights reserved. This book is for personal learning. Pipeline changes should first be practised in a test repository, never directly on a production system. The author and publisher accept no liability for data loss or service disruption caused by running commands without a backup or an approval gate.

Who this book is for

A learner who knows JavaScript, React, Next.js, Node.js, Express and MongoDB, understands basic Git and Linux concepts, has studied Linux server administration, and wants to design, implement, operate and troubleshoot real CI/CD systems — from a first pull-request check to a production deployment with approval, verification and rollback.

Learning outcomes

By the end of this book you will be able to: explain the software delivery lifecycle and the difference between CI, continuous delivery and continuous deployment; design pipeline architecture, stages, triggers and dependencies; write and debug GitHub Actions workflows, GitLab CI pipelines and Jenkinsfiles; manage secrets, variables, permissions and OIDC authentication safely; use caching and artifacts to make pipelines fast and reproducible; integrate automated testing, linting, static analysis and security scanning as quality gates; build, tag, scan and publish Docker images with immutable references; manage environments, versioning and release promotion; choose and operate rolling, blue-green and canary deployments with safe rollback; and troubleshoot and govern production pipelines.

সূচিপত্র — Contents

PART I — CI/CD Foundations and Architecture

Chapter 01 — Understanding Software Delivery and Release Engineering 7

Chapter 02 — Continuous Integration, Continuous Delivery and Continuous Deployment 13

Chapter 03 — CI/CD Architecture and Pipeline Lifecycle 19

Chapter 04 — Git Branching, Triggers, Events and Workflow Design 26

Chapter 05 — Build Systems, Dependencies and Artifact Management 33

PART II — GitHub Actions Engineering

Chapter 06 — GitHub Actions Architecture and Core Concepts 41

Chapter 07 — Workflows, Jobs, Steps and Runners 47

Chapter 08 — Actions, Reusable Workflows and Composite Actions 53

Chapter 09 — Workflow Triggers, Expressions, Conditions and Matrices 59

Chapter 10 — Secrets, Variables, Permissions and OIDC 66

Chapter 11 — Caching, Artifacts and Workflow Optimization 72

PART III — GitLab CI/CD and Jenkins

Chapter 12 — GitLab CI/CD Architecture and Pipeline Configuration 80

Chapter 13 — GitLab Runners, Jobs, Stages and Artifacts 88

Chapter 14 — GitLab Variables, Environments and Deployment Controls 95

Chapter 15 — Jenkins Architecture, Controller and Agents 102

Chapter 16 — Jenkins Pipelines: Declarative and Scripted 108

Chapter 17 — Jenkins Credentials, Plugins and Pipeline Security 115

PART IV — Testing, Security and Release Engineering

Chapter 18 — Automated Testing in CI/CD Pipelines 123

Chapter 19 — Code Quality, Linting, Static Analysis and Security Scanning 130

Chapter 20 — Docker Image Build, Tagging, Scanning and Registry Publishing 137

Chapter 21 — Environment Promotion, Versioning and Release Management 143

Chapter 22 — Deployment Strategies, Rollbacks and Release Reliability 149

PART V — Production CI/CD Engineering

Chapter 23 — Building End-to-End CI/CD Pipelines for MERN Applications 156

Chapter 24 — Pipeline Troubleshooting, Governance and Production Best Practices 164

Companion Volumes

Lab Workbook — 24 Guided Labs • 15 Assignments • 3 Major Projects • Capstone

Assessment and Answer Guide — 120 MCQs • Short Answers • Scenarios • Case Studies • Final Exam

Quick Reference Handbook — Glossary (241 Terms) • YAML References • Checklists • Decision Trees

Guided labs (24), major projects (3) and the capstone are in the separate *CI/CD Lab Workbook*. MCQs, assignments, examinations and the answer key are in the separate *Assessment and Answer Guide*. Glossary, YAML references, checklists and decision trees are in the separate *Quick Reference Handbook*.

PART I • DEVOPS ENGINEERING MASTERY

CI/CD Foundations and Architecture

সফটওয়্যার ডেলিভারি, রিলিজ ইঞ্জিনিয়ারিং, CI/CD ধারণা, পাইপলাইন আর্কিটেকচার, Git ব্রাঞ্চিং ও বিল্ড সিস্টেম।

অধ্যায় 1 • PART I — CI/CD Foundations and Architecture

Chapter 1 — Understanding Software Delivery and Release Engineering

১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Software Delivery Lifecycle — ধারণা থেকে প্রোডাকশন পর্যন্ত সফটওয়্যার পৌঁছানোর পুরো পথ — ব্যাখ্যা করতে; (২) Release Engineering কী এবং কেন এটি শুধু "ফাইল আপলোড করা" নয় তা বোঝাতে; (৩) Source Code, Build Output, Release Artifact ও Deployed Application-এর পার্থক্য স্পষ্ট করতে; (৪) Manual Deployment-এর ঝুঁকি চিহ্নিত করতে; (৫) একটি ছোট MERN অ্যাপের জন্য ডেলিভারি প্রবাহের রূপরেখা আঁকতে।

১.২ কেন এই বিষয়টি আসল সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

কোড লেখা সফটওয়্যার তৈরির অর্ধেক কাজ; বাকি অর্ধেক হলো সেই কোডকে নিরাপদে, বারবার, অনুমানযোগ্যভাবে ব্যবহারকারীর কাছে পৌঁছে দেওয়া। অনেক দক্ষ ডেভেলপারও এই দ্বিতীয় অর্ধেকটি অবহেলা করেন — ফলে শুক্রবার রাতে হাতে-কলমে সার্ভারে ফাইল কপি করতে গিয়ে সাইট ভেঙে যায়, কোন সংস্করণটি চলছে কেউ জানে না, আর ভুল হলে ফিরে যাওয়ার (Rollback) কোনো উপায় থাকে না। Release Engineering এই বিশৃঙ্খলাকে একটি শৃঙ্খলাবদ্ধ, পুনরাবৃত্তিযোগ্য (Repeatable) প্রক্রিয়ায় বদলে দেয়। ভালো ডেলিভারি ব্যবস্থা থাকলে একটি ছোট দলও আত্মবিশ্বাসের সঙ্গে দিনে অনেকবার রিলিজ দিতে পারে; না থাকলে মাসে একবার রিলিজও ভয়ের ব্যাপার হয়ে দাঁড়ায়। এই বইয়ের প্রতিটি অধ্যায় এই ভিত্তির ওপর দাঁড়িয়ে আছে।

১.৩ পূর্বশর্ত জ্ঞান

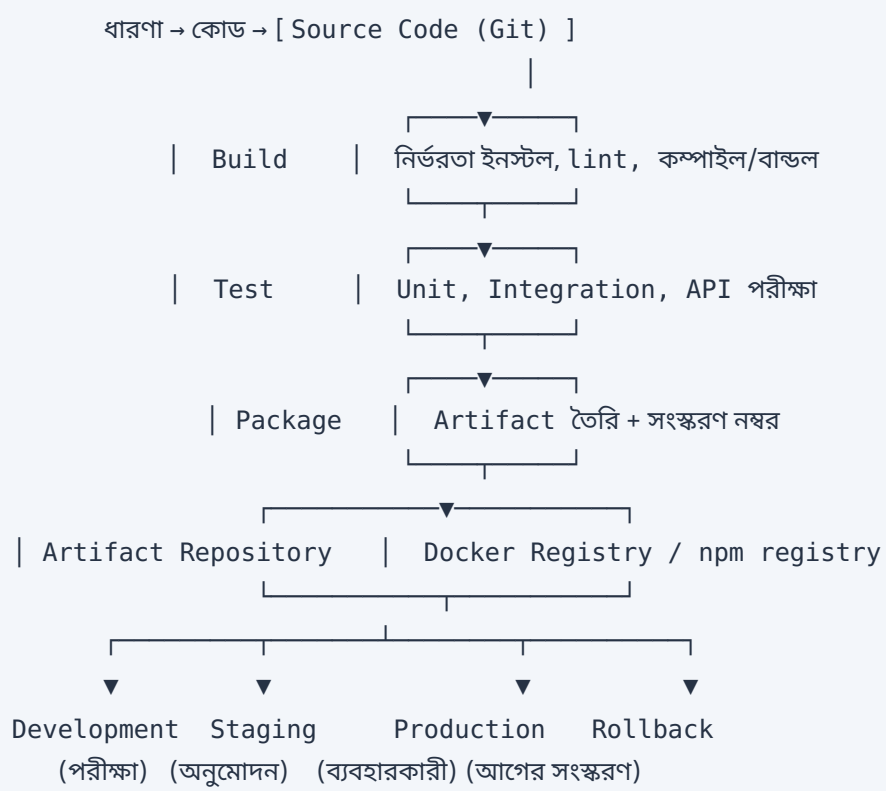
JavaScript, React/Next.js, Node.js/Express ও MongoDB-র প্রাথমিক ধারণা, Git-এর মৌলিক ব্যবহার (commit, push, branch) এবং Linux টার্মিনালে ফাইল দেখা ও কমান্ড চালানোর অভিজ্ঞতা থাকলেই চলবে। CI/CD, Docker বা Jenkins-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — এই অধ্যায়ে প্রতিটি ধারণা প্রথম নীতি থেকে পরিচয় করিয়ে দেওয়া হবে।

১.৪ প্রথম নীতি থেকে ব্যাখ্যা

সফটওয়্যার ডেলিভারি মানে একটি ধারণাকে ব্যবহারকারীর হাতে পৌঁছানোর ধারাবাহিক রূপান্তর। শুরুতে থাকে মানুষের পড়ার মতো **Source Code** — আপনার লেখা JavaScript, JSX, CSS। কম্পিউটার সরাসরি এই কোড চালায় না (বা চালালেও প্রোডাকশন-উপযোগী আকারে নয়); প্রথমে **Build** ধাপে কোডকে পরীক্ষা করে, নির্ভরতা (Dependency) জুড়ে, প্রয়োজনে কম্পাইল বা বান্ডল করে **Build Output** তৈরি করা হয়। সেই আউটপুটকে সংস্করণ-সংখ্যা (Version) দিয়ে মুড়ে, পরীক্ষা করে, স্বাক্ষরযোগ্য আকারে প্যাকেজ করলে তা হয় **Release Artifact** — যেমন একটি Docker Image বা একটি zip ফাইল। আর সেই Artifact যখন কোনো পরিবেশে (Environment) — যেমন Staging বা Production সার্ভারে — চালু করা হয়, তখন তা হয় **Deployed Application**। এই চারটি জিনিস এক নয়: Source Code বদলালেই Deployed Application বদলায় না; মাঝখানে Build, Test, Package ও Deploy ধাপগুলো পার হতে হয়। Release Engineering হলো এই পুরো পথটিকে নির্ভরযোগ্য, স্বচ্ছ ও পুনরাবৃত্তিযোগ্য করার শিল্প ও বিজ্ঞান।

> **CONCEPT:** "It works on my machine" — এই বাক্যটিই ডেলিভারি সমস্যার সারমর্ম। আপনার ল্যাপটপের পরিবেশ আর প্রোডাকশন সার্ভারের পরিবেশ আলাদা। Release Engineering-এর কাজ হলো এই ব্যবধানটি বন্ধ করা, যাতে যেকোনো মেশিনে একই কোড থেকে একই ফল আসে।

১.৫ আর্কিটেকচার / ধারণাগত চিত্র



প্রতিটি ধাপের আউটপুট পরের ধাপের ইনপুট। কোনো ধাপ ব্যর্থ হলে পরের ধাপে যাওয়া যাবে না — এটাই Pipeline-এর মূল নিয়ম।

১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Software Delivery Lifecycle (SDLC)

একটি ফিচারের জীবনচক্র: পরিকল্পনা (Plan) → কোড (Code) → Build → Test → Release → Deploy → Operate → Monitor → Feedback। আধুনিক দলে এই চক্র ছোট ছোট অংশে বারবার ঘোরে — সপ্তাহে বা দিনে একবার নয়, দিনে অনেকবার। চক্র যত ছোট, প্রতিটি পরিবর্তন তত ছোট ও নিরাপদ, আর ভুল ধরা পড়ে তত তাড়াতাড়ি। এই বইয়ে আমরা এই চক্রের Build থেকে Deploy পর্যন্ত অংশটি — অর্থাৎ CI/CD — গভীরভাবে শিখব।

Source Code, Build Output, Artifact ও Deployed Version — পার্থক্য

ধারণা	এটি কী	উদাহরণ (MERN অ্যাপ)	কোথায় থাকে
Source Code	মানুষের লেখা মূল কোড	`src/App.jsx`, `server.js`	Git repository

Build Output	Build ধাপের ফল — চালানোর উপযোগী ফাইল	Next.js-এর `.next/`, বান্ডল করা JS/CSS	Build মেশিনের অস্থায়ী ডিরেক্টরি
Release Artifact	সংস্করণ-সহ প্যাকেজ করা, অপরিবর্তনীয় (Immutable) বস্তু	Docker image `technova-web:1.4.0`	Container Registry
Deployed Application	কোনো পরিবেশে চলমান Artifact-এর কপি	Production সার্ভারে চলমান v1.4.0	Staging / Production সার্ভার

মনে রাখবেন: Build Output প্রতিবার Build করলে সামান্য বদলাতে পারে; কিন্তু একবার তৈরি হওয়া Release Artifact আর বদলানো যায় না — বদলাতে হলে নতুন সংস্করণ তৈরি করতে হয়। এই অপরিবর্তনীয়তা (Immutability) ডেলিভারির নির্ভরযোগ্যতার মূল চাবিকাঠি। Staging-এ যা পরীক্ষা করেছেন, Production-এ হুবহু সেটাই চালাতে চাইলে একই Artifact দুই জায়গায় Deploy করতে হবে।

Release Engineering-এর দায়িত্ব

Release Engineer বা DevOps Engineer এই কাজগুলো করেন: (১) সংস্করণ নির্ধারণ — Semantic Versioning অনুযায়ী `1.4.0` ধরনের নম্বর দেওয়া; (২) Build-এর পুনরাবৃত্তিযোগ্যতা নিশ্চিত করা — একই কোড থেকে সবসময় একই Artifact; (৩) Release Note লেখা — এই সংস্করণে কী বদলেছে; (৪) Deployment সমন্বয় — কখন, কোথায়, কীভাবে Deploy হবে; (৫) Rollback পরিকল্পনা — ভুল হলে আগের সংস্করণে ফেরা; (৬) Audit Trail — কে, কখন, কী Deploy করেছে তার রেকর্ড। ছোট দলে এই দায়িত্বগুলো ডেভেলপাররাই ভাগ করে নেন, তবে প্রক্রিয়াটি লিখিত ও স্বয়ংক্রিয় হওয়া চাই।

Manual Delivery বনাম Automated Delivery

Manual Delivery-তে একজন মানুষ SSH করে সার্ভারে ঢোকেন, `git pull` করেন, `npm install` করেন, সার্ভার রিস্টার্ট করেন। প্রতিটি ধাপে ভুলের সুযোগ: ভুল ব্রাঞ্চ pull করা, নির্ভরতা আপডেট না করা, রিস্টার্ট ভুলে যাওয়া। Automated Delivery-তে এই ধাপগুলো একটি Pipeline-এ লেখা থাকে — একই ধাপ, একই ক্রম, প্রতিবার। মানুষ শুধু সিদ্ধান্ত নেন (Deploy করব কি না), কাজটি করে যন্ত্র। ফলে ডেলিভারি দ্রুত, নিরাপদ ও হিসাবযোগ্য হয়।

১.৭ কনফিগারেশন উদাহরণ

একটি সহজ ডেলিভারি প্রবাহকে shell script আকারে লিখলে ধারণাটি স্পষ্ট হয়। এটি একটি ধারণাগত উদাহরণ — প্রোডাকশনে আমরা এই কাজ Pipeline দিয়ে করব:

```
#!/bin/bash
# deploy.sh — ধারণাগত উদাহরণ (সরাসরি প্রোডাকশনে চালাবেন না)
set -e  # কোনো ধাপ ব্যর্থ হলে এখানেই থামো
echo "1. কোড নেওয়া হচ্ছে..."
git pull origin main
echo "2. নির্ভরতা ইনস্টল..."
npm ci
echo "3. পরীক্ষা চালানো হচ্ছে..."
npm test
echo "4. Build..."
npm run build
echo "5. সার্ভার পুনরায় চালু..."
```

```
pm2 restart technova-api
echo "ডেলিভারি সম্পন্ন।"
```

এই স্ক্রিপ্টের প্রতিটি ধাপই পরের অধ্যায়গুলোতে একটি Pipeline Stage হবে। লক্ষ্য করুন `set -e` — পরীক্ষা ব্যর্থ হলে Deploy থেমে যায়। এটাই Pipeline-এর "Fail Fast" নীতি।

১.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: কাল্পনিক কোম্পানি TechNova-র MERN ই-কমার্স অ্যাপ app.technova.example — একজন ডেভেলপার শুক্রবার রাতে হাতে-কলমে Production সার্ভার (203.0.113.10) -এ `git pull` করে সাইট ভেঙে ফেললেন: তিনি ভুল করে একটি অসম্পূর্ণ ফিচার-ব্রাঞ্চের কোড টেনে এনেছিলেন। কোন সংস্করণটি চলছিল তার কোনো রেকর্ড ছিল না, তাই ফিরে যেতেও ঘণ্টা লেগে গেল। এই ঘটনার পর দল সিদ্ধান্ত নিল — আর হাতে Deploy নয়; প্রতিটি রিলিজ হবে সংস্করণ-সহ Artifact, Staging-এ পরীক্ষা করে, অনুমোদন নিয়ে Production-এ যাবে। এই বইয়ের শেষে আপনি ঠিক সেই ব্যবস্থাটি বানাতে পারবেন।

১.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার MERN অ্যাপের ডেলিভারি মানচিত্র আঁকা

1. একটি খাতায় বা ডায়াগ্রাম টুলে আপনার অ্যাপের বর্তমান ডেলিভারি পথ আঁকুন: কোড কোথায়, Build কোথায় হয়, Deploy কীভাবে হয়।
2. প্রতিটি ধাপের পাশে লিখুন — ধাপটি মানুষ করে, না যন্ত্র করে? ব্যর্থ হলে কী হয়?
3. Source Code, Build Output, Artifact ও Deployed Version — চারটির অবস্থান চিহ্নিত করুন।
4. একটি ভুলের দৃশ্য কল্পনা করুন (যেমন ভুল সংস্করণ Deploy) — ফিরে যেতে কত ধাপ লাগবে লিখুন।
5. কোন ধাপটি প্রথমে স্বয়ংক্রিয় করলে সবচেয়ে বেশি ঝুঁকি কমবে, তা চিহ্নিত করুন — সাধারণত Build ও Test।

১.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: "আমার ল্যাপটপে চলেছিল, সার্ভারে চলে না" — সাধারণত পরিবেশের পার্থক্য: Node.js সংস্করণ আলাদা, Environment Variable অনুপস্থিত, বা নির্ভরতার সংস্করণ ভিন্ন। সমাধান: Node সংস্করণ `.nvmrc` বা Dockerfile-এ বেঁধে দিন, Environment Variable-এর তালিকা ডকুমেন্ট করুন, আর `npm ci` ব্যবহার করুন যাতে `package-lock.json` অনুযায়ী হুবহু একই নির্ভরতা ইনস্টল হয়। আরেকটি সাধারণ ভুল: Deploy-এর পর কোন সংস্করণ চলছে তা না জানা — সমাধান: প্রতিটি Artifact-এ Git Commit Hash ও সংস্করণ নম্বর লেবেল হিসেবে লাগান।

- **ভুল:** Production সার্ভারে সরাসরি কোড সম্পাদনা করা। ফলে Git-এর কোড আর সার্ভারের কোড আলাদা হয়ে যায়।
- **ভুল:** `latest` ট্যাগ দিয়ে Deploy করা — পরে বোঝা যায় না কোন Build চলছে।
- **ভুল:** Rollback পরিকল্পনা ছাড়া রিলিজ দেওয়া।
- **ভুল ধারণা:** "Build সফল মানেই রিলিজ প্রস্তুত।" — Build সফল হওয়া শুধু প্রথম ধাপ; Test, Scan ও Staging যাচাই বাকি থাকে।

১.১১ নিরাপত্তা বিবেচনা

WARNING: ডেলিভারি প্রক্রিয়াতেই সবচেয়ে বড় নিরাপত্তা ঝুঁকি লুকিয়ে থাকে। Deploy করার ক্ষমতা মানেই Production-এ কোড চালানোর ক্ষমতা — তাই Deploy-এর অধিকার শুধু প্রয়োজনীয় মানুষের হাতে রাখুন। কখনোই Source Code-এ আসল পাসওয়ার্ড, API Key বা Database Credential লিখবেন না; সেগুলো Secret Manager বা Environment Variable-এ রাখুন। উদাহরণে শুধু কাল্পনিক প্লেসহোল্ডার ব্যবহার করুন, যেমন `CHANGE-ME-SECRET`।

১.১২ কর্মক্ষমতা ও খরচ বিবেচনা

প্রতিটি Build ও Deploy-এর খরচ আছে: Build মেশিনের সময়, Storage, নেটওয়ার্ক। ছোট দলের জন্য সবচেয়ে বড় খরচ সময় — Manual Deploy-এ একজন ডেভেলপারের ঘণ্টার পর ঘণ্টা নষ্ট হয়। স্বয়ংক্রিয় Pipeline শুরুতে সময় নেয়, কিন্তু প্রতিটি রিলিজে সেই সময় ফেরত দেয়। Artifact সংরক্ষণের খরচ নিয়ন্ত্রণে রাখতে পুরনো Artifact মুছে ফেলার নীতি (Retention Policy) রাখুন — যেমন শুধু শেষ ২০টি সংস্করণ রাখা।

১.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: প্রতিটি রিলিজে সংস্করণ নম্বর দিন; একই Artifact Staging ও Production-এ ব্যবহার করুন; প্রতিটি Deploy-এর রেকর্ড রাখুন; Rollback অনুশীলন করুন। **অ্যান্টি-প্যাটার্ন:** "Snowflake Server" — হাতে গড়া, অনন্য সার্ভার যা আর কখনো হুবহু বানানো যায় না; "Friday Deploy ভয়" — প্রক্রিয়া এত ভঙ্গুর যে সপ্তাহান্তে রিলিজ বন্ধ রাখতে হয়; ডকুমেন্টেশন ছাড়া Deploy — যেখানে পুরো প্রক্রিয়াটি একজনের মাথায় থাকে।

১.১৪ অধ্যায়ের সারাংশ

সফটওয়্যার ডেলিভারি হলো Source Code থেকে Deployed Application পর্যন্ত রূপান্তরের ধারাবাহিক পথ: Build, Test, Package, Deploy। Release Engineering এই পথকে নির্ভরযোগ্য, পুনরাবৃত্তিযোগ্য ও হিসাবযোগ্য করে। Source Code, Build Output, Release Artifact ও Deployed Application — চারটি আলাদা জিনিস; Artifact অপরিবর্তনীয় হওয়া চাই, যাতে Staging-এ পরীক্ষিত জিনিসটিই Production-এ চলে। Manual Delivery ভুলপ্রবণ; স্বয়ংক্রিয়, সংস্করণ-সহ ডেলিভারিই পেশাদার পথ।

১.১৫ মূল প্রযুক্তিগত পরিভাষা

Software Delivery Lifecycle: ধারণা থেকে ব্যবহারকারীর কাছে সফটওয়্যার পৌঁছানোর পুরো চক্র।

Release Engineering: রিলিজ তৈরি, সংস্করণ দেওয়া, Deploy ও Rollback পরিচালনার শৃঙ্খলা।

Source Code: মানুষের লেখা মূল কোড, Git-এ সংরক্ষিত।

Build Output: Build ধাপে তৈরি চালানোর উপযোগী ফাইলসমূহ।

Release Artifact: সংস্করণ-সহ, অপরিবর্তনীয়ভাবে প্যাকেজ করা বিল্ড।

Artifact Repository: Artifact সংরক্ষণ ও বিতরণের ভান্ডার (যেমন Container Registry)।

Environment: অ্যাপ চালানোর পরিবেশ — Development, Staging, Production।

Immutable: একবার তৈরি হলে আর বদলানো যায় না — এমন।

Rollback: সমস্যা হলে আগের স্থিতিশীল সংস্করণে ফিরে যাওয়া।

Repeatable / Idempotent: একই ইনপুটে প্রতিবার একই ফল দেওয়ার গুণ।

১.১৬ জ্ঞান যাচাই — প্রশ্ন

1. Source Code, Build Output, Release Artifact ও Deployed Application-এর মধ্যে পার্থক্য নিজের ভাষায় ব্যাখ্যা করুন।
2. কেন একই Artifact Staging ও Production — দুই জায়গায় Deploy করা উচিত?
3. Manual Deployment-এর তিনটি ঝুঁকি লিখুন এবং প্রতিটির জন্য স্বয়ংক্রিয় সমাধান বলুন।
4. Release Artifact অপরিবর্তনীয় (Immutable) হওয়া কেন গুরুত্বপূর্ণ?
5. Rollback পরিকল্পনা ছাড়া রিলিজ দিলে কী ধরনের ক্ষতি হতে পারে? একটি উদাহরণ দিন।

১.১৭ ব্যবহারিক অনুশীলন

1. **ডেলিভারি মানচিত্র:** আপনার একটি প্রজেক্টের বর্তমান ডেলিভারি পথের ডায়াগ্রাম আঁকুন এবং প্রতিটি ধাপে মানুষ না যন্ত্র কাজ করে তা চিহ্নিত করুন।
2. **সংস্করণ তালিকা:** একটি কাল্পনিক অ্যাপের জন্য Semantic Versioning অনুযায়ী ৫টি সংস্করণের তালিকা ও প্রতিটির পরিবর্তনের নোট লিখুন।
3. **স্ক্রিপ্ট বিশ্লেষণ:** এই অধ্যায়ের `deploy.sh` উদাহরণটি পড়ে বলুন — কোন ধাপটি ব্যর্থ হলে কী হবে, এবং স্ক্রিপ্টটিতে আর কী সুরক্ষা যোগ করা উচিত।

অধ্যায় 2 • PART I — CI/CD Foundations and Architecture

Chapter 2 — Continuous Integration, Continuous Delivery and Continuous Deployment

২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Continuous Integration (CI), Continuous Delivery (CD) ও Continuous Deployment-এর সংজ্ঞা ও পার্থক্য ব্যাখ্যা করতে; (২) প্রতিটি ধাপে কী স্বয়ংক্রিয় হয় এবং কী মানুষের সিদ্ধান্ত থাকে তা বলতে; (৩) একটি দলের জন্য কোন স্তরটি উপযুক্ত তা বিচার করতে; (৪) CI-এর মূল অভ্যাসগুলো — ছোট পরিবর্তন, ঘন ঘন মার্জ, স্বয়ংক্রিয় পরীক্ষা — প্রয়োগ করতে; (৫) Delivery ও Deployment-এর মধ্যে গুলিয়ে ফেলার ভুলটি এড়াতে।

২.২ কেন এই বিষয়টি আসল সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

CI/CD শব্দ তিনটি একসঙ্গে উচ্চারিত হয় বলে অনেকেই এগুলোকে একই জিনিস ভাবেন — কিন্তু এগুলো তিনটি আলাদা স্তর, প্রতিটির ঝুঁকি ও প্রয়োজন আলাদা। Continuous Integration না থাকলে দলের সদস্যদের কোড একসঙ্গে মেলাতে গিয়ে সপ্তাহের পর সপ্তাহ নষ্ট হয় — একে বলে Integration Hell। Continuous Delivery না থাকলে প্রতিটি রিলিজ একটি বিশেষ ঘটনা হয়ে দাঁড়ায়, যার জন্য সপ্তাহব্যাপী প্রস্তুতি লাগে। আর Continuous Deployment না বুঝে চালু করলে অপরীক্ষিত কোড সরাসরি ব্যবহারকারীর কাছে চলে যাওয়ার ঝুঁকি তৈরি হয়। এই তিন স্তর আলাদাভাবে বোঝা মানেই নিজের দলের জন্য সঠিক গতিতে এগোনোর সিদ্ধান্ত নিতে পারা।

২.৩ পূর্বশর্ত জ্ঞান

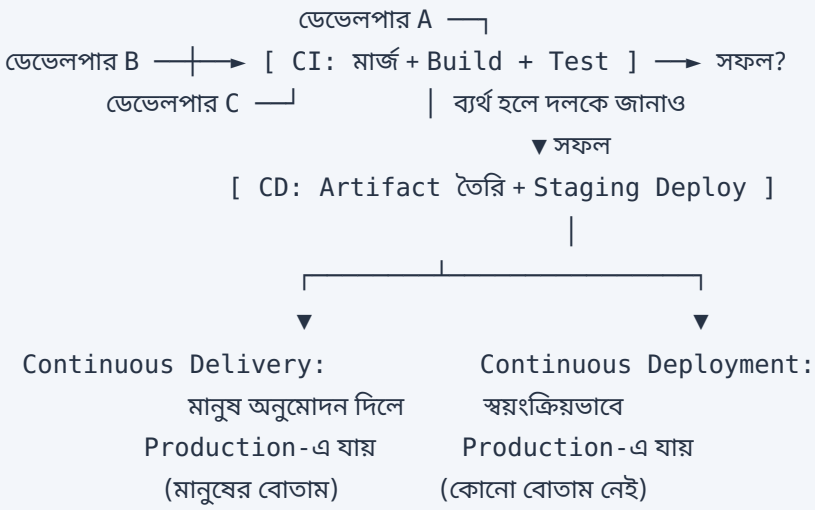
অধ্যায় ১-এর ধারণাগুলো — Source Code, Build, Artifact, Environment, Rollback — জানা থাকতে হবে। Git-এ branch ও merge-এর প্রাথমিক ধারণা এবং `npm test` বা `npm run build` চালানোর অভিজ্ঞতা থাকলে ভালো হয়।

২.৪ প্রথম নীতি থেকে ব্যাখ্যা

কল্পনা করুন চারজন ডেভেলপার একটি বাড়ি বানাচ্ছেন, কিন্তু প্রত্যেকে আলাদা ঘরে দেয়াল তুলছেন এবং মাস শেষে দেয়ালগুলো জোড়া লাগাতে চাইছেন — দেয়াল মিলবে না, এটাই নিশ্চিত। Continuous Integration বলে: প্রতিদিন, এমনকি দিনে কয়েকবার, নিজের কাজটি মূল কাঠামোর সঙ্গে মিলিয়ে দেখো — ছোট ছোট মিলন, তাই ভুল ছোট থাকে। **Continuous Integration (CI)** মানে ডেভেলপাররা ঘন ঘন (দিনে অন্তত একবার) নিজেদের কোড মূল শাখায় (Main Branch) মার্জ করেন, এবং প্রতিটি মার্জের পর স্বয়ংক্রিয়ভাবে Build ও Test চলে। **Continuous Delivery (CD)** এর পরের স্তর: কোড সবসময় Deploy-এর জন্য প্রস্তুত (Deployable) থাকে — যেকোনো মুহূর্তে একটি বোতাম চাপলেই Production-এ যাওয়া যায়, কিন্তু যাওয়ার সিদ্ধান্ত মানুষ নেন। **Continuous Deployment** আরও এক ধাপ এগিয়ে: প্রতিটি সফল পরীক্ষার পর কোড স্বয়ংক্রিয়ভাবে Production-এ চলে যায়, মানুষের বোতাম চাপার অপেক্ষা করে না। পার্থক্যটি একটি শব্দে: Delivery মানে "প্রস্তুত", Deployment মানে "চালু"।

> **CONCEPT:** CI ছাড়া CD হয় না, CD ছাড়া Continuous Deployment বিপজ্জনক। স্তরগুলো একটি সিঁড়ি: প্রথমে CI শক্ত করুন, তারপর Delivery, তারপর ভেবে-চিন্তে Deployment। অনেক দল বছরের পর বছর Delivery স্তরেই থাকে — এবং সেটাই সম্পূর্ণ বৈধ ও পেশাদার সিদ্ধান্ত।

২.৫ আর্কিটেকচার / ধারণাগত চিত্র



CI সবার জন্য বাধ্যতামূলক ভিত্তি; Delivery ও Deployment — দলের পরিপক্কতা অনুযায়ী পছন্দ।

২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Continuous Integration — গভীরে

CI-এর চারটি মূল অভ্যাস: (১) **একটি শেয়ার্ড Repository ও Main Branch** — সবাই একই মূল ধারা থেকে কাজ শুরু করেন; (২) **ঘন ঘন মার্জ** — দীর্ঘদিন আলাদা শাখায় কাজ না করে ছোট ছোট পরিবর্তন দ্রুত মার্জ করা; (৩) **স্বয়ংক্রিয় Build ও Test** — প্রতিটি মার্জ বা Pull Request-এ স্বয়ংক্রিয়ভাবে Build ও পরীক্ষা চলে; (৪) **দ্রুত প্রতিক্রিয়া (Fast Feedback)** — Build ব্যর্থ হলে কয়েক মিনিটের মধ্যে সংশ্লিষ্ট ডেভেলপার জানতে পারেন, এবং ভাঙা Build মেরামত করা দলের সর্বোচ্চ অগ্রাধিকার পায়। একটি সুস্থ CI ব্যবস্থায় Build ব্যর্থ হলে কেউ নতুন ফিচার শুরু করেন না — আগে ভাঙা Build ঠিক করা হয়। এই নিয়মের নাম "Build ভাঙা অবস্থায় রেখে যেও না"।

Continuous Delivery — গভীরে

Delivery-এর মূল প্রতিশ্রুতি: Main Branch সবসময় Production-এ যাওয়ার উপযুক্ত। এর জন্য দরকার: প্রতিটি পরিবর্তনে স্বয়ংক্রিয় পরীক্ষা, Staging পরিবেশে স্বয়ংক্রিয় Deploy, এবং Release-এর জন্য এক-ক্লিক (বা এক-কমান্ড) প্রক্রিয়া। Delivery-তে Production Deploy-এর আগে একটি **Approval Gate** (অনুমোদন ধাপ) থাকে — যেমন দলনেতা বা Product Owner অনুমোদন দেন। এই অনুমোদন একটি সিদ্ধান্ত, কোনো প্রযুক্তিগত বাধা নয়; প্রযুক্তিগতভাবে সবকিছু আগেই প্রস্তুত থাকে।

Continuous Deployment — গভীরে

Deployment-এ Approval Gate-টি সরিয়ে দেওয়া হয়: Main Branch-এ মার্জ হওয়া প্রতিটি পরিবর্তন সব পরীক্ষা পেরিয়ে সরাসরি Production-এ যায়। এটি সম্ভব হয় কেবল তখনই, যখন পরীক্ষা-ব্যবস্থা অত্যন্ত শক্তিশালী, Monitoring ও Rollback স্বয়ংক্রিয়, এবং দল ছোট পরিবর্তনে অভ্যস্ত। বড় বা নিয়ন্ত্রিত শিল্পে (যেমন ব্যাংকিং, স্বাস্থ্য) সাধারণত Continuous Deployment উপযুক্ত নয় — সেখানে মানুষের অনুমোদন আইনি বা নিয়ন্ত্রণগত প্রয়োজনে থাকে। তাই Deployment-কে "উন্নত" আর Delivery-কে "পিছিয়ে থাকা" ভাবা ভুল; এগুলো ভিন্ন প্রয়োজনের ভিন্ন পছন্দ।

তুলনা সারণি

বিষয়	Continuous Integration	Continuous Delivery	Continuous Deployment
স্বয়ংক্রিয় হয়	Build + Test	Build + Test + Staging Deploy + Artifact	সবকিছু, Production Deploy-সহ
মানুষের ভূমিকা	কোড লেখা ও মার্জ	Production Deploy-এর অনুমোদন	শুধু কোড ও পর্যবেক্ষণ
Production-এ যায়	না	অনুমোদনের পর	স্বয়ংক্রিয়ভাবে
ঝুঁকি	কম	মাঝারি — অনুমোদন সুরক্ষা দেয়	বেশি — পরীক্ষার ওপর নির্ভরশীল
কাদের জন্য	সব দল	বেশিরভাগ প্রোডাকশন দল	পরিপক্ক দল, শক্তিশালী পরীক্ষা থাকলে

CI/CD Pipeline বনাম Workflow

Pipeline হলো ধাপগুলোর ক্রম — Build, Test, Deploy — যা একটি পরিবর্তনকে Production পর্যন্ত নিয়ে যায়। **Workflow** হলো সেই Pipeline-এর সংজ্ঞা, সাধারণত YAML ফাইলে লেখা। একটি Repository-তে একাধিক Workflow থাকতে পারে: যেমন Pull Request যাচাইয়ের Workflow, রাতের পরীক্ষার Workflow, রিলিজের Workflow। এই বইয়ে আমরা GitHub Actions, GitLab CI/CD ও Jenkins — তিনটি মাধ্যমে Workflow লিখতে শিখব।

২.৭ কনফিগারেশন উদাহরণ

একটি MERN অ্যাপের জন্য সবচেয়ে সহজ CI Workflow (GitHub Actions) — ধারণাগত সম্পূর্ণ উদাহরণ। বিস্তারিত ব্যাখ্যা অধ্যায় ৬–৭-এ আসবে:

```
# .github/workflows/ci.yml — CI উদাহরণ (Node.js অ্যাপ)
name: CI
on:
  push:
    branches: [main]
  pull_request:
    branches: [main]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci
      - run: npm run lint
      - run: npm test
      - run: npm run build
```

এই Workflow-টি ব্যাখ্যা করলে: `on` বলে কখন চলবে (main-এ push বা Pull Request হলে), `jobs` বলে কী কাজ হবে, `runs-on` বলে কোন মেশিনে চলবে, আর `steps` ধাপে ধাপে কোড নেওয়া, Node সেটআপ, নির্ভরতা ইনস্টল, Lint, Test ও Build করে। কোনো ধাপ ব্যর্থ হলে Workflow ব্যর্থ হয় এবং Pull Request মার্জ আটকে যায় — এটাই CI-এর সুরক্ষা।

২.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র দলে আগে কোনো CI ছিল না। তিনজন ডেভেলপার দুই সপ্তাহ ধরে আলাদা শাখায় কাজ করে একদিনে মার্জ করতে গিয়ে দেখলেন একই ফাইলে শত শত Conflict — মেরামতে আরও তিন দিন গেল। CI চালুর পর নিয়ম হলো: প্রতিদিন অন্তত একবার main-এ মার্জ, প্রতিটি Pull Request-এ স্বয়ংক্রিয় Build ও Test। প্রথম মাসে Conflict প্রায় শূন্যে নেমে এলো। পরে দল Delivery স্তরে গেল — প্রতিটি মার্জের পর Staging-এ স্বয়ংক্রিয় Deploy (staging.technova.example), আর Production Deploy-এর জন্য দলনেতার অনুমোদন। Continuous Deployment-এ তারা এখনো যায়নি, কারণ পেমেন্ট-সংক্রান্ত ফিচারে মানুষের চোখ বুলানো তাদের নীতিমালার অংশ।

২.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার প্রজেক্টে প্রথম CI Workflow যোগ করা

1. আপনার MERN প্রজেক্টের Git Repository-তে `.github/workflows/ci.yml` ফাইল তৈরি করুন।
2. ওপরের উদাহরণটি কপি করে আপনার Node সংস্করণ ও স্ক্রিপ্টের নাম অনুযায়ী বদলান (`package.json`-এর `scripts` দেখুন)।
3. একটি ছোট পরিবর্তন করে Pull Request খুলুন — Workflow চলছে কি না দেখুন।
4. ইচ্ছাকৃতভাবে একটি পরীক্ষা ভেঙে (যেমন একটি ভুল assertion লিখে) আরেকটি Pull Request করুন — CI কীভাবে ব্যর্থতা জানায় দেখুন।
5. পরীক্ষা ঠিক করে আবার push করুন — সবুজ (সফল) চিহ্ন ফিরে আসা নিশ্চিত করুন।
6. দলের নিয়ম লিখুন: "লাল Build থাকলে নতুন ফিচার নয় — আগে Build ঠিক করো।"

২.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: CI "হঠাৎ" ব্যর্থ হলে প্রথমে দেখুন — ব্যর্থতাটি কি আপনার পরিবর্তনের কারণে, না পরিবেশের কারণে? নেটওয়ার্ক সমস্যায় নির্ভরতা ডাউনলোড ব্যর্থ হলে বা বাইরের সেবা বন্ধ থাকলে পরীক্ষা ভেঙে যেতে পারে। এমন Flaky Test (মাঝে মাঝে অকারণে ভাঙা পরীক্ষা) থাকলে তা আলাদাভাবে চিহ্নিত করে ঠিক করুন — Flaky Test-কে উপেক্ষা করার অভ্যাস CI-এর বিশ্বাস নষ্ট করে দেয়। আরেকটি সাধারণ সমস্যা: CI-তে পরীক্ষা চলে কিন্তু লোকাল মেশিনে চলে না, বা উল্টো — কারণ সাধারণত Node সংস্করণ বা Environment Variable-এর পার্থক্য। CI-তে Node সংস্করণ নির্দিষ্ট করে বেঁধে দিন।

- **ভুল:** দীর্ঘদিনের Feature Branch — মার্জের সময় Conflict-এর পাহাড়। সমাধান: ছোট ছোট Pull Request, ঘন ঘন মার্জ।
- **ভুল:** CI শুধু main-এ চালানো, Pull Request-এ নয় — ফলে ভাঙা কোড main-এ ঢুকে যায়।
- **ভুল:** পরীক্ষা ব্যর্থ হলে সেটি মুছে ফেলা বা উপেক্ষা করা। পরীক্ষা ভুল হলে পরীক্ষা ঠিক করুন, কোড ভুল হলে কোড।

- **ভুল ধারণা:** "CI মানেই শুধু স্বয়ংক্রিয় Test।" — CI হলো অভ্যাস + স্বয়ংক্রিয়তা, দুটো মিলিয়ে।

২.১১ নিরাপত্তা বিবেচনা

> **WARNING:** CI Workflow-তে বাইরের মানুষের Pull Request-এর কোড চালানো ঝুঁকিপূর্ণ — সেই কোডে লুকিয়ে থাকা নির্দেশ আপনার Secret পড়ে ফেলতে পারে। তাই Pull Request-এর Workflow-তে Production Secret দেবেন না, এবং অপরিচিত অবদানকারীর Pull Request-এ স্বয়ংক্রিয়ভাবে সংবেদনশীল ধাপ চালাবেন না। GitHub-এ Pull Request-এর জন্য সীমিত অনুমতির `GITHUB_TOKEN` ব্যবহার করা হয় — এই সুরক্ষাটি বুঝে তবেই Workflow লিখুন। বিস্তারিত অধ্যায় ১০-এ।

২.১২ কর্মক্ষমতা ও খরচ বিবেচনা

CI প্রতিটি push-এ চললে খরচ বাড়ে — বিশেষ করে হোস্টেড Runner-এর ক্ষেত্রে। খরচ কমাতে: অপ্রয়োজনীয় ফাইলের পরিবর্তনে (যেমন শুধু ডকুমেন্টেশন বদলে) Workflow না চালানোর নিয়ম (Path Filter) ব্যবহার করুন; নির্ভরতার Cache ব্যবহার করুন যাতে প্রতিবার `npm ci` শূন্য থেকে ডাউনলোড না করে; আর পরীক্ষাগুলো সমান্তরালে (Parallel) চালান। তবে সতর্কতা: গতি বাড়াতে গিয়ে পরীক্ষা কমিয়ে ফেলবেন না — CI-এর মূল্যই তার পরীক্ষায়।

২.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: দিনে অন্তত একবার মার্জ; ছোট Pull Request (কয়েকশো লাইনের বেশি নয়); সবুজ Build-কে পবিত্র মনে করা; ব্যর্থ Build দ্রুততম সময়ে ঠিক করা; Main Branch সবসময় Deployable রাখা। **অ্যান্টি-প্যাটার্ন:** "Integration Friday" — সপ্তাহের শেষে সব মার্জ একসঙ্গে করা; লাল Build-কে স্বাভাবিক মনে করা; পরীক্ষা ছাড়া CI — শুধু Build চালানোকে CI ভাবা; দীর্ঘদিনের শাখায় কাজ করে "বড় ধামাকা" মার্জ।

২.১৪ অধ্যায়ের সারাংশ

Continuous Integration মানে ঘন ঘন মার্জ ও স্বয়ংক্রিয় Build-Test; Continuous Delivery মানে কোড সবসময় Deploy-এর জন্য প্রস্তুত, মানুষের অনুমোদনে Production-এ যায়; Continuous Deployment মানে প্রতিটি সফল পরিবর্তন স্বয়ংক্রিয়ভাবে Production-এ যায়। তিনটি স্তর একটি সিঁড়ি — CI ভিত্তি, Delivery বেশিরভাগ দলের লক্ষ্য, Deployment পরিপক্ব দলের পছন্দ। Delivery ও Deployment-এর পার্থক্য একটি বোতাম: কে চাপে — মানুষ, না যন্ত্র।

২.১৫ মূল প্রযুক্তিগত পরিভাষা

Continuous Integration (CI): ঘন ঘন মার্জ ও স্বয়ংক্রিয় Build-Test-এর অভ্যাস।

Continuous Delivery (CD): কোড সবসময় Deploy-এর জন্য প্রস্তুত থাকা; Deploy মানুষের অনুমোদনে।

Continuous Deployment: প্রতিটি সফল পরিবর্তন স্বয়ংক্রিয়ভাবে Production-এ যাওয়া।

Main Branch: Repository-র মূল শাখা, যেখান থেকে রিলিজ তৈরি হয়।

Pull Request (PR): একটি শাখার পরিবর্তন মূল শাখায় মার্জ করার প্রস্তাব ও পর্যালোচনা।

Fast Feedback: পরিবর্তনের ফল কয়েক মিনিটের মধ্যে জানতে পারা।

Approval Gate: Deploy-এর আগে মানুষের অনুমোদনের ধাপ।

Flaky Test: অকারণে মাঝে মাঝে ভেঙে যাওয়া অনির্ভরযোগ্য পরীক্ষা।

Integration Hell: দীর্ঘদিন আলাদা কাজের পর মার্জ করতে গিয়ে বিশাল Conflict।

Deployable: যেকোনো মুহূর্তে Production-এ যাওয়ার উপযুক্ত অবস্থা।

২.১৬ জ্ঞান যাচাই — প্রশ্ন

1. Continuous Integration, Continuous Delivery ও Continuous Deployment-এর মধ্যে মূল পার্থক্য কী?
2. একটি দল Continuous Delivery-তে থাকলেও Continuous Deployment-এ না যাওয়ার বৈধ কারণ কী হতে পারে?
3. "ভাঙা Build মেরামত করা সর্বোচ্চ অগ্রাধিকার" — এই নিয়মটি কেন গুরুত্বপূর্ণ?
4. Flaky Test CI-এর বিশ্বাসযোগ্যতা কীভাবে নষ্ট করে?
5. ছোট Pull Request কেন বড় Pull Request-এর চেয়ে CI-এর জন্য ভালো?

২.১৭ ব্যবহারিক অনুশীলন

1. **প্রথম CI Workflow:** আপনার প্রজেক্টে ওপরের `ci.yml` যোগ করে একটি Pull Request-এ সবুজ চিহ্ন আনুন।
2. **ব্যর্থতা পরীক্ষা:** ইচ্ছাকৃতভাবে একটি পরীক্ষা ভেঙে দেখুন CI কীভাবে প্রতিক্রিয়া জানায়, তারপর ঠিক করুন।
3. **স্তর নির্বাচন:** একটি কাল্পনিক পেমেন্ট অ্যাপ ও একটি কাল্পনিক ব্লগ অ্যাপের জন্য কোন স্তর (Delivery না Deployment) উপযুক্ত, যুক্তিসহ লিখুন।

অধ্যায় 3 • PART I — CI/CD Foundations and Architecture

Chapter 3 — CI/CD Architecture and Pipeline Lifecycle

৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) CI/CD ব্যবস্থার মূল উপাদান — Version Control, CI Server, Runner, Artifact Repository, Deployment Target — চিহ্নিত করতে; (২) Pipeline, Stage, Job ও Step-এর স্তরবিন্যাস ব্যাখ্যা করতে; (৩) একটি পরিবর্তনের জীবনচক্র — Trigger থেকে Deploy ও পর্যবেক্ষণ পর্যন্ত — আঁকতে; (৪) Sequential, Parallel ও Conditional ধাপের পার্থক্য বলতে; (৫) Pipeline-এর ব্যর্থতা কোথায় ঘটেছে তা নির্ণয় করতে।

৩.২ কেন এই বিষয়টি আসল সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Pipeline-এর স্থাপত্য (Architecture) না বুঝে Workflow লেখা মানে মানচিত্র ছাড়া শহরে হাঁটা — প্রথমে কাজ করলেও সমস্যা হলে কোথায় আটকেছেন বুঝবেন না। Pipeline ধীর কেন, কোন ধাপটি ব্যর্থ হলো, কোন ধাপটি অন্য ধাপের ওপর নির্ভরশীল — এই প্রশ্নগুলোর উত্তর স্থাপত্যে লুকিয়ে থাকে। ভালো স্থাপত্য মানে দ্রুত প্রতিক্রিয়া, স্পষ্ট ব্যর্থতার বার্তা, আর সহজে বাড়ানো যায় এমন কাঠামো। খারাপ স্থাপত্য মানে একটিমাত্র বিশাল Job-এ সবকিছু গুঁজে দেওয়া — যা ধীর, বোঝা কঠিন, আর একটু ভাঙলেই পুরোটা ভেঙে যায়।

৩.৩ পূর্বশর্ত জ্ঞান

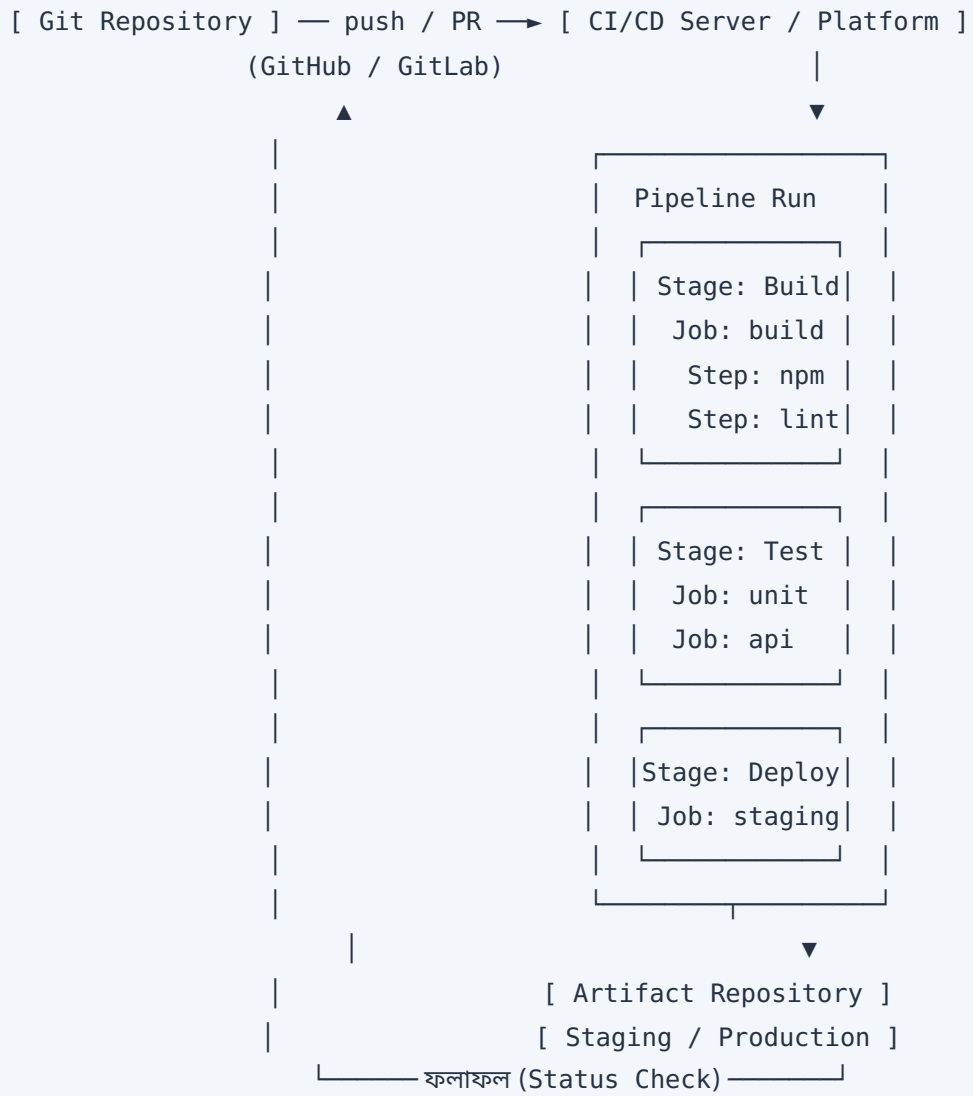
অধ্যায় ১–২-এর ধারণা — Build, Test, Artifact, Environment, CI/CD-এর তিন স্তর — জানা থাকতে হবে। Git-এ commit ও push, এবং YAML ফাইলের মৌলিক গঠন (key-value, তালিকা) চিনতে পারলেই চলবে।

৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

একটি Pipeline হলো একটি কারখানার অ্যাসেম্বলি লাইন। কাঁচামাল (Source Code) এক প্রান্তে ঢোকে, প্রতিটি স্টেশনে (Stage) কিছু কাজ হয় — যাচাই, জোড়া লাগানো, রং করা — আর শেষ প্রান্তে বেরিয়ে আসে প্রস্তুত পণ্য (Deployed Application)। কোনো স্টেশনে ত্রুটি ধরা পড়লে পণ্যটি আর এগোয় না; তাকে ফেরত পাঠানো হয়। এই লাইনের প্রতিটি স্টেশনের নাম **Stage** — যেমন Build Stage, Test Stage, Deploy Stage। প্রতিটি Stage-এর ভেতরে থাকে এক বা একাধিক **Job** — স্বাধীন কাজের একক, যেমন "Unit Test Job" বা "Lint Job"। আর প্রতিটি Job-এর ভেতরে থাকে **Step** — একেকটি নির্দেশ, যেমন "npm ci চালাও"। Stage → Job → Step — এই তিন স্তরই Pipeline-এর কঙ্কাল। আর যে মেশিনে Job চলে, তার নাম **Runner** (GitHub Actions-এ) বা **Agent** (Jenkins-এ) বা **Runner** (GitLab-এ)।

> **CONCEPT:** Pipeline-কে একটি গল্প হিসেবে ভাবুন: Trigger হলো গল্পের শুরু ("কিছু একটা ঘটল"), Stage হলো অধ্যায়, Job হলো দৃশ্য, Step হলো সংলাপ। গল্পটি শেষ হয় Deploy বা ব্যর্থতার বার্তায়। প্রতিটি Run-এর একটি লগ থাকে — গল্পটি ঠিক কীভাবে ঘটেছিল তার বিবরণ।

৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



Runner হলো সেই কর্মী, যে প্রতিটি Job চালায়; Artifact Repository হলো গুদাম, যেখানে তৈরি পণ্য রাখা হয়।

৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

মূল উপাদানসমূহ

উপাদান	কাজ	উদাহরণ
Version Control System	কোড সংরক্ষণ ও পরিবর্তনের ইতিহাস	Git, GitHub, GitLab
CI/CD Platform	Pipeline-এর সংজ্ঞা পড়ে Run পরিচালনা	GitHub Actions, GitLab CI, Jenkins
Runner / Agent	Job চালানোর মেশিন বা কন্টেইনার	Hosted Runner, Self-hosted Runner
Artifact Repository	তৈরি Artifact সংরক্ষণ ও বিতরণ	Container Registry, npm registry

Deployment Target	যেখানে অ্যাপ চলে	VPS, Kubernetes, ক্লাউড সার্ভিস
Secret Store	Credential নিরাপদে রাখা	GitHub Secrets, Vault
Observability	Pipeline ও অ্যাপের অবস্থা দেখা	লগ, মেট্রিক, অ্যালার্ট

Pipeline Lifecycle — একটি পরিবর্তনের যাত্রা

১. **Trigger:** কোনো ঘটনা Pipeline শুরু করে — push, Pull Request, সময়সূচি (Schedule), বা হাতে চালানো। ২. **Checkout:** Runner কোডের নির্দিষ্ট সংস্করণটি নামিয়ে নেয়। ৩. **Build:** নির্ভরতা ইনস্টল, Lint, কম্পাইল/বান্ডল। ৪. **Test:** Unit, Integration, API পরীক্ষা — ব্যর্থ হলে এখানেই থামে। ৫. **Package:** Artifact তৈরি ও সংস্করণ-সহ Registry-তে রাখা। ৬. **Deploy (Staging):** Staging-এ Deploy করে যাচাই। ৭. **Approval:** প্রয়োজনে মানুষের অনুমোদন। ৮. **Deploy (Production):** নির্বাচিত কৌশলে Production-এ যাওয়া। ৯. **Verify:** Health Check ও Smoke Test — অ্যাপ সত্যিই চলছে কি না। ১০. **Observe:** লগ ও মেট্রিক দেখা, সমস্যা হলে Rollback। এই চক্রের প্রতিটি ধাপের ফল লগে লেখা থাকে — তাই যেকোনো ব্যর্থতার কারণ খুঁজে পাওয়া যায়।

ধাপের সম্পর্ক: Sequential, Parallel, Conditional

Sequential ধাপ একের পর এক চলে — যেমন Test-এর আগে Build শেষ হতেই হবে। **Parallel** ধাপ একসঙ্গে চলে — যেমন Unit Test ও Lint একই সময়ে, তাতে মোট সময় কমে। **Conditional** ধাপ শুধু কোনো শর্তে চলে — যেমন Deploy ধাপ শুধু main শাখায়, শুধু সব পরীক্ষা সফল হলে। আর **Needs / Depends-on** দিয়ে বলা যায় কোন Job কোন Job-এর ওপর নির্ভরশীল। ভালো Pipeline ডিজাইন মানে নির্ভরতা যতটা সম্ভব কম রাখা, যাতে বেশি কাজ সমান্তরালে চলতে পারে।

Idempotency ও Repeatability

Idempotency মানে একই Pipeline দুইবার চালালে একই ফল — দ্বিতীয়বার চালানোয় কোনো ক্ষতি হয় না। যেমন একই Artifact দুইবার Deploy করলে একই সংস্করণই চলে। **Repeatability** মানে একই কোড থেকে প্রতিবার একই Artifact তৈরি হওয়া — এজন্য নির্ভরতার সংস্করণ বেঁধে রাখা (`package-lock.json`), Build-এ সময়-নির্ভর কিছু না ঢোকানো, আর পরিবেশ একই রাখা জরুরি। এই দুই গুণ না থাকলে "গতকাল চলেছিল, আজ চলে না" — এই রহস্যের সমাধান হয় না।

Pipeline Observability

Pipeline-এর স্বাস্থ্য বোঝা যায় চারটি মাপে: **Duration** (কতক্ষণ লাগে), **Success Rate** (কত শতাংশ Run সফল), **Time to Feedback** (ব্যর্থতা জানতে কতক্ষণ লাগে) ও **Flakiness** (অকারণ ব্যর্থতার হার)। এই মাপগুলো নিয়মিত দেখুন — Pipeline ধীরে ধীরে ধীর হয়ে যাওয়া একটি সাধারণ সমস্যা, যা নজরে না রাখলে একদিন অসহ্য হয়ে যায়।

৩.৭ কনফিগারেশন উদাহরণ

Stage, Job ও Step-এর স্তরবিন্যাস GitHub Actions YAML-এ কেমন দেখায় — ধারণাগত উদাহরণ:

```
# .github/workflows/pipeline.yml — Stage/Job/Step কাঠামো
name: Pipeline
on: [push]
jobs:
  lint:                      # Job 1 — Lint (স্বাধীন)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
```

```
      - run: npm run lint
  unit-test:                # Job 2 — Unit Test (lint-এর সঙ্গে সমান্তরালে)
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
  build:                    # Job 3 — Build (lint ও test-এর পরে)
    needs: [lint, unit-test]
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run build
```

এখানে `needs` দিয়ে Build Job-টি Lint ও Unit Test-এর ওপর নির্ভরশীল করা হয়েছে — ওরা ব্যর্থ হলে Build চলবে না। Lint ও Unit Test একসঙ্গে চলে, তাই মোট সময় কমে। এই কাঠামোটি GitLab ও Jenkins-এ ভিন্ন সিনট্যাক্সে একইভাবে প্রকাশ করা যায় — অধ্যায় ১২–১৬-এ দেখব।

৩.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র প্রথম Pipeline ছিল একটিমাত্র Job-এ সবকিছু: `npm ci && npm test && npm run build && deploy` — এক লাইনে। একদিন Deploy ব্যর্থ হলো, কিন্তু লগে বোঝা গেল না ব্যর্থতাটি Test-এ না Deploy-এ। দল Pipeline-কে ভেঙে চারটি Stage করল — Lint, Test, Build, Deploy — প্রতিটি আলাদা Job-এ। এখন ব্যর্থতা হলে GitHub-এর চেক-তালিকায় সরাসরি দেখা যায় কোন Job লাল হয়েছে, আর Lint ও Test সমান্তরালে চলায় মোট সময় ৯ মিনিট থেকে ৪ মিনিটে নেমে এলো। শিক্ষা: Pipeline-কে ছোট ছোট, নাম-সহ ধাপে ভাঙুন — ডিবাগ করা সহজ হবে।

৩.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার Pipeline-এর জীবনচক্র নথিভুক্ত করা

1. আপনার প্রজেক্টের বর্তমান Workflow ফাইলটি খুলুন (বা অধ্যায় ২-এর `ci.yml` ব্যবহার করুন)।
2. প্রতিটি Job-এর নামের পাশে লিখুন — এটি কোন Stage-এর অংশ, কীসের ওপর নির্ভরশীল, ব্যর্থ হলে কী হবে।
3. একটি কাগজে Trigger থেকে Deploy পর্যন্ত প্রবাহ আঁকুন এবং Parallel চলতে পারে এমন Job চিহ্নিত করুন।
4. একটি Job-কে দুই ভাগে ভাগ করে দেখুন — সময় কমে কি না, লগ পড়া সহজ হয় কি না।
5. Pipeline-এর Duration লিখে রাখুন — এটাই আপনার Baseline; পরে অপ্টিমাইজেশনের তুলনা এখান থেকে করবেন।

৩.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: "Pipeline হঠাৎ ধীর হয়ে গেল" — সাধারণ কারণ: Cache মিস হচ্ছে (প্রতিবার নির্ভরতা শূন্য থেকে ডাউনলোড হচ্ছে), একটি Job-এ অপ্রয়োজনীয় ভারী কাজ ঢুকে গেছে, বা সব Job Sequential হয়ে গেছে। পরীক্ষা করুন: প্রতিটি Job-এর Duration আলাদাভাবে দেখুন — সবচেয়ে ধীর Job-টিই Bottleneck। "Job আটকে আছে (Queued)" — Runner-এর অভাব বা Concurrency সীমা; অন্য Run চলছে কি না দেখুন। "Artifact পাওয়া যাচ্ছে না" — Upload ও Download Job-এর নাম ও পাথ মিলছে কি না যাচাই করুন; Artifact-এর নামে টাইপো সবচেয়ে সাধারণ ভুল।

- **ভুল:** একটিমাত্র বিশাল Job-এ সবকিছু করা — ডিবাগ কঠিন, সময় বেশি।
- **ভুল:** প্রতিটি Job-এ আলাদাভাবে `npm ci` চালিয়ে Cache না ব্যবহার করা।
- **ভুল:** Deploy Job-কে Test-এর ওপর নির্ভরশীল না করা — ফলে ব্যর্থ Build-ও Deploy হয়ে যায়।
- **ভুল ধারণা:** "Pipeline সফল মানেই অ্যাপ ঠিক আছে।" — Pipeline শুধু যা পরীক্ষা করা হয়েছে তা জানে; Health Check আলাদাভাবে দরকার।

৩.১১ নিরাপত্তা বিবেচনা

WARNING: Pipeline-এর প্রতিটি Job-কে শুধু তার প্রয়োজনীয় অনুমতি দিন (Least Privilege)। একটি Lint Job-এর Deploy-এর অধিকার থাকা উচিত নয়। Secret শুধু সেই Job-এ দিন যার সত্যিই দরকার, এবং লগে Secret কখনো ছাপাবেন না — যেমন `echo $API_KEY` কখনোই লিখবেন না। Runner-এর পরিবেশে অপরিচিত কোড চালানোর আগে ভাবুন — Runner-এর কাছে থাকা Credential সেই কোড পড়ে ফেলতে পারে।

৩.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Pipeline-এর খরচ তিন জায়গায়: Runner-এর সময়, Storage (Artifact ও Cache) ও নেটওয়ার্ক। Parallel Job সময় কমায় কিন্তু একসঙ্গে বেশি Runner ব্যবহার করে — হোস্টেড Runner-এর ক্ষেত্রে এটি খরচ বাড়াতে পারে। ভারসাম্য: দ্রুত প্রতিক্রিয়া চাইলে Parallel করুন, কিন্তু অপ্রয়োজনীয় Job চালাবেন না — যেমন শুধু README বদলালে পুরো Test Suite চালানো অপচয়। Cache-এর আকারের দিকে নজর রাখুন; বিশাল Cache ডাউনলোড করতেই Cache-এর সুবিধা শেষ হয়ে যেতে পারে।

৩.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ছোট ছোট নাম-সহ Job; স্পষ্ট নির্ভরতা (`needs`); দ্রুত ব্যর্থতা (Fail Fast) — Lint ব্যর্থ হলে ভারী Test চালাবেন না; প্রতিটি Run-এর লগ সংরক্ষণ; Pipeline-এর Duration নিয়মিত পর্যবেক্ষণ। **অ্যান্টি-প্যাটার্ন:** "God Job" — একটি Job-এ সবকিছু; লুকানো নির্ভরতা — Job A, Job B-র ফাইলের ওপর নির্ভরশীল কিন্তু তা ঘোষণা করা হয়নি; ব্যর্থতা উপেক্ষা করে `|| true` দিয়ে চাপা দেওয়া; একই Workflow-এর অনেক কপি — পরিবর্তন করতে গিয়ে একটি কপি ভুলে যাওয়া।

৩.১৪ অধ্যায়ের সারাংশ

CI/CD ব্যবস্থার মূল উপাদান: Version Control, CI/CD Platform, Runner, Artifact Repository, Deployment Target ও Secret Store। Pipeline-এর কাঠামো তিন স্তরে: Stage → Job → Step। একটি পরিবর্তনের জীবনচক্র: Trigger → Checkout → Build → Test → Package → Staging Deploy → Approval → Production Deploy → Verify → Observe। ধাপগুলো Sequential, Parallel বা Conditional হতে পারে; ভালো ডিজাইন নির্ভরতা কমিয়ে সমান্তরাল কাজ বাড়ায়। Idempotency ও Repeatability Pipeline-এর নির্ভরযোগ্যতার ভিত্তি।

৩.১৫ মূল প্রযুক্তিগত পরিভাষা

Pipeline: পরিবর্তনকে Production পর্যন্ত নিয়ে যাওয়া ধাপগুলোর ক্রম।

Stage: Pipeline-এর একটি বড় ধাপ, যেমন Build বা Test Stage।

Job: একটি Stage-এর ভেতরে স্বাধীন কাজের একক, Runner-এ চলে।

Step: একটি Job-এর ভেতরে একেকটি নির্দেশ বা Action।

Runner / Agent: Job চালানোর মেশিন বা কন্টেইনার।

Trigger: Pipeline শুরু করা ঘটনা — push, PR, সময়সূচি।

Needs / Dependency: কোন Job কোন Job-এর পরে চলবে তার ঘোষণা।

Idempotency: একই কাজ দুইবার করলে একই ফল পাওয়ার গুণ।

Observability: Pipeline ও সিস্টেমের অবস্থা লগ, মেট্রিক ও অ্যালার্ট দিয়ে দেখার ক্ষমতা।

Fail Fast: সমস্যা ধরা পড়লেই দ্রুত থেমে যাওয়া, যাতে সময় ও সম্পদ নষ্ট না হয়।

৩.১৬ জ্ঞান যাচাই — প্রশ্ন

1. Stage, Job ও Step-এর মধ্যে পার্থক্য একটি উদাহরণসহ ব্যাখ্যা করুন।
2. একটি পরিবর্তনের জীবনচক্রে Trigger থেকে Verify পর্যন্ত ধাপগুলো ক্রমানুসারে লিখুন।
3. Parallel Job কখন সময় বাঁচায়, আর কখন খরচ বাড়ায়?

4. Idempotency না থাকলে Deploy-এ কী ধরনের সমস্যা হতে পারে?
5. Pipeline-এর Duration হঠাৎ বেড়ে গেলে আপনি প্রথমে কী কী পরীক্ষা করবেন?

৩.১৭ ব্যবহারিক অনুশীলন

1. **কাঠামো আঁকা:** অধ্যায় ২-এর `ci.yml`-কে Stage/Job/Step ডায়াগ্রামে রূপান্তর করুন।
2. **সমান্তরাল করা:** Lint ও Unit Test-কে আলাদা Job-এ ভাগ করে `needs` দিয়ে Build-কে তাদের ওপর নির্ভরশীল করুন; সময়ের পার্থক্য মাপুন।
3. **ব্যর্থতা নির্ণয়:** ইচ্ছাকৃতভাবে একটি Step ভেঙে লগ পড়ে বলুন — কোন Stage, কোন Job, কোন Step ব্যর্থ হয়েছে।

অধ্যায় 4 • PART I — CI/CD Foundations and Architecture

Chapter 4 — Git Branching, Triggers, Events and Workflow Design

৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Git Branching কৌশল — Trunk-Based Development, Git Flow, GitHub Flow — ব্যাখ্যা ও তুলনা করতে; (২) দলের আকার ও রিলিজ ছন্দ অনুযায়ী সঠিক কৌশল বেছে নিতে; (৩) Workflow Trigger ও Event-এর সম্পর্ক ব্যাখ্যা করতে; (৪) Branch, Path ও Tag-ভিত্তিক Trigger লিখতে; (৫) Pull Request-ভিত্তিক Workflow ডিজাইন করতে যেখানে ভাঙা কোড main-এ ঢুকতে পারে না।

৪.২ কেন এই বিষয়টি আসল সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Branching কৌশলই ঠিক করে দেয় দল কী গতিতে কাজ করতে পারবে। ভুল কৌশলে ছোট দলও জটিল মার্জ-নীতিতে আটকে যায়; সঠিক কৌশলে বড় দলও দ্রুত এগোয়। একইভাবে Trigger ডিজাইন ঠিক করে দেয় Pipeline কখন চলবে আর কখন চলবে না — ভুল Trigger মানে হয় প্রতিটি ছোট পরিবর্তনে ভারী Pipeline চলা (খরচ ও সময়ের অপচয়), নয়তো গুরুত্বপূর্ণ পরিবর্তনে Pipeline না চলা (ঝুঁকি)। Branching ও Trigger একসঙ্গে মিলে দলের কাজের ছন্দ তৈরি করে; এই ছন্দ ভেঙে গেলে CI/CD-এর সব সুবিধাই ম্লান হয়ে যায়।

৪.৩ পূর্বশর্ত জ্ঞান

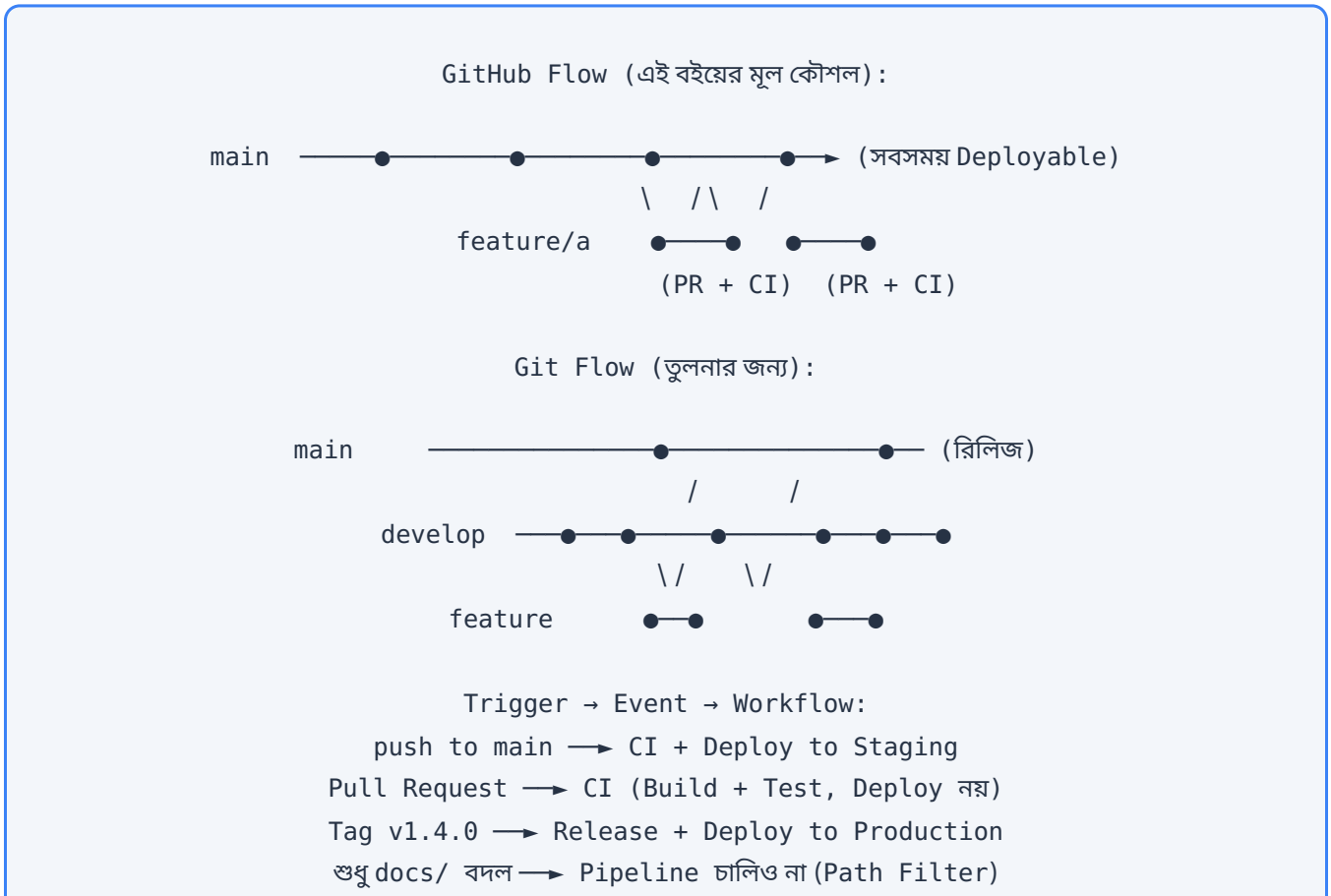
Git-এর Branch, Merge, Pull Request-এর ধারণা এবং অধ্যায় ২–৩-এর CI Workflow ও Pipeline Lifecycle জানা থাকতে হবে। YAML-এ তালিকা ও শর্ত লেখার প্রাথমিক ধারণা থাকলে ভালো হয়।

৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

Branch মানে কোডের ইতিহাসের একটি সমান্তরাল পথ — মূল ধারা থেকে আলাদা হয়ে কিছু পরিবর্তন করা, তারপর আবার মূল ধারায় মিলিয়ে দেওয়া। প্রশ্ন হলো: কতদিন আলাদা থাকবেন, কে অনুমোদন দেবে, আর কোন শাখা থেকে রিলিজ হবে? **GitHub Flow** বলে: একটি মাত্র মূল শাখা (main) আছে; প্রতিটি কাজের জন্য ছোট শাখা খুলুন, Pull Request করুন, পর্যালোচনা ও CI পেরিয়ে মার্জ করুন, তারপর Deploy করুন। সহজ, দ্রুত, ছোট দলের জন্য আদর্শ। **Trunk-Based Development** আরও এগিয়ে: শাখা খুবই স্বল্পস্থায়ী (এক দিনের কম), ডেভেলপাররা সরাসরি main-এ ছোট ছোট পরিবর্তন মার্জ করেন — Feature Flag দিয়ে অসম্পূর্ণ ফিচার লুকিয়ে রাখা হয়। এটি সবচেয়ে দ্রুত, তবে শক্তিশালী CI ও পরীক্ষা দরকার। **Git Flow** বলে: main, develop, feature, release ও hotfix — অনেকগুলো দীর্ঘস্থায়ী শাখা থাকবে, রিলিজ একটি আনুষ্ঠানিক প্রক্রিয়া। এটি নিয়ন্ত্রিত রিলিজ-চক্রের জন্য ভালো, কিন্তু জটিল ও ধীর। আর **Trigger** হলো Pipeline-কে জাগিয়ে তোলা ঘটনা — যেমন main-এ push হলে, Pull Request খুললে, Tag তৈরি হলে, বা নির্দিষ্ট সময়ে। Event হলো সেই ঘটনার বিবরণ: কে push করল, কোন শাখায়, কোন ফাইল বদলাল। Workflow ডিজাইন মানে এই দুটিকে মেলানো: কোন ঘটনায় কোন Pipeline চলবে।

> **CONCEPT:** Branching কৌশল একটি বাণিজ্য-বিনিময় (Trade-off): নিয়ন্ত্রণ বনাম গতি। Git Flow বেশি নিয়ন্ত্রণ দেয়, কম গতি; Trunk-Based বেশি গতি দেয়, কম নিয়ন্ত্রণ — কিন্তু সেই নিয়ন্ত্রণ CI, পরীক্ষা ও Feature Flag দিয়ে পূরণ করা হয়। আপনার দলের জন্য সঠিক ভারসাম্যটি বেছে নিন, অন্য দলেরটা অন্ধভাবে নকল করবেন না।

৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



ছোট দলের জন্য GitHub Flow দিয়ে শুরু করুন; প্রয়োজনে পরে জটিল করুন — উল্টোটা কঠিন।

৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Branching কৌশল তুলনা

কৌশল	শাখার আয়ু	সুবিধা	অসুবিধা	কাদের জন্য
Trunk-Based	ঘণ্টা–১ দিন	সবচেয়ে দ্রুত, Conflict কম	শক্তিশালী CI ও Feature Flag দরকার	পরিপক্ক, ছোট-মাঝারি দল
GitHub Flow	দিন–কয়েক দিন	সহজ, PR পর্যালোচনা স্পষ্ট	দীর্ঘ ফিচারে শাখা পুরনো হয়	বেশিরভাগ দল — এই বইয়ের মূল পছন্দ
Git Flow	সপ্তাহ–মাস	নিয়ন্ত্রিত রিলিজ, একাধিক সংস্করণ	জটিল, মার্জ-খরচ বেশি	নির্দিষ্ট রিলিজ-চক্রের বড় দল

Branch সুরক্ষা (Branch Protection)

Main শাখাকে সুরক্ষিত করুন: সরাসরি push নিষিদ্ধ, Pull Request বাধ্যতামূলক, অন্তত একজন পর্যালোচকের অনুমোদন, CI-এর সবুজ চিহ্ন ছাড়া মার্জ নিষিদ্ধ, আর Force Push নিষিদ্ধ। এই নিয়মগুলো GitHub-এ Repository Settings →

Branches-এ সেট করা যায়। সুরক্ষা ছাড়া CI শুধু পরামর্শ — সুরক্ষা থাকলে CI একটি দরজা, যা ভাঙা কোডকে আটকে দেয়।

Event ও Trigger — গভীরে

GitHub Actions-এ `on` দিয়ে Trigger লেখা হয়। সাধারণ Event: `push` (কোড পাঠানো), `pull_request` (PR খোলা/আপডেট), `schedule` (নির্দিষ্ট সময়ে, cron), `workflow_dispatch` (হাতে চালানো), `release` (রিলিজ তৈরি), `tag` (push-এর সঙ্গে ট্যাগ)। প্রতিটি Event-এর সঙ্গে **Context** থাকে — যেমন কোন শাখা, কে করেছে, কোন ফাইল বদলেছে। Path Filter দিয়ে বলা যায়: শুধু `src/**` বদলালে Test চলবে, শুধু `docs/**` বদলালে চলবে না। Branch Filter দিয়ে বলা যায়: Deploy শুধু main-এ।

Pull Request Workflow ডিজাইন

একটি ভালো PR Workflow: (১) PR খুললেই CI চলে — Lint, Test, Build; (২) ফলাফল PR-এ Status Check হিসেবে দেখা যায়; (৩) ব্যর্থ হলে মার্জ বোতাম বন্ধ থাকে; (৪) পর্যালোচক কোড দেখে অনুমোদন দেন; (৫) মার্জের পর Staging-এ স্বয়ংক্রিয় Deploy হয়। এই প্রবাহে main কখনো ভাঙা অবস্থায় থাকে না — কারণ ভাঙা কোড PR-স্তরেই আটকে যায়।

Feature Flag ও দীর্ঘ ফিচার

একটি বড় ফিচার এক সপ্তাহের বেশি লাগলে শাখা দীর্ঘ হয়, Conflict বাড়ে। সমাধান: ছোট ছোট অংশে মার্জ করুন, আর অসম্পূর্ণ ফিচারটি Feature Flag (একটি সেটিংস যা ফিচারটি চালু/বন্ধ করে) দিয়ে লুকিয়ে রাখুন। এতে কোড main-এ থাকে, কিন্তু ব্যবহারকারী ফিচারটি দেখে না — ফিচার শেষ হলে Flag চালু করে দিলেই হলো। Feature Flag-এর বিস্তারিত অধ্যায় ২২-এ।

৪.৭ কনফিগারেশন উদাহরণ

Trigger-এর নমুনা — কোন ঘটনায় কী চলবে:

```
# .github/workflows/ci.yml — Trigger ডিজাইন
name: CI
on:
  push:
    branches: [main]
    paths-ignore: ['docs/**', '**.md']   # শুধু ডক বদলালে চলবে না
  pull_request:
    branches: [main]
    paths: ['src/**', 'server/**', 'package*.json']
  workflow_dispatch:   # হাতে চালানোর সুযোগ
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm test
```

আর একটি Release Trigger — Tag তৈরি হলে Production Deploy:

```
# .github/workflows/release.yml — Tag-ভিত্তিক রিলিজ
name: Release
on:
```

```
  push:
    tags: ['v*.*.*']   # যেমন v1.4.0
jobs:
  release:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - run: npm ci
      - run: npm run build
      # এরপর Artifact তৈরি ও Deploy — অধ্যায় ২০–২২-এ বিস্তারিত
```

Shell-এ Tag তৈরি করা হয় এভাবে: `git tag v1.4.0 && git push origin v1.4.0`। Tag মানে কোডের একটি নির্দিষ্ট মুহূর্তে নাম লাগানো — রিলিজের জন্য অপরিহার্য।

৪.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র দলে এক ডেভেলপার একটি বড় ফিচারের জন্য তিন সপ্তাহের শাখা খুললেন। মার্জের দিন দেখা গেল main অনেক এগিয়ে গেছে — Conflict মেরামতেই দুই দিন গেল, আর তার মধ্যে নতুন বাগ ঢুকে পড়ল। এরপর দল নিয়ম করল: কোনো শাখা তিন দিনের বেশি বাঁচবে না; বড় ফিচার ভেঙে ছোট ছোট PR-এ মার্জ হবে, Feature Flag দিয়ে লুকানো থাকবে। একই সঙ্গে Trigger ঠিক করা হলো: শুধু `docs/` বদলালে আর পুরো Test Suite চলে না — এতে প্রতিদিন ঘণ্টাখানেক Runner-সময় বাঁচে।

৪.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: GitHub Flow + Branch Protection চালু করা

1. আপনার Repository-তে main শাখা থেকে `feature/short-task` নামে একটি ছোট শাখা খুলুন।
2. একটি ছোট পরিবর্তন করে Pull Request খুলুন — CI চলছে কি না দেখুন।
3. Repository Settings → Branches-এ গিয়ে main-এর জন্য Branch Protection চালু করুন: "Require a pull request before merging" ও "Require status checks to pass" নির্বাচন করুন।
4. এবার সরাসরি main-এ push করার চেষ্টা করুন — বাধা পাওয়ার কথা; এটাই সুরক্ষা কাজ করছে তার প্রমাণ।
5. PR-এ অনুমোদন দিয়ে মার্জ করুন, তারপর একটি Tag দিন: `git tag v0.1.0 && git push origin v0.1.0`।
6. দলের জন্য এক পৃষ্ঠার Branching নীতিমালা লিখুন: শাখার নামের নিয়ম, সর্বোচ্চ আয়ু, PR-এর আকার।

৪.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: "PR-এ CI চলছে না" — প্রথমে দেখুন Workflow ফাইলটি কোন শাখায় আছে: GitHub Actions Pull Request-এর Trigger পড়ে PR-এর শাখার ফাইল থেকে, main-এর ফাইল থেকে নয়। তাই নতুন Workflow যোগ করার PR-এ সেটি চলার কথা — না চললে YAML-এর Indentation বা `on.pull_request.branches` ফিল্টার যাচাই করুন। "Tag দিলাম, Release Workflow চলল না" — Tag-এর নাম `v*.*.*` প্যাটার্ন মিলছে কি না দেখুন; `v1.4` মিলবে না, `v1.4.0` মিলবে। "Conflict বারবার আসছে" — শাখার আয়ু কমান, ঘন ঘন main থেকে আপডেট নিন (`git pull origin main`)।

- **ভুল:** শাখার নামে কোনো নিয়ম না থাকা — `test123`, `final-final2` ধরনের নামে কিছু বোঝা যায় না। নিয়ম রাখুন: `feature/<কাজ>`, `fix/<বাগ>`, `hotfix/<জরুরি>`।
- **ভুল:** প্রতিটি push-এ পুরো Pipeline চালানো — ডকুমেন্টেশন বদলেও ভারী Test চলে। Path Filter ব্যবহার করুন।
- **ভুল:** Branch Protection ছাড়া CI — CI শুধু জানায়, আটকায় না।
- **ভুল ধারণা:** "বেশি শাখা মানেই বেশি নিরাপত্তা।" — বেশি শাখা মানেই বেশি মার্জ-খরচ।

৪.১১ নিরাপত্তা বিবেচনা

WARNING: Branch Protection ছাড়া main শাখায় যে কেউ সরাসরি push করতে পারে — এমনকি ভুল করে বা ক্ষতিকর উদ্দেশ্যে। Force Push main-এর ইতিহাস মুছে ফেলতে পারে, তাই তা নিষিদ্ধ রাখুন। Fork থেকে আসা Pull Request-এর Workflow-তে Secret দেবেন না — কারণ Fork-এর কোড আপনার নিয়ন্ত্রণে নেই, আর সেই কোড Secret পড়ে ফেলতে পারে। Tag-ভিত্তিক Production Deploy-এর ক্ষেত্রে Tag তৈরি করার অধিকার সীমিত রাখুন — Tag মানেই রিলিজের সিদ্ধান্ত।

৪.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Trigger ডিজাইন সরাসরি খরচের সঙ্গে জড়িত। প্রতিটি push-এ পুরো Pipeline চালালে দিনে শত শত Run হতে পারে। Path Filter, Branch Filter ও Concurrency নিয়ন্ত্রণ (একই শাখায় নতুন push এলে পুরনো Run বাতিল) দিয়ে অপ্রয়োজনীয় Run কমান। GitHub Actions-এ একই শাখার পুরনো Run বাতিল করার জন্য `concurrency` ব্যবহার করা হয় — বিস্তারিত অধ্যায় ৯-এ।

৪.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: ছোট, স্বল্পস্থায়ী শাখা; স্পষ্ট নামের নিয়ম; Branch Protection; PR-এ CI বাধ্যতামূলক; বড় ফিচারে Feature Flag; Trigger-এ Path ও Branch Filter। **অ্যান্টি-প্যাটার্ন:** দীর্ঘস্থায়ী শাখা; সরাসরি main-এ push; "মার্জের আগে একবার দেখে নেব" — কিন্তু কোনো নিয়ম না থাকা; প্রতিটি পরিবর্তনে পুরো Pipeline চালানো; Tag ছাড়া রিলিজ দেওয়া — পরে কোন কোডটি Production-এ গেল বোঝা যায় না।

৪.১৪ অধ্যায়ের সারাংশ

Branching কৌশল দলের গতির ছন্দ ঠিক করে: Trunk-Based সবচেয়ে দ্রুত, GitHub Flow সহজ ও ভারসাম্যপূর্ণ, Git Flow সবচেয়ে নিয়ন্ত্রিত। এই বইয়ের মূল পছন্দ GitHub Flow — ছোট শাখা, Pull Request, CI ও Branch Protection।

Trigger ও Event ঠিক করে কোন ঘটনায় কোন Workflow চলবে; Path ও Branch Filter দিয়ে অপ্রয়োজনীয় Run কমানো যায়। Tag দিয়ে রিলিজ চিহ্নিত করা হয় — Tag মানেই একটি নির্দিষ্ট কোড-মুহূর্তের নাম।

৪.১৫ মূল প্রযুক্তিগত পরিভাষা

Branch: কোডের ইতিহাসের সমান্তরাল পথ; আলাদা কাজের জন্য।

Trunk-Based Development: খুব ছোট, স্বল্পস্থায়ী শাখায় main-এ ঘন ঘন মার্জের কৌশল।

GitHub Flow: একটি main শাখা, ছোট Feature শাখা ও Pull Request-ভিত্তিক কৌশল।

Git Flow: main, develop, feature, release, hotfix শাখা-সহ নিয়ন্ত্রিত কৌশল।

Branch Protection: main শাখায় সরাসরি push ও Force Push আটকানোর নিয়ম।

Trigger / Event: Pipeline শুরু করা ঘটনা ও তার বিবরণ।

Path Filter: কোন ফাইল বদলালে Workflow চলবে তা নির্ধারণের নিয়ম।

Tag: কোডের নির্দিষ্ট মুহূর্তে লাগানো নাম, যেমন v1.4.0।

Feature Flag: কোড Deploy করে ফিচারটি চালু/বন্ধ করার সেটিংস।

Merge Conflict: একই জায়গায় দুটি ভিন্ন পরিবর্তন মেলাতে না পারার অবস্থা।

৪.১৬ জ্ঞান যাচাই — প্রশ্ন

1. Trunk-Based, GitHub Flow ও Git Flow-এর মধ্যে কোনটি আপনার দলের জন্য উপযুক্ত হবে — যুক্তিসহ বলুন।
2. Branch Protection ছাড়া CI কেন অসম্পূর্ণ সুরক্ষা দেয়?
3. Path Filter ব্যবহার না করলে কী ধরনের অপচয় হয়? উদাহরণ দিন।
4. Tag-ভিত্তিক রিলিজ কেন Commit Hash-ভিত্তিক রিলিজের চেয়ে বোঝা সহজ?
5. একটি বড় ফিচারকে ছোট ছোট PR-এ ভাগ করার দুটি উপায় লিখুন।

৪.১৭ ব্যবহারিক অনুশীলন

1. **Branch Protection:** আপনার Repository-তে main শাখার সুরক্ষা চালু করে সরাসরি push আটকে যাওয়া যাচাই করুন।
2. **Trigger পরীক্ষা:** Path Filter-সহ Workflow লিখে শুধু `docs/` বদলে দেখুন Pipeline চলে কি না, আর `src/` বদলে চলে কি না।
3. **Tag রিলিজ:** একটি Tag দিয়ে Release Workflow চালানোর পরীক্ষা করুন এবং Tag-এর নামের প্যাটার্ন বদলে দেখুন কী হয়।

অধ্যায় 5 • PART I — CI/CD Foundations and Architecture

Chapter 5 — Build Systems, Dependencies and Artifact Management

৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Build System কী এবং Node.js-এ Build মানে কী তা ব্যাখ্যা করতে; (২) নির্ভরতা (Dependency) পরিচালনা করতে — `package.json`, `package-lock.json`, `npm ci` বনাম `npm install`; (৩) Artifact তৈরি, সংস্করণ দেওয়া ও সংরক্ষণের নিয়ম বলতে; (৪) Semantic Versioning প্রয়োগ করতে; (৫) একটি Node.js অ্যাপের জন্য Dockerfile-এর মৌলিক কাঠামো লিখতে ও Multi-stage Build-এর ধারণা ব্যাখ্যা করতে।

৫.২ কেন এই বিষয়টি আসল সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Build হলো Pipeline-এর প্রথম ও সবচেয়ে মৌলিক ধাপ — এখানে ভুল হলে পরের সব ধাপই ভুল ভিত্তির ওপর দাঁড়ায়। নির্ভরতার সংস্করণ একটু বদলে গেলেই "গতকাল চলেছিল, আজ চলে না" — এই রহস্য তৈরি হয়। আর Artifact ব্যবস্থাপনা না থাকলে Staging-এ পরীক্ষিত জিনিসটি Production-এ যায় না — যায় অন্য কিছু, যা কেউ পরীক্ষা করেনি। পেশাদার ডেলিভারির মূলমন্ত্র: একবার Build করো, সর্বত্র একই Artifact Deploy করো (Build Once, Deploy Anywhere)। এই অধ্যায়ে সেই ভিত্তিটি তৈরি হবে।

৫.৩ পূর্বশর্ত জ্ঞান

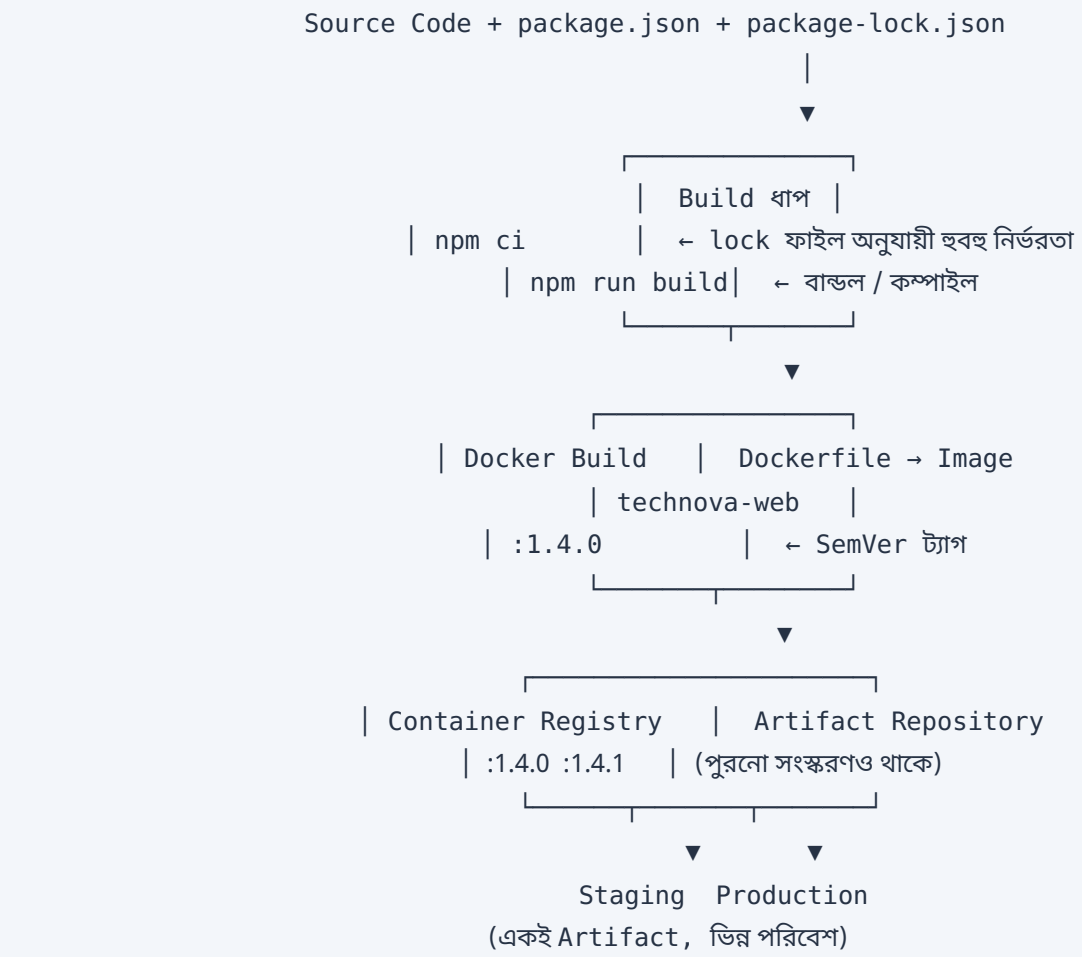
npm ও `package.json`-এর প্রাথমিক ধারণা, JavaScript মডিউল, এবং অধ্যায় ১-এর Source Code / Build Output / Artifact পার্থক্য জানা থাকতে হবে। Docker-এর পূর্ব অভিজ্ঞতা প্রয়োজন নেই — Dockerfile এখানে পরিচয় করিয়ে দেওয়া হবে, বিস্তারিত অধ্যায় ২০-এ।

৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

Build মানে Source Code-কে চালানোর উপযোগী আকারে রূপান্তর করা। Node.js-এ এই রূপান্তর কয়েক রকমের: TypeScript থাকলে তাকে JavaScript-এ কম্পাইল করা, React/Next.js-এ অনেক ফাইলকে একত্রে বান্ডল (Bundle) ও ছোট (Minify) করা, আর সব নির্ভরতা একসঙ্গে জড়ো করা। **Dependency** মানে আপনার কোড যেসব বাইরের প্যাকেজ ব্যবহার করে — যেমন Express, Mongoose, React। এগুলো `package.json`-এ তালিকাভুক্ত থাকে, আর `package-lock.json`-এ প্রতিটি প্যাকেজের হুবহু সংস্করণ লেখা থাকে। `npm install` নির্ভরতা যোগ করে ও lock ফাইল বদলাতে পারে; `npm ci` lock ফাইল অনুযায়ী হুবহু একই সংস্করণ ইনস্টল করে — তাই CI-তে সবসময় `npm ci` ব্যবহার করা হয়। Build-এর ফলকে সংস্করণ-সংখ্যা দিয়ে প্যাকেজ করলে তা হয় **Artifact** — যেমন একটি Docker Image। Artifact-কে রাখা হয় **Artifact Repository**-তে — Docker Image-এর জন্য Container Registry, npm প্যাকেজের জন্য npm Registry। আর **Semantic Versioning (SemVer)** হলো সংস্করণ-সংখ্যার ভাষা: `MAJOR.MINOR.PATCH` — যেমন `1.4.0`।

> **CONCEPT:** "Build Once, Deploy Anywhere" — একই কোড থেকে একবার Artifact তৈরি করুন, তারপর সেই একই Artifact Development, Staging ও Production — সব জায়গায় Deploy করুন। প্রতিটি পরিবেশে আলাদা Build করলে আপনি আসলে ভিন্ন ভিন্ন জিনিস পরীক্ষা ও Deploy করছেন — আর তখন Staging-এর পরীক্ষার কোনো মূল্য থাকে না।

৫.৫ আর্কিটেকচার / ধারণাগত চিত্র



লক্ষ্য করুন: Staging ও Production — দুই জায়গায় একই `:1.4.0` Image যায়, আলাদা Build নয়।

৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Build System — Node.js প্রেক্ষাপটে

Node.js অ্যাপের Build সাধারণত তিনটি কাজ করে: (১) **নির্ভরতা ইনস্টল** — `npm ci`; (২) **যাচাই** — Lint, Type Check, Test; (৩) **বান্ডল/কম্পাইল** — `npm run build` (Next.js-এ `.next/` তৈরি হয়, TypeScript থাকলে `tsc` চলে)। Build-এর পরিবেশও গুরুত্বপূর্ণ: Node.js-এর সংস্করণ `.nvmrc` ফাইলে বা Dockerfile-এ বেঁধে দিন, নইলে CI-তে এক সংস্করণ আর লোকালে আরেক সংস্করণে ভিন্ন ফল আসতে পারে। Build-কে **Deterministic** (নির্ধারিত) রাখুন — একই ইনপুটে একই আউটপুট। এজন্য Build-এ বর্তমান সময় বা Random মান ঢোকাবেন না।

নির্ভরতা পরিচালনা — গভীরে

ধারণা	অর্থ	নিয়ম
Direct Dependency	আপনি সরাসরি ব্যবহার করেন	`dependencies` -এ লিখুন

Dev Dependency	শুধু তৈরি/পরীক্ষার সময় দরকার	`devDependencies`-এ লিখুন; Production Image-এ রাখবেন না
Transitive Dependency	আপনার নির্ভরতার নির্ভরতা	lock ফাইল দিয়ে নিয়ন্ত্রণ করুন
Lock File	হুবহু সংস্করণের তালিকা	Git-এ commit করুন; কখনো মুছবেন না

`npm ci` বনাম `npm install`: CI-তে সবসময় `npm ci` — এটি দ্রুত, নির্ভরযোগ্য, আর lock ফাইলের সঙ্গে `package.json` না মিললে ব্যর্থ হয়ে জানিয়ে দেয়। লোকাল ডেভেলপমেন্টে নতুন প্যাকেজ যোগ করতে `npm install <প্যাকেজ>` ব্যবহার করুন, তারপর lock ফাইলটি commit করুন। নির্ভরতার সংস্করণে `^` (Caret) মানে Minor আপডেট অনুমোদিত, `~` (Tilde) মানে শুধু Patch — lock ফাইল থাকলে CI-তে এই চিহ্নগুলো কোনো প্রভাব ফেলে না, কারণ lock ফাইলই চূড়ান্ত।

Semantic Versioning (SemVer)

সংস্করণ `MAJOR.MINOR.PATCH` — যেমন `1.4.2`: **MAJOR** বাড়ে যখন পেছন-অসামঞ্জস্যপূর্ণ (Breaking) পরিবর্তন আসে; **MINOR** বাড়ে যখন নতুন ফিচার যোগ হয়, কিন্তু পুরনো কাজ ভাঙে না; **PATCH** বাড়ে যখন শুধু বাগ ঠিক করা হয়। এর আগে `1.4.2-rc.1` ধরনের **Release Candidate** দিয়ে পরীক্ষামূলক সংস্করণ চিহ্নিত করা যায়। SemVer শুধু সংখ্যা নয় — এটি একটি প্রতিশ্রুতি: সংখ্যাটি দেখেই ব্যবহারকারী বুঝতে পারে পরিবর্তনটি কতটা ঝুঁকিপূর্ণ।

Artifact ব্যবস্থাপনা

ভালো Artifact ব্যবস্থাপনার নিয়ম: (১) প্রতিটি Artifact-এর নামে সংস্করণ ও Git Commit Hash থাকবে — যেমন `technova-web:1.4.0-a1b2c3d`; (২) `latest` ট্যাগ কখনো Production-এ ব্যবহার করবেন না — কারণ `latest` বদলে যায়, তাই কোন Build চলছে বোঝা যায় না; (৩) Artifact অপরিবর্তনীয় — একবার Registry-তে গেলে আর বদলানো যাবে না; (৪) **Retention Policy** — পুরনো Artifact মুছে ফেলার নিয়ম, যেমন শেষ ২০টি সংস্করণ রাখা; (৫) Artifact-এর সঙ্গে মেটাডেটা — কে Build করেছে, কোন Commit থেকে, কখন। Docker Image-এর ক্ষেত্রে প্রতিটি Image-এর একটি **Digest** থাকে — একটি হ্যাশ, যা Image-এর আসল পরিচয়; Tag বদলালেও Digest একই থাকে। Production-এ Digest দিয়ে Deploy করাই সবচেয়ে নিরাপদ — বিস্তারিত অধ্যায় ২০-এ।

Multi-stage Dockerfile — ধারণা

Docker Image বানাতে **Dockerfile** লেখা হয় — ধাপে ধাপে নির্দেশ। **Multi-stage Build**-এ দুটি পর্যায় থাকে: প্রথম পর্যায়ে (Builder Stage) নির্ভরতা ইনস্টল ও Build হয়; দ্বিতীয় পর্যায়ে (Runtime Stage) শুধু Build-এর ফল ও প্রয়োজনীয় নির্ভরতা কপি করা হয়। ফলে চূড়ান্ত Image ছোট হয় — কারণ Build-এর সরঞ্জাম (যেমন TypeScript কম্পাইলার) চূড়ান্ত Image-এ থাকে না। ছোট Image মানে দ্রুত Deploy, কম Storage, আর কম আক্রমণ-তল (Attack Surface)।

৫.৭ কনফিগারেশন উদাহরণ

একটি MERN অ্যাপের জন্য Multi-stage Dockerfile (ধারণাগত সম্পূর্ণ উদাহরণ):

```
# Dockerfile — TechNova API (Node.js + Express)
# Stage 1: Build
FROM node:20-alpine AS builder
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
```

```
COPY . .
RUN npm run build
# Stage 2: Runtime — শুধু যা দরকার তাই
FROM node:20-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package.json package-lock.json ./
RUN npm ci --omit=dev
COPY --from=builder /app/dist ./dist
EXPOSE 3000
CMD ["node", "dist/server.js"]
```

প্রতিটি নির্দেশের ব্যাখ্যা: `FROM node:20-alpine` — ছোট, নির্দিষ্ট সংস্করণের বেস Image (কখনো `latest` ব্যবহার করবেন না); `WORKDIR` — কাজের ডিরেক্টরি; `COPY package*.json` আগে কপি করা হয় যাতে নির্ভরতার Cache কাজ করে — কোড বদলালেও নির্ভরতা আবার ডাউনলোড হয় না; `npm ci --omit=dev` — Production-এ শুধু প্রয়োজনীয় প্যাকেজ; `COPY --from=builder` — প্রথম Stage থেকে শুধু Build-এর ফল আনা; `EXPOSE 3000` — কোন পোর্টে অ্যাপ শুনবে তার ঘোষণা; `CMD` — কন্টেইনার চালু হলে কী কমান্ড চলবে। Image বানানো ও ট্যাগ করার কমান্ড:

```
docker build -t technova-api:1.4.0 .
docker tag technova-api:1.4.0 registry.technova.example/technova-api:1.4.0
docker push registry.technova.example/technova-api:1.4.0
```

এখানে `registry.technova.example` কাল্পনিক Registry-র ঠিকানা। আসল Registry-তে push করার আগে লগইন দরকার — সেই প্রমাণীকরণ অধ্যায় ২০-এ বিস্তারিত আসবে।

৫.৮ বাস্তব প্রকৌশল পরিস্থিতি

REAL WORLD — TechNova: TechNova-র API-র Docker Image ছিল ১.২ GB — কারণ একটিমাত্র Stage-এ Build-এর সব সরঞ্জাম, Dev Dependency ও Source Code ঢুকে গিয়েছিল। Deploy-এ সময় লাগত অনেক, আর Image-এ থাকা অপ্রয়োজনীয় প্যাকেজে নিরাপত্তা-দুর্বলতা ধরা পড়ত। Multi-stage Build-এ যাওয়ার পর Image হলো ১৮০ MB — Deploy তিন গুণ দ্রুত হলো, আর স্ক্যানে দুর্বলতার সংখ্যা অনেক কমে গেল। একই সঙ্গে দল `npm install`-এর বদলে CI-তে `npm ci` চালু করল — "গতকাল চলেছিল, আজ চলে না" অভিযোগটি প্রায় উধাও হয়ে গেল, কারণ lock ফাইল এখন হুবহু একই নির্ভরতা নিশ্চিত করে।

৫.৯ ধাপে ধাপে ব্যবহারিক উদাহরণ

উদাহরণ: আপনার অ্যাপের জন্য প্রথম Dockerfile লেখা

1. আপনার Node.js প্রজেক্টে `package.json`-এ `build` স্ক্রিপ্ট আছে কি না দেখুন; না থাকলে একটি সহজ Build স্ক্রিপ্ট যোগ করুন।
2. ওপরের Dockerfile-টি আপনার প্রজেক্টের নাম ও পোর্ট অনুযায়ী বদলে লিখুন।
3. `docker build -t technova-test:0.1.0 .` চালিয়ে Image বানান (নমুনা আউটপুট চিত্রণমূলক হবে — আপনার আউটপুটে Layer ID ও সময় ভিন্ন হবে)।
4. `docker images` দিয়ে Image-এর আকার দেখুন, তারপর `docker run -p 3000:3000 technova-test:0.1.0` দিয়ে চালিয়ে `http://localhost:3000` খুলে দেখুন।

5. একটি নির্ভরতার সংস্করণ বদলে আবার Build করুন — Cache-এর কারণে কোন ধাপগুলো আবার চলল আর কোনগুলো চলল না, লক্ষ্য করুন।
6. Image-টিতে একটি সংস্করণ ট্যাগ ও Commit Hash যোগ করার নিয়ম লিখে রাখুন।

৫.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: "CI-তে Build ব্যর্থ, লোকালে সফল" — প্রথমে Node সংস্করণ মেলান: লোকালে `node --version` আর CI-তে সেট করা সংস্করণ এক কি না দেখুন। দ্বিতীয়ত, `package-lock.json` commit করা হয়েছে কি না দেখুন — না থাকলে CI ভিন্ন সংস্করণের নির্ভরতা পেতে পারে। "Docker Build প্রতিবার ধীর" — `COPY . .` সবার আগে লিখলে প্রতিটি কোড-পরিবর্তনে নির্ভরতা আবার ডাউনলোড হয়; `package*.json` আগে কপি করুন। "`npm ci` ব্যর্থ: lock ফাইল মিলছে না" — `package.json` বদলে lock ফাইল আপডেট করা হয়নি; লোকালে `npm install` করে lock ফাইলটি commit করুন।

- **ভুল:** `package-lock.json` Git-এ না রাখা বা `.gitignore`-এ ফেলে দেওয়া।
- **ভুল:** CI-তে `npm install` ব্যবহার করা — অনির্ধারিত সংস্করণ আসতে পারে।
- **ভুল:** Production-এ `latest` ট্যাগ Deploy করা।
- **ভুল:** Dockerfile-এ `COPY . .` দিয়ে `.env` ফাইল Image-এ ঢুকিয়ে ফেলা — Secret Image-এ চিরতরে থেকে যায়। `.dockerignore` ব্যবহার করুন।

৫.১১ নিরাপত্তা বিবেচনা

WARNING: নির্ভরতা মানেই বিশ্বাস — প্রতিটি প্যাকেজ আপনার অ্যাপের ভেতরে কোড চালায়। অপরিচিত বা কম-রক্ষণাবেক্ষিত প্যাকেজ যোগ করার আগে তার রক্ষণাবেক্ষণ, ডাউনলোড-সংখ্যা ও নিরাপত্তা-ইতিহাস দেখুন। `npm audit` দিয়ে পরিচিত দুর্বলতা পরীক্ষা করুন (বিস্তারিত অধ্যায় ১৯-এ)। কখনোই Secret (`.env`, API Key, পাসওয়ার্ড) Docker Image-এ ঢোকাবেন না — Image একবার Registry-তে গেলে তা মুছে ফেলা কঠিন। Secret Runtime-এ Environment Variable বা Secret Manager থেকে দিন। উদাহরণে শুধু `CHANGE-ME-SECRET` ধরনের প্লেসহোল্ডার ব্যবহার করুন।

৫.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Build-এর সময় কমাতে: নির্ভরতার Cache ব্যবহার করুন (GitHub Actions-এ `actions/setup-node`-এর Cache অপশন — অধ্যায় ১১-এ); Dockerfile-এ Layer-এর ক্রম সাজান যাতে সবচেয়ে কম বদলানো জিনিস আগে থাকে; Dev Dependency Production Image-এ রাখবেন না। Registry-র Storage খরচ কমাতে Retention Policy রাখুন — পুরনো, অপ্রয়োজনীয় Image মুছে ফেলুন। তবে সতর্কতা: Production-এ চলমান সংস্করণের Image কখনো মুছবেন না — Rollback-এর জন্য সেটি দরকার।

৫.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা অনুশীলন: CI-তে `npm ci`; lock ফাইল Git-এ রাখা; Node সংস্করণ বেঁধে রাখা; Build Once, Deploy Anywhere; SemVer অনুযায়ী ট্যাগ; Multi-stage Dockerfile; `.dockerignore` ব্যবহার; Image-এর আকার নিয়মিত দেখা। **অ্যান্টি-**

প্যাটার্ন: "Works on my machine" Build — পরিবেশের ওপর নির্ভরশীল Build; `latest` ট্যাগের ওপর নির্ভরতা; প্রতিটি পরিবেশে আলাদা Build; Image-এ Secret ঢোকানো; একটিমাত্র বিশাল Layer-এ সবকিছু — Cache-এর সুবিধা নষ্ট হয়।

৫.১৪ অধ্যায়ের সারাংশ

Build মানে Source Code-কে চালানোর উপযোগী আকারে রূপান্তর — Node.js-এ নির্ভরতা ইনস্টল, যাচাই ও বান্ডল/কম্পাইল। নির্ভরতার হুবহু সংস্করণ নিশ্চিত করে `package-lock.json` ও CI-তে `npm ci`। Build-এর ফলকে SemVer (`MAJOR.MINOR.PATCH`) ট্যাগ দিয়ে Artifact হিসেবে Registry-তে রাখা হয়। একই Artifact সব পরিবেশে Deploy করাই নিয়ম — Build Once, Deploy Anywhere। Multi-stage Dockerfile Image-কে ছোট, দ্রুত ও নিরাপদ করে।

৫.১৫ মূল প্রযুক্তিগত পরিভাষা

Build System: Source Code থেকে চালানোর উপযোগী আউটপুট তৈরির ব্যবস্থা।

Dependency: আপনার কোড ব্যবহার করে এমন বাইরের প্যাকেজ।

package-lock.json: নির্ভরতার হুবহু সংস্করণের তালিকা; পুনরাবৃত্তিযোগ্য Build-এর চাবি।

npm ci: lock ফাইল অনুযায়ী হুবহু নির্ভরতা ইনস্টল করার কমান্ড (CI-এর জন্য)।

Semantic Versioning (SemVer): MAJOR.MINOR.PATCH নিয়মে সংস্করণ দেওয়ার পদ্ধতি।

Artifact: সংস্করণ-সহ প্যাকেজ করা, Deploy-যোগ্য Build-এর ফল।

Container Registry: Docker Image সংরক্ষণ ও বিতরণের ভান্ডার।

Dockerfile: Docker Image বানানোর ধাপে ধাপে নির্দেশের ফাইল।

Multi-stage Build: Build ও Runtime আলাদা Stage-এ রেখে ছোট Image বানানোর কৌশল।

Image Digest: Docker Image-এর অপরিবর্তনীয় হ্যাশ-পরিচয়।

৫.১৬ জ্ঞান যাচাই — প্রশ্ন

1. CI-তে `npm ci` কেন `npm install`-এর চেয়ে ভালো? দুটি কারণ দিন।

2. Semantic Versioning-এ `2.0.0` থেকে `2.1.0` ও `2.1.1`-এর মধ্যে পার্থক্য কী?
3. "Build Once, Deploy Anywhere" নীতি না মানলে Staging-এর পরীক্ষা কেন মূল্যহীন হয়ে যায়?
4. Multi-stage Dockerfile একক Stage-এর Dockerfile-এর চেয়ে কীভাবে ভালো?
5. `latest` ট্যাগ দিয়ে Production Deploy করা কেন বিপজ্জনক?

৫.১৭ ব্যবহারিক অনুশীলন

1. **Dockerfile লেখা:** আপনার Node.js অ্যাপের জন্য Multi-stage Dockerfile লিখে Image বানান ও আকার লিখে রাখুন।
2. **Lock ফাইল পরীক্ষা:** `package-lock.json` মুছে `npm install` চালিয়ে দেখুন কোন সংস্করণগুলো বদলায় — তারপর lock ফাইলটি Git-এ ফিরিয়ে আনুন।
3. **সংস্করণ পরিকল্পনা:** একটি কাল্পনিক অ্যাপের জন্য তিনটি পরিবর্তনের (Breaking ফিচার, নতুন ফিচার, বাগ-ফিক্স) SemVer সংখ্যা নির্ধারণ করুন ও যুক্তি লিখুন।

PART II • DEVOPS ENGINEERING MASTERY

GitHub Actions Engineering

GitHub Actions — workflow, job, runner, reusable workflow, trigger, secret, OIDC, caching ও অপ্টিমাইজেশন।

অধ্যায় ৬ • PART II — GitHub Actions Engineering

Chapter 6 — GitHub Actions Architecture and Core Concepts

৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) GitHub Actions কী এবং কেন এটি CI/CD-র জন্য ব্যবহৃত হয় তা ব্যাখ্যা করতে; (২) Workflow, Job, Step, Runner ও Action-এর মধ্যে সম্পর্ক আঁকতে; (৩) GitHub-hosted ও self-hosted runner-এর পার্থক্য ও ঝুঁকি বলতে; (৪) একটি Node.js অ্যাপের জন্য প্রথম workflow লিখতে; (৫) `GITHUB_TOKEN`-এর ভূমিকা ও সীমা ব্যাখ্যা করতে।

৬.২ কেন এটি গুরুত্বপূর্ণ

আপনার কোড GitHub-এ থাকলে পরীক্ষা, build ও deploy-এর জন্য আলাদা সার্ভার বসানোর দরকার নেই — GitHub Actions সেই কাজটাই করে, কোডের পাশেই, `.github/workflows/` ফোল্ডারে YAML ফাইল হিসেবে। ভুলভাবে ব্যবহার করলে একই ব্যবস্থা বিপজ্জনক হয়: অপরিচিত pull request-এর কোড আপনার secret চুরি করতে পারে, ভুল permission-এ workflow পুরো repository বদলে দিতে পারে। তাই architecture আগে বোঝা, কনফিগারেশন পরে শেখা — এটাই পেশাদার পথ।

৬.৩ পূর্বশর্ত জ্ঞান

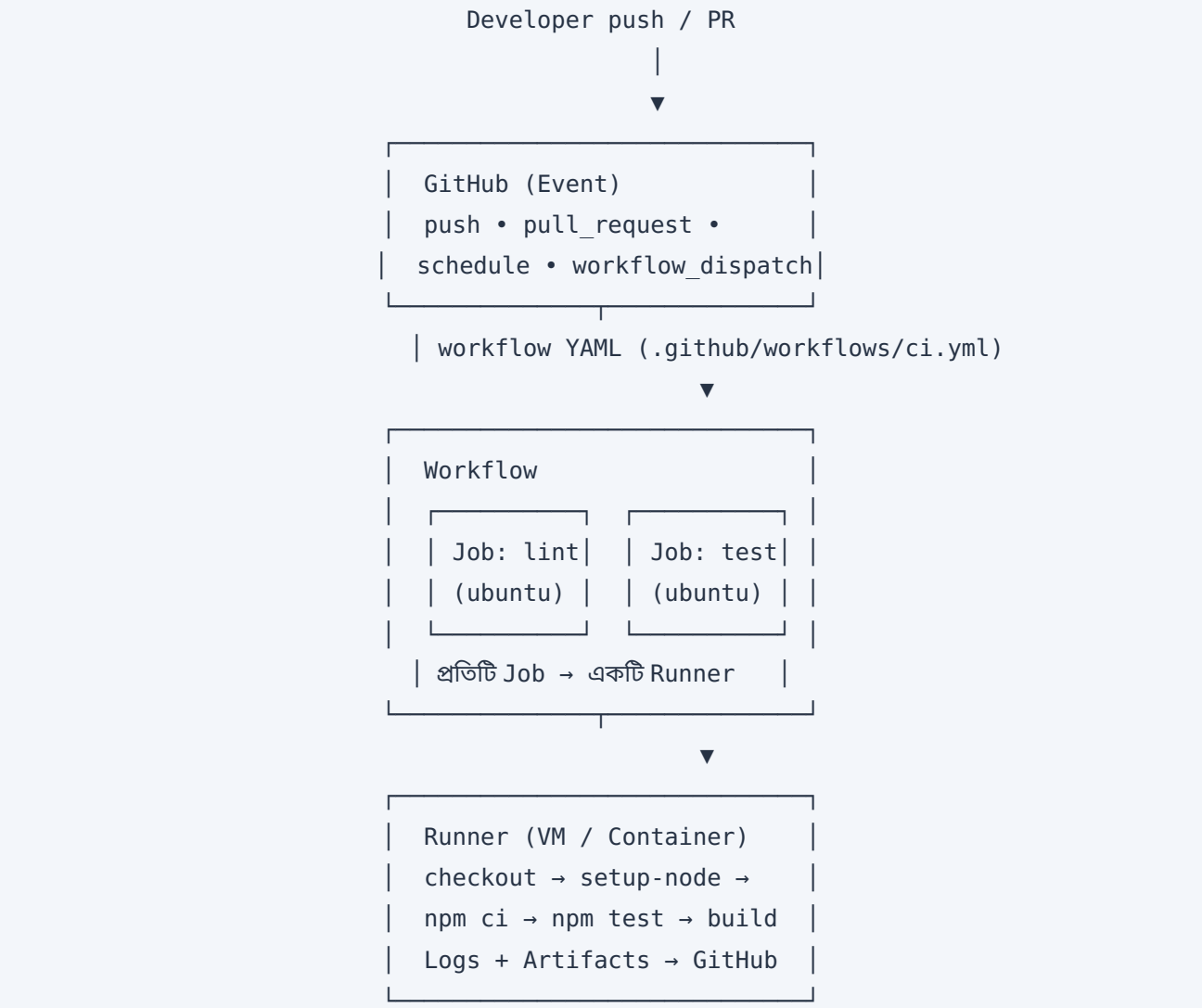
Git (commit, branch, pull request), YAML-এর মৌলিক গঠন (indentation, key-value), Node.js ও npm, এবং অধ্যায় ১–৫-এর CI/CD ধারণা — pipeline, stage, artifact, trigger। Linux কমান্ডের প্রাথমিক ধারণা থাকলে ভালো।

৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

CI/CD-র মূল প্রশ্নটি সহজ: **কোড বদলালে কী কী স্বয়ংক্রিয়ভাবে ঘটবে, কোন ক্রমে, কোথায়?** GitHub Actions এই প্রশ্নের উত্তর দেয় তিনটি স্তরে। **Event** (ঘটনা) — যেমন push বা pull request — workflow-কে জাগায়। **Workflow** — একটি YAML ফাইল, অর্থাৎ "কী করতে হবে"-র লিখিত পরিকল্পনা। **Runner** — একটি অস্থায়ী কম্পিউটার, যেখানে পরিকল্পনাটি আসলে চলে। Runner কোড checkout করে, Node.js বসায়, `npm ci` ও `npm test` চালায়, ফলাফল GitHub-কে জানায়। কাজ শেষ হলে hosted runner মুছে যায় — তাই প্রতিটি run পরিষ্কার অবস্থা থেকে শুরু হয়, এটাই repeatability-র ভিত্তি।

> **CONCEPT:** GitHub Actions কোনো আলাদা সার্ভার নয়; এটি GitHub-এর ভেতরে বসা একটি automation engine। Workflow হলো নির্দেশপত্র, Runner হলো শ্রমিক, Action হলো প্রস্তুত-করা সরঞ্জাম।

৬.৫ আর্কিটেকচার / ধারণাগত চিত্র



Event → Workflow → Job → Runner → Step: এই প্রবাহটাই পুরো GitHub Actions-এর মেরুদণ্ড।

৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Workflow ফাইল কোথায় থাকে

প্রতিটি workflow একটি YAML ফাইল: `.github/workflows/<নাম>.yml`। Repository-র default branch-এ থাকা ফাইলই চলে। এক repository-তে অনেক workflow থাকতে পারে — যেমন `ci.yml` (পরীক্ষা), `deploy.yml` (deploy), `release.yml` (release)।

Runner-এর ধরন

ধরন	অর্থ	সুবিধা	ঝুঁকি / খরচ

GitHub-hosted	GitHub-এর দেওয়া নতুন VM (ubuntu/windows/macos)	কোনো রক্ষণাবেক্ষণ নেই, পরিষ্কার অবস্থা	ব্যবহারের সীমা ও খরচ; নিজের নেটওয়ার্কে পৌঁছায় না
Self-hosted	আপনার নিজের সার্ভার/VM-এ চলা runner	নিজের হার্ডওয়্যার, প্রাইভেট নেটওয়ার্কে প্রবেশ	প্যাচ, নিরাপত্তা ও বিচ্ছিন্নতার দায়িত্ব আপনার; ভুল সেটআপে আক্রমণের পথ

GITHUB_TOKEN

প্রতিটি run-এ GitHub একটি অস্থায়ী token দেয় — `GITHUB_TOKEN`। এটি দিয়ে workflow API কল করতে পারে (যেমন PR-এ comment করা), কিন্তু run শেষে token মারা যায়। Default permission সীমিত রাখুন — অধ্যায় ১০-এ বিস্তারিত।

Action কী

Action হলো পুনর্ব্যবহারযোগ্য একটি ধাপ — যেমন `actions/checkout@v4` (কোড নামায়) বা `actions/setup-node@v4` (Node.js বসায়)। নিজে shell লেখার বদলে বিশ্বস্ত action ব্যবহার করলে ভুল কমে, কিন্তু তৃতীয় পক্ষের action-এর ওপর আস্থা রাখতে হয় — তাই version pin করা জরুরি (অধ্যায় ৮ ও ১০)।

৬.৭ কনফিগারেশন উদাহরণ — প্রথম CI Workflow

TechNova-র MERN অ্যাপের জন্য একটি সম্পূর্ণ CI workflow। প্রতিটি field-এর ব্যাখ্যা নিচে দেওয়া হলো:

```
name: CI                          # Workflow-এর নাম; GitHub UI-তে দেখা যায়
on:                               # কখন চলবে (trigger)
  push:
    branches: [main]              # main branch-এ push হলে
  pull_request:
    branches: [main]              # main-এ PR খুললে/বদলালে
permissions:
  contents: read                  # ন্যূনতম অনুমতি: শুধু কোড পড়া
jobs:
  ci:                             # Job-এর নাম (id)
    runs-on: ubuntu-latest        # কোন runner: GitHub-hosted Ubuntu
    steps:
      - uses: actions/checkout@v4        # কোড checkout করে
      - uses: actions/setup-node@v4      # Node.js বসায়
        with:
          node-version: 20               # Node.js 20 LTS
          cache: npm                     # npm cache চালু (অধ্যায় ১১)
      - run: npm ci                       # নির্ভরতা বসায় (lockfile অনুযায়ী)
      - run: npm run lint                 # ESLint চালায়
      - run: npm test                     # Jest/Vitest পরীক্ষা চালায়
      - run: npm run build                # production build যাচাই করে
```

Field ব্যাখ্যা: `name` — শনাক্তকরণ; `on` — trigger; `permissions` — token-এর অধিকার; `jobs.<id>` — কাজের একক; `runs-on` — runner নির্বাচন; `steps` — ক্রমিক ধাপ; `uses` — action ব্যবহার; `run` — shell কমান্ড; `with` — action-এর ইনপুট। `npm ci` (clean install) `npm install`-এর চেয়ে ভালো, কারণ এটি `package-lock.json` হুবহু মেনে চলে — build repeatable হয়।

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
CI / ci (pull_request) — Success
  ✓ checkout, setup-node 20, npm ci, lint, test (42 passed), build
```

৬.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র ডেভেলপার নাহিদ `main`-এ সরাসরি push করে ভাঙা কোড deploy করেছিলেন — সাইট আধা ঘণ্টা বন্ধ ছিল। সমাধান: branch protection + এই CI workflow। এখন PR ছাড়া `main`-এ কিছু যায় না, আর CI সবুজ না হলে merge বোতামই খোলে না। ভাঙা কোড আর কখনো main-এ পৌঁছায়নি।

৬.৯ ধাপে ধাপে বাস্তবায়ন

1. Repository-তে `.github/workflows/ci.yml` তৈরি করুন, ওপরের YAML বসান।
2. `package.json`-এ `lint`, `test`, `build` script আছে কি না যাচাই করুন।
3. একটি test branch থেকে PR খুলুন — Actions ট্যাবে run দেখুন।
4. Settings → Branches → Branch protection-এ "Require status checks to pass" চালু করুন।
5. ইচ্ছা করে একটি test ভেঙে PR দিন — CI লাল হয় কি না দেখুন, তারপর ঠিক করুন।

৬.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: Workflow চলছে না? প্রথমে দেখুন ফাইলটি `.github/workflows/`-এ আছে কি না এবং YAML indentation ঠিক কি না — দুটি space ব্যবহার করুন, tab নয়। `on.push.branches` ভুল branch নাম দিলে trigger হবে না। লগে "Resource not accessible by integration" মানে `GITHUB_TOKEN`-এর permission কম।

লক্ষণ	সম্ভাব্য কারণ	সমাধান
Workflow তালিকায় আসে না	YAML syntax ভুল / ভুল ফোল্ডার	YAML validator দিয়ে যাচাই করুন; ফাইল default branch-এ আছে কি না দেখুন
`npm ci` ব্যর্থ	lockfile ও package.json অমিল	স্থানীয়ভাবে `npm install` করে lockfile commit করুন
Test স্থানীয়ভাবে পাস, CI-তে ফেল	পরিবেশের পার্থক্য (Node সংস্করণ, env var)	Node সংস্করণ pin করুন; অনুপস্থিত env var খুঁজুন

৬.১১ নিরাপত্তা বিবেচনা

WARNING: Self-hosted runner-কে কখনো public repository-র pull request workflow-এ ব্যবহার করবেন না — অপরিচিত PR-এর কোড runner-এর মেশিনে চলে এবং secret চুরি বা মেশিন দখল করতে পারে। Public repo-তে শুধু GitHub-hosted runner ব্যবহার করুন।

তৃতীয় পক্ষের action সবসময় নির্দিষ্ট version-এ pin করুন (`@v4` নয়, সম্ভব হলে commit SHA)। Workflow-এ secret কখনো `echo` করে লগে ছাপাবেন না — GitHub স্বয়ংক্রিয়ভাবে secret মাস্ক করে, কিন্তু সেটাই একমাত্র ভরসা নয়।

৬.১২ কর্মক্ষমতা ও খরচ

GitHub-hosted runner প্রতি মিনিটে খরচ হয় (public repo-তে বিনামূল্যে সীমা আছে; private repo-তে plan অনুযায়ী)। অপ্রয়োজনীয় workflow বারবার চালালে খরচ ও অপেক্ষা বাড়ে। `paths` filter দিয়ে শুধু প্রাসঙ্গিক ফাইল বদলালে trigger করুন, আর cache ব্যবহার করুন (অধ্যায় ১১)।

৬.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Workflow ছোট ও একক-উদ্দেশ্যমূলক রাখুন; Node সংস্করণ pin করুন; `npm ci` ব্যবহার করুন; permissions ন্যূনতম রাখুন। **অ্যান্টি-প্যাটার্ন:** `ubuntu-latest`-এর ওপর অন্ধ নির্ভরতা ছাড়া version pin না করা; এক workflow-এ CI+deploy+release সব গুঁজে দেওয়া; secret সরাসরি YAML-এ লেখা।

৬.১৪ অধ্যায়ের সারাংশ

GitHub Actions হলো event-চালিত automation: event workflow জাগায়, workflow-র job runner-এ চলে, step-গুলো action বা shell কমান্ড চালায়। GitHub-hosted runner সহজ ও নিরাপদ শুরু; self-hosted runner শক্তিশালী কিন্তু রক্ষণাবেক্ষণ ও নিরাপত্তার দায়িত্ব আপনার। ন্যূনতম permission ও version pinning দিয়ে শুরু করুন।

৬.১৫ মূল প্রযুক্তিগত পরিভাষা

Workflow: একটি YAML ফাইলে লেখা স্বয়ংক্রিয় প্রক্রিয়া।

Event / Trigger: workflow শুরু করা ঘটনা (push, pull_request ইত্যাদি)।

Job: একই runner-এ চলা ধাপগুলোর দল।

Step: Job-এর একটি ধাপ — action বা shell কমান্ড।

Runner: workflow চালানো মেশিন/VM।

Action: পুনর্ব্যবহারযোগ্য ধাপের প্যাকেজ।

GITHUB_TOKEN: প্রতিটি run-এ দেওয়া অস্থায়ী API token।

Branch Protection: গুরুত্বপূর্ণ branch-এ সরাসরি push/merge আটকানোর নিয়ম।

Repeatability: একই ইনপুটে প্রতিবার একই ফলাফল পাওয়ার গুণ।

Self-hosted Runner: নিজের মেশিনে চালানো runner।

৬.১৬ জ্ঞান যাচাই প্রশ্ন

1. Workflow, Job, Step ও Runner-এর মধ্যে সম্পর্ক নিজের ভাষায় ব্যাখ্যা করুন।
2. GitHub-hosted runner কখন ভালো, আর self-hosted runner কখন ঝুঁকিপূর্ণ — উদাহরণ দিন।
3. `npm ci` কেন CI-তে `npm install`-এর চেয়ে ভালো?
4. `GITHUB_TOKEN` কী এবং কেন এর permission ন্যূনতম রাখা উচিত?
5. Public repository-র PR workflow-এ self-hosted runner ব্যবহার করা কেন বিপজ্জনক?

৬.১৭ ব্যবহারিক অনুশীলন

1. নিজের একটি Node.js repository-তে ওপরের `ci.yml` বসিয়ে PR খুলে সবুজ CI দেখান।
2. ইচ্ছা করে একটি test ভেঙে দেখুন CI লাল হয় কি না, তারপর ঠিক করে আবার সবুজ করুন।
3. `main` branch-এ branch protection চালু করে CI ছাড়া merge আটকে যায় কি না পরীক্ষা করুন।

অধ্যায় ৭ • PART II — GitHub Actions Engineering

Chapter 7 — Workflows, Jobs, Steps and Runners

৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) workflow YAML-এর প্রতিটি স্তর — `on`, `jobs`, `steps`, `runs-on` — ব্যাখ্যা করতে; (২) একাধিক job-এর মধ্যে নির্ভরতা (`needs`) ও সমান্তরাল চালানো ডিজাইন করতে; (৩) step-এর মধ্যে output পাস করতে; (৪) runner label ও পরিবেশ (environment) বেছে নিতে; (৫) ব্যর্থ job-এর log পড়ে ধাপ চিহ্নিত করতে।

৭.২ কেন এটি গুরুত্বপূর্ণ

একটি বাস্তব pipeline কখনো একটিমাত্র job নয় — lint, test, build, deploy আলাদা job। ভুলভাবে সাজালে test না চলেই deploy হয়ে যায়, অথবা সবকিছু ক্রমিকভাবে চলে ঘণ্টা লেগে যায়। Job-এর নির্ভরতা ও সমান্তরালতা বুঝলেই pipeline দ্রুত ও নিরাপদ হয়।

৭.৩ পূর্বশর্ত জ্ঞান

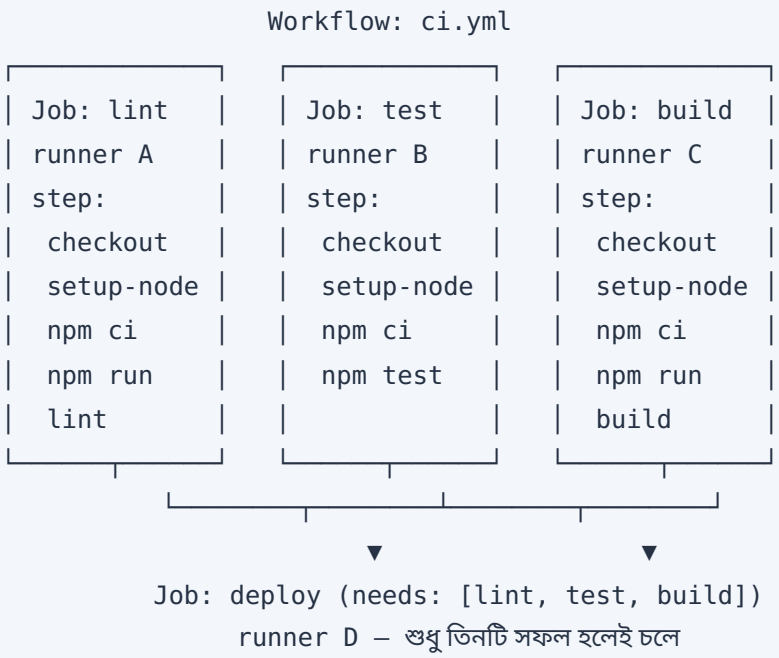
অধ্যায় ৬-এর workflow ধারণা, YAML indentation, Node.js project গঠন (`package.json` script)।

৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

Workflow-কে একটি কারখানার লাইন ভাবুন। **Job** হলো একেকটি ওয়ার্কস্টেশন — প্রতিটির নিজস্ব runner (নিজস্ব মেশিন) থাকে, তাই job-গুলো স্বাভাবিকভাবে সমান্তরাল চলে। এক job-এর ফাইল অন্য job দেখতে পায় না, যতক্ষণ না artifact দিয়ে পাস করা হয়। **Step** হলো ওয়ার্কস্টেশনের ভেতরের একেকটি কাজ — step-গুলো ক্রমিকভাবে চলে এবং একই filesystem ভাগ করে। এই পার্থক্যটাই সব ডিজাইন সিদ্ধান্তের মূল: যা ভাগ করতে হবে তা step-এ রাখুন, যা স্বাধীন তা আলাদা job-এ ভাগ করুন।

> **CONCEPT:** Job = আলাদা মেশিন, সমান্তরাল; Step = একই মেশিন, ক্রমিক। Job-এর মধ্যে ডেটা পাস করতে artifact বা output লাগে — সরাসরি ফাইল পাস হয় না।

৭.৫ আর্কিটেকচার / ধারণাগত চিত্র



`needs` ছাড়া job-গুলো সমান্তরাল চলে; `needs` দিলে নির্ভরতা তৈরি হয়।

৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Job নির্ভরতা — needs

`needs: [lint, test]` মানে এই job তখনই চলবে যখন lint ও test দুটোই সফল। একটি ব্যর্থ হলে নির্ভরশীল job বাদ পড়ে (skipped)। `needs` ছাড়া job-গুলো একসঙ্গে শুরু হয়।

Step output পাস করা

এক step-এর ফলাফল পরের step-এ নিতে step-কে `id` দিন এবং `$GITHUB_OUTPUT`-এ লিখুন। Job-এর মধ্যে output পাস করতে job-এ `outputs` ঘোষণা করুন।

runs-on ও runner label

`runs-on: ubuntu-latest` GitHub-hosted Ubuntu দেয়। Self-hosted runner-এর ক্ষেত্রে label ব্যবহার করুন, যেমন `runs-on: [self-hosted, linux, x64]`। `ubuntu-latest` সময়ের সঙ্গে বদলায় — প্রোডাকশনে `ubuntu-24.04`-এর মতো নির্দিষ্ট version ব্যবহার করা নিরাপদ।

Step-এর শর্ত ও ব্যর্থতা

কোনো step ব্যর্থ হলে পরের step-গুলো বাদ পড়ে, যদি না `if: always()` দেওয়া থাকে। `continue-on-error: true` দিলে step ব্যর্থ হলেও job সফল ধরা হয় — পরীক্ষামূলক ধাপে ব্যবহার করুন, গুরুত্বপূর্ণ ধাপে নয়।

Timeout ও concurrency

`timeout-minutes` দিয়ে job আটকে গেলে থামান। একই branch-এ একাধিক run একসঙ্গে চললে `concurrency` দিয়ে পুরনো run বাতিল করা যায় (অধ্যায় ৯)।

৭.৭ কনফিগারেশন উদাহরণ — বহু-Job Workflow

```
name: CI
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
permissions: { contents: read }
jobs:
  lint:                                  # Job 1 — lint
    runs-on: ubuntu-24.04                # নির্দিষ্ট runner version
    timeout-minutes: 10                  # ১০ মিনিটে না শেষ হলে থামবে
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run lint

  test:                                  # Job 2 — test (lint-এর সঙ্গে সমান্তরাল)
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm test -- --coverage
      - name: Upload coverage report     # ধাপের নাম (log-এ দেখা যায়)
        uses: actions/upload-artifact@v4
        with:
          name: coverage-report          # artifact-এর নাম
          path: coverage/                # কোন ফোল্ডার আপলোড হবে

  build:                                 # Job 3 — build
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm run build
      - id: version                      # step id — output পাস করতে লাগে
        run: echo "app_version=$(node -p \"require('./package.json').version\")" >>
"$GITHUB_OUTPUT"
    outputs:
      app_version: ${{ steps.version.outputs.app_version }}   # job output ঘোষণা

  deploy-staging:                        # Job 4 — deploy (নির্ভরশীল)
    needs: [lint, test, build]           # তিনটি সফল হলেই চলবে
    if: github.ref == 'refs/heads/main'  # শুধু main branch-এ
    runs-on: ubuntu-24.04
    environment: staging                 # environment (অধ্যায় ১০)
```

```
    steps:
      - run: echo "Deploying version ${{ needs.build.outputs.app_version }} to staging"
```

Field ব্যাখ্যা: `timeout-minutes` — আটকে যাওয়া job থামায়; `id` — step-কে নাম দিয়ে output ধরা; `$GITHUB_OUTPUT` — output লেখার আধুনিক উপায় (পুরনো `set-output` আর ব্যবহার করবেন না); `outputs` — job-স্তরের output; `needs.<job>.outputs` — অন্য job-এর output পড়া; `environment` — deploy পরিবেশ ও protection rule।

৭.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র প্রথম pipeline-এ lint, test, build একটাই job-এ ছিল — lint ব্যর্থ হলে test-ও চলত না, ফলে ডেভেলপার জানতেই পারত না test-ও ভাঙা কি না। Job আলাদা করার পর তিনটি সমান্তরাল চলে, মোট সময় ৯ মিনিট থেকে ৪ মিনিটে নামে, আর প্রতিটি ব্যর্থতার কারণ আলাদাভাবে দেখা যায়।

৭.৯ ধাপে ধাপে বাস্তবায়ন

1. ওপরের workflow `.github/workflows/ci.yml`-এ বসান।
2. PR খুলে Actions ট্যাবে দেখুন lint/test/build সমান্তরাল চলছে।
3. একটি job-এ ইচ্ছা করে ভুল করুন — deploy-staging skipped হয় কি না দেখুন।
4. Artifacts ট্যাবে coverage-report ডাউনলোড করে যাচাই করুন।
5. `ubuntu-latest`-এর বদলে `ubuntu-24.04` ব্যবহার করে পার্থক্য লক্ষ করুন।

৭.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: "Job skipped" দেখলে প্রথমে `needs` চেইন দেখুন — পূর্ববর্তী কোনো job ব্যর্থ হলে নির্ভরশীল job চলবে না। Output খালি এলে step-এর `id` ও `$GITHUB_OUTPUT` বানান মিলিয়ে দেখুন; `echo`-এ `>>` ভুলে `>` লিখলে ফাইল ওভাররাইট হয়ে যায়।

লক্ষণ	সম্ভাব্য কারণ	সমাধান
Job ঘণ্টার পর ঘণ্টা চলছে	timeout নেই; test hang করছে	`timeout-minutes` দিন; test-এ `--forceExit` বিবেচনা করুন
Artifact খালি	`path` ভুল	আগের step-এ `ls` দিয়ে path যাচাই করুন
`needs` output খালি	job-এ `outputs` ঘোষণা হয়নি	job-স্তরে `outputs` যোগ করুন

৭.১১ নিরাপত্তা বিবেচনা

> **WARNING:** Deploy job-কে কখনো PR trigger-এ শর্ত ছাড়া চালাবেন না — fork থেকে আসা PR-এর কোডও deploy job চালাতে পারে। Deploy job-এ সবসময় `if: github.ref == 'refs/heads/main'` শর্ত ও `environment` protection রাখুন।

Deploy job-কে অন্য job থেকে আলাদা রাখুন এবং `environment` protection ব্যবহার করুন, যাতে যেকোনো PR deploy চালাতে না পারে। `if: github.ref == 'refs/heads/main'` শর্ত ছাড়া deploy job রাখবেন না।

৭.১২ কর্মক্ষমতা ও খরচ

Job আলাদা করলে সমান্তরালতায় সময় কমে, কিন্তু প্রতিটি job-এ `npm ci` আবার চলে — cache (অধ্যায় ১১) ছাড়া মোট খরচ বাড়তে পারে। নিয়ম: স্বাধীন কাজ সমান্তরাল করুন, কিন্তু cache দিয়ে পুনরাবৃত্তি খরচ কমান।

৭.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: প্রতিটি job-এর একটিমাত্র দায়িত্ব; `needs` দিয়ে স্পষ্ট নির্ভরতা; runner version pin; প্রতিটি step-এ নাম দিন। **অ্যান্টি-প্যাটার্ন:** একটি বিশাল job-এ সবকিছু; `continue-on-error: true` গুরুত্বপূর্ণ ধাপে; নামহীন step (log পড়া কঠিন হয়)।

৭.১৪ অধ্যায়ের সারাংশ

Workflow job-এর সমষ্টি, job step-এর সমষ্টি। Job আলাদা runner-এ সমান্তরাল চলে, step একই runner-এ ক্রমিক চলে। `needs` নির্ভরতা তৈরি করে, `outputs` ও artifact job-এর মধ্যে ডেটা পাস করে। এই কাঠামো বুঝলেই pipeline দ্রুত, স্পষ্ট ও নিরাপদ হয়।

৭.১৫ মূল প্রযুক্তিগত পরিভাষা

needs: Job-এর নির্ভরতা ঘোষণা — পূর্ববর্তী job সফল হলেই চলবে।

runs-on: Job কোন runner-এ চলবে তা নির্ধারণ করে।

Step output: এক step থেকে অন্য step-এ পাস করা মান।

GITHUB_OUTPUT: Step output লেখার ফাইল-ভিত্তিক উপায়।

Artifact: Job থেকে সংরক্ষিত ফাইল (রিপোর্ট, build ফলাফল)।

timeout-minutes: Job-এর সর্বোচ্চ চলার সময়।

continue-on-error: Step ব্যর্থ হলেও job চালিয়ে যাওয়ার নির্দেশ।

Environment: Deploy-এর লক্ষ্য পরিবেশ (staging/production) ও তার নিয়ম।

Runner label: Runner শনাক্ত করার ট্যাগ (যেমন self-hosted, linux)।

Parallel job: একসঙ্গে চলা স্বাধীন job।

৭.১৬ জ্ঞান যাচাই প্রশ্ন

1. Job ও Step-এর মধ্যে মৌলিক পার্থক্য কী, এবং কেন তা ডিজাইনে গুরুত্বপূর্ণ?
2. `needs` না দিলে job-গুলো কীভাবে চলে? একটি উদাহরণ দিন।
3. এক job-এর output অন্য job-এ পাস করতে কী কী ঘোষণা লাগে?
4. `continue-on-error: true` কখন ঠিক, কখন বিপজ্জনক?
5. `ubuntu-latest`-এর বদলে `ubuntu-24.04` ব্যবহার করা কেন নিরাপদ?

৭.১৭ ব্যবহারিক অনুশীলন

1. lint/test/build আলাদা job-এ ভাগ করে `needs` দিয়ে deploy job যোগ করুন।
2. Build job থেকে version output পাস করে deploy job-এ echo করে দেখান।
3. একটি job-এ `timeout-minutes: 1` দিয়ে আটকে যাওয়া run থেমে যায় কি না পরীক্ষা করুন।

অধ্যায় ৮ • PART II — GitHub Actions Engineering

Chapter 8 — Actions, Reusable Workflows and Composite Actions

৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Action, reusable workflow ও composite action-এর পার্থক্য ব্যাখ্যা করতে; (২) Marketplace action নিরাপদে বেছে pin করতে; (৩) নিজের composite action লিখতে; (৪) একটি reusable workflow তৈরি করে একাধিক repository-তে ব্যবহার করতে; (৫) কোন ক্ষেত্রে কোন পুনর্ব্যবহার পদ্ধতি ঠিক তা সিদ্ধান্ত নিতে।

৮.২ কেন এটি গুরুত্বপূর্ণ

TechNova-র পাঁচটি repository-তে একই setup ধাপ বারবার কপি করা হলে Node সংস্করণ বদলাতে পাঁচ জায়গায় হাত দিতে হয় — এক জায়গায় ভুলে গেলেই অসঙ্গতি। পুনর্ব্যবহার ঠিকভাবে করলে পরিবর্তন এক জায়গায় হয়, সব জায়গায় পৌঁছায়। ভুলভাবে করলে একটিমাত্র আপস করা action পাঁচটি repository-র secret-ই ফাঁস করে দিতে পারে।

৮.৩ পূর্বশর্ত জ্ঞান

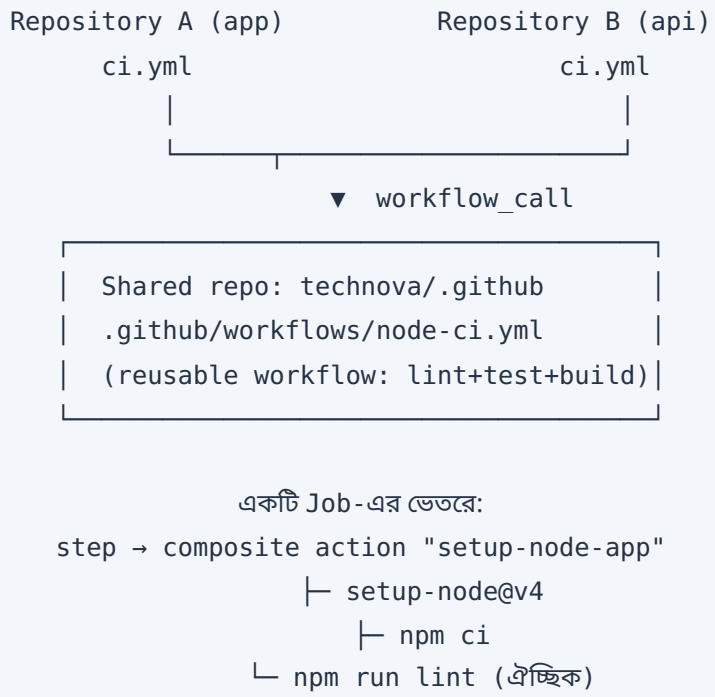
অধ্যায় ৬–৭-এর workflow/job/step ধারণা, `uses` ও `with`-এর ব্যবহার, GitHub repository গঠন।

৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

পুনর্ব্যবহারের তিনটি স্তর আছে। **Action** — একটি ধাপের পুনর্ব্যবহার (যেমন setup-node)। **Composite action** — কয়েকটি step-এর পুনর্ব্যবহার, একই job-এর ভেতরে চলে। **Reusable workflow** — পুরো job-এর পুনর্ব্যবহার, অন্য workflow থেকে `workflow_call` দিয়ে ডাকা হয়। নিয়ম: ধাপ পুনর্ব্যবহার করতে composite, পুরো job পুনর্ব্যবহার করতে reusable workflow।

> **CONCEPT:** Action = একটি সরঞ্জাম; Composite action = সরঞ্জামের সেট; Reusable workflow = পুরো কাজের লাইন। ছোট পুনর্ব্যবহারে composite, বড় পুনর্ব্যবহারে reusable workflow।

৮.৫ আর্কিটেকচার / ধারণাগত চিত্র



Reusable workflow job-স্তরে, composite action step-স্তরে কাজ করে।

৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Action-এর ধরন

ধরন	অর্থ	উদাহরণ
JavaScript action	Node.js-এ লেখা, দ্রুত চলে	actions/checkout, actions/setup-node
Docker action	Container-এ চলে	কাস্টম টুলের action
Composite action	YAML-এ কয়েকটি step-এর সমষ্টি	নিজের setup action

Version pinning

`actions/checkout@v4` একটি major version tag — tag সরানো যেতে পারে। সর্বোচ্চ নিরাপত্তায় commit SHA ব্যবহার করুন: `actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683` (উদাহরণস্বরূপ; ব্যবহারের আগে অফিসিয়াল repository থেকে বর্তমান SHA যাচাই করুন)। Dependabot action-এর আপডেট জানাতে পারে।

Reusable workflow — inputs, secrets, outputs

Reusable workflow `on.workflow_call` দিয়ে শুরু হয়। Caller `with` দিয়ে input ও `secrets` দিয়ে secret পাস করে (secret স্বয়ংক্রিয়ভাবে যায় না — স্পষ্টভাবে পাস করতে হয়)। ফলাফল `outputs` দিয়ে ফেরত আসে।

Composite action — inputs ও outputs

Composite action-এর `action.yml`-এ `runs.using: composite` লিখতে হয়। এর step-গুলো caller job-এর runner-এই চলে — তাই আলাদা runner লাগে না, কিন্তু `uses`-এ অন্য action ডাকা যায়, `if` শর্ত ব্যবহার করা যায়।

৮.৭ কনফিগারেশন উদাহরণ

(ক) Composite action — `.github/actions/setup-node-app/action.yml`:

```
name: Setup Node app                  # action-এর নাম
description: Node.js বসিয়ে নির্ভরতা ইনস্টল করে
inputs:
  node-version:                       # ইনপুট ঘোষণা
    description: Node.js সংস্করণ
    required: false
    default: "20"
runs:
  using: composite                    # composite ধরন
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ inputs.node-version }}
        cache: npm
    - run: npm ci
      shell: bash                     # composite-এ shell উল্লেখ বাধ্যতামূলক
```

(খ) Reusable workflow — `.github/workflows/node-ci.yml`:

```
name: Reusable Node CI
on:
  workflow_call:                      # শুধু অন্য workflow ডাকলে চলে
    inputs:
      node-version:
        required: false
        type: string
        default: "20"
    outputs:
      app_version:
        description: অ্যাপের সংস্করণ
        value: ${{ jobs.build.outputs.app_version }}
jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: ./.github/actions/setup-node-app   # নিজের composite action
        with: { node-version: ${{ inputs.node-version }} }
      - run: npm test
      - run: npm run build
      - id: version
        run: echo "app_version=$(node -p \"require('./package.json').version\")" >>
"$GITHUB_OUTPUT"
    outputs:
      app_version: ${{ steps.version.outputs.app_version }}
```

(গ) Caller workflow:

```
name: CI
on: { push: { branches: [main] }, pull_request: { branches: [main] } }
permissions: { contents: read }
jobs:
  ci:
    uses: ./.github/workflows/node-ci.yml   # একই repo-র reusable workflow
    with: { node-version: "20" }
  # অন্য repo থেকে: uses: technova/.github/.github/workflows/node-ci.yml@v1
```

Field ব্যাখ্যা: `workflow_call` — reusable workflow-এর trigger; `inputs` / `outputs` — চুক্তি (contract); `shell: bash` — composite-এ run step-এ বাধ্যতামূলক; `uses: ./...` — স্থানীয় action/workflow-এর পথ।

৮.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র তিনটি MERN repository-তে Node 18, 20 ও 22 মিশে ছিল — একই কোড এক জায়গায় পাস, আরেক জায়গায় ফেল। Reusable workflow চালু করে Node সংস্করণ এক জায়গায় pin করা হলো; এখন সংস্করণ বদলাতে শুধু shared workflow বদলালেই হয়।

৮.৯ ধাপে ধাপে বাস্তবায়ন

1. Composite action ফাইল তৈরি করে একটি workflow-তে ব্যবহার করুন।
2. Reusable workflow তৈরি করে caller workflow থেকে ডাকুন।
3. Input বদলে (Node 20 → 22) দেখুন সব জায়গায় প্রভাব পড়ে কি না।
4. Dependabot-এ action আপডেটের নোটিফিকেশন চালু করুন।

৮.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: Reusable workflow-এ secret পৌঁছাচ্ছে না? মনে রাখবেন secret স্বয়ংক্রিয়ভাবে যায় না — caller-এ `secrets:` দিয়ে স্পষ্টভাবে পাস করতে হয়। Composite action-এ "shell is required" ত্রুটি মানে `run` step-এ `shell: bash` দিতে ভুলেছেন।

লক্ষণ	সম্ভাব্য কারণ	সমাধান
Action পাওয়া যাচ্ছে না	ভুল repo/path বা version tag	Marketplace-এ নাম যাচাই করুন; tag/SHA মিলান
Reusable workflow ডাকা যাচ্ছে না	Caller-এ `uses` ভুল	একই repo-তে `./.github/workflows/...` পথ ব্যবহার করুন
Input খালি আসছে	`inputs.<নাম>` বানান অমিল	ঘোষণা ও ব্যবহারের নাম হুবহু মিলান

৮.১১ নিরাপত্তা বিবেচনা

> **WARNING:** তৃতীয় পক্ষের action আপনার workflow-র মতোই ক্ষমতা পায় — secret পড়তে পারে, কোড বদলাতে পারে। অপরিচিত লেখকের action ব্যবহার করবেন না; জনপ্রিয় action-ও commit SHA-তে pin করুন এবং নিয়মিত আপডেট পরীক্ষা করুন। Supply-chain আক্রমণে আপস করা action-ই সবচেয়ে সাধারণ পথ।

৮.১২ কর্মক্ষমতা ও খরচ

Reusable workflow ডুপ্লিকেশন কমায়, কিন্তু প্রতিটি caller-এ আলাদা run হয় — খরচ কমে না, রক্ষণাবেক্ষণ কমে। Composite action অতিরিক্ত runner খরচ যোগ করে না, কারণ এটি একই job-এ চলে।

৮.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Shared workflow একটিমাত্র repository-তে রাখুন; version tag (`@v1`) দিয়ে ডাকুন; input/output-এর চুক্তি লিখে রাখুন। **অ্যান্টি-প্যাটার্ন:** Copy-paste workflow পাঁচ জায়গায়; `@main` branch দিয়ে action ডাকা (যেকোনো সময় বদলে যেতে পারে); secret reusable workflow-এ না জানিয়ে পাস করা।

৮.১৪ অধ্যায়ের সারাংশ

Action ধাপ পুনর্ব্যবহার করে, composite action কয়েকটি ধাপ পুনর্ব্যবহার করে, reusable workflow পুরো job পুনর্ব্যবহার করে। Pin করা version, স্পষ্ট input/output চুক্তি ও বিশ্বস্ত উৎস — এই তিনটি মেনে চললে পুনর্ব্যবহার নিরাপদ ও টেকসই হয়।

৮.১৫ মূল প্রযুক্তিগত পরিভাষা

Composite Action: কয়েকটি step নিয়ে গঠিত পুনর্ব্যবহারযোগ্য action।

Reusable Workflow: অন্য workflow থেকে ডাকা যায় এমন পুরো workflow।

workflow_call: Reusable workflow চালু করার trigger।

Marketplace: GitHub Actions-এর action-এর ভান্ডার।

Version Pinning: Action-কে নির্দিষ্ট version/SHA-তে আটকে রাখা।

Dependabot: নির্ভরতা ও action আপডেট জানানো GitHub টুল।

Input / Output (contract): পুনর্ব্যবহারযোগ্য অংশের দেওয়া-নেওয়ার নিয়ম।

Supply-chain Attack: নির্ভরতা বা টুলের মাধ্যমে আক্রমণ।

Docker Action: Container-এ চলা action।

JavaScript Action: Node.js-এ লেখা action।

৮.১৬ জ্ঞান যাচাই প্রশ্ন

1. Composite action ও reusable workflow-এর পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
2. Action-কে commit SHA-তে pin করা কেন `@v4` tag-এর চেয়ে নিরাপদ?
3. Reusable workflow-এ secret কীভাবে পাস করতে হয় এবং কেন স্বয়ংক্রিয়ভাবে যায় না?
4. Composite action-এ `shell: bash` কেন বাধ্যতামূলক?
5. কখন copy-paste-এর বদলে reusable workflow ব্যবহার করা উচিত?

৮.১৭ ব্যবহারিক অনুশীলন

1. নিজের setup ধাপগুলো দিয়ে একটি composite action লিখে দুটি workflow-তে ব্যবহার করুন।
2. একটি reusable workflow তৈরি করে দুটি ভিন্ন repository (বা branch) থেকে ডাকুন।
3. একটি action-কে tag থেকে SHA-তে pin করে run সফল হয় কি না যাচাই করুন।

অধ্যায় ৯ • PART II — GitHub Actions Engineering

Chapter 9 — Workflow Triggers, Expressions, Conditions and Matrices

৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) event, trigger ও filter-এর পার্থক্য ব্যাখ্যা করতে; (২) `push`, `pull_request`, `schedule`, `workflow_dispatch` ও `workflow_call` ব্যবহার করতে; (৩) expression ও context (`github`, `env`, `matrix`) লিখতে; (৪) `if` শর্ত ও `needs` দিয়ে job নিয়ন্ত্রণ করতে; (৫) matrix strategy দিয়ে একাধিক Node সংস্করণ ও OS-এ পরীক্ষা চালাতে; (৬) concurrency দিয়ে একই branch-এর পুরনো run বাতিল করতে।

৯.২ কেন এটি গুরুত্বপূর্ণ

ভুল trigger মানে ভুল সময়ে ভুল কাজ: documentation বদলালেও পুরো test suite চলে, অথবা production deploy হয়ে যায় feature branch থেকে। Matrix ছাড়া Node 20-এ পাস করা কোড Node 22-এ ভেঙে প্রোডাকশনে ধরা পড়ে। শর্ত ও trigger-ই pipeline-কে বুদ্ধিমান করে।

৯.৩ পূর্বশর্ত জ্ঞান

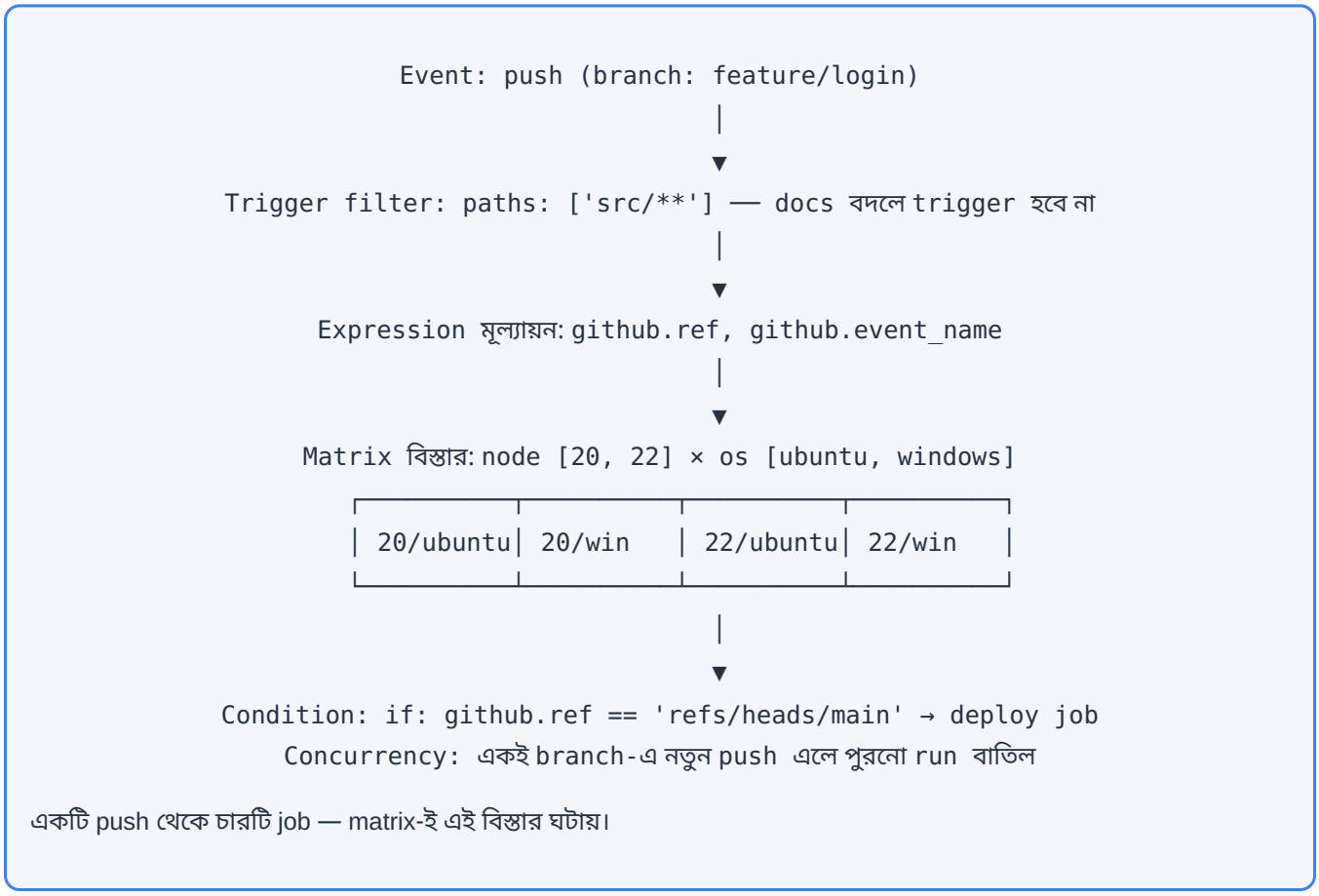
অধ্যায় ৬–৮, YAML, GitHub event ধারণা, `${{ }}` expression-এর প্রাথমিক পরিচয়।

৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

Trigger প্রশ্নের উত্তর দেয় **কখন** চলবে; expression উত্তর দেয় **কী মান** ব্যবহার হবে; condition উত্তর দেয় **চলবে কি না**; matrix উত্তর দেয় **কত রকমভাবে** চলবে। `github` context হলো run-সম্পর্কিত তথ্যের ভান্ডার — কোন branch, কে trigger করল, কোন PR। Expression `${{ }}`-এর ভেতরে লেখা হয় এবং YAML পার্স হওয়ার আগে মূল্যায়ন হয়। এই মূল্যায়ন ক্রম না বুঝলে শর্ত "কাজ করছে না" মনে হবে।

> **CONCEPT:** Trigger = দরজার ঘণ্টা; Expression = ঘণ্টা বাজানো ব্যক্তির পরিচয়পত্র; Condition = দরজা খুলবে কি না; Matrix = একই পরীক্ষা অনেক ঘরে একসঙ্গে।

৯.৫ আর্কিটেকচার / ধারণাগত চিত্র



একটি push থেকে চারটি job — matrix-ই এই বিস্তার ঘটায়।

৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Trigger ও filter

`on.push.branches`, `on.push.paths`, `on.pull_request.types: [opened, synchronize]` — এগুলো দিয়ে trigger সীমিত করুন। `paths-ignore` দিয়ে docs বদলে CI বাদ দেওয়া যায়। মনে রাখবেন: `pull_request` event fork থেকে আসা কোডও চালায় — secret সেখানে পাওয়া যায় না (অধ্যায় ১০)।

Expression ও context

Context	কী থাকে	উদাহরণ
`github`	Run, repo, branch, actor তথ্য	`github.ref`, `github.event_name`
`env`	Workflow/env variable	`env.NODE_ENV`
`matrix`	Matrix-এর বর্তমান মান	`matrix.node-version`
`secrets`	Secret (শুধু `if` বা input-এ)	`secrets.CHANGE_ME_TOKEN`
`needs`	পূর্ববর্তী job-এর output/ফলাফল	`needs.test.result`

Condition — if

`if: success()` (default — আগের ধাপ সফল হলে), `if: failure()` (ব্যর্থ হলে, যেমন Slack-এ জানাতে), `if: always()` (যাই হোক), `if: cancelled()`। Job-স্তরে `if: needs.test.result == 'success'` লেখা যায়।

Matrix

`strategy.matrix` দিয়ে একাধিক মানের সমন্বয় তৈরি হয়। `fail-fast: false` দিলে একটি combination ব্যর্থ হলেও বাকিগুলো চলে। `include` / `exclude` দিয়ে নির্দিষ্ট combination যোগ/বাদ দেওয়া যায়। `max-parallel` দিয়ে একসঙ্গে কতটি job চলবে তা সীমিত করুন।

Concurrency

`concurrency: { group: ci-${{ github.ref }}, cancel-in-progress: true }` — একই branch-এ নতুন push এলে পুরনো run বাতিল হয়, সময় ও খরচ বাঁচে। Production deploy-এ `cancel-in-progress: false` রাখুন — deploy মাঝপথে বাতিল করা বিপজ্জনক।

schedule ও workflow_dispatch

`schedule` cron syntax-এ চলে (UTC) — যেমন রাতে dependency পরীক্ষা। `workflow_dispatch` হাতে চালানোর বোতাম দেয়, input নিতে পারে।

৯.৭ কনফিগারেশন উদাহরণ — Matrix + Condition + Concurrency

```
name: CI Matrix
on:
  push:
    branches: [main]
    paths: ["src/**", "package.json", "package-lock.json"]  # শুধু কোড বদলে
  pull_request:
    branches: [main]
    types: [opened, synchronize, reopened]
  schedule:
    - cron: "0 2 * * 1"        # প্রতি সোমবার ০২:০০ UTC — nightly check
  workflow_dispatch:           # হাতে চালানোর বোতাম
    inputs:
      node-version:
        description: Node সংস্করণ
        required: false
        default: "20"
permissions: { contents: read }
concurrency:
  group: ci-${{ github.ref }}  # প্রতি branch-এ একটিমাত্র run
  cancel-in-progress: true     # নতুন push এলে পুরনো বাতিল
jobs:
  test:
    runs-on: ${{ matrix.os }}  # matrix থেকে OS নেওয়া
    strategy:
      fail-fast: false         # একটি ব্যর্থ হলেও বাকি চলবে
      max-parallel: 4          # একসঙ্গে সর্বোচ্চ ৪টি
      matrix:
        node-version: [20, 22]
        os: [ubuntu-24.04, windows-latest]
```

```
        exclude:
          - node-version: 22
            os: windows-latest  # এই combination বাদ
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: ${{ matrix.node-version }}, cache: npm }
      - run: npm ci
      - run: npm test
  deploy-staging:
    needs: test
    if: github.ref == 'refs/heads/main' && github.event_name == 'push'
    runs-on: ubuntu-24.04
    environment: staging
    steps:
      - run: echo "Deploying to staging from ${{ github.sha }}"
  notify-on-failure:
    needs: test
    if: failure()              # test ব্যর্থ হলেই চলবে
    runs-on: ubuntu-24.04
    steps:
      - run: echo "Test failed on ${{ github.ref }} — notify team (example)"
```

Field ব্যাখ্যা: `paths` — trigger filter; `cron` — UTC সময়সূচি; `workflow_dispatch.inputs` — হাতে চালানোর ইনপুট; `concurrency.group` — run-এর দল; `strategy.matrix` — combination-এর তালিকা; `exclude` — combination বাদ; `fail-fast` — প্রথম ব্যর্থতায় সব থামবে কি না; `if: failure()` — শুধু ব্যর্থতায় চলা job।

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
test (20, ubuntu-24.04) — Success
test (20, windows-latest) — Success
test (22, ubuntu-24.04) — Success
test (22, windows-latest) — Skipped (excluded)
```

৯.৮ বাস্তব পরিস্থিতি — TechNova

> **REAL WORLD:** TechNova-র ডেভেলপাররা দিনে ২০ বার push করতেন, প্রতিবার পুরনো run-ও চলতে থাকত — queue জমে PR-এর ফলাফল আসতে ৪০ মিনিট লাগত। Concurrency group যোগ করার পর প্রতিটি branch-এ শুধু সর্বশেষ run চলে; ফলাফল আসে ৫ মিনিটে।

৯.৯ ধাপে ধাপে বাস্তবায়ন

1. Matrix workflow বসিয়ে Node 20 ও 22-এ test চালান।
2. `exclude` দিয়ে একটি combination বাদ দিয়ে দেখুন।
3. দ্রুত দুটি push দিয়ে পুরনো run বাতিল হয় কি না দেখুন।
4. `workflow_dispatch` দিয়ে হাতে run চালান।
5. `schedule` দিয়ে সাপ্তাহিক nightly run যোগ করুন।

৯.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: Matrix job-এর নামে `(20, ubuntu-24.04)` দেখা যায় — এটাই স্বাভাবিক। `if` শর্তে `${{ }}` লেখা লাগে না (GitHub নিজে যোগ করে), তবে লিখলেও চলে। Cron run না হলে মনে রাখবেন schedule শুধু default branch-এ চলে এবং GitHub ব্যস্ত সময়ে দেরি করতে পারে।

লক্ষণ	সম্ভাব্য কারণ	সমাধান
PR-এ deploy job চলছে	`if` শর্তে `event_name` যাচাই হয়নি	`github.event_name == 'push'` যোগ করুন
Matrix-এ অপ্রত্যাশিত combination	`exclude` ভুল	Matrix মান ও exclude-এর key হুবহু মিলান
সব job একসঙ্গে ব্যর্থ	`fail-fast: true` (default)	ডিবাগের সময় `fail-fast: false` দিন

৯.১১ নিরাপত্তা বিবেচনা

WARNING: `pull_request` event-এ fork-এর কোড চলে — সেখানে `if` শর্ত দিয়ে secret ব্যবহার করা step চালাবেন না। `pull_request_target` event আরও বিপজ্জনক: এটি base branch-এর কোডে চলে কিন্তু PR-এর কোড checkout করতে পারে — না বুঝে ব্যবহার করলে আক্রমণকারী আপনার secret চুরি করতে পারে।

৯.১২ কর্মক্ষমতা ও খরচ

Matrix combination যত বাড়বে, runner-মিনিট তত বাড়বে: ২টি Node × ২টি OS = ৪টি job। শুধু প্রয়োজনীয় combination রাখুন; Windows runner Linux-এর চেয়ে ব্যয়বহুল হতে পারে (plan অনুযায়ী)। `paths` filter ও concurrency দিয়ে অপ্রয়োজনীয় run কমান।

৯.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Trigger filter দিয়ে অপ্রয়োজনীয় run বাদ দিন; matrix-এ শুধু সমর্থিত সংস্করণ রাখুন; deploy-এ কঠোর `if` শর্ত; production concurrency-তে `cancel-in-progress: false`। **অ্যান্টি-প্যাটার্ন:** সব push-এ সবকিছু চালানো; `pull_request_target`-এ PR কোড checkout করে secret-সহ step চালানো; matrix-এ ১০টি combination রেখে খরচ বাড়ানো।

৯.১৪ অধ্যায়ের সারাংশ

Trigger কখন চলবে ঠিক করে, expression তথ্য দেয়, condition চলবে কি না ঠিক করে, matrix কত রকমভাবে চলবে ঠিক করে, concurrency একসঙ্গে কতটি run চলবে নিয়ন্ত্রণ করে। এই পাঁচটি একসঙ্গে মিলেই pipeline-কে নির্ভুল, দ্রুত ও নিরাপদ করে।

৯.১৫ মূল প্রযুক্তিগত পরিভাষা

Trigger / Event: workflow শুরু করা ঘটনা।

Expression: `${{ }}` -এর ভেতরে লেখা মূল্যায়নযোগ্য অভিব্যক্তি।

Context: run-সম্পর্কিত তথ্যের ভান্ডার (github, matrix, env)।

Matrix Strategy: একাধিক মানের সমন্বয়ে job বিস্তার।

fail-fast: একটি combination ব্যর্থ হলে বাকি থামবে কি না।

Concurrency: একই group-এ একসঙ্গে চলা run নিয়ন্ত্রণ।

workflow_dispatch: হাতে workflow চালানোর trigger।

schedule (cron): সময়সূচি অনুযায়ী চালানো।

pull_request_target: base branch প্রসঙ্গে চলা PR event — সতর্কতায় ব্যবহার করুন।

Condition (if): job/step চলবে কি না তা নির্ধারণ করা শর্ত।

৯.১৬ জ্ঞান যাচাই প্রশ্ন

1. `push` ও `pull_request` trigger-এর মধ্যে নিরাপত্তাগত পার্থক্য কী?
2. Matrix-এ ৩টি Node সংস্করণ ও ২টি OS থাকলে কয়টি job তৈরি হয়? `exclude` কীভাবে তা বদলায়?
3. `if: failure()` কখন ব্যবহার করবেন — একটি উদাহরণ দিন।
4. Concurrency group কেন production deploy-এ `cancel-in-progress: false` হওয়া উচিত?
5. `pull_request_target` কেন `pull_request`-এর চেয়ে ঝুঁকিপূর্ণ?

৯.১৭ ব্যবহারিক অনুশীলন

1. Node 20 ও 22-এর matrix workflow লিখে দুটি সংস্করণেই test চালান।
2. `paths` filter যোগ করে README বদলে CI না চলা যাচাই করুন।

3. Concurrency group যোগ করে দ্রুত দুটি push দিয়ে পুরনো run বাতিল হওয়া দেখান।

অধ্যায় ১০ • PART II — GitHub Actions Engineering

Chapter 10 — Secrets, Variables, Permissions and OIDC

১০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) secret, variable ও environment-এর পার্থক্য ব্যাখ্যা করতে; (২) repository/organization/environment secret নিরাপদে সেট করতে; (৩) `GITHUB_TOKEN`-এর permission ন্যূনতম (least privilege) করতে; (৪) environment protection rule ও approval সেট করতে; (৫) OIDC দিয়ে short-lived credential ব্যবহার করে দীর্ঘমেয়াদি secret বাদ দিতে; (৬) secret ফাঁসের ঝুঁকি চিহ্নিত করতে।

১০.২ কেন এটি গুরুত্বপূর্ণ

CI/CD pipeline-এ secret-ই সবচেয়ে মূল্যবান জিনিস — API key, registry password, cloud credential। একটি ভুল `echo`, একটি ভুল permission, একটি অপরিচিত PR — যেকোনো একটিই পুরো production দখলের পথ খুলে দিতে পারে। আবার secret ছাড়া deploy-ও হয় না। তাই secret ব্যবস্থাপনা শুধু কনফিগারেশন নয়, এটি pipeline নিরাপত্তার কেন্দ্র।

১০.৩ পূর্বশর্ত জ্ঞান

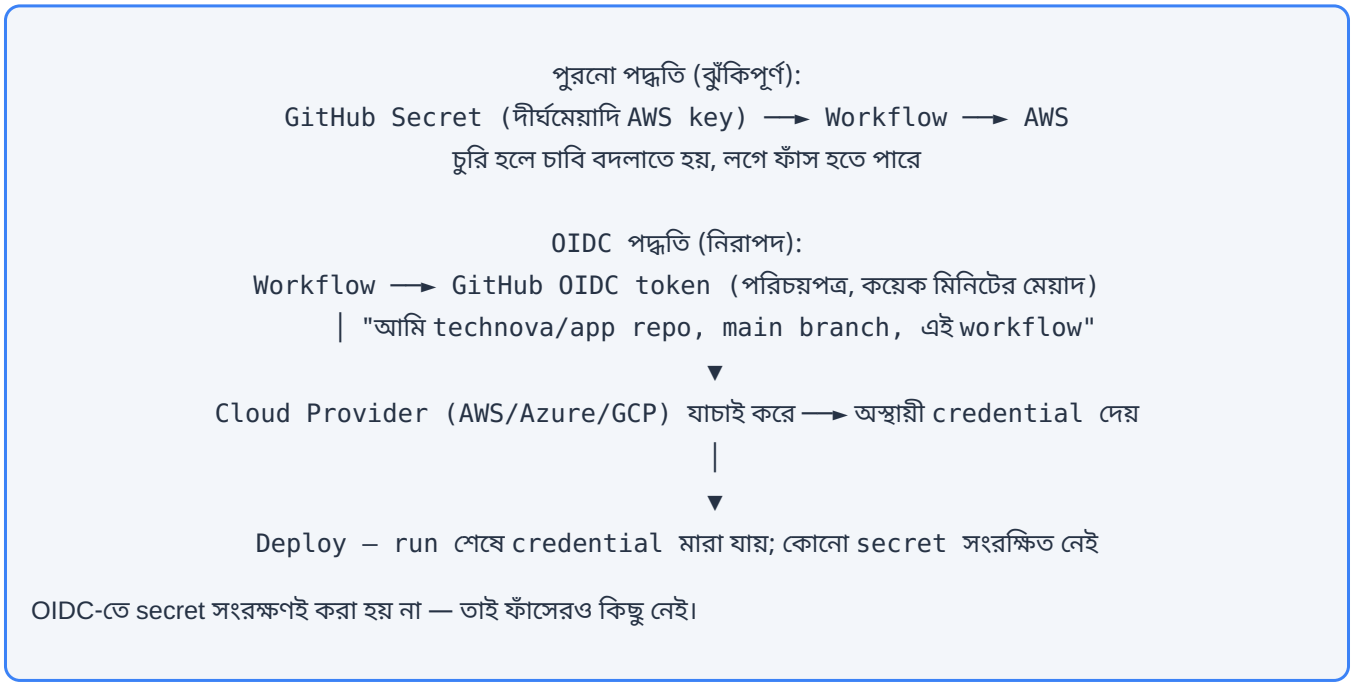
অধ্যায় ৬–৯, environment variable ধারণা, GitHub repository settings-এর প্রাথমিক পরিচয়।

১০.৪ প্রথম নীতি থেকে ব্যাখ্যা

Secret মানে এমন মান যা জানলে কেউ আপনার হয়ে কাজ করতে পারে। তাই তিনটি নিয়ম: **(১) Secret কোডে লিখবেন না** — GitHub-এর secret store-এ রাখুন; **(২) Secret যত কম জায়গায় যায় তত ভালো** — শুধু যে job-এর দরকার সেই job-এই পাস করুন; **(৩) Secret যত কম সময় বাঁচে তত ভালো** — এখানেই OIDC আসে। **OIDC** (OpenID Connect) মানে GitHub নিজেই পরিচয় দেয়: cloud provider GitHub-কে বিশ্বাস করে একটি অস্থায়ী token দেয়, কোনো দীর্ঘমেয়াদি password ছাড়াই। Password চুরি হওয়ার সুযোগই থাকে না, কারণ password-ই নেই।

> **CONCEPT:** দীর্ঘমেয়াদি secret = বাড়ির চাবি হারালে যে কেউ ঢুকতে পারে; OIDC = দারোয়ান পরিচয় যাচাই করে একবারের পাস দেয়। চাবি না থাকলে চাবি চুরির ভয় নেই।

১০.৫ আর্কিটেকচার / ধারণাগত চিত্র



OIDC-তে secret সংরক্ষণই করা হয় না — তাই ফাঁসেরও কিছু নেই।

১০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Secret বনাম Variable বনাম Environment

ধরন	কী	কোথায়	উদাহরণ
Secret	গোপন মান, লগে মাস্ক হয়	Repo / Org / Environment	CHANGE-ME-REGISTRY-PASSWORD
Variable	গোপন নয় এমন কনফিগারেশন	Repo / Org / Environment	APP_NAME, REGION
Environment	Deploy লক্ষ্য + নিয়মের সমষ্টি	Repo settings	staging, production (+approval)

GITHUB_TOKEN permission

Workflow-স্তরে `permissions` ঘোষণা করুন — যেমন `contents: read`, `pull-requests: write` (শুধু comment দরকার হলে)। `id-token: write` শুধু OIDC job-এ দিন। Repository settings-এ default permission "read-only" রাখুন। `permissions: write-all` কখনো ব্যবহার করবেন না।

Environment protection

Production environment-এ **required reviewers** (অনুমোদন ছাড়া deploy নয়) ও **wait timer** সেট করুন। `environment: production` লেখা job তখনই চলবে যখন অনুমোদন আসে।

OIDC — ধাপে ধাপে

(১) Cloud provider-এ identity provider হিসেবে GitHub যোগ করুন ও trust policy লিখুন (কোন repo/branch/workflow বিশ্বাসযোগ্য)। (২) Workflow-এ `permissions: id-token: write` দিন। (৩) `actions/configure-aws-`

`credentials`-এর মতো action দিয়ে অস্থায়ী credential নিন। (৪) Deploy করুন। কোনো দীর্ঘমেয়াদি key সংরক্ষণ করবেন না।

Secret rotation ও ফাঁসের প্রতিক্রিয়া

Secret নিয়মিত বদলান (rotation)। Secret ফাঁস হয়েছে সন্দেহ হলে সঙ্গে সঙ্গে বাতিল (revoke) করুন, লগ পরীক্ষা করুন, নতুন secret বসান। GitHub secret push protection চালু করলে commit-এ secret ধরা পড়ে।

১০.৭ কনফিগারেশন উদাহরণ — OIDC Deploy + Environment Protection

```
name: Deploy to Production
on:
  push: { branches: [main] }
permissions:
  contents: read          # শুধু কোড পড়া
  id-token: write         # OIDC token নেওয়ার অনুমতি (শুধু এই workflow-এ)
jobs:
  deploy:
    runs-on: ubuntu-24.04
    environment: production    # অনুমোদন ছাড়া চলবে না (required reviewers)
    steps:
      - uses: actions/checkout@v4
      - name: Configure AWS credentials via OIDC
        uses: aws-actions/configure-aws-credentials@v4
        with:
          role-to-assume: arn:aws:iam::123456789012:role/technova-deploy-role  # কাল্পনিক ARN
          aws-region: ap-south-1
      # কোনো দীর্ঘমেয়াদি AWS key এখানে নেই — credential অস্থায়ী
      - run: echo "Deploying with temporary credentials (example)"
      - name: Use a secret safely
        env:
          REGISTRY_PASSWORD: ${{ secrets.CHANGE_ME_REGISTRY_PASSWORD }}  # কাল্পনিক placeholder
        run: echo "$REGISTRY_PASSWORD" | docker login ghcr.io -u technova --password-stdin
      # ভুল: run: echo ${{ secrets.CHANGE_ME_REGISTRY_PASSWORD }}  — কখনো করবেন না
```

Field ব্যাখ্যা: `id-token: write` — OIDC token চাওয়ার অনুমতি; `role-to-assume` — cloud-এ কোন role নেওয়া হবে (trust policy-তে repo/branch সীমিত থাকবে); `environment: production` — protection rule চালু করে; `env:` দিয়ে secret environment variable হিসেবে পাস করা নিরাপদ, সরাসরি `run`-এ লেখা নয়; `--password-stdin` — password command-line-এ দেখা যায় না।

১০.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র এক ডেভেলপার deploy script-এ `echo $API_KEY` ডিবাগ করতে গিয়ে লগে key ছাপিয়ে ফেলেছিলেন — GitHub মাস্ক করলেও key-টি base64-এ এনকোড করা ছিল, তাই মাস্ক হয়নি। শিক্ষা: secret কখনো print করবেন না, আর OIDC-তে যাওয়ার পর deploy-এ কোনো দীর্ঘমেয়াদি key-ই রইল না।

১০.৯ ধাপে ধাপে বাস্তবায়ন

1. Settings → Secrets and variables → Actions-এ একটি test secret বসান (মান: CHANGE-ME-SECRET)।

2. Workflow-এ `permissions: contents: read` ঘোষণা করুন।
3. Settings → Environments-এ `production` তৈরি করে required reviewer যোগ করুন।
4. Deploy job-এ `environment: production` দিয়ে PR থেকে deploy আটকে যায় কি না দেখুন।
5. Cloud provider-এর ডকুমেন্টেশন পড়ে OIDC trust policy-র নমুনা বুঝুন (lab-এ শুধু পরিকল্পনা করুন, আসল cloud অ্যাকাউন্ট ছাড়া নয়)।

১০.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: Secret খালি আসছে? Fork থেকে আসা PR-এ secret পাওয়া যায় না — এটাই স্বাভাবিক নিরাপত্তা। "Error: Could not assume role" মানে OIDC trust policy-তে repo/branch/workflow শর্ত মেলেনি — provider-এর trust policy ও workflow-র `permissions` মিলিয়ে দেখুন। লগে `***` দেখা মানে secret মাস্ক হয়েছে — ভালো লক্ষণ।

লক্ষণ	সম্ভাব্য কারণ	সমাধান
Secret fork PR-এ খালি	নিরাপত্তা নিয়ম	Fork PR-এ secret দরকার এমন step চালাবেন না
OIDC token পাওয়া যাচ্ছে না	`id-token: write` নেই	Workflow-স্তরে permission যোগ করুন
Deploy অনুমোদন ছাড়াই চলছে	Environment protection সেট হয়নি	Environment-এ required reviewer সেট করুন

১০.১১ নিরাপত্তা বিবেচনা

WARNING: Secret কখনো কোডে, লগে বা `pull_request_target` workflow-এ PR-এর কোডের কাছে দেবেন না। `permissions: write-all` ও `contents: write` অপ্রয়োজনে দেবেন না। Secret ফাঁস হলে ধরে নিন তা আপস হয়েছে — বাতিল করুন, বদলান, লগ পরীক্ষা করুন।

১০.১২ কর্মক্ষমতা ও খরচ

OIDC-তে অতিরিক্ত খরচ নেই — বরং secret rotation-এর পরিচালন খরচ কমে। Environment approval-এ মানুষের অপেক্ষা deploy-কে ধীর করতে পারে — সেটাই উদ্দেশ্য: production-এ গতি নয়, নিরাপত্তা আগে।

১০.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Least privilege permission; environment-ভিত্তিক secret; OIDC যেখানে সম্ভব; secret rotation; push protection চালু। **অ্যান্টি-প্যাটার্ন:** Secret YAML-এ লেখা; `echo` দিয়ে secret ছাপানো; এক secret সব environment-এ ব্যবহার; দীর্ঘমেয়াদি cloud key workflow-এ রাখা।

১০.১৪ অধ্যায়ের সারাংশ

Secret গোপন মান, variable গোপন নয় এমন কনফিগারেশন, environment deploy-এর নিয়ম। GITHUB_TOKEN-এর permission ন্যূনতম রাখুন, production-এ approval বাধ্যতামূলক করুন, আর যেখানে সম্ভব OIDC দিয়ে দীর্ঘমেয়াদি secret বাদ দিন — কারণ সবচেয়ে নিরাপদ secret হলো যেটা অস্তিত্বই নেই।

১০.১৫ মূল প্রযুক্তিগত পরিভাষা

Secret: গোপন মান (API key, password) — লগে মাস্ক হয়।

Variable: গোপন নয় এমন কনফিগারেশন মান।

Least Privilege: শুধু প্রয়োজনীয় অনুমতিটুকু দেওয়ার নীতি।

OIDC: OpenID Connect — পরিচয় যাচাই করে অস্থায়ী credential দেওয়ার ব্যবস্থা।

id-token: OIDC পরিচয়পত্র চাওয়ার permission।

Environment Protection: Deploy-এর আগে অনুমোদন/অপেক্ষার নিয়ম।

Required Reviewer: Deploy অনুমোদনকারী ব্যক্তি।

Secret Rotation: নিয়মিত secret বদলানো।

Push Protection: Commit-এ secret ধরা পড়লে আটকানো।

Trust Policy: Cloud provider-এ কোন পরিচয় বিশ্বাসযোগ্য তার নিয়ম।

১০.১৬ জ্ঞান যাচাই প্রশ্ন

1. Secret, variable ও environment-এর পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
2. OIDC কীভাবে দীর্ঘমেয়াদি secret ছাড়া deploy সম্ভব করে?
3. `permissions: contents: read` কেন `write-all`-এর চেয়ে ভালো?
4. Fork থেকে আসা PR-এ secret পাওয়া যায় না কেন?
5. Secret ফাঁস হয়েছে সন্দেহ হলে প্রথম তিনটি কাজ কী করবেন?

১০.১৭ ব্যবহারিক অনুশীলন

1. একটি test secret বসিয়ে workflow-এ `env:` দিয়ে নিরাপদে ব্যবহার করুন; লগে মাস্ক হয় কি না দেখুন।
2. `production` environment-এ required reviewer সেট করে deploy আটকে যায় কি না পরীক্ষা করুন।
3. একটি workflow-এর permission `contents: read`-এ সীমিত করে run সফল হয় কি না যাচাই করুন।

অধ্যায় ১১ • PART II — GitHub Actions Engineering

Chapter 11 — Caching, Artifacts and Workflow Optimization

১১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) cache ও artifact-এর পার্থক্য ব্যাখ্যা করতে; (২) `actions/cache` ও `setup-node`-এর cache ব্যবহার করতে; (৩) artifact upload/download দিয়ে job-এর মধ্যে build ফলাফল পাস করতে; (৪) cache key ও restore key ডিজাইন করতে; (৫) workflow-র সময় ও খরচ কমানোর কৌশল প্রয়োগ করতে; (৬) workflow debug করতে (debug logging, `tmate` নয় — শুধু নিরাপদ উপায়ে)।

১১.২ কেন এটি গুরুত্বপূর্ণ

প্রতিটি run-এ `npm ci` শূন্য থেকে নির্ভরতা নামালে ২–৩ মিনিট লেগে যায় — দিনে ২০ বার push করলে ঘণ্টার পর ঘণ্টা নষ্ট হয়, আর runner-মিনিটের খরচ বাড়ে। Cache সেই সময় ৩০ সেকেন্ডে নামায়। আবার ভুল cache পুরনো নির্ভরতা দিয়ে ভুল build তৈরি করতে পারে। দ্রুত pipeline মানে দ্রুত feedback, আর দ্রুত feedback মানেই ভালো ডেভেলপার অভিজ্ঞতা।

১১.৩ পূর্বশর্ত জ্ঞান

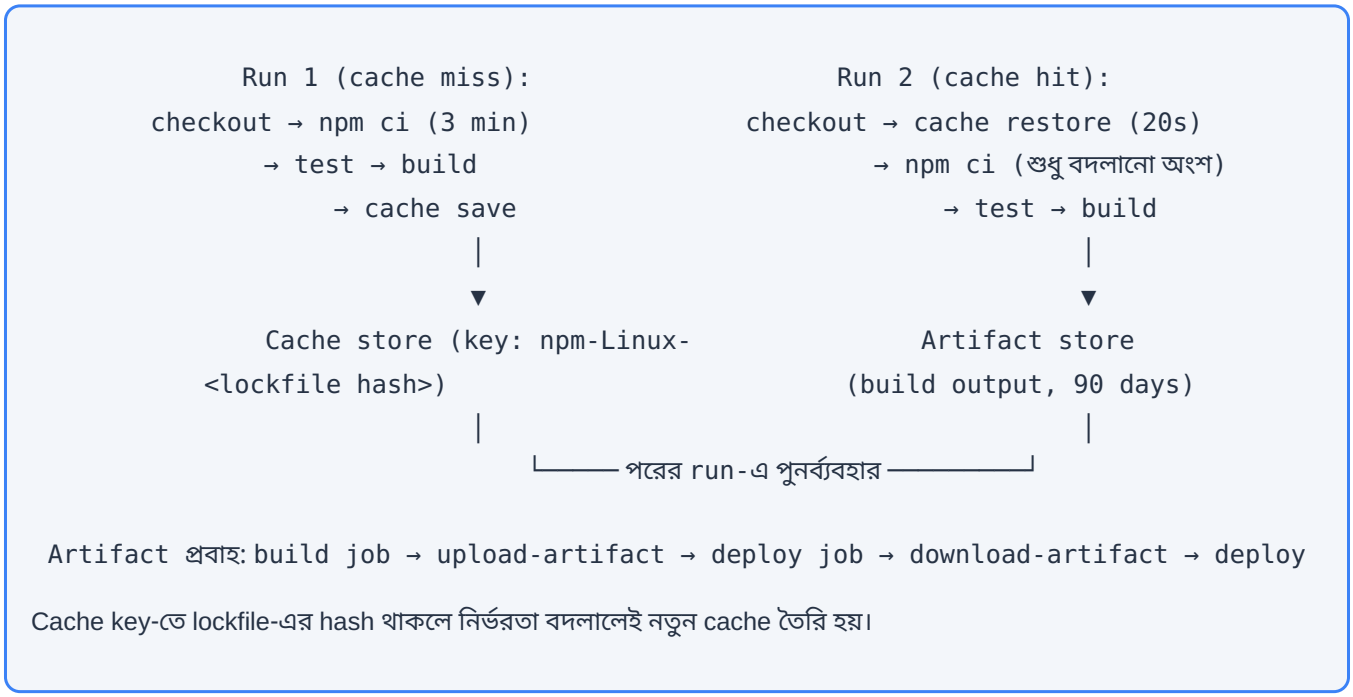
অধ্যায় ৬–১০, `npm ci` ও `node_modules` ধারণা, artifact-এর প্রাথমিক পরিচয় (অধ্যায় ৭)।

১১.৪ প্রথম নীতি থেকে ব্যাখ্যা

Runner প্রতিবার নতুন মেশিন — তাই কিছুই মনে রাখে না। **Cache** হলো runner-এর বাইরে রাখা "মনে রাখার" ব্যবস্থা: lockfile একই থাকলে আগেরবারের `node_modules` আবার ব্যবহার করা যায়। **Artifact** হলো run-এর ফলাফল সংরক্ষণ: build করা ফাইল, test রিপোর্ট — যা পরে ডাউনলোড করা যায় বা অন্য job ব্যবহার করে। পার্থক্য: cache পুনরাবৃত্তি এড়ায় (ভবিষ্যতের run দ্রুত করে), artifact ফলাফল সংরক্ষণ করে (এই run-এর আউটপুট রাখে)। Cache ভুল হলে শুধু ধীর হয়; artifact ভুল হলে ভুল জিনিস deploy হয় — তাই artifact-এর সতর্কতা বেশি।

> **CONCEPT:** Cache = রান্নাঘরের প্রস্তুত উপকরণ (পরের রান্না দ্রুত হয়); Artifact = রান্না করা খাবার (পরিবেশন/সংরক্ষণের জন্য)। উপকরণ পুরনো হলে রান্না ধীর হয়, খাবার ভুল হলে বিপদ হয়।

১১.৫ আর্কিটেকচার / ধারণাগত চিত্র



Cache key-তে lockfile-এর hash থাকলে নির্ভরতা বদলালেই নতুন cache তৈরি হয়।

১১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Cache key ডিজাইন

`key: npm-${{ runner.os }}-${{ hashFiles('**/package-lock.json') }}` — OS ও lockfile hash মিলিয়ে key। Lockfile বদলালে নতুন key, পুরনো cache মেলে না। `restore-keys` দিয়ে আংশিক মিল (prefix) থেকে পুরনো cache নেওয়া যায় — দ্রুত, কিন্তু পুরনো হতে পারে। Cache-এর আকার সীমা আছে (GitHub-এর নির্ধারিত সীমা; অফিসিয়াল ডকুমেন্টেশন দেখুন) এবং পুরনো cache স্বয়ংক্রিয়ভাবে মুছে যায়।

setup-node cache বনাম actions/cache

`actions/setup-node@v4`-এর `cache: npm` সবচেয়ে সহজ — Node.js project-এর জন্য এটাই ব্যবহার করুন। `actions/cache@v4` দিয়ে যেকোনো ফোল্ডার cache করা যায় (যেমন Playwright browser, Turbo build cache)।

Artifact — upload/download

`actions/upload-artifact@v4` দিয়ে ফাইল সংরক্ষণ করুন, `actions/download-artifact@v4` দিয়ে অন্য job-এ নামান। `retention-days` দিয়ে কতদিন রাখা হবে ঠিক করুন (default ৯০ দিন)। বড় artifact (যেমন `node_modules`) আপলোড করবেন না — শুধু build output ও রিপোর্ট রাখুন।

Optimization কৌশল

কৌশল	প্রভাব	সতর্কতা
Cache (npm)	Install সময় ২–৩ মিনিট → ২০–৩০ সেকেন্ড	Lockfile hash key-তে রাখুন
সমান্তরাল job	মোট সময় কমে	প্রতিটি job-এ install আবার চলে
	অপ্রয়োজনীয় run বাদ	Filter ভুল হলে প্রয়োজনীয় run বাদ পড়ে

Paths filter		
Concurrency cancel	পুরনো run বাতিল	Deploy-এ ব্যবহার করবেন না
ছোট runner image	শুরু দ্রুত	প্রয়োজনীয় টুল আছে কি না যাচাই করুন

Debugging

লগে প্রতিটি step-এর আউটপুট দেখুন; `ACTIONS_STEP_DEBUG` secret `true` করলে বিস্তারিত debug লগ আসে। `run: ls -la` বা `cat package.json` দিয়ে runner-এর অবস্থা যাচাই করুন। `act` টুল দিয়ে স্থানীয়ভাবে workflow চালিয়ে দেখা যায় (lab-এ ঐচ্ছিক)। Debug-এর জন্য SSH-ভিত্তিক টুল (যেমন tmate) ব্যবহার করলে secret ফাঁসের ঝুঁকি থাকে — public repo-তে কখনো ব্যবহার করবেন না।

১১.৭ কনফিগারেশন উদাহরণ — Optimized CI + Artifact

```
name: Optimized CI
on:
  push: { branches: [main] }
  pull_request: { branches: [main] }
permissions: { contents: read }
concurrency:
  group: ci-${{ github.ref }}
  cancel-in-progress: true
jobs:
  build:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: npm                    # npm cache — সবচেয়ে সহজ উপায়
      - run: npm ci
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: build-output            # artifact-এর নাম
          path: dist/                   # শুধু build output
          retention-days: 7             # ৭ দিন রাখা হবে
          if-no-files-found: error      # ফাইল না থাকলে ব্যর্থ হবে

  test:
    runs-on: ubuntu-24.04
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm }
      - run: npm ci
      - run: npm test -- --coverage
      - uses: actions/upload-artifact@v4
        with:
          name: coverage-report
          path: coverage/
          retention-days: 7
```

```
  deploy-staging:
    needs: [build, test]
    if: github.ref == 'refs/heads/main'
    runs-on: ubuntu-24.04
    environment: staging
    steps:
      - uses: actions/download-artifact@v4
        with:
          name: build-output            # build job-এর artifact নামায়
          path: dist/
      - run: ls -la dist/               # যাচাই: ফাইল এসেছে কি না
      - run: echo "Deploying dist/ to staging (example)"
```

Field ব্যাখ্যা: `cache: npm` — setup-node-এর অন্তর্নির্মিত cache; `retention-days` — artifact কতদিন থাকবে; `if-no-files-found: error` — খালি artifact ধরা পড়ে; `download-artifact` — অন্য job-এর artifact নামায়; `needs` — deploy শুধু build+test সফল হলে চলে।

নমুনা আউটপুট — চিত্রণমূলক:

```
নমুনা আউটপুট — চিত্রণমূলক
build: cache hit (npm) — npm ci 24s (আগে 2m 40s)
test:  coverage-report artifact uploaded (7 days)
deploy-staging: dist/ ডাউনলোড সফল — 12টি ফাইল
```

১১.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র CI ১১ মিনিট লাগত — প্রতিটি job-এ `npm ci` শূন্য থেকে চলত। Cache যোগ করার পর ৪ মিনিটে নামে। কিন্তু একবার lockfile বদল না করে `package.json` বদলানো হয়েছিল — cache পুরনো নির্ভরতা দিয়েছিল, test ভুলভাবে পাস করেছিল। শিক্ষা: cache key-তে lockfile hash রাখুন, আর `npm ci` lockfile-এর সঙ্গে অমিল হলে নিজেই ব্যর্থ হয় — সেটাই নিরাপত্তা।

১১.৯ ধাপে ধাপে বাস্তবায়ন

1. ওপরের workflow বসিয়ে প্রথম run-এর সময় লিখে রাখুন (cache miss)।
2. দ্বিতীয় run-এ সময় কমে কি না দেখুন (cache hit)।
3. Artifacts ট্যাবে build-output ডাউনলোড করে যাচাই করুন।
4. `retention-days: 7` সেট করে পুরনো artifact মুছে যায় কি না লক্ষ করুন।
5. `ACTIONS_STEP_DEBUG` secret `true` করে একটি run-এর বিস্তারিত লগ দেখুন, তারপর secret মুছে দিন।

১১.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: Cache hit হচ্ছে না? key-তে `hashFiles('**/package-lock.json')` আছে কি না দেখুন — lockfile-এর পথ ভুল হলে hash বদলে যায়। Artifact ডাউনলোডে ফাইল নেই? `upload-artifact`-এর `path` ও `download-artifact`-এর `name` হুবহু মিলিয়ে দেখুন। "Cache size exceeded" হলে বড় ফোল্ডার (যেমন `node_modules` সরাসরি) cache করবেন না।

লক্ষণ	সম্ভাব্য কারণ	সমাধান
প্রতিবার cache miss	Key-তে lockfile hash নেই / lockfile বদলাচ্ছে	Key-তে hashFiles যোগ করুন; lockfile commit করুন
পুরনো নির্ভরতা দিয়ে build	restore-keys পুরনো cache দিচ্ছে	Key-তে hash রাখুন; প্রয়োজনে cache মুছে আবার চালান
Artifact খালি	`path` ভুল বা build ব্যর্থ	`if-no-files-found: error` দিন; build log দেখুন
Workflow ধীর, কারণ অজানা	কোন step ধীর তা জানা নেই	প্রতিটি step-এর সময় লগে দেখুন; debug logging চালু করুন

১১.১১ নিরাপত্তা বিবেচনা

WARNING: Cache-এ secret বা সংবেদনশীল ফাইল রাখবেন না — cache অন্য branch-এর run-ও ব্যবহার করতে পারে। Artifact-এ secret বা `.env` ফাইল আপলোড করবেন না — artifact যে কেউ ডাউনলোড করতে পারে যার repository access আছে। Debug logging চালু রাখলে secret লগে আসতে পারে — debug শেষে বন্ধ করুন।

১১.১২ কর্মক্ষমতা ও খরচ

Cache ও সমান্তরাল job মিলে CI সময় ৫০–৭০% কমাতে পারে (TechNova-র উদাহরণে ১১ → ৪ মিনিট)। কিন্তু cache সংরক্ষণ ও artifact সংরক্ষণেরও সীমা আছে — `retention-days` কম রাখুন, বড় artifact এড়িয়ে চলুন। অপ্রয়োজনীয় matrix combination ও অপ্রয়োজনীয় run বাদ দেওয়াই সবচেয়ে বড় সাশ্রয়।

১১.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: `setup-node`-এর cache ব্যবহার করুন; cache key-তে lockfile hash রাখুন; artifact-এ শুধু প্রয়োজনীয় ফাইল রাখুন; retention সীমিত করুন; debug শেষে debug logging বন্ধ করুন। **অ্যান্টি-প্যাটার্ন:** `node_modules` artifact হিসেবে আপলোড করা; cache-এ secret রাখা; retention ৯০ দিন রেখে storage ভরানো; debug logging স্থায়ীভাবে চালু রাখা।

১১.১৪ অধ্যায়ের সারাংশ

Cache ভবিষ্যতের run দ্রুত করে, artifact এই run-এর ফলাফল সংরক্ষণ করে। Lockfile hash-ভিত্তিক cache key, সীমিত retention, সমান্তরাল job ও trigger filter মিলে pipeline দ্রুত ও সাশ্রয়ী হয়। কিন্তু cache ও artifact-এ কখনো secret রাখবেন না — গতির জন্য নিরাপত্তা বিসর্জন দেওয়া যাবে না।

১১.১৫ মূল প্রযুক্তিগত পরিভাষা

Cache: পুনরাবৃত্তি এড়াতে সংরক্ষিত ফাইল (যেমন node_modules)।

Cache Key: cache শনাক্ত করার নাম — সাধারণত lockfile hash-সহ।

Restore Key: আংশিক মিলে পুরনো cache নেওয়ার prefix।

Cache Hit / Miss: cache পাওয়া গেছে / পাওয়া যায়নি।

Artifact: Run-এর সংরক্ষিত ফলাফল (build output, রিপোর্ট)।

Retention Days: Artifact/cache কতদিন রাখা হবে।

Debug Logging: বিস্তারিত লগ চালু করার ব্যবস্থা (ACTIONS_STEP_DEBUG)।

Runner-minute: Runner ব্যবহারের সময়ের একক — খরচের ভিত্তি।

Optimization: একই ফলাফল কম সময়/খরচে পাওয়ার কৌশল।

act: স্থানীয়ভাবে GitHub Actions চালানোর টুল (ঐচ্ছিক)।

১১.১৬ জ্ঞান যাচাই প্রশ্ন

1. Cache ও artifact-এর পার্থক্য উদাহরণসহ ব্যাখ্যা করুন।
2. Cache key-তে lockfile hash রাখা কেন জরুরি?
3. `retention-days` কম রাখা কেন ভালো?
4. Debug logging চালু রাখলে কী ঝুঁকি থাকে?
5. CI সময় কমানোর তিনটি কৌশল লিখুন এবং প্রতিটির সতর্কতা বলুন।

১১.১৭ ব্যবহারিক অনুশীলন

1. Cache ছাড়া ও cache-সহ run-এর সময় মেপে পার্থক্য লিখুন।
2. Build artifact upload করে অন্য job-এ download করে deploy ধাপ সম্পূর্ণ করুন।
3. ইচ্ছা করে cache key ভুল করে cache miss তৈরি করুন, তারপর ঠিক করুন।

PART III • DEVOPS ENGINEERING MASTERY

GitLab CI/CD and Jenkins

GitLab CI/CD — pipeline, runner, variable, environment — এবং Jenkins — controller/agent, Pipeline, credential ও নিরাপত্তা।

অধ্যায় 12 • PART III — GitLab CI/CD and Jenkins

Chapter 12 — GitLab CI/CD Architecture and Pipeline Configuration

১২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) GitLab-এর একীভূত (integrated) DevOps প্ল্যাটফর্মের স্থাপত্য ব্যাখ্যা করতে; (২) `.gitlab-ci.yml` ফাইলের গঠন ও প্রতিটি গুরুত্বপূর্ণ কীওয়ার্ডের অর্থ বলতে; (৩) Stage, Job ও Pipeline-এর সম্পর্ক আঁকতে; (৪) Pipeline-এর ধরন — Branch, Merge Request, Scheduled, Manual ও Parent-Child — আলাদা করতে; (৫) একটি MERN অ্যাপের জন্য সম্পূর্ণ Node.js পাইপলাইন লিখতে; (৬) Pipeline ব্যর্থ হলে লগ পড়ে মূল কারণ খুঁজতে।

১২.২ কেন এটি বাস্তব সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

GitHub Actions-এ CI/CD একটি আলাদা সেবা হলেও GitLab-এ রিপোজিটরি, Issue, Merge Request, Registry ও CI/CD একই প্ল্যাটফর্মের ভেতরে থাকে। এর অর্থ — কোড push করা থেকে শুরু করে deploy ও monitoring পর্যন্ত এক জায়গায় হয়, আলাদা টুল জোড়া দেওয়ার ঝামেলা কমে। TechNova-র মতো ছোট দলের জন্য এটি বড় সুবিধা: একজন ডেভেলপার Merge Request খুললেই স্বয়ংক্রিয়ভাবে পরীক্ষা চলে, রিভিউয়ার একই পর্দায় কোড, পরীক্ষার ফল ও নিরাপত্তা রিপোর্ট দেখতে পান। কিন্তু এই একীভূত সুবিধার মূল্য হলো — YAML-এর একটি ভুলে পুরো delivery থেমে যেতে পারে। তাই `.gitlab-ci.yml`-এর প্রতিটি ক্ষেত্র বোঝা শুধু কনফিগারেশন নয়, এটি release engineering-এর মূল দক্ষতা।

১২.৩ পূর্বশর্ত জ্ঞান

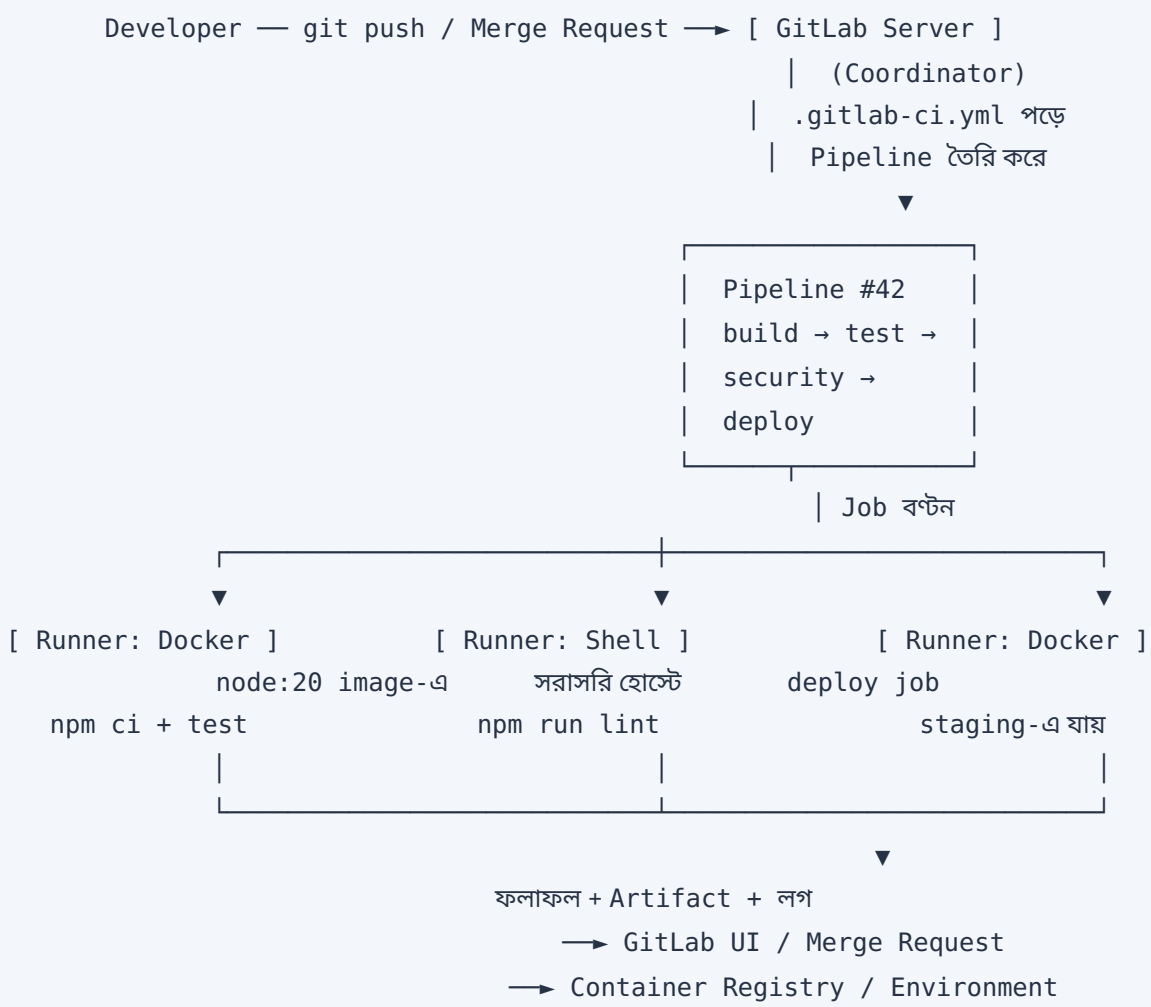
Git-এর branch, commit ও merge ধারণা, YAML-এর indentation ও key-value গঠন, অধ্যায় ৩-এর pipeline lifecycle এবং অধ্যায় ৫-এর build ও artifact ধারণা জানা থাকতে হবে। Docker-এর প্রাথমিক ধারণা (image, container) থাকলে Runner অধ্যায়টি সহজ হবে, তবে এই অধ্যায়ে প্রতিটি ধারণা প্রথম ব্যবহারে ব্যাখ্যা করা হয়েছে। একটি GitLab অ্যাকাউন্ট (SaaS — gitlab.com — অথবা self-managed) ও Node.js 20 LTS থাকলে হাতে-কলমে অনুশীলন করা যাবে।

১২.৪ প্রথম নীতি থেকে ব্যাখ্যা

মূল প্রশ্নটি সহজ: **কোড রিপোজিটরিতে গেলেই কী কী স্বয়ংক্রিয়ভাবে ঘটবে, কে সিদ্ধান্ত নেবে, এবং কোথায় চলবে?** GitLab-এর উত্তর তিনটি স্তরে ভাগ করা। প্রথমে **ঘোষণা (declaration)**: আপনি `.gitlab-ci.yml`-এ লিখে দেন কী করতে হবে — এটি একটি রেসিপি। দ্বিতীয়ত **অর্কেস্ট্রেশন (orchestration)**: GitLab Server (Coordinator) push বা Merge Request-এর ঘটনা দেখে রেসিপি পড়ে, Job-গুলোকে Stage অনুযায়ী সাজায় ও Runner-দের কাজ বণ্টন করে। তৃতীয়ত **সম্পাদন (execution)**: GitLab Runner নামের এজেন্ট আসল কমান্ড চালায় — `npm install`, `npm test`, Docker build — এবং ফলাফল ফেরত পাঠায়। এই তিন স্তর আলাদা রাখার কারণেই আপনি YAML বদলে পুরো delivery বদলাতে পারেন, Runner বদলে চলার জায়গা বদলাতে পারেন, কিন্তু দুটো একসঙ্গে গুলিয়ে ফেলেন না।

> **CONCEPT:** GitLab CI/CD-তে Pipeline হলো রেসিপির একবারের চলমান রূপ (a single run); `.gitlab-ci.yml` হলো রেসিপির বই। একই বই থেকে প্রতিদিন শত শত Pipeline চলতে পারে, প্রতিটির নিজস্ব নম্বর, লগ ও ফলাফল থাকে।

১২.৫ আর্কিটেকচার ও কার্যপ্রবাহ চিত্র



Coordinator কখনোই নিজে build চালায় না; সে শুধু সিদ্ধান্ত নেয় ও ফলাফল সংরক্ষণ করে। ভারী কাজ Runner-দের।

১২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

.gitlab-ci.yml-এর মৌলিক গঠন

ফাইলটি রিপোজিটরির root-এ থাকে এবং প্রতিটি commit-এ সংস্করণ নিয়ন্ত্রণের আওতায় আসে — অর্থাৎ pipeline-এর সংজ্ঞা নিজেই কোডের মতো review ও rollback করা যায়। শীর্ষ স্তরের প্রধান কীওয়ার্ড: `stages` (পর্যায়ের ক্রম), `default` (সব job-এর সাধারণ সেটিং), `variables`, `include` (অন্য ফাইল আনা), `workflow` (কখন pipeline চলবে তার নিয়ম) এবং প্রতিটি job-এর সংজ্ঞা। একটি job-এর ভেতরে `stage`, `script` (চালানোর কমান্ড), `image` (Docker image), `rules` বা `only/except`, `artifacts`, `cache`, `needs`, `environment` ও `allow_failure` থাকে।

Stage, Job ও Pipeline-এর সম্পর্ক

Stage হলো যৌক্তিক পর্যায় — যেমন `build`, `test`, `security`, `deploy`। একটি stage-এর সব job একসঙ্গে (parallel) চলে; একটি stage সফল হলেই পরের stage শুরু হয়। **Job** হলো একক কাজের একক — যেমন `unit-test`। `needs` ব্যবহার করলে একটি job আগের stage শেষ হওয়ার আগেই, শুধু নির্দিষ্ট job-এর ফলাফল পেলেই শুরু হতে পারে

— একে বলে DAG (Directed Acyclic Graph) pipeline। এটি সময় বাঁচায়, তবে নির্ভরতা ভুল হলে অসম্পূর্ণ artifact নিয়ে job চলতে পারে।

Pipeline-এর ধরন

Pipeline ধরন	কখন তৈরি হয়	ব্যবহার
Branch pipeline	যেকোনো branch-এ push হলে	প্রতিটি push-এ দ্রুত পরীক্ষা
Merge Request pipeline	Merge Request খোলা/আপডেট হলে	merge-এর আগে গুণমান যাচাই; detached pipeline-ও সম্ভব
Scheduled pipeline	নির্ধারিত সময়ে (cron)	রাতের regression, dependency পরীক্ষা
Manual / Triggered pipeline	UI, API বা অন্য pipeline থেকে	release, hotfix, পুনরায় চালানো
Parent-Child / Multi-project	এক pipeline অন্য pipeline ডাকে	মনোরেপো বা মাইক্রোসার্ভিসে আলাদা দলের pipeline

Rules বনাম Only/Except

`only/except` পুরোনো পদ্ধতি; `rules` আধুনিক ও শক্তিশালী। `rules`-এ শর্তগুলো ক্রমানুসারে মূল্যায়ন হয় — প্রথম মিলে যাওয়া নিয়মই জেতে। যেমন: `if: $CI_COMMIT_BRANCH == "main"` হলে deploy job চলবে, নইলে `when: never`। `workflow: rules` দিয়ে পুরো pipeline-ই নিয়ন্ত্রণ করা যায় — যেমন শুধু Merge Request ও `main` branch-এ pipeline চালানো, প্রতিটি ছোট push-এ নয়।

Include ও পুনঃব্যবহার

`include` দিয়ে একই প্রজেক্টের অন্য ফাইল, অন্য প্রজেক্টের ফাইল বা দূরবর্তী (remote) টেমপ্লেট আনা যায়। GitLab নিজেই অনেক Component ও Template দেয় — যেমন Security Scanning টেমপ্লেট। তবে remote include মানে বাইরের কোড আপনার pipeline-এ চলছে — বিশ্বাসযোগ্য উৎস ছাড়া ব্যবহার করবেন না (অধ্যায় ১৭ দেখুন)।

১২.৭ সম্পূর্ণ ব্যবহারিক Node.js পাইপলাইন

TechNova-র MERN অ্যাপ `app.technova.example`-এর জন্য সম্পূর্ণ `.gitlab-ci.yml`। প্রতিটি ক্ষেত্র নিচে ব্যাখ্যা করা হয়েছে।

```
# .gitlab-ci.yml — TechNova MERN অ্যাপ (Node.js 20 LTS)
# ফাইলটি রিপোজিটরির root-এ রাখুন।

default:
  image: node:20-alpine          # সব job ডিফল্টভাবে এই Docker image-এ চলবে
  cache:                          # node_modules ক্যাশ — প্রতিবার npm install এড়াতে
    key: ${CI_COMMIT_REF_SLUG}    # প্রতি branch-এ আলাদা ক্যাশ
    paths: [node_modules/]

stages:                           # পর্যায়ের ক্রম — বাম থেকে ডানে চলে
  - install
  - lint
  - test
```

```
  - security
  - build
  - deploy

workflow:                          # কখন pipeline চলবে
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
    - if: $CI_COMMIT_BRANCH == "main"
    - if: $CI_COMMIT_TAG

install-deps:
  stage: install
  script:
    - npm ci                       # package-lock.json অনুযায়ী নির্ভুল ইনস্টল
  artifacts:
    paths: [node_modules/]         # পরের job-গুলো এই ফোল্ডার পাবে
    expire_in: 1 hour

lint:
  stage: lint
  needs: [install-deps]
  script:
    - npm run lint
  allow_failure: false             # lint ব্যর্থ হলে pipeline থামবে

unit-test:
  stage: test
  needs: [install-deps]
  script:
    - npm test -- --coverage
  artifacts:
    reports:
      junit: junit.xml             # GitLab UI-তে পরীক্ষার রিপোর্ট দেখায়
      coverage_report:
        coverage_format: cobertura
        path: coverage/cobertura-coverage.xml
  coverage: '/All files[^|]*\|[^|]*\s+([\d\.]+)/'

security-scan:
  stage: security
  needs: [install-deps]
  script:
    - npm audit --audit-level=high
  allow_failure: true              # সতর্কতা দেখাবে, তবে pipeline আটকাবে না (শুরুতে)

build-image:
  stage: build
  image: docker:24                 # Docker-in-Docker ব্যবহারের জন্য
  services: [docker:24-dind]
  needs: [unit-test, lint]
  script:
    - docker build -t registry.technova.example/technova/web:$CI_COMMIT_SHORT_SHA .
    - docker push registry.technova.example/technova/web:$CI_COMMIT_SHORT_SHA
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy-staging:
  stage: deploy
  needs: [build-image]
```

```
  environment:
    name: staging
    url: https://staging.technova.example
  script:
    - echo "Deploying $CI_COMMIT_SHORT_SHA to staging"
  rules:
    - if: $CI_COMMIT_BRANCH == "main"
      when: manual               # মানুষ অনুমোদন দিলে তবেই চলবে
```

ক্ষেত্র-ব্যাখ্যা: `default.image` — সব job-এর ভিত্তি image; `cache.key` — branch অনুযায়ী ক্যাশ আলাদা রাখে, নইলে এক branch-এর ক্যাশ অন্য branch দূষিত করতে পারে; `npm ci` — `npm install`-এর চেয়ে নির্ভুল, কারণ lock ফাইলই চূড়ান্ত; `needs` — stage-এর অপেক্ষা না করে শুধু প্রয়োজনীয় job শেষ হলেই শুরু; `artifacts.reports.junit` — GitLab পরীক্ষার ফলাফল UI-তে সাজিয়ে দেখায়; `coverage` — regex দিয়ে কভারেজ শতাংশ বের করে; `rules.when: manual` — deploy মানুষের ক্লিকে আটকে থাকে; `$CI_COMMIT_SHORT_SHA` — প্রতিটি build-কে অনন্য tag দেয়, `latest`-এর মতো পরিবর্তনশীল tag নয়।

> **PRO TIP:** YAML লেখার পর GitLab-এর **CI Lint** টুলে (CI/CD → Pipelines → CI Lint বা `gitlab-ci-lint` ধারণা) যাচাই করুন। একটি ভুল indentation-এ job নীরবে ভিন্ন stage-এ চলে যেতে পারে — lint ছাড়া push করবেন না।

১২.৮ বাস্তব পরিস্থিতি — TechNova

> **REAL WORLD — TechNova:** কাল্পনিক কোম্পানি TechNova-র দলে আগে deploy মানে ছিল একজন সিনিয়র ডেভেলপারের ল্যাপটপ থেকে `scp` করে ফাইল তোলা। একদিন তিনি ছুটিতে থাকায় ভুল সংস্করণ production-এ গেল। GitLab CI চালু করার পর নিয়ম হলো: `main`-এ merge হলেই pipeline চলে — lint, test, security scan, Docker build — এবং staging deploy-এর আগে মানুষ অনুমোদন দেন। Merge Request-এই সবুজ টিক না দেখলে কেউ merge করতে পারেন না (protected branch নিয়ম)। ফলে deploy আর কোনো একজনের স্মৃতির ওপর নির্ভর করে না; এটি লিখিত, পুনরাবৃত্তিযোগ্য প্রক্রিয়া।

১২.৯ ধাপে ধাপে বাস্তবায়ন

1. GitLab-এ প্রজেক্ট তৈরি করে Node.js অ্যাপ push করুন; root-এ `.gitlab-ci.yml` যোগ করুন।
2. Settings → CI/CD → Runners-এ গিয়ে একটি Runner সক্রিয় আছে কি না নিশ্চিত করুন (অধ্যায় ১৩)।
3. CI Lint দিয়ে YAML যাচাই করুন, তারপর একটি ছোট commit push করুন।
4. CI/CD → Pipelines-এ গিয়ে Pipeline-এর প্রতিটি job-এর লগ পড়ুন; ব্যর্থ job-এ ক্লিক করে ঠিক কোন কমান্ড ব্যর্থ হয়েছে দেখুন।
5. Merge Request খুলে দেখুন — pipeline-এর অবস্থা MR পর্দায় দেখা যাচ্ছে কি না।
6. `main` branch-কে Protected করুন (Settings → Repository → Protected branches) যাতে pipeline ছাড়া সরাসরি push না হয়।
7. Pipeline-এর সময় মাপুন; ধীর job থাকলে অধ্যায় ১৩-এর cache ও `needs` প্রয়োগ করুন।

১২.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "No pipeline created" — YAML syntax ভুল বা `workflow: rules` কোনো শর্ত মেলেনি; CI Lint দিয়ে যাচাই করুন। **(২) Job "pending" হয়ে আটকে আছে** — কোনো Runner সক্রিয় নেই বা Runner-এর tag job-এর tag-এর সঙ্গে মিলছে না; Runner-এর অবস্থা দেখুন। **(৩) `npm ci` ব্যর্থ** — `package-lock.json` commit করা হয়নি বা Node সংস্করণ মিলছে না; lock ফাইল commit করুন। **(৪) Cache কাজ করছে না** — `cache.key` ভুল বা Runner-এর cache সংরক্ষণ বন্ধ; key ও Runner সেটিং পরীক্ষা করুন। **(৫) Artifact পরের job-এ পাওয়া যাচ্ছে না** — `needs` ব্যবহার করলে artifact স্বয়ংক্রিয়ভাবে আসে না; `needs: [{job: install-deps, artifacts: true}]` লিখুন।

১২.১১ নিরাপত্তা বিবেচনা

WARNING: `.gitlab-ci.yml` রিপোজিটরিতে থাকে — অর্থাৎ যেকেউ Merge Request-এ YAML বদলে pipeline-এ নিজের কমান্ড চালাতে পারে। Protected branch ও Merge Request approval ছাড়া `main`-এর pipeline-এ secret ব্যবহার করবেন না। Secret কখনো YAML-এ লিখবেন না; CI/CD Variables (masked ও protected) ব্যবহার করুন (অধ্যায় ১৪)। Fork থেকে আসা Merge Request-এ protected variable পাওয়া যায় না — এটিই নিরাপত্তা, ত্রুটি নয়।

১২.১২ কর্মক্ষমতা ও খরচ বিবেচনা

প্রতিটি job একটি Runner-এর CPU/RAM ও সময় খরচ করে। SaaS Runner-এ মিনিটের হিসাব থাকে; self-managed Runner-এ আপনার নিজের সার্ভারের খরচ। খরচ কমাতে: অপ্রয়োজনীয় branch-এ pipeline বন্ধ করুন (`workflow: rules`), `node_modules` ক্যাশ করুন, `needs` দিয়ে অযথা অপেক্ষা কমান, এবং ভারী job (যেমন Docker build) শুধু `main`-এ চালান। তবে ক্যাশ বেশি বড় হলে ডাউনলোডের সময়ই বেড়ে যায় — ১ GB-এর বেশি ক্যাশ সন্দেহজনক।

১২.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: ছোট ছোট job-এ ভাগ করুন; প্রতিটি job-এর একটিই দায়িত্ব রাখুন; commit SHA দিয়ে image tag করুন; pipeline-কে কোডের মতো review করুন। **অ্যান্টি-প্যাটার্ন:** একটিমাত্র বিশাল job-এ সবকিছু করা (ব্যর্থ হলে কিছুই বোঝা যায় না); `latest` tag-এ deploy করা (কোন সংস্করণ চলছে জানা যায় না); `allow_failure: true` সব job-এ দিয়ে দেওয়া (লাল pipeline-ও সবুজ দেখায়); YAML-এ secret লেখা।

১২.১৪ অধ্যায়ের সারাংশ

GitLab CI/CD-তে `.gitlab-ci.yml` হলো delivery-র ঘোষণা, Coordinator হলো সিদ্ধান্তকারী, Runner হলো সম্পাদনকারী। Stage পর্যায় সাজায়, Job একক কাজ করে, `needs` নির্ভরতা ছোট করে, `rules` কখন চলবে ঠিক করে, আর artifact ও cache কাজের ফলাফল ও গতি সামলায়। এই ভিত্তির ওপর দাঁড়িয়ে পরের অধ্যায়গুলোতে Runner, variable, environment ও deployment নিয়ন্ত্রণ শেখা হবে।

১২.১৫ মূল প্রযুক্তিগত পরিভাষা

Pipeline: একটি commit বা ঘটনার জন্য একবার চলা সম্পূর্ণ CI/CD প্রক্রিয়া।

Stage: Pipeline-এর যৌক্তিক পর্যায় (build, test, deploy); এক stage-এর সব job সমান্তরালে চলে।

Job: Pipeline-এর একক কাজের একক; একটি Runner-এ একটি script চালায়।

Runner: Job সম্পাদনকারী এজেন্ট; GitLab Server থেকে কাজ নিয়ে চালায়।

Coordinator: GitLab Server-এর অংশ যা pipeline তৈরি করে ও Runner-দের কাজ বণ্টন করে।

Artifact: Job-এর তৈরি ফলাফল (build output, রিপোর্ট) যা পরের job বা ডাউনলোডের জন্য সংরক্ষিত হয়।

Cache: Job-গুলোর মধ্যে পুনঃব্যবহারের জন্য সংরক্ষিত ফাইল (যেমন node_modules), গতি বাড়াতে ব্যবহৃত হয়।

Rules: কোন শর্তে job বা pipeline চলবে তা নির্ধারণকারী নিয়মের তালিকা।

Needs (DAG): Job-এর নির্ভরতা ঘোষণা; শুধু প্রয়োজনীয় job শেষ হলেই শুরু হয়।

Merge Request Pipeline: Merge Request-এর জন্য তৈরি pipeline; merge-এর আগের গুণমান যাচাই।

১২.১৬ জ্ঞান যাচাই প্রশ্ন

1. Pipeline, Stage ও Job-এর মধ্যে পার্থক্য কী? একটি উদাহরণ দিয়ে বোঝান।
2. Coordinator নিজে কেন build চালায় না? Runner আলাদা রাখার সুবিধা কী?
3. `rules` ও `only/except`-এর মধ্যে কোনটি আধুনিক এবং কেন?
4. `needs` ছাড়া ও `needs` সহ pipeline-এর চলার সময়ের পার্থক্য কী?
5. Artifact ও Cache-এর মধ্যে পার্থক্য কী? কখন কোনটি ব্যবহার করবেন?

১২.১৭ ব্যবহারিক অনুশীলন

1. **প্রথম pipeline:** একটি খালি Node.js প্রজেক্টে শুধু `install` ও `test` stage-সহ `.gitlab-ci.yml` লিখে push করুন; Pipeline পর্দায় প্রতিটি job-এর লগ পড়ে ব্যাখ্যা লিখুন।

2. **Rules পরীক্ষা:** `workflow: rules` দিয়ে শুধু Merge Request ও `main`-এ pipeline চালান; একটি feature branch-এ push করে pipeline তৈরি হয় কি না পরীক্ষা করুন।
3. **DAG রূপান্তর:** অধ্যায়ের উদাহরণ pipeline-এ `needs` সরিয়ে দিয়ে ও রেখে দিয়ে দুইবার চালান; মোট সময়ের পার্থক্য মেপে লিখুন।

অধ্যায় 13 • PART III — GitLab CI/CD and Jenkins

Chapter 13 — GitLab Runners, Jobs, Stages and Artifacts

১৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) GitLab Runner-এর স্থাপত্য ও নিবন্ধন (registration) প্রক্রিয়া ব্যাখ্যা করতে; (২) Shell ও Docker executor-এর পার্থক্য ও ঝুঁকি বলতে; (৩) Job-এর lifecycle — queued, running, passed, failed, canceled — ব্যাখ্যা করতে; (৪) Stage-এর সমান্তরালতা ও `needs`-ভিত্তিক DAG ডিজাইন করতে; (৫) Artifact ও Cache সঠিকভাবে কনফিগার করতে; (৬) Runner-এর লগ ও ত্রুটি পড়ে সমস্যার সমাধান করতে।

১৩.২ কেন এটি বাস্তব সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Pipeline-এর YAML যতই সুন্দর হোক, কাজ চলে Runner-এ। Runner ধীর হলে deploy দেরি হয়, Runner অনিরাপদ হলে secret চুরি যায়, Runner-এর executor ভুল হলে এক job অন্য job-এর ফাইল দূষিত করে। TechNova-র প্রথম GitLab সেটআপে একজন ডেভেলপারের ল্যাপটপেই Shell Runner চলত — ল্যাপটপ বন্ধ হলেই pipeline আটকে যেত, আর ল্যাপটপের ব্যক্তিগত ফাইলও job-এর নাগালে চলে আসত। Runner বোঝা মানে delivery-র আসল ইঞ্জিন বোঝা: কোথায় চলছে, কতটা বিচ্ছিন্ন (isolated), কতটা দ্রুত, এবং কীভাবে স্কেল করবেন।

১৩.৩ পূর্বশর্ত জ্ঞান

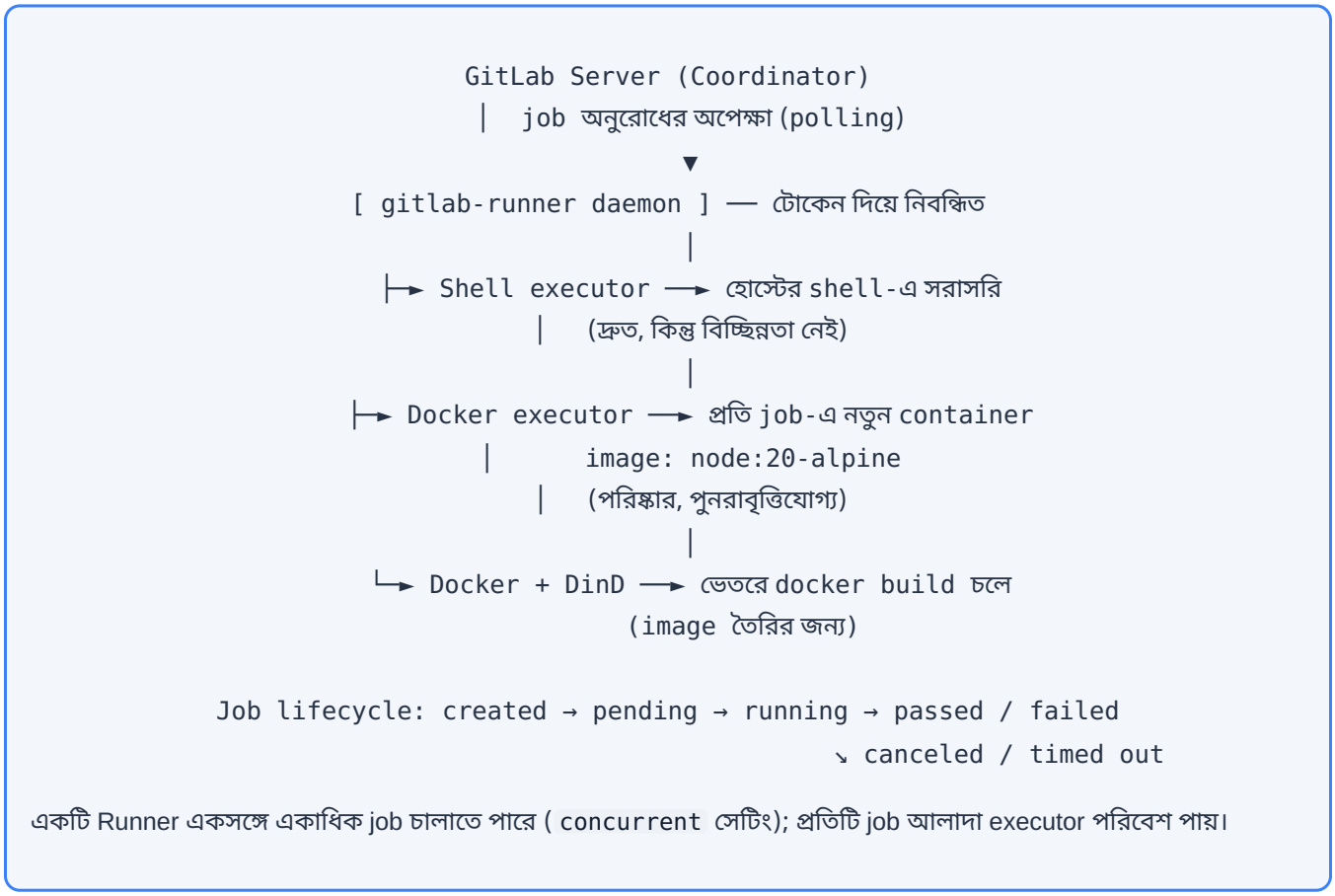
অধ্যায় ১২-এর `.gitlab-ci.yml`, stage/job ধারণা, Docker-এর image/container ধারণা এবং Linux-এ service চালানোর প্রাথমিক ধারণা। Docker executor বুঝতে `docker run`-এর অভিজ্ঞতা সহায়ক, তবে আবশ্যক নয়।

১৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

Runner একটি **কর্মী (worker)** — সে নিজে থেকে কাজ খোঁজে না, GitLab Server-কে জিজ্ঞেস করে: "আমার জন্য কোনো job আছে?" — একে বলে **polling**। Server job দিলে Runner সেটি একটি executor-এর ভেতরে চালায়, লগ ফেরত পাঠায়, আর artifact আপলোড করে। এই নকশার বড় সুবিধা: Runner আপনার নিজের নেটওয়ার্কের ভেতরে থাকলেও Server-কে তার কাছে পৌঁছাতে হয় না — Runner নিজেই বাইরে সংযোগ করে। অর্থাৎ firewall-এর পেছনে, নিজের সার্ভারে বা ক্লাউডে — যেকোনো জায়গায় Runner বসানো যায়। Executor হলো job চলার পরিবেশ: Shell executor হোস্টের shell-এ সরাসরি চালায়, Docker executor প্রতিটি job-কে নতুন container-এ চালায়।

> **CONCEPT:** Runner-কে পোষা প্রাণীর মতো নয়, গবাদি পশুর মতো ভাবুন (cattle, not pets): কোনো Runner নষ্ট হলে সারিয়ে তোলার চেষ্টা না করে নতুন Runner বসান। Runner-এ স্থায়ী কিছু রাখবেন না — সবকিছু YAML ও artifact-এ থাকবে।

১৩.৫ আর্কিটেকচার ও কার্যপ্রবাহ চিত্র



একটি Runner একসঙ্গে একাধিক job চালাতে পারে (`concurrent` সেটিং); প্রতিটি job আলাদা executor পরিবেশ পায়।

১৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Runner-এর ধরন ও নিবন্ধন

Runner তিন স্তরে থাকতে পারে: **Shared Runner** — পুরো GitLab instance-এর জন্য (SaaS-এ GitLab দেয়); **Group Runner** — একটি group-এর সব প্রজেক্টের জন্য; **Project Runner** — একটি প্রজেক্টের জন্য। নিবন্ধনের সময় Runner একটি টোকেন পায় ও `config.toml`-এ সংরক্ষিত হয়। আধুনিক GitLab-এ Runner নিবন্ধনে `gitlab-runner register` কমান্ড বা UI-তে তৈরি টোকেন ব্যবহৃত হয়। **Tag** দিয়ে Runner চিহ্নিত করা যায় — যেমন `docker, linux` — আর job-এ `tags: [docker]` লিখলে শুধু সেই Runner-ই job নেবে। Tag না মিললে job চিরকাল pending থাকে — এটি সবচেয়ে সাধারণ আটকে যাওয়ার কারণ।

Shell বনাম Docker Executor

বিষয়	Shell Executor	Docker Executor
চলার জায়গা	হোস্ট মেশিনের shell-এ সরাসরি	প্রতিটি job-এ নতুন container
বিচ্ছিন্নতা	নেই — job হোস্টের সব ফাইল দেখতে পারে	ভালো — container মুছে যায়
গতি	দ্রুত (image টানতে হয় না)	প্রথমবার image টানতে সময় লাগে
পুনরাবৃত্তিযোগ্যতা	হোস্টের অবস্থার ওপর নির্ভরশীল	উচ্চ — একই image, একই ফল

কখন ব্যবহার	বিশ্বস্ত অভ্যন্তরীণ কাজ, Windows/macOS build	সাধারণ CI কাজের জন্য ডিফল্ট পছন্দ

Shell executor-এ একটি job হোস্টের `~/.ssh` বা পরিবেশ ফাইল পড়তে পারে — তাই অবিশ্বস্ত কোড (যেমন বাইরের Merge Request) Shell Runner-এ চালাবেন না। Docker executor-এও `privileged` মোড বা Docker socket মাউন্ট করলে container হোস্টের Docker নিয়ন্ত্রণ করতে পারে — Docker-in-Docker (DinD) ব্যবহারে সতর্ক থাকুন।

Job-এর Lifecycle ও Retry

Job তৈরি হয় (`created`), Runner না পেলে অপেক্ষা করে (`pending`), চলতে থাকে (`running`), শেষে `passed` বা `failed` হয়। `timeout` (ডিফল্ট সাধারণত ১ ঘণ্টা) পেরোলে job জোর করে থামে। `retry` দিয়ে নির্দিষ্ট ব্যর্থতায় (যেমন নেটওয়ার্ক ত্রুটি) স্বয়ংক্রিয়ভাবে আবার চালানো যায় — তবে পরীক্ষার ব্যর্থতায় retry দিলে আসল ত্রুটি লুকিয়ে যায়, তাই `retry: max: 2, when: runner_system_failure`-এর মতো সীমিত শর্ত ব্যবহার করুন।

Stage ডিজাইন ও DAG

ডিফল্টভাবে stage ক্রমানুসারে চলে; `needs` দিয়ে DAG বানালে lint ও test একসঙ্গে শুরু হতে পারে। তবে DAG-এ চক্র (cycle) তৈরি করা যাবে না — A, B-এর ওপর নির্ভরশীল আর B, A-এর ওপর নির্ভরশীল হলে pipeline অবৈধ। বড় pipeline-এ `needs: [job]`-এর পরিবর্তে `needs: {job: ..., artifacts: true}` দিয়ে স্পষ্ট করুন artifact দরকার কি না।

Artifact বনাম Cache — গভীরে

Artifact হলো job-এর ফলাফল যা সংরক্ষণ করতে হয়: build করা `dist/`, পরীক্ষার রিপোর্ট, Docker image-এর বাইরের ফাইল। Artifact-এর মেয়াদ থাকে (`expire_in`); মেয়াদ শেষ হলে GitLab তা মুছে দেয়। **Cache** হলো গতি বাড়ানোর উপায়: `node_modules` পরের pipeline-এ আবার ব্যবহার করা। পার্থক্য মনে রাখুন: artifact না থাকলে deploy ব্যর্থ হয়, cache না থাকলে শুধু ধীর হয়। `cache:policy: pull` দিয়ে শুধু পড়া যায় (লেখা নয়) — deploy job-এ এটি নিরাপদ।

১৩.৭ কনফিগারেশন উদাহরণ — Runner ও Artifact

```
# /etc/gitlab-runner/config.toml — Runner-এর কনফিগারেশন (সার্ভারে)
concurrent = 4              # একসঙ্গে সর্বোচ্চ ৪টি job
check_interval = 3          # প্রতি ৩ সেকেন্ডে নতুন job আছে কি না দেখে

[[runners]]
  name = "technova-docker-runner"
  url = "https://gitlab.technova.example/"
  token = "RUNNER-TOKEN-PLACEHOLDER"   # আসল টোকেন UI থেকে নিন; ফাইলে রাখা টোকেন শেয়ার করবেন না
  executor = "docker"
  [runners.docker]
    image = "node:20-alpine"           # job-এ image না দিলে এই image ব্যবহৃত হয়
    privileged = false                 # DinD ছাড়া false রাখুন
    volumes = ["/cache"]               # ক্যাশ সংরক্ষণের জায়গা
```

```
# .gitlab-ci.yml — artifact ও cache-এর নমুনা
build-app:
  stage: build
  script:
    - npm ci
    - npm run build
  artifacts:
```

```
    paths: [dist/]                      # deploy job dist/ পাবে
    expire_in: 1 week                   # এক সপ্তাহ পর মুছে যাবে
    name: "web-$CI_COMMIT_SHORT_SHA"    # ডাউনলোড করা ফাইলের নাম

e2e-test:
  stage: test
  needs:
    - job: build-app
      artifacts: true                   # dist/ স্পষ্টভাবে নেওয়া হচ্ছে
  cache:
    key: ${CI_COMMIT_REF_SLUG}
    paths: [node_modules/]
    policy: pull                        # শুধু পড়বে, ক্যাশ দূষিত করবে না
  script:
    - npm run test:e2e
```

নমুনা আউটপুট — চিত্রণমূলক: `Job succeeded — Duration: 1m 12s — Artifacts: dist/ (2.4 MB), junit.xml`। আসল আউটপুটে সময় ও আকার ভিন্ন হবে।

১৩.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD — TechNova: TechNova-র pipeline হঠাৎ ধীর হয়ে গেল — প্রতিটি job ৬ মিনিট করে `npm install`-এ কাটাচ্ছিল। কারণ: cache কনফিগার করা হয়নি। `node_modules` ক্যাশ করার পর গড় pipeline সময় ৯ মিনিট থেকে ৩ মিনিটে নামল। পরে দেখা গেল Shell Runner-এ একটি job আগের job-এর ফাইল দেখে ভুলভাবে পাস করছে — কারণ হোস্টের workspace পরিষ্কার হচ্ছিল না। Docker executor-এ যাওয়ার পর প্রতিটি job পরিষ্কার পরিবেশে চলতে লাগল, ফলাফল নির্ভরযোগ্য হলো। শিক্ষা: গতি ও নির্ভরযোগ্যতা — দুটোই Runner ও executor-এর সিদ্ধান্ত।

১৩.৯ ধাপে ধাপে বাস্তবায়ন

1. একটি Linux VM-এ `gitlab-runner` ইনস্টল করুন (অফিসিয়াল GitLab Runner ডকুমেন্টেশন অনুযায়ী)।
2. GitLab UI থেকে Runner টোকেন নিয়ে `gitlab-runner register` চালান; executor হিসেবে `docker` বেছে নিন।
3. `gitlab-runner verify` দিয়ে সংযোগ পরীক্ষা করুন; Settings → CI/CD → Runners-এ Runner "online" দেখাচ্ছে কি না দেখুন।
4. একটি পরীক্ষামূলক job-এ `tags` দিয়ে শুধু এই Runner-এ চালান।
5. Artifact-সহ build job লিখুন; পরের job-এ artifact এসেছে কি না UI থেকে ডাউনলোড করে যাচাই করুন।
6. Cache যোগ করে pipeline-এর সময় আগে-পরে মাপুন ও লিখে রাখুন।

১৩.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) Job "pending" / "stuck" — Runner offline, tag মিলছে না, বা `concurrent` সীমা পূর্ণ; Runner-এর অবস্থা ও tag পরীক্ষা করুন। **(২) "Cannot connect to the Docker daemon"** — Docker executor-এ DinD সার্ভিস যোগ করা হয়নি বা `privileged` বন্ধ; DinD সার্ভিস ও Runner সেটিং দেখুন। **(৩) "No such file: dist/"** — আগের job-এর artifact মেয়াদ শেষ বা `needs`-এ `artifacts: true` নেই। **(৪) Cache প্রতিবার নতুন তৈরি হচ্ছে** — `cache.key` প্রতি commit-এ বদলে যাচ্ছে (যেমন `$CI_COMMIT_SHA` ব্যবহার); branch-ভিত্তিক key ব্যবহার করুন। **(৫) Runner-এ ডিস্ক ভরে গেছে** — পুরোনো Docker image/container জমেছে; `docker system prune` নিয়মিত চালান (ল্যাবে, প্রোডাকশন Runner-এ সাবধানে)।

১৩.১১ নিরাপত্তা বিবেচনা

WARNING: Runner যে মেশিনে চলে, job সেই মেশিনের অনেক কিছুই দেখতে পারে — বিশেষ করে Shell executor ও privileged Docker executor-এ। অবিশ্বস্ত কোড (fork-এর Merge Request) কখনোই production Runner বা secret-সহ Runner-এ চালাবেন না। Runner টোকেন চুরি হলে আক্রমণকারী আপনার Runner হিসেবে job নিতে পারে — টোকেন ঘুরিয়ে (rotate) দিন ও Runner-এর অ্যাক্সেস সীমিত রাখুন। Docker socket (`/var/run/docker.sock`) job-এর ভেতরে মাউন্ট করবেন না — এটি container-কে হোস্টের সম্পূর্ণ Docker নিয়ন্ত্রণ দেয়।

১৩.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Runner-এর সংখ্যা ও আকার সরাসরি খরচ। `concurrent` বেশি হলে একসঙ্গে অনেক job চলে, কিন্তু CPU/RAM ভাগ হয়ে প্রতিটি job ধীর হয়। ছোট দলের জন্য ২–৪ concurrent যথেষ্ট; বড় দলে Runner autoscale (যেমন Docker Machine বা Kubernetes executor) বিবেচনা করুন। Artifact বেশিদিন রাখলে স্টোরেজ খরচ বাড়ে — `expire_in` ছোট রাখুন (build artifact ১ সপ্তাহ, রিপোর্ট ১ দিন যথেষ্ট)। Cache-ও স্টোরেজ খায়; বড় `node_modules` ক্যাশের ডাউনলোড সময় মাপুন — কখনো ক্যাশ না থাকাই দ্রুত হয়।

১৩.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Docker executor ডিফল্ট করুন; প্রতিটি job-কে স্বাধীন রাখুন (আগের job-এর অবশিষ্ট ফাইলের ওপর নির্ভর করবেন না); artifact-এ শুধু প্রয়োজনীয় ফাইল রাখুন; Runner-এর সংস্করণ নিয়মিত আপডেট করুন। **অ্যান্টি-প্যাটার্ন:** ব্যক্তিগত ল্যাপটপে production Runner চালানো; tag ছাড়া সব Runner-এ সব job চালানো; `cache`-কে artifact-এর বিকল্প ভাবা; Runner-এ হাতে ফাইল রেখে pipeline "ঠিক" করা।

১৩.১৪ অধ্যায়ের সারাংশ

Runner হলো pipeline-এর ইঞ্জিন: Coordinator সিদ্ধান্ত নেয়, Runner কাজ করে। Docker executor বিচ্ছিন্নতা ও পুনরাবৃত্তিযোগ্যতা দেয়, Shell executor দ্রুত কিন্তু ঝুঁকিপূর্ণ। Stage সমান্তরালতা দেয়, `needs` নির্ভরতা ছোট করে, artifact ফলাফল বহন করে, cache গতি বাড়ায় — কিন্তু artifact ও cache এক জিনিস নয়। Runner-কে গবাদি পশুর মতো ভাবুন: বদলানো যায় এমন, স্থায়ী কিছু রাখবেন না।

১৩.১৫ মূল প্রযুক্তিগত পরিভাষা

Executor: Runner-এর ভেতরে job চালানোর পরিবেশ (Shell, Docker, Kubernetes)।

Shared Runner: পুরো instance-এর সব প্রজেক্টের জন্য উপলব্ধ Runner।

Group / Project Runner: নির্দিষ্ট group বা প্রজেক্টের জন্য নিবন্ধিত Runner।

Tag: Runner ও job মেলানোর লেবেল; tag না মিললে job pending থাকে।

Docker-in-Docker (DinD): Docker container-এর ভেতরে Docker daemon চালিয়ে image build করার পদ্ধতি।

Concurrent: একটি Runner একসঙ্গে কতটি job চালাতে পারবে তার সীমা।

Job Lifecycle: created → pending → running → passed/failed/canceled — job-এর অবস্থার ক্রম।

Expire_in: Artifact কতদিন সংরক্ষিত থাকবে তার মেয়াদ।

Cache Policy: cache শুধু পড়া হবে (pull) নাকি লেখাও হবে (pull-push) তার নিয়ম।

Polling: Runner-এর নিয়মিত Server-কে জিজ্ঞেস করা — "আমার জন্য job আছে কি?"

১৩.১৬ জ্ঞান যাচাই প্রশ্ন

1. Shell ও Docker executor-এর মধ্যে নিরাপত্তার পার্থক্য কী? কোনটি কখন বেছে নেবেন?
2. একটি job "pending" হয়ে আটকে থাকলে প্রথমে কী কী পরীক্ষা করবেন?
3. Artifact ও Cache-এর মধ্যে তিনটি পার্থক্য লিখুন।
4. `needs`-এ `artifacts: true` না লিখলে কী সমস্যা হয়?
5. Runner টোকেন চুরি হলে আক্রমণকারী কী করতে পারে? প্রতিকার কী?

১৩.১৭ ব্যবহারিক অনুশীলন

1. **নিজের Runner:** ল্যাব VM-এ Docker executor-সহ একটি Runner নিবন্ধন করুন; একটি test job শুধু সেই Runner-এ চালান (tag ব্যবহার করে)।

2. **Artifact যাচাই:** build job-এ `dist/` artifact তৈরি করে deploy job-এ তা ব্যবহার করুন; UI থেকে artifact ডাউনলোড করে বিষয়বস্তু মিলিয়ে দেখুন।
3. **Cache পরিমাপ:** cache ছাড়া ও cache-সহ একই pipeline দুইবার চালিয়ে সময়ের পার্থক্য লিখুন; ক্যাশের আকারও নোট করুন।

অধ্যায় 14 • PART III — GitLab CI/CD and Jenkins

Chapter 14 — GitLab Variables, Environments and Deployment Controls

১৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) CI/CD Variable-এর ধরন — project, group, instance — ও তার অগ্রাধিকার (precedence) ব্যাখ্যা করতে; (২) Masked, Protected ও Hidden variable-এর পার্থক্য বলতে; (৩) Environment (development, staging, production) সংজ্ঞায়িত করতে ও deploy করতে; (৪) Manual approval, deployment freeze ও protected environment দিয়ে production নিয়ন্ত্রণ করতে; (৫) `rules` ও `environment`-এর সমন্বয়ে নিরাপদ promotion ডিজাইন করতে; (৬) Variable সংক্রান্ত ত্রুটি নির্ণয় করতে।

১৪.২ কেন এটি বাস্তব সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

একই কোড development, staging ও production — তিন জায়গায় চলে, কিন্তু ডেটাবেসের ঠিকানা, API key ও ডোমেইন তিন জায়গায় তিন রকম। এই পার্থক্য যদি কোডে লেখা থাকে, তবে ভুল করে production secret development branch-এ ফাঁস হয়, বা staging-এর ডেটাবেস production-এ মুছে যায়। Variable ও Environment-এর কাজ হলো এই পার্থক্যকে কোড থেকে আলাদা করে নিরাপদে পরিচালনা করা। TechNova-তে একবার staging-এর ডেটাবেস URL ভুল করে production job-এ ব্যবহৃত হয়েছিল — Environment আলাদা না থাকায় কেউ বুঝতেই পারেনি কোন পরিবেশে deploy হচ্ছে। Deployment control ছাড়া pipeline শুধু দ্রুত ভুল করার যন্ত্র।

১৪.৩ পূর্বশর্ত জ্ঞান

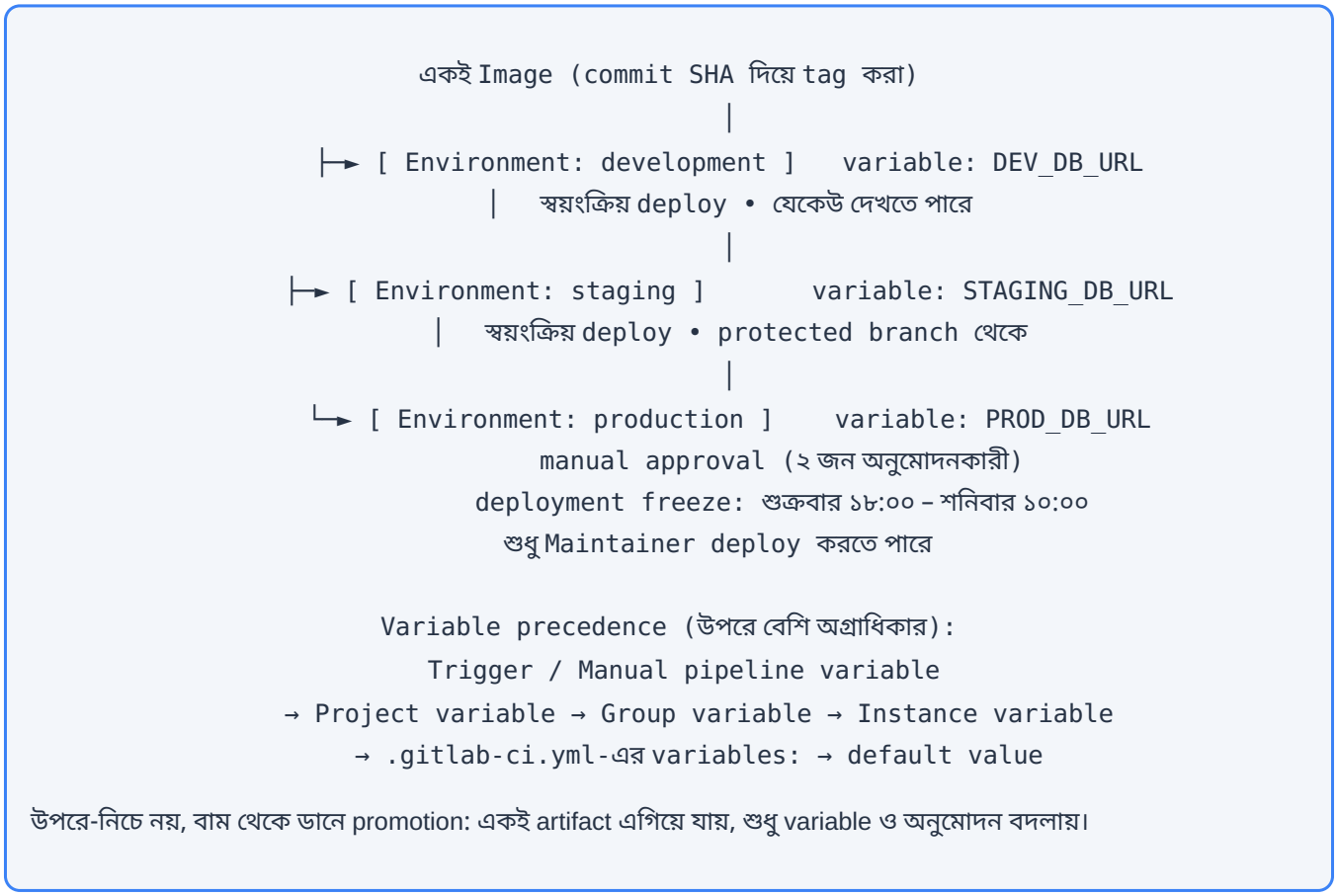
অধ্যায় ১২–১৩-এর pipeline, job ও artifact ধারণা; পরিবেশ (environment) ধারণা — development/staging/production-এর পার্থক্য (অধ্যায় ৩); এবং secret ব্যবস্থাপনার মৌলিক ধারণা (secret কখনো কোডে লেখা যায় না)।

১৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

মূল নীতিটি হলো **পৃথকীকরণ (separation)**: কোড এক, কনফিগারেশন আলাদা। একই Docker image তিনটি পরিবেশে চলে; শুধু পরিবেশ-ভেদে variable বদলায়। দ্বিতীয় নীতি **ন্যূনতম সুযোগ (least privilege)**: staging-এর job-এর কাছে production-এর secret থাকবে না; development branch-এর pipeline production variable দেখতেই পাবে না। তৃতীয় নীতি **দরজায় পাহারা (gate)**: production-এ যাওয়ার আগে একজন মানুষ অনুমোদন দেবেন, এবং নির্দিষ্ট সময়ে (যেমন ব্যবসায়িক ব্যস্ত সময়ে) deploy বন্ধ থাকবে। GitLab-এ এই তিন নীতির যন্ত্র হলো Variable, Environment ও Deployment Control।

> **CONCEPT:** Environment শুধু একটি নাম নয় — এটি একটি রেকর্ড: কোন commit, কোন সময়ে, কে deploy করল, কোন URL-এ চলছে — সব GitLab সংরক্ষণ করে। Deploy করা মানে ইতিহাসে একটি স্বাক্ষর রাখা, যাতে rollback-এর সময় জানা যায় কোথায় ফিরতে হবে।

১৪.৫ আর্কিটেকচার ও কার্যপ্রবাহ চিত্র



উপরে-নিচে নয়, বাম থেকে ডানে promotion: একই artifact এগিয়ে যায়, শুধু variable ও অনুমোদন বদলায়।

১৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Variable-এর ধরন ও অগ্রাধিকার

Variable তিন স্তরে সংজ্ঞায়িত হয়: **Instance** (সব প্রজেক্টে), **Group** (একটি group-এর প্রজেক্টে), **Project** (একটি প্রজেক্টে)। একই নামের variable একাধিক স্তরে থাকলে নিচের স্তর (project) জেতে — একে বলে precedence। YAML-এর `variables:` ব্লকেও মান দেওয়া যায়, তবে সেখানে secret লেখা যাবে না — YAML রিপোজিটরিতে থাকে। এছাড়া **File type variable** আছে — মানটি সরাসরি নয়, একটি অস্থায়ী ফাইলের পথ হিসেবে দেওয়া হয় (যেমন TLS certificate বা kubeconfig); job-এ `$MY_CERT_FILE` দিয়ে ফাইলটি পড়া যায়।

Masked, Protected ও Hidden

বৈশিষ্ট্য	অর্থ	কখন ব্যবহার
Masked	লগে মানটি `***` দিয়ে ঢাকা থাকে	সব secret-এ
Protected	শুধু protected branch/tag-এর pipeline-এ পাওয়া যায়	production secret-এ — feature branch দেখতে পাবে না
Hidden	UI-তে মান দেখা যায় না (শুধু সেট করা যায়)	অত্যন্ত সংবেদনশীল টোকেনে
	মানের ভেতরে `$VAR` প্রসারিত হবে কি না	অন্য variable-এর ওপর নির্ভরশীল মান এড়াতে raw ব্যবহার করুন

Expanded / Raw		

সতর্কতা: Masked মানে নিরাপদ নয় — একটি job `echo $SECRET | base64` করলে masked মানও ফাঁস হতে পারে (encoding-এর পর মানটি আর চেনা যায় না)। তাই secret শুধু বিশ্বস্ত job-এই ব্যবহার করুন, এবং job-এর লগে secret print করা নিষিদ্ধ করুন।

Environment সংজ্ঞা ও Deployment

`environment: {name: staging, url: https://staging.technova.example}` লিখলে GitLab ওই নামে Environment তৈরি করে ও deploy-এর ইতিহাস রাখে। `environment:action: stop` দিয়ে review app বন্ধ করার job লেখা যায়। **Dynamic environment** — যেমন প্রতিটি Merge Request-এর জন্য `review/$CI_COMMIT_REF_SLUG` — রিভিউয়ারকে সরাসরি লাইভ প্রিভিউ দেয়; Merge Request বন্ধ হলে `on_stop` job পরিবেশটি মুছে দেয়। `auto_stop_in: 1 day` দিয়ে মেয়াদও বেঁধে দেওয়া যায়।

Deployment Controls

Protected environment: শুধু নির্দিষ্ট ভূমিকা (যেমন Maintainer) বা নির্দিষ্ট ব্যবহারকারী production-এ deploy করতে পারে। **Manual approval:** `when: manual` job-এ ক্লিক না করলে deploy থেমে থাকে; `allowed_to_fail: false` রাখলে অনুমোদন ছাড়া pipeline এগোয় না। **Deployment freeze:** নির্ধারিত সময়ে (যেমন উৎসবের সময় বা রাতে) deploy সম্পূর্ণ বন্ধ থাকে — জরুরি hotfix-এর জন্য freeze ভাঙার আলাদা অনুমতি লাগে। **Deployment approval (GitLab Premium বৈশিষ্ট্য):** একাধিক অনুমোদনকারীর প্রয়োজন — একজন deploy করলে অন্যজন অনুমোদন দেন।

Rules দিয়ে Promotion ডিজাইন

Promotion মানে একই artifact-কে পরের পরিবেশে পাঠানো, নতুন করে build করা নয়। নিয়ম: `main`-এ merge হলে staging-এ স্বয়ংক্রিয় deploy; production-এ শুধু tag (`v1.2.0`) তৈরি হলে manual deploy। এতে প্রতিটি production release-এর একটি নাম (version) থাকে ও rollback-এর লক্ষ্য স্পষ্ট হয়।

১৪.৭ কনফিগারেশন উদাহরণ

```
# .gitlab-ci.yml — environment ও deployment control
stages: [build, deploy]

variables:
  APP_NAME: "technova-web"        # সাধারণ মান — secret নয়
  # DATABASE_URL এখানে নয় — Settings → CI/CD → Variables-এ রাখুন

build-image:
  stage: build
  script:
    - docker build -t registry.technova.example/technova/web:$CI_COMMIT_SHA .
    - docker push registry.technova.example/technova/web:$CI_COMMIT_SHA

deploy-staging:
  stage: deploy
  environment:
    name: staging
    url: https://staging.technova.example
  script:
```

```
    - echo "Deploying $CI_COMMIT_SHA to staging"
    - ./scripts/deploy.sh staging $CI_COMMIT_SHA
  rules:
    - if: $CI_COMMIT_BRANCH == "main"

deploy-review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    url: https://$CI_ENVIRONMENT_SLUG.review.technova.example
    on_stop: stop-review
    auto_stop_in: 1 day
  script:
    - ./scripts/deploy.sh review $CI_COMMIT_SHA
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"

stop-review:
  stage: deploy
  environment:
    name: review/$CI_COMMIT_REF_SLUG
    action: stop
  script: [./scripts/teardown-review.sh]
  rules:
    - if: $CI_PIPELINE_SOURCE == "merge_request_event"
      when: manual

deploy-production:
  stage: deploy
  environment:
    name: production
    url: https://app.technova.example
  script:
    - ./scripts/deploy.sh production $CI_COMMIT_TAG
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
      when: manual                # tag + মানুষের অনুমোদন — দুটোই লাগবে
  # Protected environment: Settings → CI/CD → Protected environments-এ
  # শুধু Maintainer-দের deploy অনুমতি দিন।
```

ক্ষেত্র-ব্যাখ্যা: `$CI_ENVIRONMENT_SLUG` — environment নামের URL-নিরাপদ রূপ; `on_stop` — পরিবেশ বন্ধ করার job-এর নাম; `auto_stop_in` — review পরিবেশ নিজে থেকে মুছে যাওয়ার সময়; `$CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/` — শুধু semantic version tag-এ production deploy; `when: manual` — tag থাকলেও মানুষের ক্লিক ছাড়া চলবে না।

১৪.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD — TechNova: TechNova-র staging ও production-এর ডেটাবেস URL একই variable নামে (`DATABASE_URL`) রাখা হয়েছিল, শুধু মান আলাদা — কিন্তু variable-টি protected ছিল না। একটি feature branch-এর pipeline ভুল করে production URL পড়ে staging ডেটা production-এ লিখতে যাচ্ছিল। ধরা পড়ার পর নিয়ম বদলানো হলো: production variable protected + masked, শুধু `main` ও tag-এর pipeline-এ দৃশ্যমান; staging variable আলাদা নামে (`STAGING_DATABASE_URL`)। আর production deploy-এ দুইজনের অনুমোদন বাধ্যতামূলক করা হলো — deploy করা ব্যক্তি নিজে অনুমোদন দিতে পারেন না।

১৪.৯ ধাপে ধাপে বাস্তবায়ন

1. Settings → CI/CD → Variables-এ `STAGING_API_KEY` (masked) ও `PROD_API_KEY` (masked + protected) তৈরি করুন — মান হিসেবে কাল্পনিক placeholder ব্যবহার করুন।
2. YAML-এ staging ও production environment যোগ করুন; production job-এ `when: manual` দিন।
3. Settings → CI/CD → Protected environments-এ production-এ শুধু Maintainer-দের অনুমতি দিন।
4. একটি feature branch-এ pipeline চালিয়ে যাচাই করুন — production variable সেখানে পাওয়া যাচ্ছে কি না (পাওয়া যাওয়া উচিত নয়)।
5. Review app-এর জন্য dynamic environment যোগ করুন; Merge Request বন্ধ করে `on_stop` কাজ করে কি না দেখুন।
6. Deployment freeze-এর সময় নির্ধারণ করুন ও দলকে জানান।

১৪.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) Variable খালি দেখাচ্ছে — protected variable unprotected branch-এর pipeline-এ পাওয়া যায় না; branch protected কি না দেখুন। **(২) ভুল পরিবেশের মান ব্যবহৃত হচ্ছে** — একই নামের variable একাধিক স্তরে আছে; precedence তালিকা দেখে কোনটি জিতছে বের করুন। **(৩) লগে secret দেখা যাচ্ছে** — masked থাকলেও `echo` বা error message-এ ফাঁস হতে পারে; secret print করা কোড সরান ও টোকেন rotate করুন। **(৪) Manual job "skipped" হচ্ছে** — আগের stage ব্যর্থ হয়েছে বা `rules` মেলেনি; job-এর "Why skipped" তথ্য দেখুন। **(৫) Review পরিবেশ মুছছে না** — `on_stop` job-এর নাম ভুল বা `action: stop` নেই।

১৪.১১ নিরাপত্তা বিবেচনা

WARNING: Variable-ই সবচেয়ে বেশি চুরি যাওয়া সম্পদ। নিয়ম: (১) secret কখনো YAML, কোড বা লগে নয় — শুধু Variables UI-তে; (২) production secret সবসময় masked + protected; (৩) টোকেনের মেয়াদ বেঁধে দিন ও নিয়মিত rotate করুন; (৪) কে variable দেখতে ও বদলাতে পারে তা নিয়মিত নিরীক্ষা করুন; (৫) কোনো সন্দেহ হলেই টোকেন বাতিল করে নতুন টোকেন দিন — "হয়তো ফাঁস হয়নি" ধরে নেবেন না। Fork-এর Merge Request-এ protected variable না দেওয়া GitLab-এর নিরাপত্তা বৈশিষ্ট্য — এটি বন্ধ করবেন না।

১৪.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Review app প্রতিটি Merge Request-এ একটি করে পরিবেশ তৈরি করে — বন্ধ না করলে সার্ভার ও খরচ বাড়তেই থাকে। `auto_stop_in` দিয়ে মেয়াদ বেঁধে দিন ও `on_stop` নিশ্চিত করুন। Production deploy-এর manual approval সময় নেয় — এটি খরচ নয়, বীমা। তবে অনুমোদনকারী ছুটিতে থাকলে deploy আটকে যায় — তাই প্রতিটি পরিবেশে অন্তত দুইজন অনুমোদনকারী রাখুন। Deployment freeze-এর সময়সূচি ব্যবসায়িক ক্যালেন্ডারের সঙ্গে মিলিয়ে নির্ধারণ করুন।

১৪.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: একই artifact সব পরিবেশে পাঠান; variable-এর নাম পরিবেশ অনুযায়ী স্পষ্ট রাখুন; production-এ দুইজনের অনুমোদন; প্রতিটি deploy-এর ইতিহাস সংরক্ষণ করুন। **অ্যান্টি-প্যাটার্ন:** প্রতিটি পরিবেশের জন্য আলাদা করে build করা (একই কোডের ভিন্ন ভিন্ন build — কোনটি production-এ গেল জানা যায় না); secret-কে সাধারণ variable হিসেবে রাখা; একজন ব্যক্তির ওপর production deploy নির্ভর করা; freeze-এর সময়ও "শুধু এবার" deploy করা।

১৪.১৪ অধ্যায়ের সারাংশ

Variable কোড থেকে কনফিগারেশন আলাদা করে; masked ও protected সেই কনফিগারেশনকে সুরক্ষিত করে; Environment deploy-এর ইতিহাস রাখে; protected environment, manual approval ও deployment freeze production-এর দরজায় পাহারা দেয়। Promotion মানে একই artifact এগিয়ে নেওয়া — নতুন করে build করা নয়। এই নিয়ন্ত্রণ ছাড়া pipeline দ্রুত, কিন্তু অনিরাপদ।

১৪.১৫ মূল প্রযুক্তিগত পরিভাষা

CI/CD Variable: Pipeline-এ ব্যবহৃত কনফিগারেশন মান; কোডের বাইরে সংরক্ষিত হয়।

Masked Variable: লগে মান ঢেকে রাখা variable।

Protected Variable: শুধু protected branch/tag-এর pipeline-এ দৃশ্যমান variable।

Precedence: একই নামের variable একাধিক স্তরে থাকলে কোনটি জিতবে তার ক্রম।

Environment: deploy-এর লক্ষ্য পরিবেশ (development/staging/production) ও তার ইতিহাস।

Review App: প্রতিটি Merge Request-এর জন্য স্বয়ংক্রিয়ভাবে তৈরি অস্থায়ী পরিবেশ।

Protected Environment: শুধু নির্দিষ্ট ভূমিকার ব্যবহারকারীরা deploy করতে পারে এমন পরিবেশ।

Deployment Freeze: নির্ধারিত সময়ে deploy বন্ধ রাখার নিয়ম।

Promotion: একই artifact-কে এক পরিবেশ থেকে পরের পরিবেশে পাঠানো।

File Variable: মানের পরিবর্তে ফাইলের পথ হিসেবে দেওয়া variable (যেমন certificate)।

১৪.১৬ জ্ঞান যাচাই প্রশ্ন

1. Project, group ও instance variable-এর precedence ক্রমটি লিখুন ও একটি উদাহরণ দিন।
2. Masked variable-ও কীভাবে ফাঁস হতে পারে? একটি উদাহরণ দিন।
3. Protected variable feature branch-এর pipeline-এ পাওয়া যায় না কেন? এটি কি ত্রুটি?
4. Review app কী? `on_stop` ও `auto_stop_in` কেন দরকার?
5. একই artifact সব পরিবেশে পাঠানো কেন আলাদা করে build করার চেয়ে ভালো?

১৪.১৭ ব্যবহারিক অনুশীলন

1. **Variable পরীক্ষা:** একটি masked + protected variable তৈরি করে feature branch ও `main` branch-এর pipeline-এ তার দৃশ্যমানতা পরীক্ষা করুন; ফলাফল লিখুন।
2. **Review app:** একটি Merge Request-এর জন্য dynamic review environment তৈরি করুন; MR বন্ধ করে পরিবেশ মুছে যায় কি না যাচাই করুন।
3. **Promotion ডিজাইন:** staging-এ স্বয়ংক্রিয় ও production-এ tag + manual approval — এমন pipeline লিখে একটি tag তৈরি করে deploy অনুমোদন করুন।

অধ্যায় 15 • PART III — GitLab CI/CD and Jenkins

Chapter 15 — Jenkins Architecture, Controller and Agents

১৫.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Jenkins-এর ইতিহাস ও কেন এটি এখনো ব্যবহৃত হয় তা ব্যাখ্যা করতে; (২) Controller ও Agent মডেলের স্থাপত্য আঁকতে; (৩) Jenkins LTS ইনস্টল ও প্রাথমিক কনফিগারেশন করতে; (৪) Agent সংযোগের পদ্ধতি — SSH, JNLP, Docker — আলাদা করতে; (৫) Build-এর workspace, executor ও queue ধারণা ব্যাখ্যা করতে; (৬) Jenkins-এর পরিচালনাগত ঝুঁকি ও রক্ষণাবেক্ষণের কাজ তালিকাভুক্ত করতে।

১৫.২ কেন এটি বাস্তব সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Jenkins সবচেয়ে পুরোনো ও বহুল ব্যবহৃত CI সার্ভার — ২০০৪ সালের Hudson থেকে শুরু, আজ হাজার হাজার কোম্পানির delivery এর ওপর চলে। GitHub Actions বা GitLab-এর মতো এটি SaaS নয় — আপনি নিজের সার্ভারে চালান, নিজে আপডেট করেন, নিজে ব্যাকআপ নেন। এই স্বাধীনতাই এর শক্তি ও দুর্বলতা: যেকোনো টুল, যেকোনো ভাষা, যেকোনো অবকাঠামো জোড়া দেওয়া যায়, কিন্তু সার্ভারটি রক্ষণাবেক্ষণের দায়িত্বও আপনার। অনেক কোম্পানি Jenkins ছাড়তে পারে না — কারণ তাদের pipeline, plugin ও ইতিহাস এতে বাঁধা। Jenkins বোঝা মানে স্ব-পরিচালিত (self-hosted) CI-এর আসল খরচ ও নিয়ন্ত্রণ বোঝা।

১৫.৩ পূর্বশর্ত জ্ঞান

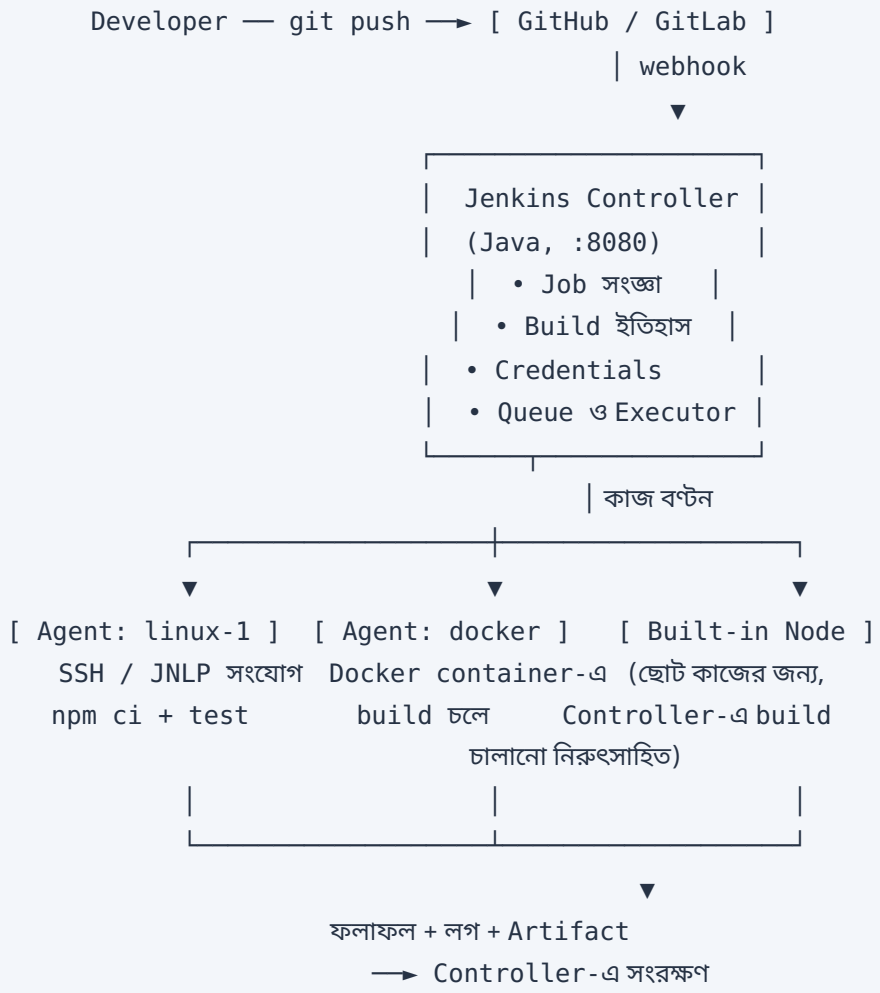
Linux সার্ভার প্রশাসনের প্রাথমিক ধারণা (Volume 04), Java চালানোর ধারণা (Jenkins Java-তে লেখা), Git ও pipeline lifecycle (অধ্যায় ৩), এবং Docker-এর প্রাথমিক ধারণা। Jenkins-এর UI ইংরেজিতে — প্রযুক্তিগত শব্দগুলো ইংরেজিতেই রাখা হয়েছে।

১৫.৪ প্রথম নীতি থেকে ব্যাখ্যা

Jenkins-এর মূল ধারণা একটিই: **একটি কেন্দ্রীয় সার্ভার (Controller) কাজের তালিকা রাখে, আর কর্মীরা (Agent) কাজ করে।** Controller নিজে build চালায় না — সে job-এর সংজ্ঞা, ইতিহাস, credential ও সময়সূচি রাখে। Agent হলো একটি প্রক্রিয়া (process) যা Controller-এর সঙ্গে সংযোগ করে, build চালায় ও ফলাফল ফেরত পাঠায়। এই বিভাজনের কারণ: Controller-কে হালকা ও সুরক্ষিত রাখা যায়, আর ভারী build-এর কাজ অনেক Agent-এর মধ্যে ভাগ করা যায়। Agent মারা গেলে build থামে, কিন্তু Controller ও ইতিহাস বেঁচে থাকে। উল্টোভাবে — Controller মারা গেলে সবকিছু থামে। তাই Controller-ই সবচেয়ে গুরুত্বপূর্ণ একক ব্যর্থতার বিন্দু (single point of failure)।

> **CONCEPT:** Jenkins-কে একটি রেস্তোরাঁর মতো ভাবুন: Controller হলো কাউন্টার — অর্ডার নেয়, হিসাব রাখে, বিল দেয়। Agent হলো রান্নাঘরের রাঁধুনি — আসল রান্না করে। কাউন্টার বন্ধ হলে কেউ অর্ডার দিতে পারে না, কিন্তু রাঁধুনি যতই থাকুক লাভ নেই। তাই Controller-এর সুরক্ষা ও ব্যাকআপ সবার আগে।

১৫.৫ আর্কিটেকচার ও কার্যপ্রবাহ চিত্র



Agent Controller-এর দিকে সংযোগ করে (inbound) অথবা Controller Agent-এর দিকে SSH করে (outbound) — দুই পদ্ধতির নিরাপত্তা ভিন্ন।

১৫.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Controller: যা যা রাখে

Controller-এর হোম ডিরেক্টরি (`JENKINS_HOME`, সাধারণত `/var/lib/jenkins`) Jenkins-এর মস্তিষ্ক: `jobs/`-এ প্রতিটি job-এর সংজ্ঞা ও build ইতিহাস, `plugins/`-এ ইনস্টল করা plugin, `credentials.xml`-এ এনক্রিপ্ট করা credential, `config.xml`-এ সার্ভারের সেটিং। এই ডিরেক্টরি হারিয়ে গেলে Jenkins হারিয়ে যায় — তাই ব্যাকআপ মানে `JENKINS_HOME`-এর ব্যাকআপ। Controller-এ **executor** হলো একসঙ্গে কতটি build চলতে পারবে তার স্লট; built-in node-এ executor ০ রাখাই সেরা অনুশীলন — Controller-এ build চালালে Controller ধীর হয়ে যায় ও নিরাপত্তা ঝুঁকি বাড়ে।

Agent সংযোগের পদ্ধতি

পদ্ধতি	কীভাবে কাজ করে	কখন ভালো

SSH	Controller SSH করে Agent-এ agent প্রক্রিয়া চালায়	স্থায়ী Linux সার্ভার; সহজ ও পরিচিত
JNLP / Inbound Agent	Agent নিজে Controller-এর দিকে সংযোগ করে	Agent firewall-এর পেছনে থাকলে; ক্লাউড Agent-এ
Docker / Kubernetes	প্রতিটি build-এর জন্য নতুন container/pod	পরিষ্কার পরিবেশ, স্বয়ংক্রিয় স্কেল; আধুনিক পছন্দ
Windows Agent	JNLP বা SSH (OpenSSH) দিয়ে	Windows-নির্দিষ্ট build (.NET, ডেস্কটপ অ্যাপ)

Agent-এর **workspace** হলো build চলার ফোল্ডার — কোড checkout, `node_modules`, build output সব এখানে থাকে। প্রতিটি job-এর আলাদা workspace থাকে; build শেষে workspace মুছে ফেলা যায় (`cleanWs()`) — নইলে পুরোনো ফাইল পরের build-কে দূষিত করতে পারে।

ইনস্টলেশন ও প্রাথমিক কনফিগারেশন

Jenkins LTS (Long-Term Support) ব্যবহার করুন — সাপ্তাহিক (weekly) সংস্করণে নতুন বৈশিষ্ট্য থাকলেও স্থিতিশীলতা কম। Ubuntu-তে অফিসিয়াল Jenkins repository যোগ করে `apt install jenkins` করলে systemd service হিসেবে চলে। প্রথম চালুতে `/var/lib/jenkins/secrets/initialAdminPassword` ফাইলে একটি অস্থায়ী পাসওয়ার্ড থাকে — সেটি দিয়ে লগইন করে admin ব্যবহারকারী তৈরি করুন। "Install suggested plugins" দিয়ে শুরু করা সহজ, তবে production-এ শুধু প্রয়োজনীয় plugin রাখুন — প্রতিটি plugin একটি সম্ভাব্য ঝুঁকি ও রক্ষণাবেক্ষণের বোঝা।

```
# Ubuntu Server 24.04 LTS-এ Jenkins LTS ইনস্টল (অফিসিয়াল পদ্ধতির সারাংশ)
# বিস্তারিত ধাপের জন্য Jenkins অফিসিয়াল ইনস্টলেশন ডকুমেন্টেশন দেখুন।
sudo apt update
sudo apt install -y openjdk-17-jre       # Jenkins-এর জন্য Java 17 (সংস্করণ ডকুমেন্টেশনে যাচাই করুন)
sudo systemctl enable --now jenkins
sudo systemctl status jenkins
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
# ব্রাউজারে http://SERVER-IP:8080 খুলে পাসওয়ার্ড দিয়ে সেটআপ সম্পূর্ণ করুন
# প্রথম কাজ: admin ব্যবহারকারী তৈরি, URL নির্ধারণ, অপ্রয়োজনীয় executor বন্ধ করা
```

Queue, Executor ও Build ইতিহাস

Job চালু হলে প্রথমে **queue**-তে যায় — উপযুক্ত Agent ও executor খালি না থাকলে অপেক্ষা করে। Executor খালি হলে build শুরু হয়, প্রতিটি build-এর একটি নম্বর (#1, #2, ...) ও লগ (console output) থাকে। Build-এর ফলাফল: `SUCCESS`, `FAILURE`, `UNSTABLE` (build হয়েছে কিন্তু পরীক্ষা ব্যর্থ), `ABORTED`। `UNSTABLE` ও `FAILURE`-এর পার্থক্য গুরুত্বপূর্ণ — পরীক্ষা ব্যর্থ হলে build "অস্থিতিশীল" হয়, কিন্তু artifact তৈরি হতে পারে; কম্পাইল ব্যর্থ হলে build সম্পূর্ণ ব্যর্থ।

কেন Production Jenkins-এর যত্নশীল পরিচালনা দরকার

Jenkins একটি জীবন্ত সার্ভার: Java heap memory, ডিস্ক (build ইতিহাস ও artifact), plugin আপডেট, নিরাপত্তা প্যাচ — সবই নিয়মিত দেখতে হয়। Controller-এর ডিস্ক ভরে গেলে Jenkins থেমে যায়; পুরোনো plugin-এ দুর্বলতা থাকলে আক্রমণকারী build-এর মাধ্যমে সার্ভার দখল করতে পারে; ব্যাকআপ না থাকলে Controller নষ্ট হলে বছরের ইতিহাস হারিয়ে যায়। SaaS CI-তে এই দায়িত্ব প্রদানকারীর; Jenkins-এ এটি আপনার। এই অধ্যায়ের বাকি অংশ ও অধ্যায় ১৭ এই পরিচালনারই বিস্তারিত।

১৫.৭ বাস্তব পরিস্থিতি — TechNova

REAL WORLD — TechNova: TechNova-র Jenkins Controller একটি ছোট VPS-এ চলত — আর সব build-ও Controller-এই চলত (built-in node-এ executor ৪)। একদিন একটি ভারী Docker build Controller-এর সব RAM খেয়ে ফেলল — Jenkins UI খুলছিল না, কোনো build চলছিল না, এমনকি লগইনও করা যাচ্ছিল না। সমাধান: Controller-এর executor ০ করা হলো, আলাদা দুটি Agent সার্ভার যোগ করা হলো, আর Controller-এর ডিস্ক ও heap মনিটরিং চালু হলো। শিক্ষা: Controller হলো মস্তিষ্ক — মস্তিষ্ক দিয়ে ভারী কাজ করাবেন না।

১৫.৮ ধাপে ধাপে বাস্তবায়ন

1. ল্যাব VM-এ (Ubuntu 24.04) অফিসিয়াল ডকুমেন্টেশন অনুযায়ী Jenkins LTS ইনস্টল করুন।
2. প্রাথমিক পাসওয়ার্ড দিয়ে admin তৈরি করুন; Jenkins URL নির্ধারণ করুন।
3. Manage Jenkins → Nodes-এ built-in node-এর executor সংখ্যা ০ করুন।
4. একটি Agent যোগ করুন (SSH পদ্ধতিতে): Agent সার্ভারে Java ইনস্টল করুন, SSH credential যোগ করুন, Agent online কি না দেখুন।
5. একটি Freestyle job তৈরি করে Agent-এ `node --version` চালিয়ে যাচাই করুন।
6. `JENKINS_HOME`-এর ব্যাকআপ নেওয়ার পরীক্ষা করুন — ফোল্ডারটি কপি করে অন্য VM-এ চালিয়ে দেখুন।

১৫.৯ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) Jenkins UI খুলছে না — service চলছে কি না (`systemctl status jenkins`), পোর্ট 8080 firewall-এ খোলা কি না, Java সংস্করণ মিলছে কি না দেখুন। **(২) Agent offline** — SSH key/credential ভুল, Agent-এ Java নেই, বা নেটওয়ার্ক বন্ধ; Agent-এর লগে সঠিক ত্রুটি দেখুন। **(৩) Build queue-তে আটকে আছে** — কোনো Agent-এ খালি executor নেই বা job-এর label-এর সঙ্গে কোনো Agent মিলছে না। **(৪) "No space left on device"** — পুরোনো build ও workspace জমেছে; পুরোনো build মুছুন ও Discard Old Builds সেট করুন। **(৫) Plugin ইনস্টলের পর Jenkins চালু হচ্ছে না** — অসামঞ্জস্যপূর্ণ plugin; safe mode-এ চালু করে সমস্যার plugin নিষ্ক্রিয় করুন।

১৫.১০ নিরাপত্তা বিবেচনা

WARNING: Jenkins-কে কখনোই সরাসরি ইন্টারনেটে খুলে দেবেন না — reverse proxy ও TLS-এর পেছনে রাখুন, শক্তিশালী প্রমাণীকরণ চালু করুন, এবং ডিফল্ট admin পাসওয়ার্ড বদলান। Controller-এ build চালালে যেকোনো job Controller-এর credential পড়তে পারে — তাই built-in node-এ executor ০ রাখুন। Agent-এর সঙ্গে সংযোগে SSH key ব্যবহার করুন, পাসওয়ার্ড নয়। Jenkins-এর নিরাপত্তা পরামর্শ (security advisory) নিয়মিত পড়ুন — Jenkins ও plugin-এ দুর্বলতা প্রায়ই প্রকাশিত হয়।

১৫.১১ কর্মক্ষমতা ও খরচ বিবেচনা

Controller-এর জন্য ২–৪ GB RAM ও নিয়মিত ডিস্ক পরিষ্কার যথেষ্ট হতে পারে ছোট দলের জন্য; Agent-এর আকার build-এর চাহিদা অনুযায়ী নির্ধারণ করুন। Docker/Kubernetes Agent ব্যবহার করলে build না থাকলে খরচ শূন্যের কাছাকাছি — স্থায়ী Agent সার্ভার সবসময় চলে, খরচও সবসময়। Build ইতিহাস সীমিত রাখুন (যেমন শেষ ৩০টি build) — প্রতিটি build-এর artifact ডিস্ক খায়। Java heap (`JAVA_OPTS=-Xmx`) খুব ছোট হলে Jenkins ধীর হয়, খুব বড় হলে সার্ভারের RAM শেষ হয় — মনিটর করে নির্ধারণ করুন।

১৫.১২ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: LTS ব্যবহার করুন; Controller-এ executor ০; Agent-এ build চালান; নিয়মিত ব্যাকআপ নিন; plugin সংখ্যা কম রাখুন; build ইতিহাস সীমিত করুন। **অ্যান্টি-প্যাটার্ন:** Controller-এ build চালানো; Jenkins-কে ইন্টারনেটে উন্মুক্ত রাখা; বছরের পর বছর plugin আপডেট না করা; ব্যাকআপ ছাড়া Controller চালানো; প্রতিটি build-এর artifact চিরকাল রাখা।

১৫.১৩ অধ্যায়ের সারাংশ

Jenkins-এ Controller সিদ্ধান্ত ও ইতিহাস রাখে, Agent কাজ করে। SSH, JNLP ও Docker — তিনভাবে Agent যুক্ত হয়; Docker Agent পরিষ্কার ও স্কেলযোগ্য। Built-in node-এ build চালানো বিপজ্জনক ও ধীর। Jenkins-এর স্বাধীনতার মূল্য হলো পরিচালনা: ব্যাকআপ, আপডেট, মনিটরিং ও নিরাপত্তা — সবই আপনার দায়িত্ব। পরের অধ্যায়ে Jenkinsfile দিয়ে pipeline লেখা শেখা হবে।

১৫.১৪ মূল প্রযুক্তিগত পরিভাষা

Controller: Jenkins-এর কেন্দ্রীয় সার্ভার; job, ইতিহাস ও credential রাখে।

Agent (Node): Controller-এর নির্দেশে build চালানো কর্মী মেশিন বা container।

Executor: Agent-এ একসঙ্গে কতটি build চলতে পারবে তার স্লট।

Workspace: Agent-এ build চলার ফোল্ডার; কোড ও build ফাইল এখানে থাকে।

Queue: চালুর অপেক্ষায় থাকা build-এর তালিকা।

JENKINS_HOME: Controller-এর ডেটা ডিরেক্টরি; ব্যাকআপের মূল লক্ষ্য।

LTS: Long-Term Support — স্থিতিশীল, দীর্ঘদিন সমর্থিত Jenkins সংস্করণ।

JNLP / Inbound Agent: Agent নিজে Controller-এর দিকে সংযোগ করার পদ্ধতি।

UNSTABLE: Build হয়েছে কিন্তু পরীক্ষা ব্যর্থ — এমন build-এর অবস্থা।

Single Point of Failure: একটি অংশ নষ্ট হলে পুরো ব্যবস্থা থেমে যাওয়ার ঝুঁকি।

১৫.১৫ জ্ঞান যাচাই প্রশ্ন

1. Controller ও Agent-এর দায়িত্ব আলাদা কেন? Controller-এ build চালালে কী ঝুঁকি?
2. SSH, JNLP ও Docker Agent-এর মধ্যে কোনটি কখন বেছে নেবেন?
3. Executor সংখ্যা কী? Built-in node-এ executor ০ রাখা কেন সেরা অনুশীলন?
4. `UNSTABLE` ও `FAILURE`-এর পার্থক্য কী?
5. `JENKINS_HOME`-এ কী কী থাকে? ব্যাকআপে কোনটি সবচেয়ে গুরুত্বপূর্ণ?

১৫.১৬ ব্যবহারিক অনুশীলন

1. **ইনস্টলেশন:** ল্যাব VM-এ Jenkins LTS ইনস্টল করে admin তৈরি করুন; built-in node-এর executor ০ করুন।
2. **Agent যোগ:** SSH পদ্ধতিতে একটি Agent যোগ করে তার ওপর একটি পরীক্ষামূলক job চালান; Agent-এর লগ পড়ুন।
3. **ব্যাকআপ পরীক্ষা:** `JENKINS_HOME`-এর ব্যাকআপ নিয়ে একটি job মুছে ফেলুন, তারপর ব্যাকআপ থেকে ফিরিয়ে আনুন।

অধ্যায় 16 • PART III — GitLab CI/CD and Jenkins

Chapter 16 — Jenkins Pipelines: Declarative and Scripted

১৬.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Declarative ও Scripted Pipeline-এর পার্থক্য ও কখন কোনটি ব্যবহার করবেন তা বলতে; (২) Jenkinsfile-এর গঠন — pipeline, agent, stages, steps, post — ব্যাখ্যা করতে; (৩) সম্পূর্ণ Declarative Jenkinsfile লিখতে (Node.js অ্যাপের জন্য); (৪) Scripted Pipeline-এর Groovy ভিত্তি বুঝতে ও সহজ উদাহরণ লিখতে; (৫) Shared Library, parameter ও post condition ব্যবহার করতে; (৬) Pipeline-এর ত্রুটি পড়ে সমাধান করতে।

১৬.২ কেন এটি বাস্তব সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Freestyle job-এ ক্লিক করে করে pipeline বানানো যায়, কিন্তু সেটি কোড নয় — review করা যায় না, সংস্করণ নিয়ন্ত্রণে রাখা যায় না, অন্য প্রজেক্টে নকল করা যায় না। Jenkinsfile এই সমস্যার সমাধান: pipeline-ই কোড, রিপোজিটরিতে থাকে, Merge Request-এ review হয়। TechNova-র Freestyle job একবার ভুল ক্লিকে মুছে গিয়েছিল — কোনো ব্যাকআপ ছিল না, আবার বানাতে দুই দিন লেগেছিল। Jenkinsfile থাকলে `git checkout` করলেই ফিরে আসত। Declarative Pipeline বেশিরভাগ কাজের জন্য যথেষ্ট ও পড়তে সহজ; Scripted Pipeline জটিল যুক্তির জন্য শক্তিশালী — দুটোর সীমা জানা পেশাদার Jenkins ব্যবহারকারীর চিহ্ন।

১৬.৩ পূর্বশর্ত জ্ঞান

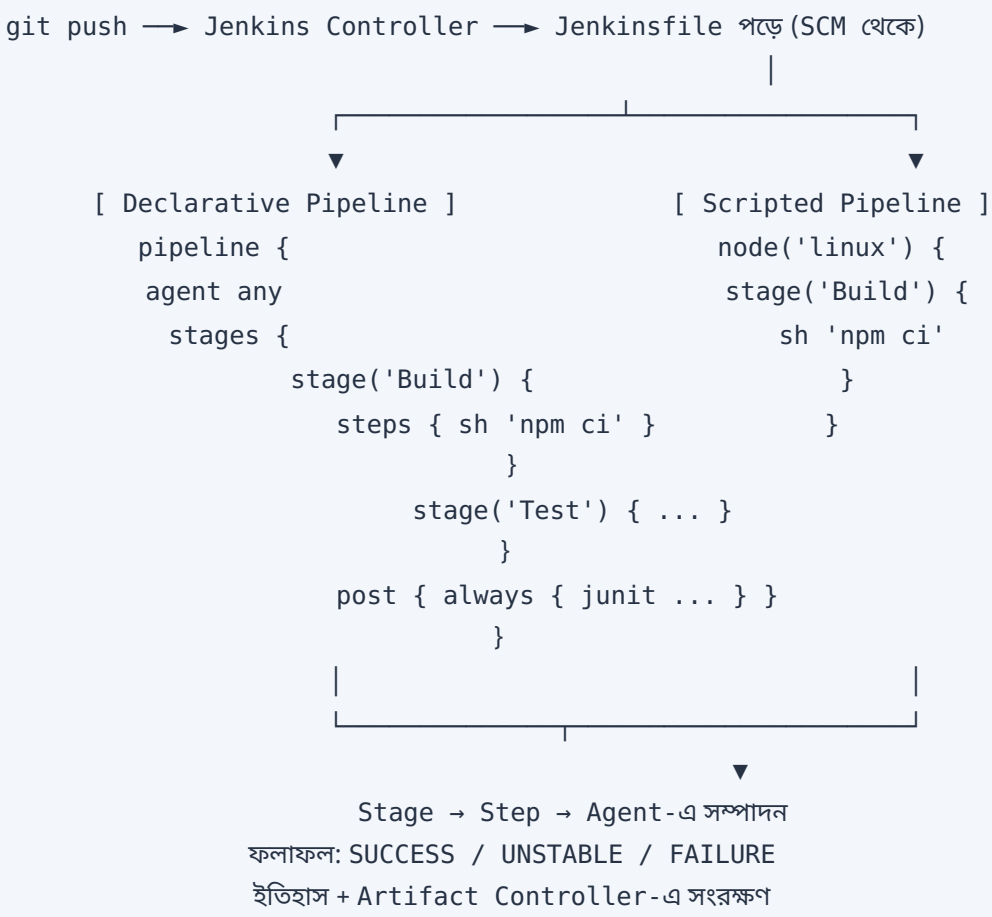
অধ্যায় ১৫-এর Controller/Agent স্থাপত্য, Git, YAML/JSON-এর মতো গঠনমূলক ভাষার ধারণা, এবং Groovy-র প্রাথমিক ধারণা (Java-সদৃশ স্ক্রিপ্ট ভাষা) সহায়ক — তবে Scripted অংশে প্রয়োজনীয় Groovy ব্যাখ্যা করা হয়েছে।

১৬.৪ প্রথম নীতি থেকে ব্যাখ্যা

Pipeline-কে কোড হিসেবে লেখার (Pipeline as Code) মূল যুক্তি: **delivery-র সংজ্ঞা যদি কোড হয়, তবে delivery-ও কোডের মতো আচরণ করে** — review, পরীক্ষা, rollback, পুনঃব্যবহার — সব সম্ভব। Declarative Pipeline একটি **ঘোষণামূলক (declarative)** ভাষা: আপনি বলেন "কী চাই" — কোন stage, কোন agent, কোন শর্তে — Jenkins নিজে "কীভাবে" ঠিক করে। Scripted Pipeline একটি **আদেশমূলক (imperative)** ভাষা: আপনি ধাপে ধাপে বলেন "কীভাবে করবে" — Groovy কোড দিয়ে যেকোনো যুক্তি লেখা যায়। ঘোষণামূলক পদ্ধতি পড়তে সহজ ও ভুল কম হয়; আদেশমূলক পদ্ধতি নমনীয়, কিন্তু জটিল ও ভুলের সুযোগ বেশি। বেশিরভাগ দলের নিয়ম: Declarative দিয়ে শুরু করুন; শুধু যখন Declarative দিয়ে করা যাচ্ছে না, তখন Scripted-এর ছোট অংশ ব্যবহার করুন।

> **CONCEPT:** Jenkinsfile রিপোজিটরির root-এ থাকে এবং প্রতিটি branch-এর নিজস্ব Jenkinsfile থাকতে পারে। অর্থাৎ feature branch-এর pipeline বদলালেও `main`-এর pipeline বদলায় না — প্রতিটি branch নিজের delivery নিয়ম নিজে বহন করে।

১৬.৫ আর্কিটেকচার ও কার্যপ্রবাহ চিত্র



দুটো পদ্ধতিই একই Agent-এ চলে; পার্থক্য শুধু লেখার ভাষা ও নমনীয়তায়।

১৬.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Declarative Pipeline-এর গঠন

শীর্ষে `pipeline { }` ব্লক; তার ভেতরে `agent` (কোথায় চলবে — `any`, `label 'linux'`, বা `docker { image 'node:20-alpine' }`), `environment` (পরিবেশ variable), `parameters` (ব্যবহারকারীর ইনপুট), `stages` (প্রতিটি `stage`-এ `steps`), `post` (শেষে কী হবে)। `when` দিয়ে stage-এর শর্ত (যেমন শুধু `main` branch-এ deploy), `parallel` দিয়ে একসঙ্গে চলা stage, `input` দিয়ে মানুষের অনুমোদন, `options` দিয়ে timeout ও retry — সবই Declarative-এ আছে। `post`-এর শর্ত: `always` (সবসময়), `success`, `failure`, `unstable`, `changed` (আগের build-এর চেয়ে ফল বদলালে)।

Scripted Pipeline-এর গঠন

শীর্ষে `node('label') { }` ব্লক; তার ভেতরে `stage('নাম') { }` ও Groovy কোড — `if`, `for`, ফাংশন, try/catch সব লেখা যায়। Scripted-এ কোনো `post` নেই — try/finally দিয়ে একই কাজ করতে হয়। শক্তি: যেকোনো জটিল যুক্তি — যেমন পরীক্ষার ফলাফল পড়ে গতিশীলভাবে deploy সিদ্ধান্ত। দুর্বলতা: ভুল করা সহজ, পড়া কঠিন, এবং Jenkins-এর নতুন বৈশিষ্ট্য (যেমন Blue Ocean ভিজ্যুয়ালাইজেশন) Declarative-এ ভালো কাজ করে।

Agent নির্বাচন

Agent নির্দেশ	অর্থ
`agent any`	যেকোনো উপলব্ধ Agent
`agent { label 'docker' }`	শুধু ওই label-এর Agent
`agent { docker { image 'node:20-alpine' } }`	প্রতিটি build-এ নতুন container
`agent none` (শীর্ষে) + প্রতি stage-এ agent	প্রতিটি stage আলাদা Agent-এ চলতে পারে

Shared Library

একই ধাপ (যেমন "Docker image build করে push করো") দশটি প্রজেক্টে থাকলে প্রতিটিতে নকল না করে **Shared Library** তৈরি করুন: একটি আলাদা Git রিপোজিটরিতে `vars/deployApp.groovy` লিখে Jenkins-এ লাইব্রেরি হিসেবে যুক্ত করুন; তারপর যেকোনো Jenkinsfile থেকে `deployApp('staging')` ডাকা যায়। তবে Shared Library মানে সব pipeline একটি কেন্দ্রীয় কোডের ওপর নির্ভরশীল — লাইব্রেরি ভাঙলে সব pipeline ভাঙে; তাই লাইব্রেরির সংস্করণ (version/tag) বেঁধে ব্যবহার করুন ও পরীক্ষা করুন।

Parameter ও Build ইতিহাস

`parameters { string(name: 'VERSION', defaultValue: '1.0.0') }` দিয়ে build চালুর সময় ইনপুট নেওয়া যায় — যেমন কোন সংস্করণ deploy হবে। `buildDiscarder(logRotator(numToKeepStr: '30'))` দিয়ে পুরোনো build মুছে ডিস্ক বাঁচান। `archiveArtifacts artifacts: 'dist/**'` দিয়ে build output সংরক্ষণ করুন; `junit 'junit.xml'` দিয়ে পরীক্ষার রিপোর্ট UI-তে দেখান।

১৬.৭ সম্পূর্ণ Declarative Jenkinsfile — TechNova MERN অ্যাপ

```
// Jenkinsfile — TechNova MERN অ্যাপ (Declarative Pipeline)
pipeline {
  agent { docker { image 'node:20-alpine' } }  // প্রতিটি build নতুন container-এ

  options {
    timeout(time: 30, unit: 'MINUTES')         // ৩০ মিনিটের বেশি চললে থামবে
    buildDiscarder(logRotator(numToKeepStr: '30'))  // শেষ ৩০টি build রাখবে
    timestamps()                               // লগে সময় দেখাবে
  }

  environment {
    APP_NAME = 'technova-web'
    // DATABASE_URL এখানে নয় — Jenkins Credentials-এ (অধ্যায় ১৭)
  }

  parameters {
    string(name: 'DEPLOY_ENV', defaultValue: 'staging',
           description: 'কোন পরিবেশে deploy হবে: staging বা production')
  }

  stages {
    stage('Install') {
      steps { sh 'npm ci' }
```

```
    }
    stage('Lint') {
      steps { sh 'npm run lint' }
    }
    stage('Test') {
      steps { sh 'npm test -- --coverage' }
      post {
        always { junit 'junit.xml' }           // পরীক্ষার রিপোর্ট সংরক্ষণ
      }
    }
    stage('Build') {
      steps {
        sh 'npm run build'
        archiveArtifacts artifacts: 'dist/**', fingerprint: true
      }
    }
    stage('Deploy') {
      when { branch 'main' }                   // শুধু main branch-এ deploy
      input {                                  // মানুষের অনুমোদন
        message "Deploy ${params.DEPLOY_ENV}-এ করবেন?"
        ok 'Deploy করুন'
      }
      steps {
        echo "Deploying to ${params.DEPLOY_ENV}"
        sh "./scripts/deploy.sh ${params.DEPLOY_ENV}"
      }
    }
  }

  post {
    success { echo 'Pipeline সফল হয়েছে।' }
    failure { echo 'Pipeline ব্যর্থ — লগ দেখুন।' }
    always  { cleanWs() }                      // workspace পরিষ্কার
  }
}
```

ক্ষেত্র-ব্যাখ্যা: `agent docker` — পরিষ্কার পরিবেশ; `timeout` — আটকে যাওয়া build চিরকাল চলবে না; `buildDiscarder` — ডিস্ক বাঁচায়; `when branch` — feature branch-এ deploy নয়; `input` — মানুষের অনুমোদন ছাড়া deploy নয়; `archiveArtifacts fingerprint: true` — কোন build থেকে কোন artifact এসেছে তা শনাক্ত করা যায়; `cleanWs()` — পরের build-এর জন্য workspace পরিষ্কার।

```
// Scripted Pipeline — একই কাজের সংক্ষিপ্ত রূপ (তুলনার জন্য)
node('linux') {
  try {
    stage('Install') { sh 'npm ci' }
    stage('Test')    { sh 'npm test' }
    if (env.BRANCH_NAME == 'main') {
      stage('Deploy') {
        input 'Deploy করবেন?'
        sh './scripts/deploy.sh staging'
      }
    }
  } finally {
    cleanWs()          // post.always-এর বিকল্প
```

```
    }
}
```

Scripted-এ `if` দিয়ে যেকোনো শর্ত লেখা যায় — কিন্তু `post`, `when` ও `options`-এর মতো ঘোষণামূলক সুবিধা নেই; সবকিছু হাতে লিখতে হয়।

১৬.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD — TechNova: TechNova-র দলে পাঁচটি মাইক্রোসার্ভিসের প্রতিটিতে প্রায় একই Jenkinsfile ছিল — নকল করে বানানো, একটিতে ভুল থাকলে পাঁচটিতেই ভুল। দল Shared Library তৈরি করল: `nodePipeline()` নামে একটি ফাংশন — lint, test, build, Docker push — সব প্রজেক্টে একই। তারপর প্রতিটি Jenkinsfile মাত্র কয়েক লাইনের হয়ে গেল। কিন্তু একদিন লাইব্রেরিতে একটি ভুল commit সব পাঁচটি pipeline একসঙ্গে ভেঙে দিল। শিক্ষা: Shared Library শক্তিশালী, তবে তার সংস্করণ বেঁধে (tag দিয়ে) ব্যবহার করুন ও লাইব্রেরির নিজেরও পরীক্ষা রাখুন।

১৬.৯ ধাপে ধাপে বাস্তবায়ন

1. অধ্যায় ১৫-এর Jenkins-এ একটি Pipeline job তৈরি করুন; Definition হিসেবে "Pipeline script from SCM" বেছে Git রিপোজিটরি ও branch দিন।
2. রিপোজিটরির root-এ উপরের Declarative Jenkinsfile রাখুন ও push করুন।
3. Job চালিয়ে প্রতিটি stage-এর লগ পড়ুন; Blue Ocean বা Stage View-তে stage-এর সময় দেখুন।
4. একটি feature branch-এ push করে যাচাই করুন — Deploy stage চলছে কি না (চলা উচিত নয়)।
5. `main`-এ merge করে Deploy stage-এর `input` অনুমোদন দিন।
6. একটি ছোট Shared Library তৈরি করে একটি ধাপ সেখানে সরিয়ে নিন।

১৬.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "WorkflowScript: expecting '}'" — Groovy syntax ভুল; Declarative Pipeline Linter বা Replay-তে ছোট অংশ পরীক্ষা করুন। **(২) "No such DSL method 'sh'"** — `sh` শুধু `steps` ব্লকের ভেতরে চলে; বাইরে লিখলে এই ত্রুটি হয়। **(৩) Stage "skipped"** — `when` শর্ত মেলেনি; branch নাম বা expression পরীক্ষা করুন। **(৪) `input` আটকে আছে** — কেউ অনুমোদন দেয়নি; build-এর পর্দায় Approve বোতাম দেখুন; timeout না দিলে চিরকাল অপেক্ষা করবে। **(৫) Docker agent-এ "docker: not found"** — Agent-এ Docker নেই বা image-এ Docker CLI নেই; Agent-এর label ও image যাচাই করুন।

১৬.১১ নিরাপত্তা বিবেচনা

WARNING: Jenkinsfile রিপোজিটরিতে থাকে — যে কেউ Jenkinsfile বদলে pipeline-এ নিজের কমান্ড চালাতে পারে। তাই `main` branch-এর Jenkinsfile বদলাতে Merge Request approval বাধ্যতামূলক করুন। Scripted Pipeline-এ **Script Security** plugin অননুমোদিত Groovy পদ্ধতি আটকায় — অনুমোদন ছাড়া বিপজ্জনক পদ্ধতি ব্যবহার করবেন না। `input` অনুমোদনকারী যেন deploy করা ব্যক্তি না হন — তা নিশ্চিত করুন (অধ্যায় ১৭)।

১৬.১২ কর্মক্ষমতা ও খরচ বিবেচনা

Docker agent প্রতিটি build-এ নতুন container তৈরি করে — পরিষ্কার, কিন্তু image টানার সময় লাগে; image ক্যাশ করে রাখুন। `parallel` stage দিয়ে lint ও test একসঙ্গে চালালে সময় কমে, কিন্তু একসঙ্গে বেশি executor লাগে। `timeout` ছাড়া আটকে যাওয়া build executor দখল করে রাখে — সব pipeline-এ timeout দিন। Replay বৈশিষ্ট্য দিয়ে Jenkinsfile বদলে আবার চালানো যায় — তবে এটি শুধু পরীক্ষার জন্য, স্থায়ী পরিবর্তন Git-এ commit করুন।

১৬.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Declarative দিয়ে শুরু করুন; Jenkinsfile Git-এ রাখুন; ছোট ছোট stage রাখুন; `post`-এ সবসময় পরিষ্কার করুন; Shared Library-র সংস্করণ বেঁধে দিন। **অ্যান্টি-প্যাটার্ন:** UI-তে Pipeline script লেখা (সংস্করণ নিয়ন্ত্রণ নেই); একটিমাত্র বিশাল stage; `input` ছাড়া production deploy; Scripted-এ সবকিছু লেখা যখন Declarative-ই যথেষ্ট; লাইব্রেরির `master` branch সরাসরি ব্যবহার করা।

১৬.১৪ অধ্যায়ের সারাংশ

Declarative Pipeline "কী চাই" বলে — পড়তে সহজ, বেশিরভাগ কাজের জন্য যথেষ্ট। Scripted Pipeline "কীভাবে" বলে — Groovy দিয়ে যেকোনো যুক্তি সম্ভব, কিন্তু জটিল। Jenkinsfile pipeline-কে কোড বানায় — review, rollback ও পুনঃব্যবহার সম্ভব হয়। Shared Library পুনরাবৃত্তি কমায়, তবে সংস্করণ বেঁধে ব্যবহার করতে হয়। `agent`, `when`, `input` ও `post` — এই চারটি দিয়েই বেশিরভাগ production pipeline তৈরি করা যায়।

১৬.১৫ মূল প্রযুক্তিগত পরিভাষা

Jenkinsfile: Pipeline-এর সংজ্ঞা লেখা Groovy-ভিত্তিক ফাইল; রিপোজিটরিতে থাকে।

Declarative Pipeline: "কী চাই" ঘোষণা করে লেখা pipeline; সহজ ও নিরাপদ।

Scripted Pipeline: Groovy কোড দিয়ে ধাপে ধাপে লেখা pipeline; সর্বোচ্চ নমনীয়তা।

Stage / Step: Pipeline-এর পর্যায় / পর্যায়ের ভেতরের একক নির্দেশ।

Post Condition: Pipeline বা stage শেষে শর্ত অনুযায়ী চালানো কাজ (always, success, failure)।

Shared Library: একাধিক pipeline-এ পুনঃব্যবহারযোগ্য Groovy কোডের ভান্ডার।

Input Step: মানুষের অনুমোদনের জন্য pipeline থামানোর ধাপ।

Parameter: Build চালুর সময় দেওয়া ইনপুট মান।

Archive Artifacts: Build-এর ফলাফল Controller-এ সংরক্ষণ করা।

Pipeline as Code: Pipeline-কে সংস্করণ-নিয়ন্ত্রিত কোড হিসেবে রাখার নীতি।

১৬.১৬ জ্ঞান যাচাই প্রশ্ন

1. Declarative ও Scripted Pipeline-এর মধ্যে তিনটি পার্থক্য লিখুন। কখন Scripted দরকার হয়?
2. `post { always { cleanWs() } }`-এর কাজ কী? এটি না করলে কী সমস্যা?
3. `agent none` কখন ব্যবহার করবেন? একটি উদাহরণ দিন।
4. Shared Library-র সুবিধা ও ঝুঁকি কী? ঝুঁকি কমাতে কী করবেন?
5. `when { branch 'main' }` ও `input` একসঙ্গে কী নিরাপত্তা দেয়?

১৬.১৭ ব্যবহারিক অনুশীলন

1. **Declarative Jenkinsfile:** অধ্যায়ের উদাহরণটি নিজের Node.js প্রজেক্টে চালান; প্রতিটি stage-এর সময় লিখুন।
2. **Scripted রূপান্তর:** একই pipeline-কে Scripted-এ লিখে দুটোর পড়ার সহজতা তুলনা করুন।
3. **Shared Library:** একটি ছোট `vars/` ফাংশন তৈরি করে দুটি job-এ ব্যবহার করুন; লাইব্রেরির tag বেঁধে দিন।

অধ্যায় 17 • PART III — GitLab CI/CD and Jenkins

Chapter 17 — Jenkins Credentials, Plugins and Pipeline Security

১৭.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Jenkins Credential-এর ধরন — Username/Password, Secret Text, SSH Key, Certificate — ব্যাখ্যা করতে; (২) `withCredentials` দিয়ে নিরাপদে secret ব্যবহার করতে; (৩) Plugin-এর ভূমিকা, ঝুঁকি ও আপডেট নীতি বলতে; (৪) Role-Based Access Control ও Matrix Authorization দিয়ে অনুমতি সাজাতে; (৫) Pipeline-এ untrusted কোড চালানোর ঝুঁকি ব্যাখ্যা করতে; (৬) Jenkins-এর ব্যাকআপ, আপডেট ও রক্ষণাবেক্ষণ পরিকল্পনা তৈরি করতে।

১৭.২ কেন এটি বাস্তব সফটওয়্যার ডেলিভারিতে গুরুত্বপূর্ণ

Jenkins-এ একটি ভুলই যথেষ্ট: একটি credential লগে ফাঁস হলে আক্রমণকারী আপনার সার্ভার, রেজিস্ট্রি বা ক্লাউডে ঢুকতে পারে; একটি পুরোনো plugin-এর দুর্বলতা দিয়ে পুরো Controller দখল করা যায়; একটি অবিশ্বস্ত Pull Request-এর Jenkinsfile production secret পড়ে ফেলতে পারে। TechNova-র এক ডেভেলপার একবার API key সরাসরি Jenkinsfile-এ লিখে Git-এ push করেছিলেন — key-টি ইতিহাসে চিরকাল থেকে গেল, বাতিল করতে হলো। নিরাপত্তা Jenkins-এর "পরে দেখা যাবে" বিষয় নয় — এটি প্রথম দিনের সিদ্ধান্ত। এই অধ্যায় Part III-এর সমাপ্তি: GitLab ও Jenkins — দুই প্ল্যাটফর্মের নিরাপত্তা নীতিই এখানে একসঙ্গে বাঁধা হয়েছে।

১৭.৩ পূর্বশর্ত জ্ঞান

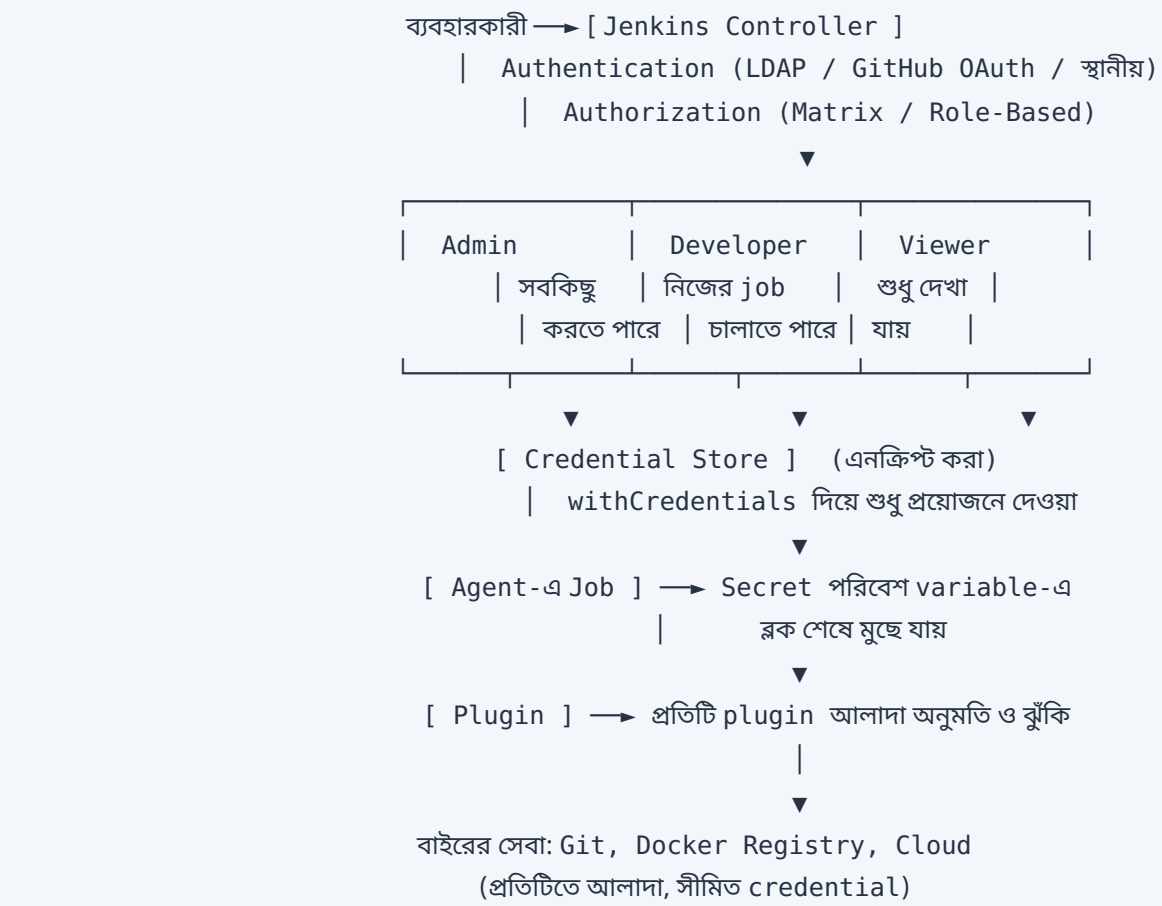
অধ্যায় ১৫–১৬-এর Jenkins স্থাপত্য ও Jenkinsfile; অধ্যায় ১৪-এর GitLab variable ও least privilege ধারণা; secret ব্যবস্থাপনার মৌলিক নীতি — secret কখনো কোডে বা লগে নয়।

১৭.৪ প্রথম নীতি থেকে ব্যাখ্যা

Jenkins নিরাপত্তার তিনটি প্রশ্ন: **(১) কে কী করতে পারে?** — প্রমাণীকরণ (authentication) ও অনুমোদন (authorization)। **(২) Secret কোথায় থাকে ও কে দেখতে পারে?** — credential সংরক্ষণ ও ব্যবহার। **(৩) বাইরের কোড কতটা বিশ্বাসযোগ্য?** — pipeline-এ চলা কোডের সীমানা। মূল নীতি **least privilege**: প্রতিটি ব্যবহারকারী, job ও plugin শুধু ততটুকুই করতে পারবে যতটুকু তার কাজের জন্য দরকার — একটুও বেশি নয়। দ্বিতীয় নীতি **defense in depth**: একটি স্তর ভাঙলেও পরের স্তর আটকাবে — যেমন credential masked থাকলেও job-এর অনুমতি সীমিত থাকবে।

> **CONCEPT:** Jenkins-এ credential একটি "ধার দেওয়া চাবি" — job চাবিটি দেখতে পায় না, শুধু ব্যবহার করতে পারে। `withCredentials` ব্লকের ভেতরে secret পরিবেশ variable হিসেবে থাকে, ব্লক শেষ হলে মুছে যায়। কেউ যদি `echo $SECRET` লেখে, Jenkins লগে সেটি `****` দিয়ে ঢেকে দেয় — তবে encoding করে ফাঁস করা যায়, তাই শুধু masking-এর ওপর ভরসা করবেন না।

১৭.৫ আর্কিটেকচার ও কার্যপ্রবাহ চিত্র



কোনো স্তরই একা যথেষ্ট নয় — প্রমাণীকরণ, অনুমোদন, credential সুরক্ষা ও plugin নিয়ন্ত্রণ একসঙ্গে কাজ করে।

১৭.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Credential-এর ধরন

ধরন	ব্যবহার	উদাহরণ
Username with password	Git, Nexus, সাধারণ লগইন	Git HTTPS credential
Secret text	API token, webhook secret	Docker Registry token
SSH Username with private key	Git SSH, Agent SSH সংযোগ	Deploy সার্ভারের key
Certificate	TLS/mTLS প্রমাণীকরণ	অভ্যন্তরীণ সেবার client cert
Vault / বাইরের secret store	কেন্দ্রীয় secret ব্যবস্থাপনা	HashiCorp Vault plugin

Credential-এর **scope** থাকে: Global (সব job) বা System (শুধু Controller/Agent সংযোগ)। নীতিমালা: credential-কে যতটা সম্ভব সংকীর্ণ scope-এ রাখুন; একটি job-এর credential অন্য job-এ ব্যবহার করবেন না। Credential-এর **ID** দিয়ে Jenkinsfile-এ ডাকা হয় — আসল মান Jenkinsfile-এ কখনো লেখা হয় না।

withCredentials ব্যবহার

```
// Jenkinsfile — নিরাপদ credential ব্যবহার
pipeline {
  agent any
  stages {
    stage('Publish Image') {
      steps {
        // REGISTRY_USER / REGISTRY_PASS শুধু এই ব্লকের ভেতরে থাকে
        withCredentials([usernamePassword(
          credentialsId: 'docker-registry-creds',   // Jenkins-এ তৈরি ID
          usernameVariable: 'REGISTRY_USER',
          passwordVariable: 'REGISTRY_PASS')]) {
          sh '''
            echo "$REGISTRY_PASS" | docker login \
              registry.technova.example -u "$REGISTRY_USER" --password-stdin
            docker push registry.technova.example/technova/web:${BUILD_NUMBER}
          '''
        }   // ব্লক শেষ — variable মুছে যায়
      }
    }
    stage('Deploy') {
      steps {
        withCredentials([string(
          credentialsId: 'deploy-api-token',
          variable: 'API_TOKEN')]) {
          sh 'curl -H "Authorization: Bearer $API_TOKEN" https://deploy.technova.example/hook'
        }
      }
    }
  }
}
```

ব্যাখ্যা: `credentialsId` — Jenkins-এ সংরক্ষিত credential-এর নাম; `--password-stdin` — পাসওয়ার্ড কমান্ড লাইনে নয়, stdin দিয়ে দেওয়া (process list-এ দেখা যায় না); `echo` দিয়ে secret print করবেন না — Jenkins masking করলেও `echo "$REGISTRY_PASS" | base64` করলে ফাঁস হয়।

Plugin: শক্তি ও ঝুঁকি

Plugin-ই Jenkins-কে সর্বজনীন বানায় — Git, Docker, Kubernetes, Slack, SonarQube — সব plugin দিয়ে আসে। কিন্তু প্রতিটি plugin: (১) আক্রমণের পৃষ্ঠতল (attack surface) বাড়ায়; (২) আপডেট না করলে পরিচিত দুর্বলতা থাকে; (৩) পরিত্যক্ত (abandoned) plugin-এ কেউ প্যাচ দেয় না; (৪) একটি plugin-এর ত্রুটি পুরো Controller থামিয়ে দিতে পারে। নিয়ম: শুধু প্রয়োজনীয় plugin ইনস্টল করুন; **Plugin Manager**-এ নিয়মিত আপডেট পরীক্ষা করুন; আপডেটের আগে ব্যাকআপ নিন ও staging Jenkins-এ পরীক্ষা করুন; Jenkins Security Advisory পড়ুন; অপ্রয়োজনীয় plugin মুছে ফেলুন।

অ্যাক্সেস নিয়ন্ত্রণ

Matrix-based security: প্রতিটি ব্যবহারকারী/group-এর জন্য আলাদা অনুমতি (Job/Build, Job/Configure, Overall/Administer)। **Role-Based Authorization Strategy plugin:** ভূমিকা (admin, developer, viewer) তৈরি করে ব্যবহারকারীকে ভূমিকায় বসানো — বড় দলে এটি পরিচালনা সহজ। **Project-based Matrix:** প্রতিটি job-এর নিজস্ব অনুমতি — সংবেদনশীল job (যেমন production deploy) শুধু নির্দিষ্ট দল দেখতে ও চালাতে পারে। নিয়ম: কাউকে Overall/Administer দেবেন না যদি না সে Jenkins পরিচালক হয়; job-এর Configure অনুমতি শুধু সেই job-এর মালিকদের দিন।

Untrusted কোডের ঝুঁকি — Pull Request

Pull Request-এর Jenkinsfile বা কোড আক্রমণকারীর হতে পারে। সেই কোড যদি production credential-সহ Agent-এ চলে, তবে আক্রমণকারী credential চুরি করতে পারে। প্রতিকার: (১) Fork-এর Pull Request-এ pipeline চালানোর আগে অনুমোদন নিন (GitHub Branch Source plugin-এর "trusted" সেটিং); (২) PR build-এ production credential দেবেন না — আলাদা, সীমিত credential ব্যবহার করুন; (৩) PR build আলাদা, বিচ্ছিন্ন Agent-এ চালান; (৪) Script Security-এর অনুমোদন ছাড়া Groovy পদ্ধতি ব্যবহার করবেন না। GitLab-এ protected variable-এর মতোই — Jenkins-এও অবিশ্বস্ত কোডকে বিশ্বস্ত secret দেবেন না।

ব্যাকআপ ও রক্ষণাবেক্ষণ

ব্যাকআপের তালিকা: `JENKINS_HOME` (job, ইতিহাস, credential, config) — প্রতিদিন; plugin তালিকা — `plugins.txt` হিসেবে সংরক্ষণ; Jenkinsfile Git-এই থাকে। পুনরুদ্ধার পরীক্ষা করুন — ব্যাকআপ নেওয়া হয়েছে কিন্তু ফেরানো যায়নি, এমন ব্যাকআপের কোনো মূল্য নেই। রক্ষণাবেক্ষণের ক্যালেন্ডার: সাপ্তাহিক — ডিস্ক ও build ইতিহাস পরীক্ষা; মাসিক — plugin ও Jenkins LTS আপডেট; ত্রৈমাসিক — credential rotation ও অনুমতি নিরীক্ষা; বার্ষিক — পুনরুদ্ধার মহড়া (disaster recovery drill)।

১৭.৭ বাস্তব পরিস্থিতি — TechNova

> **REAL WORLD — TechNova:** TechNova-র Jenkins-এ একজন প্রাক্তন কর্মীর অ্যাকাউন্ট মাসের পর মাস সক্রিয় ছিল — তার credential-এ production deploy-এর অনুমতি ছিল। একটি নিরীক্ষায় ধরা পড়ার পর নিয়ম হলো: কর্মী চলে গেলে সঙ্গে সঙ্গে অ্যাকাউন্ট নিষ্ক্রিয়; প্রতি ত্রৈমাসিকে অনুমতি নিরীক্ষা; production deploy job-এ শুধু দুইজন Maintainer-এর অনুমতি। একই নিরীক্ষায় দেখা গেল ১২টি plugin-এর মধ্যে ৪টির আপডেট বাকি — দুটিতে পরিচিত দুর্বলতা ছিল। শিক্ষা: নিরাপত্তা একবারের কাজ নয় — এটি নিয়মিত নিরীক্ষার অভ্যাস।

১৭.৮ ধাপে ধাপে বাস্তবায়ন

1. Manage Jenkins → Credentials-এ একটি Secret Text credential তৈরি করুন (মান: কাল্পনিক placeholder)।
2. উপরের Jenkinsfile-এর মতো `withCredentials` দিয়ে একটি পরীক্ষামূলক job-এ ব্যবহার করুন; লগে মানটি `****` দেখাচ্ছে কি না যাচাই করুন।
3. Matrix Authorization চালু করে একটি Viewer ব্যবহারকারী তৈরি করুন — সে job দেখতে পারে, কিন্তু চালাতে বা বদলাতে পারে না।
4. Plugin Manager-এ গিয়ে আপডেট বাকি আছে কি না দেখুন; একটি অপ্রয়োজনীয় plugin নিষ্ক্রিয় করুন।
5. `JENKINS_HOME`-এর ব্যাকআপ নিন ও অন্য VM-এ পুনরুদ্ধার পরীক্ষা করুন।
6. একটি credential rotate করুন — পুরোনোটি বাতিল করে নতুন ID-তে job আপডেট করুন।

১৭.৯ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "Credentials not found" — `credentialsId` ভুল বা scope মিলছে না; ID হুবহু মিলিয়ে লিখুন। **(২) লগে `****` দেখা যাচ্ছে কিন্তু build ব্যর্থ** — secret-এর মান ভুল বা মেয়াদ শেষ; মানটি বাইরে যাচাই করে rotate করুন। **(৩) "Access denied" / 403** — ব্যবহারকারীর অনুমতি নেই; Matrix-এ Job/Build অনুমতি আছে কি না দেখুন। **(৪) Plugin আপডেটের পর Jenkins চালু হচ্ছে না** — অসামঞ্জস্য; safe mode-এ চালু করে plugin-এর সংস্করণ ফিরিয়ে নিন। **(৫) Agent-এ credential কাজ করছে না** — credential-এর scope System নয় Global কি না, এবং job-এর Agent-এ নেটওয়ার্ক আছে কি না দেখুন।

১৭.১০ নিরাপত্তা বিবেচনা

WARNING: এই অধ্যায়ের সবচেয়ে গুরুত্বপূর্ণ নিয়মগুলো: (১) আসল secret কখনো Jenkinsfile, Git, লগ বা চ্যাটে নয় — শুধু Credential Store-এ; (২) Secret print করবেন না — `echo`, `printenv` বা error message-এ secret ফাঁস হতে পারে; (৩) Credential নিয়মিত rotate করুন — চুরি হলে ক্ষতি সীমিত হয়; (৪) Fork-এর PR-এ production credential দেবেন না; (৫) Jenkins-কে TLS ও reverse proxy-এর পেছনে রাখুন; (৬) Security Advisory পড়ুন ও দ্রুত প্যাচ দিন। কোনো secret ফাঁস হয়েছে সন্দেহ হলে সঙ্গে সঙ্গে বাতিল করুন — "হয়তো কেউ দেখেনি" ধরে নেবেন না।

১৭.১১ কর্মক্ষমতা ও খরচ বিবেচনা

নিরাপত্তার খরচ আছে — rotation, নিরীক্ষা ও আলাদা Agent সময় নেয়। কিন্তু একটি নিরাপত্তা ঘটনার খরচ তার চেয়ে অনেক বেশি। বাইরের secret store (যেমন Vault) ব্যবহার করলে পরিচালনার খরচ বাড়ে, তবে rotation ও নিরীক্ষা সহজ হয় — বড় দলে এটি বিনিয়োগের যোগ্য। Plugin সংখ্যা কম রাখলে Jenkins দ্রুত চালু হয় ও আপডেট সহজ হয়। Credential-এর সংখ্যাও কম রাখুন — প্রতিটি credential একটি ঝুঁকি; একই উদ্দেশ্যে একাধিক credential রাখবেন না।

১৭.১২ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: `withCredentials` দিয়ে সীমিত সময়ের জন্য secret ব্যবহার; নিয়মিত rotation; Role-Based অনুমতি; plugin আপডেট ও নিরীক্ষা; ব্যাকআপ ও পুনরুদ্ধার পরীক্ষা; PR build-এ সীমিত credential। **অ্যান্টি-প্যাটার্ন:** Jenkinsfile-এ secret লেখা; সবাইকে admin দেওয়া; plugin আপডেট না করা; ব্যাকআপ ছাড়া Controller চালানো; `printenv` দিয়ে সব variable লগে ছাপানো; প্রাক্তন কর্মীর অ্যাকাউন্ট সক্রিয় রাখা।

১৭.১৩ অধ্যায়ের সারাংশ

Jenkins নিরাপত্তা তিন স্তম্ভের ওপর দাঁড়িয়ে: কে কী করতে পারে (অ্যাক্সেস নিয়ন্ত্রণ), secret কীভাবে সংরক্ষিত ও ব্যবহৃত হয় (Credential Store + `withCredentials`), এবং বাইরের কোড কতটা বিশ্বাসযোগ্য (PR সুরক্ষা)। Plugin শক্তি দেয়, কিন্তু প্রতিটি plugin একটি ঝুঁকি — নিয়মিত আপডেট ও নিরীক্ষা ছাড়া Jenkins নিরাপদ থাকে না। ব্যাকআপ ও rotation — এই দুটি অভ্যাসই production Jenkins-কে টিকিয়ে রাখে। Part III এখানেই শেষ: GitLab-এর একীভূত pipeline ও Jenkins-এর স্ব-পরিচালিত pipeline — দুটোর স্থাপত্য, কনফিগারেশন ও নিরাপত্তা এখন আপনার হাতে। Part IV-এ এই ভিত্তির ওপর testing, security scanning ও release engineering শেখা হবে।

১৭.১৪ মূল প্রযুক্তিগত পরিভাষা

Credential Store: Jenkins-এ এনক্রিপ্ট করা secret সংরক্ষণের জায়গা।

withCredentials: সীমিত সময়ের জন্য secret পরিবেশ variable হিসেবে দেওয়ার ধাপ।

Secret Text: API token-এর মতো একটিমাত্র গোপন মানের credential ধরন।

Scope: Credential কোথায় ব্যবহার করা যাবে তার সীমা (Global / System)।

Matrix Authorization: প্রতিটি ব্যবহারকারীর জন্য আলাদা অনুমতির ছক।

Role-Based Access Control (RBAC): ভূমিকা অনুযায়ী অনুমতি দেওয়ার পদ্ধতি।

Script Security: অননুমোদিত Groovy পদ্ধতি আটকানোর Jenkins plugin।

Security Advisory: Jenkins ও plugin-এর দুর্বলতা সম্পর্কে অফিসিয়াল সতর্কবার্তা।

Rotation: নিয়মিতভাবে পুরোনো secret বাতিল করে নতুন secret দেওয়া।

Attack Surface: আক্রমণকারী যে সব পথে ব্যবস্থায় ঢুকতে পারে তার মোট পরিমাণ।

১৭.১৫ জ্ঞান যাচাই প্রশ্ন

1. `withCredentials` ব্লকের বাইরে secret কেন পাওয়া যায় না? এটি কী নিরাপত্তা দেয়?
2. Masking থাকা সত্ত্বেও secret কীভাবে ফাঁস হতে পারে? একটি উদাহরণ দিন।
3. Fork-এর Pull Request-এ production credential দেওয়া কেন বিপজ্জনক?
4. Matrix Authorization ও Role-Based Authorization-এর মধ্যে কোনটি বড় দলে ভালো এবং কেন?
5. Plugin আপডেটের আগে কী কী করা উচিত? কেন staging Jenkins-এ পরীক্ষা করবেন?

১৭.১৬ ব্যবহারিক অনুশীলন

1. **Credential পরীক্ষা:** একটি Secret Text credential তৈরি করে `withCredentials`-এ ব্যবহার করুন; লগে masking যাচাই করুন ও `printenv` দিয়ে ফাঁসের ঝুঁকি ব্যাখ্যা লিখুন।

2. **অনুমতি নিরীক্ষা:** তিনটি ভূমিকার (admin, developer, viewer) ব্যবহারকারী তৈরি করে প্রতিটির অনুমতি পরীক্ষা করুন; ফলাফল একটি ছকে লিখুন।
3. **ব্যাকআপ ও পুনরুদ্ধার:** `JENKINS_HOME`-এর ব্যাকআপ নিয়ে একটি job মুছে পুনরুদ্ধার করুন; পুনরুদ্ধারের সময় মাপুন ও পরিকল্পনায় লিখুন।

PART IV • DEVOPS ENGINEERING MASTERY

Testing, Security and Release Engineering

স্বয়ংক্রিয় টেস্টিং, কোড কোয়ালিটি, সিকিউরিটি স্ক্যানিং, Docker ইমেজ, রিলিজ ব্যবস্থাপনা ও ডেপ্লয়মেন্ট কৌশল।

অধ্যায় ১৮ • PART IV

Chapter 18 — Automated Testing in CI/CD Pipelines

১৮.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Unit, Integration, API ও End-to-End (E2E) টেস্টের পার্থক্য ও সীমা ব্যাখ্যা করতে; (২) Jest/Vitest দিয়ে Node.js অ্যাপের Unit ও API টেস্ট লিখতে; (৩) Playwright দিয়ে E2E টেস্ট চালাতে; (৪) টেস্ট পিরামিড অনুযায়ী পাইপলাইনে টেস্ট সাজাতে; (৫) Flaky টেস্ট চিনে স্থিতিশীল করতে; (৬) টেস্ট ব্যর্থ হলে পাইপলাইন আটকে দেওয়ার (Quality Gate) নকশা করতে।

১৮.২ কেন এটি গুরুত্বপূর্ণ

CI/CD-র মূল প্রতিশ্রুতি — "প্রতিদিন অনেকবার নিরাপদে রিলিজ" — টেস্ট ছাড়া মিথ্যা প্রতিশ্রুতি। টেস্ট না থাকলে পাইপলাইন শুধু ভাঙা কোড দ্রুত প্রোডাকশনে পৌঁছে দেয়। TechNova-র মতো MERN অ্যাপে এক লাইনের ভুলে চেকআউট ভেঙে গেলে ক্ষতি টাকায় মাপা যায়; অটোমেটেড টেস্ট সেই ভুলকে প্রোডাকশনের আগেই ধরে। মানুষের হাতে টেস্ট করা ধীর, ভুলপ্রবণ ও পুনরাবৃত্তিযোগ্য নয় — পাইপলাইনে টেস্ট মানে প্রতিটি কমিটে একই মানের যাচাই, স্বয়ংক্রিয়ভাবে।

১৮.৩ পূর্বশর্ত জ্ঞান

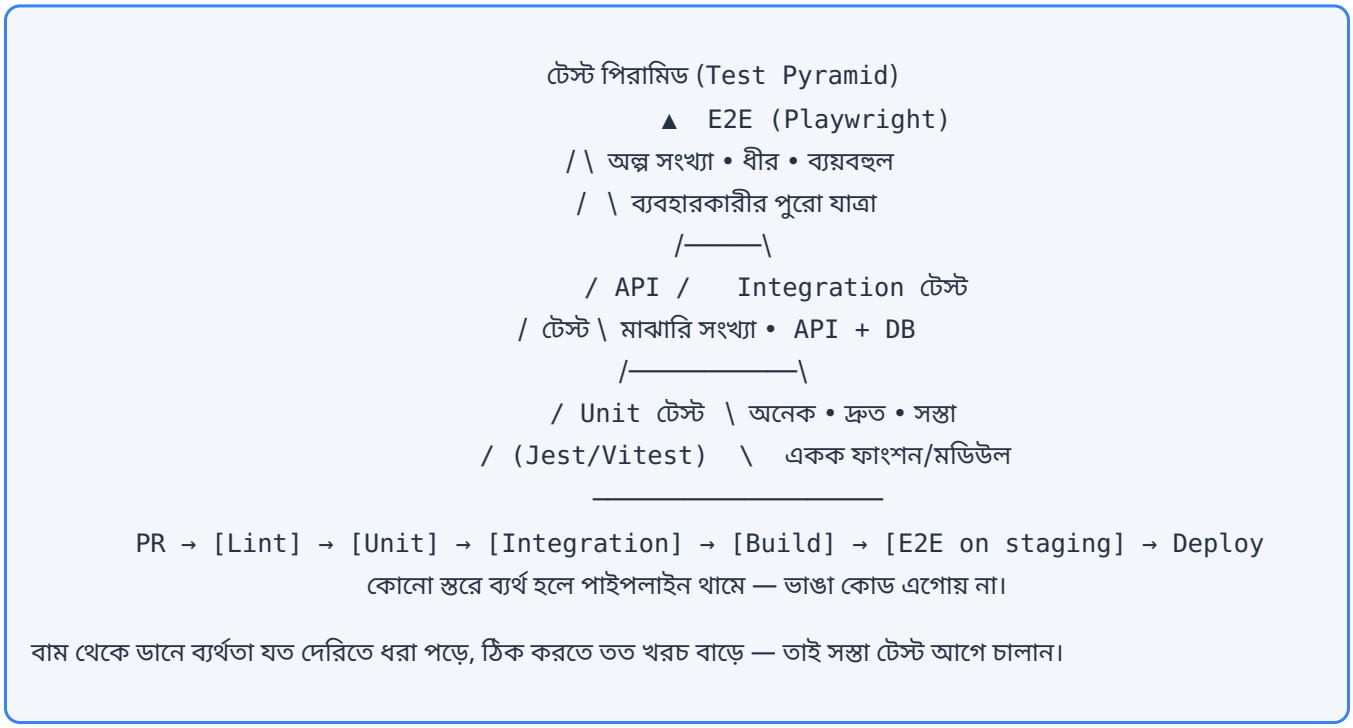
JavaScript/Node.js, Express রাউট, MongoDB-র মৌলিক ধারণা, Git ও আগের অধ্যায়ের GitHub Actions/GitLab/Jenkins পাইপলাইনের ধারণা থাকতে হবে। টেস্টিং ফ্রেমওয়ার্কের পূর্ব অভিজ্ঞতা প্রয়োজন নেই — Jest ও Vitest এখানেই পরিচয় করানো হবে।

১৮.৪ প্রথম নীতি থেকে ব্যাখ্যা

টেস্ট মানে একটি দাবি যাচাই করা: "এই ইনপুট দিলে এই আউটপুট আসবে।" কম্পিউটার এই যাচাই মানুষের চেয়ে দ্রুত ও নির্ভুলভাবে করতে পারে। প্রশ্ন শুধু — কী যাচাই করবেন, কতটা যাচাই করবেন, আর কত খরচে? একটি ফাংশন যাচাই করা সস্তা ও দ্রুত; পুরো ব্রাউজার চালিয়ে ব্যবহারকারীর পথ যাচাই করা ব্যয়বহুল ও ধীর। এই বাণিজ্য-বিনিময় (trade-off) বুঝলেই টেস্ট কৌশল বোঝা যায়। **টেস্ট পিরামিড** (Test Pyramid) বলে: নিচে অনেক ছোট দ্রুত Unit টেস্ট, মাঝে কম Integration টেস্ট, ওপরে অল্প E2E টেস্ট। উল্টো পিরামিড — অর্থাৎ শুধু E2E টেস্টের ওপর ভরসা — ধীর, ভঙ্গুর ও ব্যয়বহুল হয়; একে বলে Ice-Cream Cone অ্যান্টি-প্যাটার্ন।

> **CONCEPT:** টেস্ট পাস করা মানে "বাগ নেই" নয় — মানে "আমরা যা যা যাচাই করেছি, তাতে ভুল ধরা পড়েনি।" টেস্টের গুণগত মান নির্ভর করে আপনি কী প্রশ্ন করেছেন তার ওপর, কতগুলো টেস্ট আছে তার ওপর নয়।

১৮.৫ আর্কিটেকচার / ধারণাগত চিত্র



বাম থেকে ডানে ব্যর্থতা যত দেরিতে ধরা পড়ে, ঠিক করতে তত খরচ বাড়ে — তাই সস্তা টেস্ট আগে চালান।

১৮.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Unit টেস্ট — একক অংশের যাচাই

Unit টেস্ট একটি ফাংশন বা মডিউলকে আলাদা করে পরীক্ষা করে, বাইরের নির্ভরতা (ডেটাবেস, নেটওয়ার্ক) ছাড়া। Jest ও Vitest দুটোই Node.js-এ জনপ্রিয়; Vitest দ্রুত ও ESM-বান্ধব, Jest পরিণত ও ব্যাপক ইকোসিস্টেমসমৃদ্ধ। Mock (ভান করা নির্ভরতা) দিয়ে ডেটাবেস কলকে অনুকরণ করা হয়। ভালো Unit টেস্ট দ্রুত (মিলিসেকেন্ড), নির্ধারক (Deterministic — প্রতিবার একই ফল) ও স্বাধীন (একটির ব্যর্থতা অন্যকে প্রভাবিত করে না)।

Integration টেস্ট — অংশগুলোর সংযোগ

Integration টেস্ট যাচাই করে রাউট + সার্ভিস + ডেটাবেস একসঙ্গে ঠিকমতো কাজ করে কি না। Express অ্যাপে `supertest` দিয়ে HTTP অনুরোধ পাঠিয়ে আসল রাউট চালানো যায়, আর পরীক্ষার জন্য ইন-মেমরি বা আলাদা টেস্ট ডেটাবেস ব্যবহার করা হয়। এখানেই ধরা পড়ে ভুল স্কিমা, ভুল কুয়েরি বা ভুল কনফিগারেশনের বাগ — যা Unit টেস্টে ধরা পড়ে না।

API টেস্ট — চুক্তির যাচাই

API টেস্ট HTTP স্তরে চুক্তি (Contract) যাচাই করে: স্ট্যাটাস কোড, রেসপন্স আকৃতি, ভ্যালিডেশন ত্রুটি, অথেনটিকেশন। এটি ফ্রন্টএন্ড ও ব্যাকএন্ড দলের মধ্যে চুক্তি ভাঙা ধরে। Contract Testing (যেমন Pact) আরও এক ধাপ এগিয়ে — কনজিউমারের প্রত্যাশা প্রোভাইডারের বিরুদ্ধে যাচাই করে।

E2E টেস্ট — ব্যবহারকারীর চোখে

Playwright আসল ব্রাউজার (Chromium/Firefox/WebKit) চালিয়ে লগইন → পণ্য দেখা → কার্ট → চেকআউটের পুরো যাত্রা পরীক্ষা করে। সবচেয়ে বাস্তবসম্মত, কিন্তু সবচেয়ে ধীর ও ভঙ্গুর: UI-তে ছোট পরিবর্তনেই টেস্ট ভাঙতে পারে। তাই E2E অল্প রাখুন — শুধু মূল্যবান ব্যবহারকারী-যাত্রা (happy path) ও গুরুত্বপূর্ণ ব্যর্থতার পথ।

প্রতিটি ধরনের সীমা

ধরন	যা ধরে	যা ধরে না / সীমা
Unit	লজিকের ভুল, রিগ্রেশন	সংযোগের ভুল, ভুল কনফিগারেশন, আসল DB আচরণ
Integration	রাউট-সার্ভিস-DB সংযোগ	ব্রাউজার আচরণ, আসল নেটওয়ার্ক, লোডের আচরণ
API	চুক্তি ভাঙা, ভ্যালিডেশন	UI সমস্যা, পারফরম্যান্স, ডেটা মাইগ্রেশন
E2E	ব্যবহারকারীর পুরো যাত্রা	ধীর, ভঙ্গুর, ব্যর্থতার মূল কারণ খুঁজে পাওয়া কঠিন

কোনো এক ধরনের টেস্টই যথেষ্ট নয় — স্তরগুলো একসঙ্গে কাজ করলেই আত্মবিশ্বাস আসে।

Flaky টেস্ট ও Test Data

Flaky টেস্ট মানে কখনো পাস কখনো ফেল — কারণ সময়-নির্ভরতা, নেটওয়ার্ক, ভাগ করা ডেটাবেস বা অপর্যাপ্ত অপেক্ষা (wait)। Flaky টেস্ট দলের আস্থা নষ্ট করে: মানুষ ব্যর্থতাকে "আবার flaky" বলে উপেক্ষা করতে শেখে, আর তখন আসল বাগও উপেক্ষিত হয়। নীতি: flaky টেস্ট পেলে হয় ঠিক করুন, নয়তো quarantine (আলাদা) করে দ্রুত ঠিক করুন — কখনোই চুপচাপ রেখে দেবেন না। টেস্ট ডেটা প্রতিটি রানে আলাদা করুন (unique suffix, আলাদা DB) যাতে সমান্তরাল রান একে অপরকে নষ্ট না করে।

১৮.৭ কনফিগারেশন উদাহরণ — Jest Unit + API টেস্ট

```
// price.js — ব্যবসায়িক লজিক (পরীক্ষার বিষয়)
function finalPrice(amount, discountPct) {
  if (amount < 0 || discountPct < 0 || discountPct > 100)
    throw new Error('invalid input');
  return Math.round(amount * (1 - discountPct / 100));
}
module.exports = { finalPrice };

// price.test.js — Jest unit test
const { finalPrice } = require('./price');
describe('finalPrice', () => {
  test('10% ছাড়ে 1000 → 900', () => {
    expect(finalPrice(1000, 10)).toBe(900);
  });
  test('অবৈধ ইনপুটে ত্রুটি ছোড়ে', () => {
    expect(() => finalPrice(-5, 10)).toThrow('invalid input');
  });
  test('সীমান্ত মান: 0% ও 100% ছাড়', () => {
    expect(finalPrice(500, 0)).toBe(500);
    expect(finalPrice(500, 100)).toBe(0);
  });
});
```

```
// api.test.js — supertest দিয়ে Express API টেস্ট (integration/API স্তর)
const request = require('supertest');
const app = require('./app'); // টেস্টে আসল সার্ভার listen করে না
describe('GET /api/products', () => {
  test('পণ্যের তালিকা 200 ও array দেয়', async () => {
```

```
    const res = await request(app).get('/api/products');
    expect(res.status).toBe(200);
    expect(Array.isArray(res.body)).toBe(true);
  });
  test('অননুমোদিত অনুরোধে 401', async () => {
    const res = await request(app).get('/api/admin/orders');
    expect(res.status).toBe(401);
  });
});
```

```
# package.json — টেস্ট স্ক্রিপ্ট (Node.js 20 LTS)
{
  "scripts": {
    "test": "jest --ci --coverage",
    "test:unit": "jest price.test.js --ci",
    "test:api": "jest api.test.js --ci --runInBand",
    "test:e2e": "playwright test"
  }
}
# --ci মানে CI মোড: নতুন snapshot নেবে না, ব্যর্থতায় সঙ্গে সঙ্গে থামবে না,
# --runInBand মানে সমান্তরাল নয় — DB-ভাগ করা টেস্টে নিরাপদ।
```

```
# Playwright E2E — চেকআউটের মূল যাত্রা (tests/checkout.spec.js)
import { test, expect } from '@playwright/test';
test('ব্যবহারকারী লগইন করে চেকআউট সম্পূর্ণ করে', async ({ page }) => {
  await page.goto('https://staging.technova.example/login');
  await page.fill('[name=email]', 'test-user@technova.example');
  await page.fill('[name=password]', 'CHANGE-ME-TEST-PASSWORD');
  await page.click('button[type=submit]');
  await expect(page).toHaveURL(/\/products/);
  await page.click('text=Add to Cart');
  await page.click('text=Checkout');
  await expect(page.locator('text=Order confirmed')).toBeVisible();
});
```

১৮.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র MERN দোকানে (app.technova.example) ছাড়ের হিসাব ভুলে গ্রাহকরা ১০%-এর বদলে ৯০% ছাড় পাচ্ছিল — কারণ `finalPrice`-এ শতাংশ উল্টো প্রয়োগ হয়েছিল। Unit টেস্ট থাকলে PR-এই ধরা পড়ত। ঘটনার পর দল তিন স্তর যোগ করল: Unit টেস্ট (সীমান্ত মানসহ), API টেস্ট (চেকআউট রেসপন্স যাচাই) ও একটি E2E টেস্ট (পুরো চেকআউট)। এরপর থেকে ছাড়-সংক্রান্ত বাগ আর প্রোডাকশনে যায়নি। শিক্ষা: বাগের ধরন দেখে বুঝুন কোন স্তরের টেস্ট অনুপস্থিত ছিল।

১৮.৯ ধাপে ধাপে বাস্তবায়ন

1. টেস্ট পিরামিড আঁকুন: কোন ফিচারের কোন স্তরে টেস্ট লাগবে ঠিক করুন।
2. `npm install -D jest supertest` ও প্রয়োজনে `@playwright/test` যোগ করুন।
3. Unit টেস্ট লিখুন ব্যবসায়িক লজিকের জন্য; সীমান্ত মান (0, ঋণাত্মক, সর্বোচ্চ) অন্তর্ভুক্ত করুন।

4. API টেস্টে আলাদা টেস্ট ডেটাবেস ব্যবহার করুন; প্রোডাকশন DB কখনো নয়।
5. পাইপলাইনে ধাপ সাজান: lint → unit → integration/API → build → E2E (staging-এ)।
6. Coverage রিপোর্ট আর্টিফ্যাক্ট হিসেবে সংরক্ষণ করুন (অধ্যায় ১১-এর আর্টিফ্যাক্ট ধারণা)।
7. Flaky টেস্টের জন্য retry নয়, মূল কারণ ঠিক করুন; সাময়িকভাবে quarantine লেবেল দিন।

১৮.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "টেস্ট আমার মেশিনে পাস, CI-তে ফেল" — কারণ: সময় অঞ্চল, Node সংস্করণ, বা পরিবেশ ভেরিয়েবলের পার্থক্য। সমাধান: CI-তে Node সংস্করণ পিন করুন, টেস্টে UTC ব্যবহার করুন। (২) "DB সংযোগ নেই" — টেস্ট DB সার্ভিস (MongoDB service container) চালু আছে কি না দেখুন; সংযোগ স্ট্রিং টেস্ট পরিবেশের কি না যাচাই করুন। (৩) "E2E টেস্ট timeout" — সিলেক্টর ভঙ্গুর বা পেজ ধীরে লোড হচ্ছে; `expect().toBeVisible()`-এর মতো auto-waiting assertion ব্যবহার করুন, নির্দিষ্ট sleep নয়। (৪) "Coverage হঠাৎ কমে গেল" — নতুন কোডে টেস্ট যোগ হয়নি; coverage threshold ব্যর্থ হলে PR আটকান।

১৮.১১ নিরাপত্তা বিবেচনা

WARNING: টেস্টে কখনো আসল পাসওয়ার্ড, আসল API কি বা প্রোডাকশন ডেটাবেসের সংযোগ স্ট্রিং ব্যবহার করবেন না — টেস্ট ফাইল Git-এ যায়, আর Git ইতিহাস থেকে গোপন তথ্য মুছে ফেলা কঠিন। টেস্টের গোপন তথ্য CI secrets থেকে নিন, আর টেস্ট ডেটা কাল্পনিক রাখুন। E2E টেস্টে আসল পেমেন্ট গেটওয়ে চালাবেন না — sandbox/test মোড ব্যবহার করুন।

১৮.১২ পারফরম্যান্স ও খরচ

Unit টেস্ট সস্তা — সেকেন্ডে হাজারো চলে। E2E ব্যয়বহুল — প্রতিটি রানে ব্রাউজার, staging পরিবেশ ও সময় লাগে। কৌশল: PR-এ দ্রুত স্তরগুলো (lint + unit + API) চালান; E2E শুধু main শাখায় বা staging ডেপ্লয়ের পরে চালান। টেস্ট সমান্তরালকরণ (sharding) সময় কমায় কিন্তু খরচ বাড়ায় — মাপজোখ করে সিদ্ধান্ত নিন। Coverage ১০০% তাড়া করা ব্যয়বহুল ও প্রায়শই অর্থহীন; গুরুত্বপূর্ণ পথে উচ্চ coverage-ই লক্ষ্য।

১৮.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: টেস্ট পিরামিড মেনে চলুন; টেস্টকে প্রোডাকশন কোডের মতো যত্নে লিখুন; ব্যর্থ টেস্টকে সঙ্গে সঙ্গে ঠিক করুন; প্রতিটি বাগ ঠিক করার আগে তার পুনরুৎপাদনকারী টেস্ট লিখুন (regression test)। **অ্যান্টি-প্যাটার্ন:** শুধু E2E-র ওপর ভরসা; ব্যর্থ টেস্ট উপেক্ষা করে "retry" দিয়ে পাস করানো; টেস্টে `sleep(5000)` লিখে সময়ের ওপর ভরসা; coverage সংখ্যা বাড়াতে অর্থহীন টেস্ট লেখা।

১৮.১৪ অধ্যায়ের সারাংশ

অটোমেটেড টেস্ট CI/CD-র নিরাপত্তা-জাল: Unit টেস্ট লজিক ধরে, Integration/API টেস্ট সংযোগ ও চুক্তি ধরে, E2E ব্যবহারকারীর যাত্রা ধরে। প্রতিটি ধরনের সীমা আছে — তাই পিরামিড আকারে একসঙ্গে ব্যবহার করুন, ব্যর্থতায় পাইপলাইন থামান, flaky টেস্টকে অবহেলা করবেন না, আর টেস্টে কখনো আসল গোপন তথ্য ব্যবহার করবেন না।

১৮.১৫ মূল প্রযুক্তিগত পরিভাষা

Unit Test: একটি ফাংশন/মডিউলের আলাদা যাচাই।

Integration Test: একাধিক অংশ একসঙ্গে কাজ করে কি না তার যাচাই।

API Test / Contract Test: HTTP চুক্তি ও রেসপন্স আকৃতির যাচাই।

E2E Test: আসল ব্রাউজারে ব্যবহারকারীর পুরো যাত্রার যাচাই।

Test Pyramid: অনেক Unit, মাঝারি Integration, অল্প E2E — টেস্ট সাজানোর নকশা।

Flaky Test: কখনো পাস কখনো ফেল করা অনির্ধারক টেস্ট।

Mock: পরীক্ষার জন্য আসল নির্ভরতার ভান করা বিকল্প।

Code Coverage: টেস্টে কোডের কত শতাংশ চলেছে তার পরিমাপ।

Quality Gate: শর্ত পূরণ না হলে পাইপলাইন এগোতে না দেওয়ার নিয়ম।

Regression Test: পুরোনো বাগ আবার ফিরে আসেনি তা যাচাই করা টেস্ট।

১৮.১৬ জ্ঞান যাচাই প্রশ্ন

1. টেস্ট পিরামিডে Unit টেস্ট সবচেয়ে নিচে ও সবচেয়ে বেশি কেন থাকে?
2. একটি বাগ Unit টেস্টে ধরা না পড়ে Integration টেস্টে ধরা পড়ার একটি উদাহরণ দিন।
3. Flaky টেস্ট কেন শুধু বিরক্তিকর নয়, নিরাপত্তা-ঝুঁকিও?
4. E2E টেস্টে নির্দিষ্ট `sleep` ব্যবহার কেন ভঙ্গুর?
5. Coverage ১০০% হলেও বাগ থাকতে পারে কেন?

১৮.১৭ ব্যবহারিক অনুশীলন

1. `finalPrice`-এর মতো একটি ব্যবসায়িক ফাংশন লিখে সীমান্ত মানসহ ৫টি Jest টেস্ট লিখুন; একটি ইচ্ছাকৃত ভুল ঢুকিয়ে টেস্ট ব্যর্থ হতে দেখুন।

2. supertest দিয়ে একটি Express রাউটের সফল ও অননুমোদিত — দুই ক্ষেত্রেই API টেস্ট লিখুন।

3. Playwright দিয়ে আপনার অ্যাপের লগইন যাত্রার একটি E2E টেস্ট লিখুন এবং CI-তে চালানোর ধাপ নোট করুন।

অধ্যায় ১৯ • PART IV

Chapter 19 — Code Quality, Linting, Static Analysis and Security Scanning

১৯.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Linting, Formatting, Type Checking ও Static Analysis (SAST)-এর পার্থক্য ব্যাখ্যা করতে; (২) ESLint ও TypeScript দিয়ে কোড গুণমান পরীক্ষা করতে; (৩) Dependency, Secret ও Container স্ক্যানিং পাইপলাইনে যোগ করতে; (৪) Quality Gate নকশা করতে; (৫) স্ক্যানের ফলাফল ব্যাখ্যা করে false positive থেকে আসল ঝুঁকি আলাদা করতে।

১৯.২ কেন এটি গুরুত্বপূর্ণ

টেস্ট বলে "কোড কাজ করে কি না"; গুণমান-পরীক্ষা বলে "কোড নিরাপদ, পড়ার মতো ও রক্ষণাবেক্ষণযোগ্য কি না।" একটি অ্যাপ সব টেস্ট পাস করেও SQL injection-এর মতো দুর্বলতা নিয়ে প্রোডাকশনে যেতে পারে, বা এমন জটিল কোড নিয়ে যেতে পারে যা কেউ ঠিক করতে পারে না। TechNova-র অভিজ্ঞতা: একবার একটি ডিপেন্ডেন্সিতে জানা দুর্বলতা থাকা সত্ত্বেও ডেপ্লয় হয়েছিল — কারণ কেউ স্ক্যানই চালায়নি। গুণমান-গেট পাইপলাইনে থাকলে সেই ডেপ্লয় আটকে যেত।

১৯.৩ পূর্বশর্ত জ্ঞান

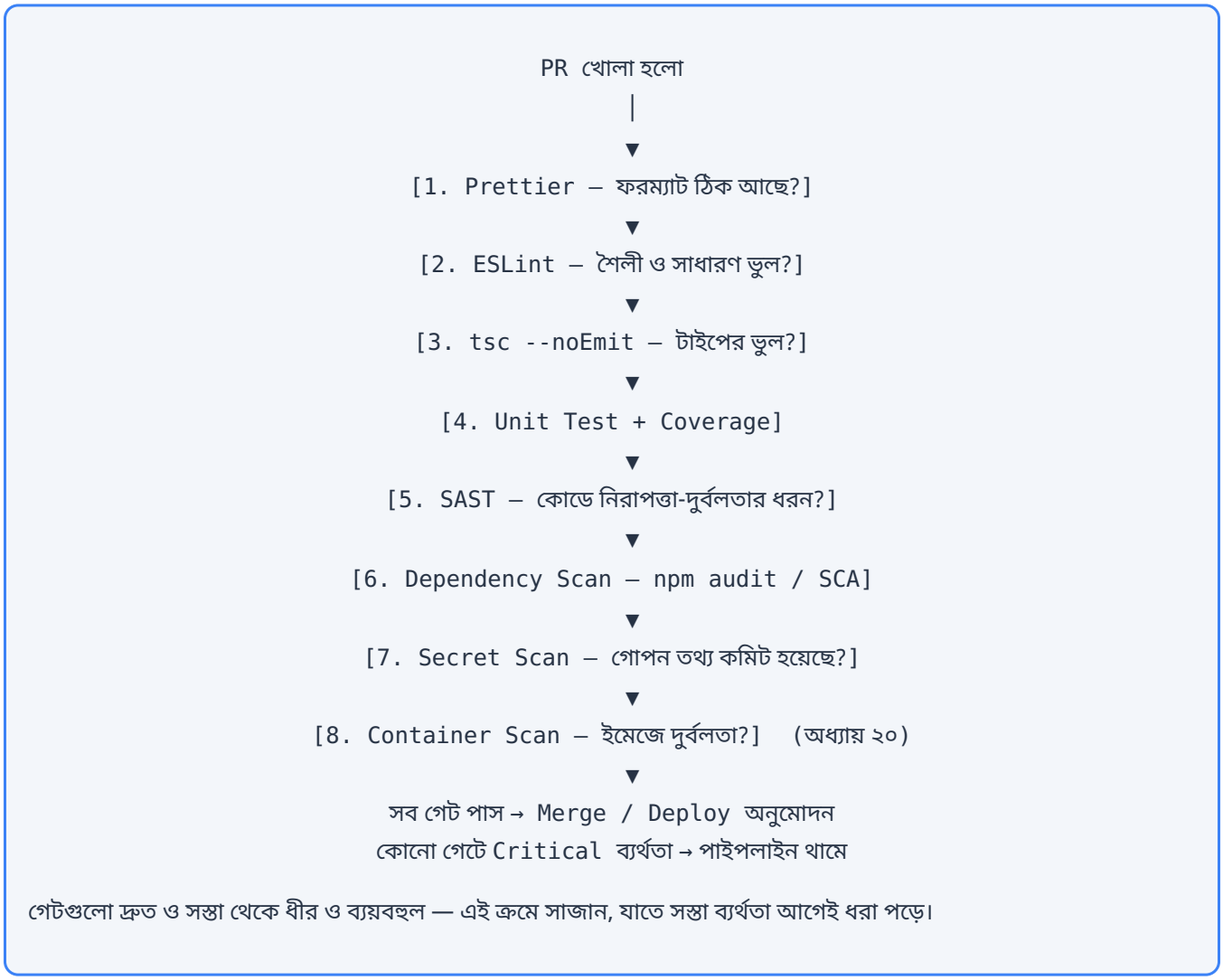
অধ্যায় ১৮-এর টেস্টিং ধারণা, npm ও `package.json`, TypeScript-এর প্রাথমিক ধারণা এবং CI পাইপলাইনের ধাপ সাজানোর অভিজ্ঞতা থাকতে হবে।

১৯.৪ প্রথম নীতি থেকে ব্যাখ্যা

কোড চালানোর আগেই কোড পড়ে ভুল খোঁজা যায় — এটাই Static Analysis। কম্পাইলার যেমন বানান ভুল ধরে, তেমনি linter শৈলীর ভুল ধরে, type checker ধরনের ভুল ধরে, SAST (Static Application Security Testing) নিরাপত্তার ধরন ধরে। প্রতিটি সরঞ্জাম কোডকে একটি ভিন্ন প্রশ্ন করে: "এটা কি সুসংগত?", "এটা কি ধরন-নিরাপদ?", "এটা কি আক্রমণযোগ্য?" — আর Dependency Scanning প্রশ্ন করে: "আপনি যেসব লাইব্রেরি ব্যবহার করছেন, সেগুলোতে কি জানা দুর্বলতা আছে?" কোনো সরঞ্জামই সব প্রশ্নের উত্তর দেয় না; স্তরে স্তরে সাজানোই কৌশল।

> **CONCEPT:** Linting শৈলী ও সাধারণ ভুল ধরে; SAST নিরাপত্তা-দুর্বলতার ধরন ধরে; Dependency Scanning জানা দুর্বলতার তালিকা (CVE) মেলায়। তিনটি আলাদা কাজ — একটিকে অন্যটির বদলে ব্যবহার করবেন না।

১৯.৫ আর্কিটেকচার / ধারণাগত চিত্র



গেটগুলো দ্রুত ও সস্তা থেকে ধীর ও ব্যয়বহুল — এই ক্রমে সাজান, যাতে সস্তা ব্যর্থতা আগেই ধরা পড়ে।

১৯.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

ESLint ও Prettier

ESLint নিয়ম-ভিত্তিক যাচাই: অব্যবহৃত ভেরিয়েবল, `==`-এর বদলে `===` না ব্যবহার, অনিরাপদ `eval` ইত্যাদি ধরে। Prettier শুধু ফরম্যাট ঠিক করে — ইন্ডেন্ট, কোট, সেমিকোলন — যাতে PR-এ ফরম্যাট নিয়ে তর্ক না হয়। দুটোকে আলাদা রাখুন: Prettier ফরম্যাট করে, ESLint ভুল ধরে।

TypeScript — `tsc --noEmit`

TypeScript ধরন-পরীক্ষা (type checking) চালালে ভুল ধরনের ডেটা পাঠানো, `undefined`-এর ওপর প্রপার্টি পড়া — এই ধরনের রানটাইম ত্রুটি আগেই ধরা পড়ে। `strict: true` চালু করলে সুরক্ষা বাড়ে, তবে শুরুতে অনেক ত্রুটি আসতে পারে; ধীরে ধীরে কঠোর করুন। মনে রাখবেন: TypeScript কম্পাইলের সময়ের নিরাপত্তা দেয়, রানটাইমে নয় — বাইরে থেকে আসা ডেটা (API ইনপুট) সবসময় runtime-এ যাচাই (validation) করতে হবে, যেমন Zod দিয়ে।

SAST — কোডে নিরাপত্তা-দুর্বলতার ধরন

SAST সরঞ্জাম (যেমন CodeQL, Semgrep) কোডে আক্রমণের ধরন খোঁজে: SQL injection, XSS, পথ-ভ্রমণ (path traversal), অনিরাপদ `crypto` ব্যবহার। এটি ডেটা-প্রবাহ (taint tracking) অনুসরণ করে: ব্যবহারকারীর ইনপুট কোথা থেকে কোথায় যায়। SAST-এর সীমা: এটি সম্ভাব্য দুর্বলতা দেখায়, প্রমাণ নয় — false positive হয়। তাই প্রতিটি ফলাফল মানুষের পর্যালোচনা লাগে।

Dependency Scanning ও SCA

Software Composition Analysis (SCA) আপনার ডিপেন্ডেন্সি-গাছের প্রতিটি প্যাকেজকে জানা দুর্বলতার তালিকার (CVE ডেটাবেস) সঙ্গে মেলায়। `npm audit` npm-এর নিজস্ব সরঞ্জাম; Dependabot/Renovate স্বয়ংক্রিয়ভাবে দুর্বল প্যাকেজ আপডেটের PR তোলে। শুধু direct dependency নয়, transitive dependency-তেও দুর্বলতা লুকিয়ে থাকে — তাই lockfile (`package-lock.json`) স্ক্যান করাই সঠিক।

Secret Scanning

Secret scanning কমিট করা API কি, টোকেন, পাসওয়ার্ড খোঁজে (যেমন Gitleaks, TruffleHog, GitHub-এর নিজস্ব secret scanning)। একবার Git ইতিহাসে গোপন তথ্য গেলে তা মুছে ফেলা কঠিন — তাই প্রতিরোধই (pre-commit hook + CI স্ক্যান) সেরা। ধরা পড়লে কি-টি সঙ্গে সঙ্গে revoke/rotate করুন; শুধু ফাইল থেকে মুছে ফেলা যথেষ্ট নয়।

Quality Gate — কখন আটকাবেন

ফলাফল	গেটের আচরণ
Critical নিরাপত্তা দুর্বলতা	পাইপলাইন ব্যর্থ — merge/ডেপ্লয় বন্ধ
High দুর্বলতা	ব্যর্থ, তবে নথিভুক্ত ব্যতিক্রম (exception) থাকলে সাময়িক ছাড়
Medium/Low	সতর্কতা (warning) — নির্দিষ্ট সময়ে ঠিক করার দায়িত্ব
Coverage কমে যাওয়া	PR-এ সতর্কতা; threshold ভাঙলে ব্যর্থ
False positive	নথিভুক্ত করে suppress — চুপচাপ নয়

গেট খুব কঠোর হলে দল তাকে এড়িয়ে যেতে শেখে; খুব ঢিলা হলে অর্থহীন হয়। মাঝপথ বেছে নিন ও নিয়মিত পর্যালোচনা করুন।

১৯.৭ কনফিগারেশন উদাহরণ

```
# ESLint — ফ্ল্যাট কনফিগ (eslint.config.js, ESLint 9)
import js from '@eslint/js';
export default [
  js.configs.recommended,
  {
    rules: {
      'no-unused-vars': 'error',
      'no-eval': 'error',
      'eqeqeq': ['error', 'always'],
      'no-console': ['warn', { allow: ['warn', 'error'] }]
    }
  }
```

```
];
# চালানো: npx eslint .        ঠিক করা: npx eslint . --fix
```

```
# package.json — গুণমান স্ক্রিপ্ট
{
  "scripts": {
    "lint": "eslint .",
    "format:check": "prettier --check .",
    "typecheck": "tsc --noEmit",
    "audit": "npm audit --audit-level=high",
    "scan:secrets": "gitleaks detect --no-git -v"
  }
}
# npm audit --audit-level=high মানে high ও critical দুর্বলতা থাকলে ব্যর্থ হবে।
```

```
# GitHub Actions — গুণমান গেট (আংশিক; সম্পূর্ণ workflow অধ্যায় ২৩-এ)
quality:
  runs-on: ubuntu-latest
  permissions:
    contents: read
    security-events: write   # SARIF আপলোডের জন্য
  steps:
    - uses: actions/checkout@v4
    - uses: actions/setup-node@v4
      with: { node-version: '20', cache: 'npm' }
    - run: npm ci
    - run: npm run lint
    - run: npm run typecheck
    - run: npm run test -- --coverage
    - run: npm audit --audit-level=high
    - uses: github/codeql-action/init@v3
      with: { languages: javascript-typescript }
    - uses: github/codeql-action/analyze@v3
# CodeQL SAST ফলাফল GitHub-এর Security ট্যাবে SARIF আকারে যায়।
```

```
# Semgrep — হালকা SAST, CI-তে চালানো যায়
# semgrep ci           # CI পরিবেশে স্বয়ংক্রিয় নিয়ম
# semgrep --config auto --error   # ব্যর্থতা হলে exit code 1
# .semgrepignore-এ test ফাইল বাদ দিলে শব্দ (noise) কমে।
```

১৯.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র এক ডেভেলপার ভুল করে `.env` ফাইলে আসল MongoDB Atlas পাসওয়ার্ডসহ কমিট করে ফেলেছিলেন। Secret scanning না থাকায় সেটা main শাখায় চলে যায়। পরের দিন দল Gitleaks CI-তে যোগ করে, Atlas পাসওয়ার্ড rotate করে, পুরোনো কি revoke করে এবং pre-commit hook-এ secret চেক বসায়। ক্ষতি হয়নি — কিন্তু শিক্ষা: গোপন তথ্য ফাঁসের পর "মুছে ফেলেছি" যথেষ্ট নয়; কি-টি বাতিল করতেই হবে, কারণ Git ইতিহাস যে কেউ clone করে দেখতে পারে।

১৯.৯ ধাপে ধাপে বাস্তবায়ন

1. ESLint + Prettier বসান; `npm run lint` ও `format:check` PR-এ বাধ্যতামূলক করুন।
2. TypeScript প্রকল্পে `tsc --noEmit` CI-তে যোগ করুন; `strict` ধাপে ধাপে কঠোর করুন।
3. `npm audit --audit-level=high` দিয়ে dependency গেট বসান; Dependabot চালু করুন।
4. Gitleaks বা সমতুল্য secret scanning CI-তে ও pre-commit hook-এ যোগ করুন।
5. CodeQL বা Semgrep দিয়ে SAST চালু করুন; প্রথম সপ্তাহে শুধু warning, তারপর গেট কঠোর করুন।
6. প্রতিটি গেটের ব্যর্থতার মালিক ও ঠিক করার সময়সীমা নথিভুক্ত করুন।

১৯.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "ESLint হাজারো ত্রুটি দেখাচ্ছে" — সম্ভবত পুরোনো কোডে একসঙ্গে কঠোর নিয়ম চালু হয়েছে। সমাধান: নতুন কোডে কঠোর নিয়ম, পুরোনো কোডে ধাপে ধাপে ঠিক করা; `--fix` দিয়ে স্বয়ংক্রিয় ঠিকযোগ্য অংশ সারুন। (২) "npm audit-এ শুধু transitive দুর্বলতা" — direct আপডেটে না সারলে `npm audit fix` সাবধানে চালান, টেস্ট চালিয়ে নিশ্চিত হোন; কখনোই অন্ধভাবে `--force` নয়। (৩) "SAST-এ অনেক false positive" — নিয়মের সেট কমান, suppress-এর কারণ লিখে রাখুন, আর সত্যিকারের দুর্বলতা চিনতে ডেটা-প্রবাহ পড়তে শিখুন। (৪) "Secret স্ক্যানে টেস্টের ভুয়া কি ধরা পড়ল" — টেস্টের ভুয়া গোপন তথ্যে স্পষ্ট চিহ্ন দিন (যেমন `TEST-ONLY-FAKE-KEY`) ও allowlist-এ নথিভুক্ত করুন।

১৯.১১ নিরাপত্তা বিবেচনা

WARNING: স্ক্যান সরঞ্জাম নিজেই আস্থার বিষয়: তৃতীয় পক্ষের action বা স্ক্যানারকে অন্ধভাবে `contents: write` অনুমতি দেবেন না — অধ্যায় ১০-এর least-privilege নীতি এখানেও প্রযোজ্য। স্ক্যান ফলাফলে কখনো গোপন তথ্য ছাপাবেন না (লগে কি ছাপানো নিষেধ)। আর মনে রাখবেন: SAST/SCA পাস মানেই নিরাপদ নয় — অজানা (zero-day) দুর্বলতা ও যুক্তির ভুল (business-logic flaw) স্ক্যানার ধরে না; মানুষের code review তার বিকল্প নয়।

১৯.১২ পারফরম্যান্স ও খরচ

ESLint/Prettier/tsc সস্তা ও দ্রুত — প্রতিটি PR-এ চালান। CodeQL-এর মতো গভীর SAST ধীর — main শাখায় বা দিনে একবার চালানো যুক্তিসংগত হতে পারে। Dependency স্ক্যান প্রায় বিনামূল্যে, তবে দুর্বলতা ঠিক করার সময়ই আসল খরচ। একটি কৌশল: PR-এ শুধু পরিবর্তিত কোডের স্ক্যান (diff-aware) চালিয়ে শব্দ কমান, আর পূর্ণ স্ক্যান রাতের scheduled workflow-তে চালান।

১৯.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: গেট ধাপে ধাপে কঠোর করুন (warn → fail); প্রতিটি suppress-এর কারণ ও মেয়াদ লিখুন; Dependabot-এর PR নিয়মিত পর্যালোচনা করুন; স্ক্যান ফলাফল Security ট্যাবে কেন্দ্রীভূত করুন। **অ্যান্টি-প্যাটার্ন:** সব সতর্কতা একসঙ্গে fail করে দলকে অচল করা; false positive-কে চুপচাপ উপেক্ষা করা; স্ক্যান পাসকে "সম্পূর্ণ নিরাপদ" মনে করা; lockfile কমিট না করে স্ক্যানের ভিত্তি নষ্ট করা।

১৯.১৪ অধ্যায়ের সারাংশ

গুণমান-গেট হলো পাইপলাইনের ফিল্টার: Linting শৈলী, TypeScript ধরন, SAST কোডের নিরাপত্তা-ধরন, SCA ডিপেন্ডেন্সির জানা দুর্বলতা, আর Secret Scanning ফাঁস হওয়া গোপন তথ্য ধরে। একটিও একা যথেষ্ট নয়; স্তরে সাজিয়ে, ধাপে ধাপে কঠোর করে, false positive নথিভুক্ত করে ব্যবহার করলেই গেট দলকে ধীর না করে নিরাপদ রাখে।

১৯.১৫ মূল প্রযুক্তিগত পরিভাষা

Linting (ESLint): নিয়ম দিয়ে কোডের শৈলী ও সাধারণ ভুল যাচাই।

Static Analysis / SAST: কোড না চালিয়ে পড়ে দুর্বলতার ধরন খোঁজা।

Type Checking: ধরন-সংক্রান্ত ভুল কম্পাইলের সময় ধরা।

SCA: Software Composition Analysis — ডিপেন্ডেন্সির দুর্বলতা মেলানো।

CVE: Common Vulnerabilities and Exposures — জানা দুর্বলতার তালিকার পরিচয়।

Secret Scanning: কমিট করা গোপন তথ্য খোঁজার স্বয়ংক্রিয় পরীক্ষা।

False Positive: সরঞ্জাম ভুল করে দুর্বলতা বলে চিহ্নিত করা নিরাপদ কোড।

Quality Gate: নির্দিষ্ট মান না পেরোলে পাইপলাইন আটকে দেওয়ার নিয়ম।

Dependabot: দুর্বল ডিপেন্ডেন্সি আপডেটের স্বয়ংক্রিয় PR তোলার সরঞ্জাম।

SARIF: Static Analysis Results Interchange Format — স্ক্যান ফলাফলের আদর্শ ফরম্যাট।

১৯.১৬ জ্ঞান যাচাই প্রশ্ন

1. Linting, SAST ও Dependency Scanning — প্রতিটি কী ভিন্ন প্রশ্নের উত্তর দেয়?
2. TypeScript type checking থাকলেও API ইনপুট runtime-এ যাচাই করতে হয় কেন?
3. Secret ফাঁসের পর শুধু ফাইল থেকে মুছে ফেলা যথেষ্ট নয় কেন?
4. False positive-কে চুপচাপ উপেক্ষা করা কেন বিপজ্জনক?
5. Quality Gate খুব কঠোর হলে দল কীভাবে তাকে এড়িয়ে যেতে শেখে?

১৯.১৭ ব্যবহারিক অনুশীলন

1. একটি Node.js প্রকল্পে ESLint + Prettier বসিয়ে ইচ্ছাকৃত শৈলী-ভুল ঢুকিয়ে CI গেট ব্যর্থ হতে দেখুন, তারপর ঠিক করুন।
2. `npm audit` চালিয়ে একটি high দুর্বলতা খুঁজে বের করুন ও আপডেটের পর টেস্ট চালিয়ে নিশ্চিত হোন।
3. Gitleaks দিয়ে একটি ভুয়া কি-সহ কমিট স্ক্যান করুন; ধরা পড়ার পর সঠিক প্রতিকার (rotate + hook) নথিভুক্ত করুন।

অধ্যায় ২০ • PART IV

Chapter 20 — Docker Image Build, Tagging, Scanning and Registry Publishing

২০.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Docker image build lifecycle ব্যাখ্যা করতে; (২) Multi-stage Dockerfile লিখতে; (৩) ট্যাগ বনাম digest-এর পার্থক্য ও immutable artifact-এর গুরুত্ব বোঝাতে; (৪) Registry-তে authenticate করে image publish করতে; (৫) Image scanning পাইপলাইনে যোগ করতে; (৬) Build reproducibility ও supply-chain নিরাপত্তা (SLSA) ব্যাখ্যা করতে।

২০.২ কেন এটি গুরুত্বপূর্ণ

CI পাইপলাইনের আউটপুট কোড নয় — artifact। Docker image-ই সেই artifact যা staging থেকে production পর্যন্ত একইভাবে চলে। Image যদি প্রতিবার আলাদা হয়, ট্যাগ যদি `latest`-এর মতো বদলে যায়, বা স্ক্যান না করা image ডেপ্লয় হয় — তবে "আমার মেশিনে চলেছিল" সমস্যা আবার ফিরে আসে, এবার কনটেইনার আকারে। TechNova-র একবার staging-এ পাস করা image-ই production-এ ভিন্ন আচরণ করেছিল — কারণ দুই পরিবেশে দুই ভিন্ন `latest` ট্যাগ pull হয়েছিল। শিক্ষা: যা পরীক্ষা করেছেন, হুবহু সেটাই ডেপ্লয় করুন — digest দিয়ে।

২০.৩ পূর্বশর্ত জ্ঞান

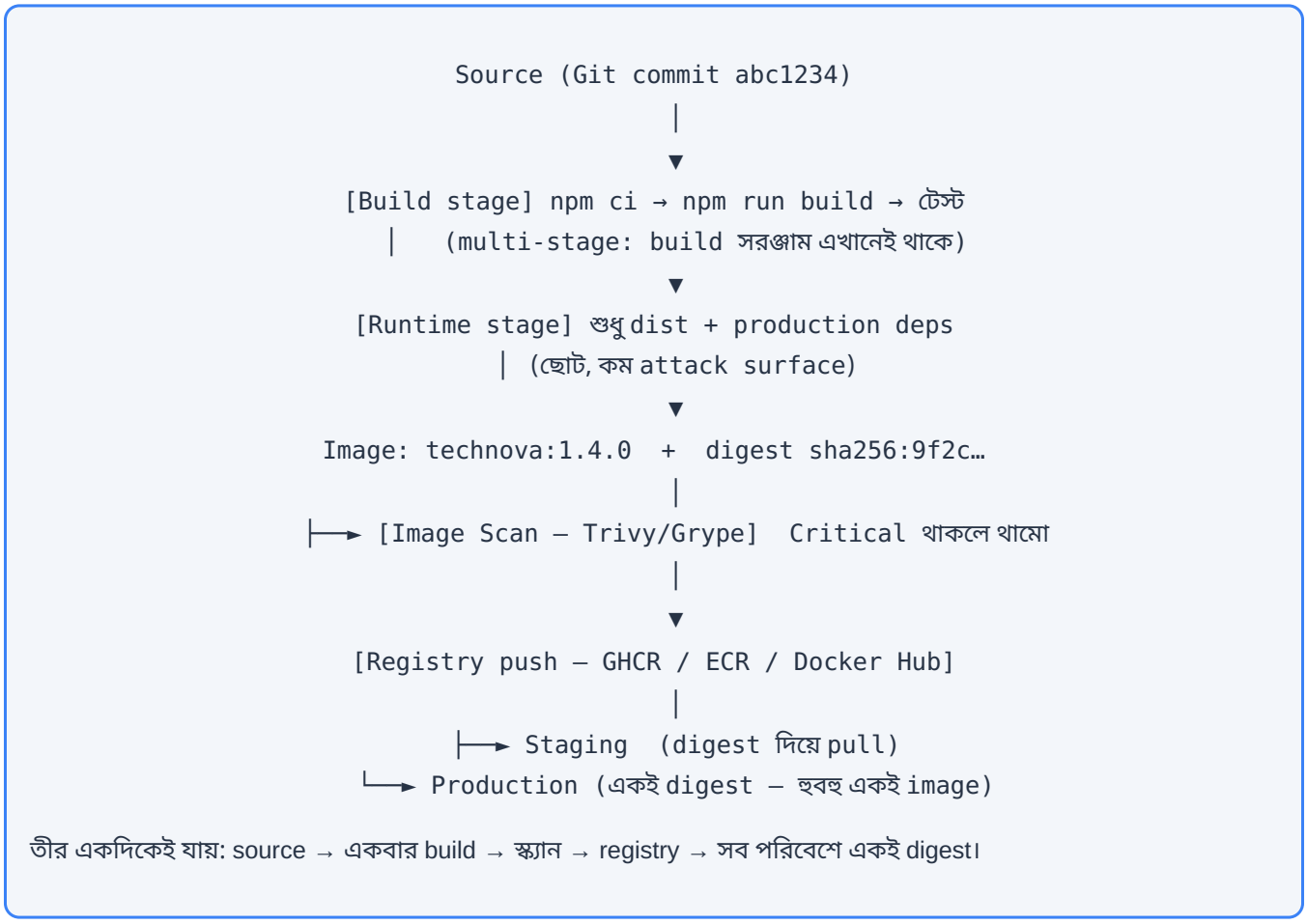
Docker-এর মৌলিক ধারণা (image, container, Dockerfile), অধ্যায় ১৮–১৯-এর টেস্ট ও স্ক্যানিং, এবং registry (যেমন Docker Hub, GitHub Container Registry, AWS ECR) সম্পর্কে প্রাথমিক ধারণা থাকতে হবে।

২০.৪ প্রথম নীতি থেকে ব্যাখ্যা

Docker image হলো স্তরে স্তরে (layer) সাজানো একটি অপরিবর্তনীয় (immutable) প্যাকেজ: প্রতিটি Dockerfile নির্দেশ একটি নতুন layer যোগ করে। Layer ক্যাশ (cache) হওয়ার কারণে একই নির্দেশ আবার চালালে পুনর্নির্মাণ হয় না — এটাই build দ্রুত করে, আবার ভুল ক্রমে সাজালে ক্যাশ নষ্ট হয়ে ধীর করে। Image-র পরিচয় দুভাবে: **ট্যাগ** — মানুষের পড়ার মতো নাম (`technova:1.4.0`), যা বদলানো যায়; আর **digest** — কনটেন্টের হ্যাশ (`sha256:...`), যা বদলানো যায় না। ট্যাগ মানে "এই নামের সর্বশেষ যা আছে", digest মানে "হুবহু এই বাইটগুলো"। প্রোডাকশনে বিশ্বাসযোগ্য পরিচয় শুধু digest।

> **CONCEPT:** "Build once, deploy many times" — একবার build করে একই artifact সব পরিবেশে চালান। প্রতিটি পরিবেশে নতুন করে build করলে আপনি আর একই জিনিস পরীক্ষা করছেন না।

২০.৫ আর্কিটেকচার / ধারণাগত চিত্র



তীর একদিকেই যায়: source → একবার build → স্ক্যান → registry → সব পরিবেশে একই digest।

২০.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Multi-stage Build

প্রথম stage-এ build সরঞ্জাম (TypeScript compiler, dev dependencies) থাকে; শেষ stage-এ শুধু চলমান অ্যাপ ও production dependency কপি করা হয়। ফলে চূড়ান্ত image ছোট হয়, attack surface কমে, আর গোপন তথ্য বা build সরঞ্জাম image-এ থাকে না। স্তরের ক্রম গুরুত্বপূর্ণ: কম বদলানো জিনিস (dependency ইনস্টল) আগে, বেশি বদলানো জিনিস (source কপি) পরে — যাতে ক্যাশ বেশি কাজ করে।

ট্যাগিং কৌশল

ট্যাগ ধরন	উদাহরণ	ব্যবহার
Semantic version	`1.4.0`	রিলিজ চিহ্নিত করা; মানুষের পড়ার জন্য
Git SHA	`abc1234`	কোন কমিট থেকে build তা খুঁজে পাওয়া
`latest`	`latest`	শুধু সুবিধার জন্য; প্রোডাকশনে নয় — mutable
Digest	`sha256:9f2c…`	প্রোডাকশন ডেপ্লয় — immutable, যাচাইযোগ্য

ভালো অভ্যাস: প্রতিটি build-এ একাধিক ট্যাগ দিন (`1.4.0` , `1.4` , `abc1234`), কিন্তু ডেপ্লয় করুন digest দিয়ে।

Registry ও প্রমাণীকরণ

Registry হলো image-র সংরক্ষণাগার। GitHub Container Registry (GHCR)-তে GitHub-এর OIDC বা `GITHUB_TOKEN` দিয়ে, AWS ECR-তে IAM/OIDC দিয়ে authenticate করা যায় — দীর্ঘস্থায়ী পাসওয়ার্ড এড়িয়ে স্বল্পস্থায়ী (short-lived) credential ব্যবহার করাই আধুনিক নীতি (অধ্যায় ১০)। Registry-তে push করার আগে image স্ক্যান করুন; registry-তেও স্ক্যান চালু করা যায়, তবে পাইপলাইনের স্ক্যানই প্রথম প্রতিরক্ষা।

Image Scanning ও SLSA

Trivy/Grype image-র প্রতিটি layer-এর প্যাকেজকে CVE তালিকার সঙ্গে মেলায়। Critical দুর্বলতা থাকলে build ব্যর্থ করুন। SLSA (Supply-chain Levels for Software Artifacts) ফ্রেমওয়ার্ক বলে: build যেন পুনরুৎপাদনযোগ্য (reproducible), উৎস যাচাইযোগ্য (provenance — কে, কী থেকে, কীভাবে build করল তার স্বাক্ষরিত রেকর্ড) হয়। Provenance ছাড়া আপনি জানেন না image-টি আসলেই আপনার কোড থেকে তৈরি কি না।

Artifact Retention ও Reproducibility

পুরোনো image অনির্দিষ্টকাল রাখা খরচ বাড়ায় ও বিভ্রান্তি তৈরি করে; retention নীতি ঠিক করুন (যেমন শেষ ২০টি রিলিজ রাখা, বাকি মুছে ফেলা)। Reproducibility মানে একই source থেকে আবার build করলে একই digest পাওয়া — এর জন্য lockfile, পিন করা base image সংস্করণ (`node:20.11-alpine`, `node:20` নয়) ও নির্দিষ্ট সময় ব্যবহার করুন।

২০.৭ কনফিগারেশন উদাহরণ — Multi-stage Dockerfile

```
# Dockerfile — MERN API সার্ভারের জন্য (Node.js 20, Alpine)
# Stage 1: build — TypeScript কম্পাইল ও dev deps এখানেই
FROM node:20.11-alpine AS build
WORKDIR /app
COPY package.json package-lock.json ./
RUN npm ci
COPY tsconfig.json ./
COPY src ./src
RUN npm run build

# Stage 2: runtime — শুধু প্রয়োজনীয় জিনিস
FROM node:20.11-alpine AS runtime
WORKDIR /app
ENV NODE_ENV=production
COPY package.json package-lock.json ./
RUN npm ci --omit=dev && npm cache clean --force
COPY --from=build /app/dist ./dist
# root নয় — non-root ব্যবহারকারী
RUN addgroup -S app && adduser -S app -G app
USER app
EXPOSE 3000
HEALTHCHECK --interval=30s --timeout=5s --start-period=10s \
  CMD node -e "fetch('http://localhost:3000/health').then(r => process.exit(r.ok ? 0 : 1))"
CMD ["node", "dist/index.js"]
```

ব্যাখ্যা: `npm ci` lockfile অনুযায়ী হুবহু ইনস্টল করে (`npm install`-এর মতো নয়); `--omit=dev` dev dependency বাদ দেয়; `.dockerignore`-এ `node_modules`, `.git`, `.env` রাখতে হবে, যাতে সেগুলো image-এ না যায়। Base image-র সংস্করণ পিন করা হয়েছে (`20.11-alpine`) — `latest` নয়।

```
# Build + scan + push (GitHub Actions — আংশিক)
- name: Build and push
  uses: docker/build-push-action@v6
  with:
    context: .
    push: true
    tags: |
      ghcr.io/technova/api:1.4.0
      ghcr.io/technova/api:abc1234
    cache-from: type=gha
    cache-to: type=gha,mode=max
- name: Scan image
  uses: aquasecurity/trivy-action@0.24.0
  with:
    image-ref: ghcr.io/technova/api:1.4.0
    format: sarif
    output: trivy-results.sarif
    severity: CRITICAL,HIGH
    exit-code: '1'   # Critical/High পেলে build ব্যর্থ
# cache-from/to (GitHub Actions cache) layer ক্যাশ করে build দ্রুত করে।
# ধাপের ক্রম: build → scan → push — স্ক্যান ব্যর্থ হলে registry-তে যায় না।
```

```
# digest দিয়ে pull (প্রোডাকশন ডেপ্লয় — উদাহরণ)
docker pull ghcr.io/technova/
api@sha256:9f2c4a1b8e7d6c5b4a39281706f5e4d3c2b1a09f8e7d6c5b4a39281706f5e4d3c
docker run -d --name api -p 3000:3000 \
  --env-file /etc/technova/api.env \
  ghcr.io/technova/
api@sha256:9f2c4a1b8e7d6c5b4a39281706f5e4d3c2b1a09f8e7d6c5b4a39281706f5e4d3c
# উপরের digest উদাহরণমূলক; আসল digest build লগ থেকে নিন।
```

২০.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র API image 1.2 GB ছিল — কারণ একটিমাত্র stage-এ build সরঞ্জাম, dev dependency ও Git ইতিহাস সব ঢুকেছিল। Multi-stage-এ যাওয়ার পর image 180 MB হলো, pull সময় ৪ মিনিট থেকে ৪০ সেকেন্ডে নামল, আর Trivy স্ক্যানে দুর্বলতার সংখ্যাও কমে গেল — কারণ অপ্রয়োজনীয় প্যাকেজই আর image-এ নেই। একইসঙ্গে দল `latest` দিয়ে ডেপ্লয় বন্ধ করে digest বাধ্যতামূলক করল; এরপর staging-এ পরীক্ষিত image-ই production-এ চলতে লাগল।

২০.৯ ধাপে ধাপে বাস্তবায়ন

1. `.dockerignore` লিখুন (`node_modules`, `.git`, `.env`, `*.md`)।
2. Multi-stage Dockerfile লিখুন; base image সংস্করণ পিন করুন।
3. স্থানীয়ভাবে build করে আকার দেখুন (`docker images`) ও `docker run` করে health endpoint পরীক্ষা করুন।
4. CI-তে build → Trivy scan (exit-code 1) → push ক্রম বসান।
5. Registry-তে push হওয়া image-র digest রেকর্ড করুন; staging ও production একই digest দিয়ে ডেপ্লয় করুন।
6. Retention নীতি ঠিক করুন ও পুরোনো untagged image নিয়মিত পরিষ্কার করুন।

২০.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "Build হঠাৎ ধীর" — `.dockerignore` নেই বা COPY ক্রম ভুল; dependency layer-এর আগে source কপি করলে প্রতিবার ক্যাশ ভাঙে। (২) "Registry-তে push-এ 401" — credential মেয়াদোত্তীর্ণ বা ভুল scope; OIDC/short-lived টোকেন ব্যবহার করুন, দীর্ঘস্থায়ী পাসওয়ার্ড নয়। (৩) "Image চলে না: module not found" — runtime stage-এ `--omit=dev` দিয়ে production-এ দরকারি প্যাকেজ বাদ পড়েছে; dependency শ্রেণি যাচাই করুন। (৪) "স্ক্যানে অনেক দুর্বলতা" — base image পুরোনো; পিন করা সংস্করণ আপডেট করে আবার build করুন, আর অপ্রয়োজনীয় প্যাকেজ বাদ দিন।

২০.১১ নিরাপত্তা বিবেচনা

WARNING: Image-তে কখনো `.env`, API কি, বা SSH কি রাখবেন না — image যে কেউ pull করে layer খুলে দেখতে পারে। Root ব্যবহারকারী হিসেবে container চালাবেন না; non-root ব্যবহারকারী ও read-only filesystem ব্যবহার করুন। `latest` ট্যাগ দিয়ে ডেপ্লয় করলে আপনি জানেন না আসলে কী চলছে — প্রোডাকশনে digest বাধ্যতামূলক করুন। তৃতীয় পক্ষের base image ব্যবহারের আগে তার উৎস ও স্ক্যান ফলাফল যাচাই করুন।

২০.১২ পারফরম্যান্স ও খরচ

ছোট image মানে দ্রুত pull, কম স্টোরেজ খরচ ও দ্রুত স্কেল। Layer ক্যাশ (GitHub Actions cache বা registry cache) build সময় অনেক কমায়। তবে ক্যাশ নিজেও খরচ ও জটিলতা বাড়ায় — বড় প্রকল্পে BuildKit cache পরিমাপ করে সিদ্ধান্ত নিন। Registry স্টোরেজ ও ডেটা ট্রান্সফার খরচের জন্য retention নীতি জরুরি; অপ্রয়োজনীয় পুরোনো image মুছে ফেলুন।

২০.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: Multi-stage build; পিন করা base image; `.dockerignore`; non-root ব্যবহারকারী; স্ক্যান-তারপর-push; digest দিয়ে ডেপ্লয়; provenance রেকর্ড। **অ্যান্টি-প্যাটার্ন:** এক stage-এ সবকিছু; `latest` দিয়ে প্রোডাকশন ডেপ্লয়; image-তে গোপন তথ্য; স্ক্যান ছাড়া push; প্রতিটি পরিবেশে আলাদা build।

২০.১৪ অধ্যায়ের সারাংশ

Docker image হলো CI/CD-র artifact: multi-stage build তাকে ছোট ও নিরাপদ করে, ট্যাগ মানুষের জন্য আর digest যাচাইয়ের জন্য, স্ক্যান তাকে নিরাপদ প্রমাণ করে, আর registry থেকে একই digest সব পরিবেশে ডেপ্লয় করলেই "build once, deploy many times" সত্য হয়।

২০.১৫ মূল প্রযুক্তিগত পরিভাষা

Multi-stage Build: একাধিক stage-এ image তৈরি — build সরঞ্জাম শেষ image-এ থাকে না।

Layer: Image-র একটি স্তর; প্রতিটি Dockerfile নির্দেশ একটি layer যোগ করে।

Tag: Image-র মানুষের পড়ার মতো নাম; বদলানো যায় (mutable)।

Digest: Image কনটেন্টের হ্যাশ; বদলানো যায় না (immutable)।

Registry: Image সংরক্ষণ ও বিতরণের সংরক্ষণাগার (যেমন GHCR, ECR)।

Image Scanning: Image-র প্যাকেজে জানা দুর্বলতা খোঁজা।

Provenance: Artifact কী থেকে, কীভাবে, কে তৈরি করল তার যাচাইযোগ্য রেকর্ড।

SLSA: Supply-chain Levels for Software Artifacts — সরবরাহ-শৃঙ্খল নিরাপত্তার কাঠামো।

Build Cache: আগের build-এর layer পুনঃব্যবহার করে build দ্রুত করা।

Attack Surface: আক্রমণকারী যেসব অংশে আঘাত করতে পারে তার মোট পরিমাণ।

২০.১৬ জ্ঞান যাচাই প্রশ্ন

1. `latest` ট্যাগ দিয়ে প্রোডাকশন ডেপ্লয় কেন ঝুঁকিপূর্ণ?
2. Multi-stage build কীভাবে image-র আকার ও নিরাপত্তা দুটোই উন্নত করে?
3. Digest ও ট্যাগের মধ্যে পার্থক্য কী? ডেপ্লয়ে কোনটি ব্যবহার করবেন?
4. স্ক্যানের পরেও image-তে দুর্বলতা থাকতে পারে কেন?
5. Provenance ছাড়া image-র ওপর ভরসা করা কেন কঠিন?

২০.১৭ ব্যবহারিক অনুশীলন

1. একটি Node.js অ্যাপের এক-stage ও multi-stage Dockerfile লিখে দুটি image-র আকার তুলনা করুন।
2. Trivy দিয়ে আপনার image স্ক্যান করুন; একটি দুর্বলতা খুঁজে base image আপডেট করে আবার স্ক্যান করুন।
3. GHCR বা Docker Hub-এ image push করে digest দিয়ে pull ও run করুন।

অধ্যায় ২১ • PART IV

Chapter 21 — Environment Promotion, Versioning and Release Management

২১.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Development, Staging ও Production পরিবেশের ভূমিকা ও পার্থক্য ব্যাখ্যা করতে; (২) Semantic Versioning (SemVer) ও release candidate ব্যবহার করতে; (৩) Environment promotion নকশা করতে; (৪) Approval gate ও release note তৈরি করতে; (৫) রিলিজ ব্যবস্থাপনার ঝুঁকি ও পরিবর্তন-নিয়ন্ত্রণ (change management) ব্যাখ্যা করতে।

২১.২ কেন এটি গুরুত্বপূর্ণ

কোড লেখা শেষ হওয়া মানেই রিলিজ নয়। কোডকে development থেকে staging হয়ে production-এ পৌঁছাতে হয় — প্রতিটি ধাপে যাচাই, অনুমোদন ও রেকর্ডসহ। Promotion ছাড়া দল সরাসরি production-এ কোড ছোড়ে, আর তখন একটি ভুলের দায় কার, কোন সংস্করণ চলছে, কী বদলেছে — কিছুই জানা যায় না। ভালো release management মানে প্রতিটি রিলিজ পুনরাবৃত্তিযোগ্য, যাচাইযোগ্য ও ফিরিয়ে নেওয়ার মতো।

২১.৩ পূর্বশর্ত জ্ঞান

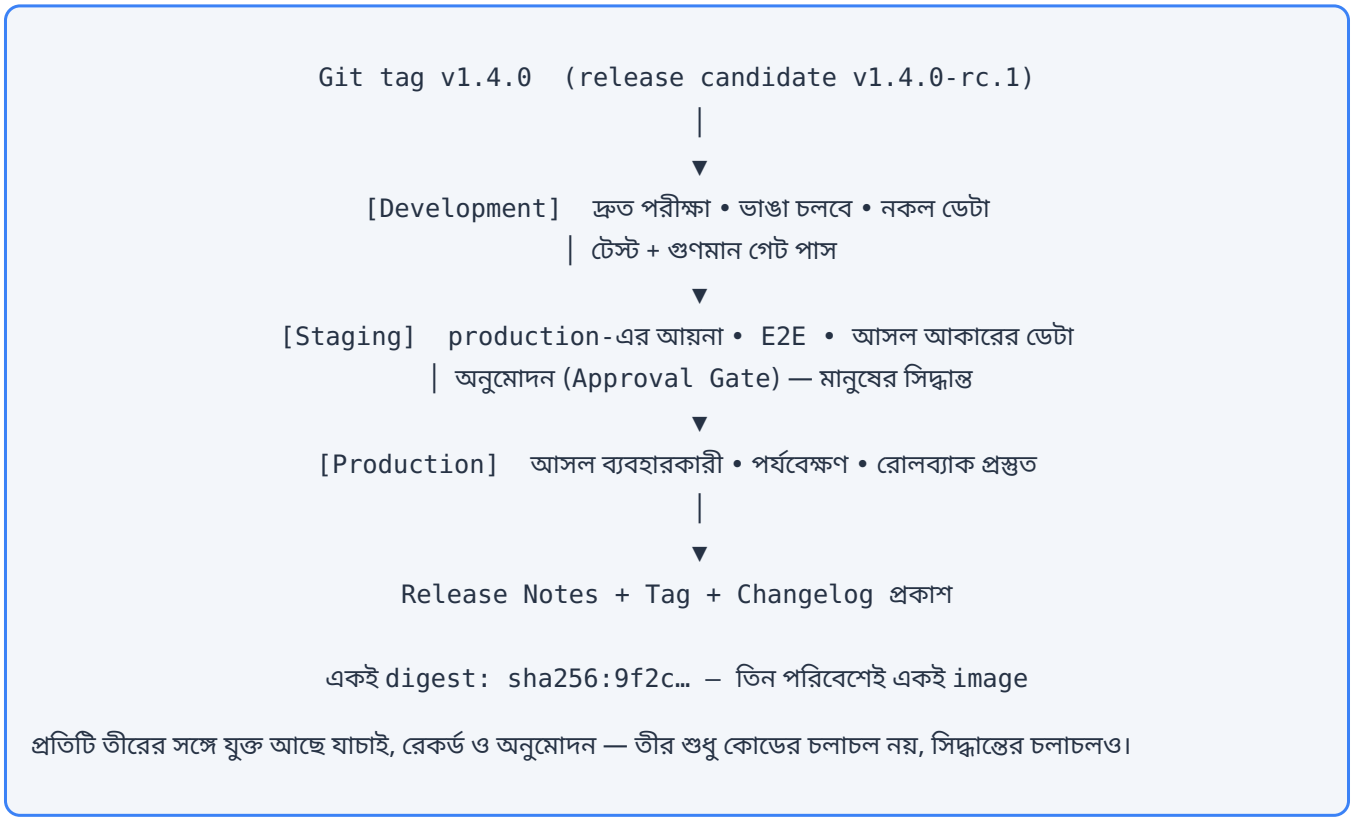
Git ট্যাগ, অধ্যায় ২০-এর artifact ও digest ধারণা, অধ্যায় ১৮–১৯-এর টেস্ট ও গুণমান-গেট, এবং পরিবেশ-ভিত্তিক কনফিগারেশন (environment variable) সম্পর্কে ধারণা থাকতে হবে।

২১.৪ প্রথম নীতি থেকে ব্যাখ্যা

পরিবেশ (Environment) মানে একই কোড চলার ভিন্ন প্রেক্ষাপট — ভিন্ন ডেটাবেস, ভিন্ন গোপন তথ্য, ভিন্ন ট্রাফিক। Development হলো পরীক্ষার জায়গা, staging হলো production-এর আয়না (একই কনফিগারেশন, নকল ডেটা), production হলো আসল ব্যবহারকারী। Promotion মানে artifact-কে এক পরিবেশ থেকে পরের পরিবেশে এগিয়ে দেওয়া — কোড আবার build না করে, একই digest নিয়ে। সংস্করণ (Version) হলো সেই artifact-এর নাম, যাতে মানুষ বুঝতে পারে কী বদলেছে: SemVer-এ `MAJOR.MINOR.PATCH` — MAJOR মানে ভাঙা পরিবর্তন, MINOR মানে নতুন ফিচার (পেছনে সামঞ্জস্যপূর্ণ), PATCH মানে বাগ ঠিক।

> **CONCEPT:** Promotion-এ কোড বদলায় না, পরিবেশ বদলায়। একই artifact, ভিন্ন কনফিগারেশন। যে দল প্রতিটি পরিবেশে আলাদা build করে, তারা আসলে প্রতিবার ভিন্ন জিনিস পরীক্ষা করে।

২১.৫ আর্কিটেকচার / ধারণাগত চিত্র



প্রতিটি তীরের সঙ্গে যুক্ত আছে যাচাই, রেকর্ড ও অনুমোদন — তীর শুধু কোডের চলাচল নয়, সিদ্ধান্তের চলাচলও।

২১.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

পরিবেশের নকশা

পরিবেশ	উদ্দেশ্য	ডেটা	কে ডেপ্লয় করে
Development	দ্রুত পরীক্ষা	নকল/স্থানীয়	স্বয়ংক্রিয় (প্রতিটি PR)
Staging	production-এর মতো যাচাই	নকল, production-আকারের	স্বয়ংক্রিয় + পর্যালোচনা
Production	আসল সেবা	আসল	অনুমোদনসহ স্বয়ংক্রিয়

Staging যত production-এর কাছাকাছি (একই কনফিগারেশন, একই আকারের ডেটা, একই গোপন-তথ্য ব্যবস্থাপনা), তত তার যাচাই মূল্যবান। Staging-এ আসল ব্যবহারকারীর ডেটা ব্যবহার করবেন না — গোপনীয়তা ও নিরাপত্তার ঝুঁকি।

Semantic Versioning ও Release Candidate

SemVer: `1.4.0` — ১ = MAJOR, ৪ = MINOR, ০ = PATCH। নতুন ফিচার যোগ হলে MINOR বাড়ে, ভাঙা পরিবর্তনে MAJOR, বাগ ঠিক হলে PATCH। Release candidate (`1.4.0-rc.1`) মানে "প্রায় প্রস্তুত, পরীক্ষা চলছে" — rc-তে বড় বাগ পেলে `rc.2` তৈরি হয়, না পেলে একই কোড `1.4.0` হয়। Pre-release ট্যাগ (`-rc.1`, `-beta.1`) ব্যবহারকারীদের জানায় এটি এখনো চূড়ান্ত নয়।

Approval Gate

Approval gate মানে production-এ যাওয়ার আগে একজন নির্দিষ্ট মানুষের (বা দলের) স্পষ্ট অনুমোদন। GitHub Actions-এ environment protection rule, GitLab-এ manual job ও protected environment, Jenkins-এ `input` ধাপ দিয়ে এটি করা হয়। অনুমোদনকারীর কাছে যা যা থাকতে হবে: কী বদলেছে (changelog), কোন টেস্ট পাস করেছে, রোলব্যাক পরিকল্পনা কী। অনুমোদন ছাড়া production ডেপ্লয় মানে নিয়ন্ত্রণহীন রিলিজ।

Release Notes ও Changelog

প্রতিটি রিলিজে লিখুন: নতুন ফিচার, বাগ ঠিক, ভাঙা পরিবর্তন, মাইগ্রেশন নির্দেশ, পরিচিত সমস্যা। Conventional Commits (`feat:`, `fix:`, `BREAKING CHANGE:`) ব্যবহার করলে changelog স্বয়ংক্রিয়ভাবে তৈরি করা যায়। Release note শুধু ডেভেলপারদের জন্য নয় — সাপোর্ট দল ও ব্যবহারকারীরাও তা পড়ে।

পরিবর্তন-নিয়ন্ত্রণ (Change Management)

Production পরিবর্তন নথিভুক্ত করুন: কে, কখন, কী, কেন, রোলব্যাক কী। ছোট দলে এটি একটি PR + release note-ই যথেষ্ট; বড় সংস্থায় অনুমোদন-প্রক্রিয়া লাগতে পারে। উদ্দেশ্য একটাই — কোনো পরিবর্তন অদৃশ্য না থাকা, যাতে সমস্যা হলে কারণ খুঁজে পাওয়া যায়।

২১.৭ কনফিগারেশন উদাহরণ

```
# Git tag দিয়ে রিলিজ চিহ্নিত করা
git tag -a v1.4.0-rc.1 -m "Release candidate 1 for 1.4.0"
git push origin v1.4.0-rc.1
# rc পরীক্ষা পাস করলে একই কমিটে চূড়ান্ত ট্যাগ:
git tag -a v1.4.0 -m "Release 1.4.0 — checkout discount fix, new search"
git push origin v1.4.0
```

```
# GitHub Actions — environment protection সহ production ডেপ্লয়
deploy-production:
  needs: [build, test, scan]
  runs-on: ubuntu-latest
  environment:
    name: production
    url: https://app.technova.example
  permissions:
    contents: read
    id-token: write   # OIDC দিয়ে registry/cloud-এ প্রমাণীকরণ
  steps:
    - name: Deploy image by digest
      run: |
        echo "Deploying digest: ${{ needs.build.outputs.image_digest }}"
        # ডেপ্লয় স্ক্রিপ্ট এখানে digest দিয়ে pull করে
# environment: production-এ GitHub-এ required reviewers বসাতে হয় —
# তখন এই job অনুমোদন না পাওয়া পর্যন্ত অপেক্ষা করে।
```

```
# CHANGELOG.md — Conventional Commits থেকে স্বয়ংক্রিয় (উদাহরণ)
## [1.4.0] - 2026-11-08
### Added
- পণ্য অনুসন্ধানে ফিল্টার (feat)
### Fixed
- ছাড়ের হিসাবে ভুল শতাংশ প্রয়োগ (fix)
```

```
### BREAKING CHANGE
- /api/v1/orders রেসপন্সে `total` এখন সংখ্যা, স্ট্রিং নয়
```

২১.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-তে একবার production-এ কোন সংস্করণ চলছে তা কেউ বলতে পারেনি — কারণ ডেপ্লয় হতো `latest` ট্যাগে, ট্যাগও ছিল না, changelog-ও ছিল না। একটি বাগের পর দল তিন দিন ধরে খুঁজেছিল কোন কমিট থেকে সমস্যা শুরু। এরপর নীতি বদলাল: প্রতিটি production ডেপ্লয় একটি Git ট্যাগ (`vX.Y.Z`), একই digest, একটি changelog ও একজন অনুমোদনকারীর নামসহ হয়। পরের বাগে কারণ খুঁজতে সময় লেগেছিল ২০ মিনিট।

২১.৯ ধাপে ধাপে বাস্তবায়ন

1. তিনটি পরিবেশের কনফিগারেশন আলাদা করুন (environment variable, secret) — কোড একই রাখুন।
2. SemVer গ্রহণ করুন; `package.json`-এর সংস্করণ ও Git ট্যাগ একই রাখুন।
3. Release candidate প্রবাহ চালু করুন: `rc.1` → staging পরীক্ষা → চূড়ান্ত ট্যাগ।
4. Production environment-এ required reviewers বসান।
5. প্রতিটি রিলিজে changelog ও release note প্রকাশ করুন।
6. রিলিজ রেকর্ড রাখুন: ট্যাগ, digest, অনুমোদনকারী, ডেপ্লয়ের সময়।

২১.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "Staging-এ চলে, production-এ চলে না" — পরিবেশের কনফিগারেশনে পার্থক্য (ভিন্ন env var, ভিন্ন DB সংস্করণ)। সমাধান: কনফিগারেশনও কোডের মতো সংস্করণ-নিয়ন্ত্রণে রাখুন, পার্থক্য তালিকা করুন। (২) "কোন সংস্করণ চলছে জানি না" — ট্যাগ/digest রেকর্ড নেই; `/health` endpoint-এ সংস্করণ দেখান। (৩) "অনুমোদন আটকে আছে" — অনুমোদনকারী ছুটিতে; বিকল্প অনুমোদনকারী (backup reviewer) আগেই ঠিক রাখুন। (৪) "rc ও চূড়ান্ত রিলিজে কোড আলাদা" — rc-র পর কোড বদলালে নতুন rc তৈরি করুন; একই কোডেই চূড়ান্ত ট্যাগ দিন।

২১.১১ নিরাপত্তা বিবেচনা

WARNING: Production পরিবেশের গোপন তথ্য শুধু production-এই থাকবে — staging-এ আসল কি ব্যবহার করবেন না। Production ডেপ্লয়ের অনুমতি শুধু কয়েকজনের হাতে রাখুন, আর প্রতিটি ডেপ্লয় লগ করুন। Staging-এ আসল ব্যবহারকারীর ডেটা কপি করলে গোপনীয়তা লঙ্ঘন হতে পারে — নকল বা anonymized ডেটা ব্যবহার করুন।

২১.১২ পারফরম্যান্স ও খরচ

তিনটি পরিবেশ চালানো মানে তিন গুণ অবকাঠামো খরচ — তবে staging ছোট আকারে চালানো যায় (কম replica, ছোট DB)। খরচের বিনিময়ে যা পাওয়া যায়: production-এ ভুলের ঝুঁকি কমা। Release candidate প্রবাহ সময় নেয়, কিন্তু জরুরি hotfix-এর জন্য আলাদা দ্রুত পথ (fast-track) রাখুন — নিয়ম ভেঙে নয়, নিয়মের ভেতরে।

২১.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: একই artifact সব পরিবেশে; SemVer; rc প্রবাহ; অনুমোদন গেট; changelog; রিলিজ রেকর্ড। **অ্যান্টি-প্যাটার্ন:** সরাসরি production-এ ডেপ্লয়; `latest` দিয়ে রিলিজ চিহ্নিত করা; ট্যাগ ছাড়া ডেপ্লয়; অনুমোদন ছাড়া production পরিবর্তন; changelog না লেখা।

২১.১৪ অধ্যায়ের সারাংশ

Release management মানে নিয়ন্ত্রিত যাত্রা: একই artifact development → staging → production যায়, প্রতিটি ধাপে যাচাই ও অনুমোদনসহ। SemVer বলে কী বদলেছে, ট্যাগ ও digest বলে কোনটি চলছে, changelog বলে কেন বদলেছে — আর এই তিনটি রেকর্ড ছাড়া production-এর কোনো পরিবর্তনই পেশাদার নয়।

২১.১৫ মূল প্রযুক্তিগত পরিভাষা

Environment Promotion: একই artifact-কে এক পরিবেশ থেকে পরের পরিবেশে এগিয়ে দেওয়া।

Semantic Versioning (SemVer): MAJOR.MINOR.PATCH — পরিবর্তনের ধরন বোঝানো সংস্করণ-পদ্ধতি।

Release Candidate (RC): চূড়ান্ত রিলিজের আগে পরীক্ষাধীন প্রায়-প্রস্তুত সংস্করণ।

Approval Gate: মানুষের অনুমোদন ছাড়া পরের ধাপে যেতে না দেওয়ার নিয়ম।

Changelog: প্রতিটি সংস্করণে কী বদলেছে তার তালিকা।

Staging: production-এর আয়না হিসেবে পরীক্ষার পরিবেশ।

Change Management: production পরিবর্তন নথিভুক্ত ও অনুমোদনের প্রক্রিয়া।

Hotfix: জরুরি বাগ ঠিক করতে দ্রুত পথে করা ছোট রিলিজ।

Conventional Commits: কমিট বার্তার আদর্শ রূপ, যা changelog তৈরিতে সাহায্য করে।

Environment Parity: staging ও production-এর কনফিগারেশনের মিল।

২১.১৬ জ্ঞান যাচাই প্রশ্ন

1. Promotion-এ একই artifact ব্যবহার করা কেন জরুরি? প্রতিটি পরিবেশে আলাদা build করলে কী ভুল হয়?
2. SemVer-এ MAJOR, MINOR ও PATCH কখন বাড়ে — উদাহরণসহ বলুন।
3. Release candidate কেন দরকার, সরাসরি চূড়ান্ত ট্যাগ দিলে কী ক্ষতি?
4. Staging production-এর আয়না না হলে তার যাচাই কেন কম মূল্যবান?
5. Approval gate-এ অনুমোদনকারীর কী কী তথ্য দেখা উচিত?

২১.১৭ ব্যবহারিক অনুশীলন

1. একটি প্রকল্পে SemVer অনুযায়ী `v1.0.0`, `v1.1.0-rc.1` ও `v1.1.0` ট্যাগ তৈরি করে push করুন।
2. Conventional Commits দিয়ে ৫টি কমিট লিখে একটি ছোট changelog হাতে তৈরি করুন।
3. GitHub-এ একটি environment-এ required reviewer বসিয়ে অনুমোদন ছাড়া ডেপ্লয় আটকে যায় কি না পরীক্ষা করুন।

অধ্যায় ২২ • PART IV

Chapter 22 — Deployment Strategies, Rollbacks and Release Reliability

২২.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) Rolling, Blue-Green, Canary ও Recreate কৌশলের পার্থক্য ও বাণিজ্য-বিনিময় ব্যাখ্যা করতে; (২) Feature flag দিয়ে ধাপে ধাপে ফিচার চালু করতে; (৩) Health check ও deployment verification নকশা করতে; (৪) Rollback কৌশল বেছে নিতে ও ডেটাবেস মাইগ্রেশন নিরাপদে করতে; (৫) প্রতিটি কৌশলের পরিচালন-পূর্বশর্ত ও ঝুঁকি মূল্যায়ন করতে।

২২.২ কেন এটি গুরুত্বপূর্ণ

ডেপ্লয় করা মানেই ঝুঁকি নেওয়া — নতুন কোড আসল ব্যবহারকারীর সামনে যায়। ভালো কৌশল সেই ঝুঁকিকে ছোট ছোট অংশে ভাগ করে: প্রথমে ৫% ব্যবহারকারী, তারপর ৫০%, তারপর সবাই; আর কিছু ভুল হলে সেকেন্ডে ফিরে যাওয়া যায়। TechNova-র একবারের শিক্ষা: সরাসরি সব সার্ভারে নতুন কোড বসানোর (big-bang) পর চেকআউট ভেঙে গিয়েছিল, আর ফিরে যেতে সময় লেগেছিল ৪০ মিনিট — কারণ পুরোনো সংস্করণ ফিরিয়ে আনার পরিকল্পনাই ছিল না। কৌশল মানে ডেপ্লয়ের আগে ফেরার পথ ঠিক করে রাখা।

২২.৩ পূর্বশর্ত জ্ঞান

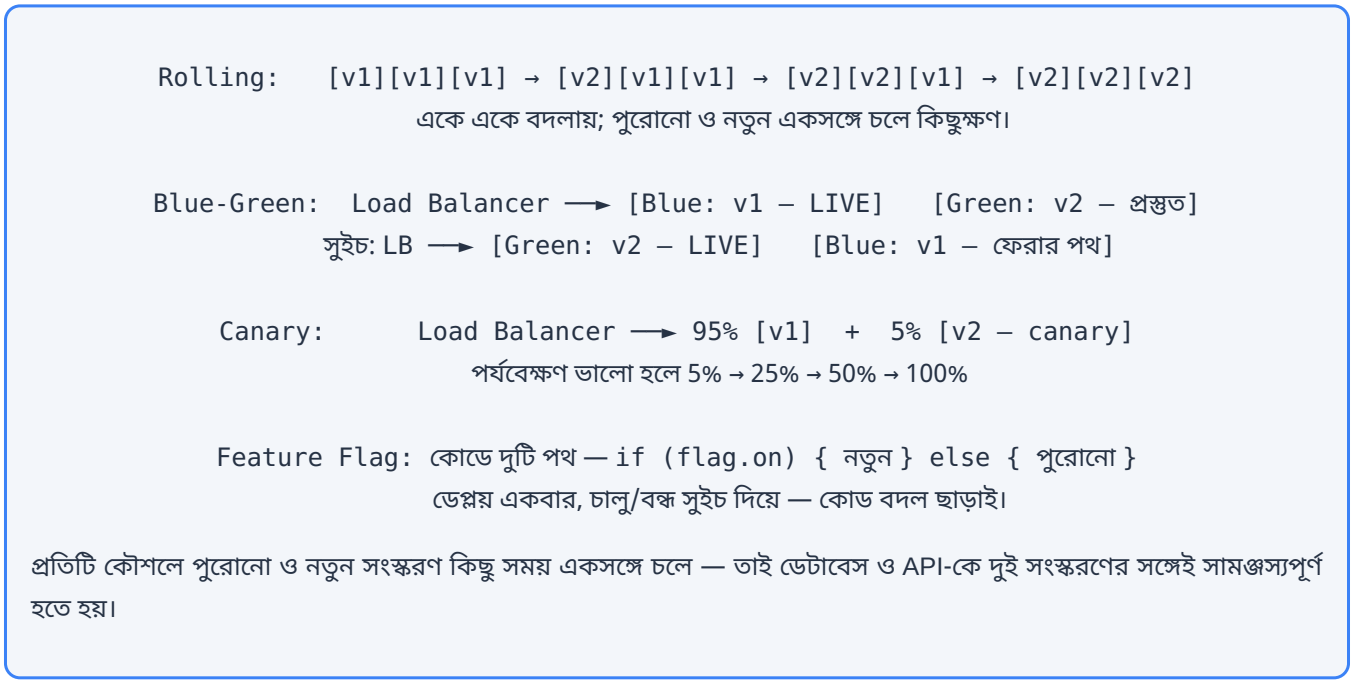
অধ্যায় ২০–২১-এর artifact, digest ও promotion ধারণা, Docker container চালানো, load balancer/reverse proxy-র প্রাথমিক ধারণা এবং MongoDB migration-এর মৌলিক ধারণা থাকতে হবে।

২২.৪ প্রথম নীতি থেকে ব্যাখ্যা

ডেপ্লয় কৌশলের মূল প্রশ্ন দুটি: (১) নতুন সংস্করণ কতজন ব্যবহারকারীর কাছে যাবে, কত দ্রুত? (২) ভুল হলে কত দ্রুত ফেরা যাবে? সব কৌশল এই দুই প্রশ্নের ভিন্ন উত্তর। Rolling ধীরে ধীরে বদলায়; Blue-Green পুরো নতুন পরিবেশ তৈরি করে একবারে সুইচ করে; Canary অল্প ট্রাফিক দিয়ে পরীক্ষা করে; Feature Flag কোড ডেপ্লয় ও ফিচার চালু করা আলাদা করে দেয়। কোনো কৌশলই সব অ্যাপের জন্য সেরা নয় — ডেটা, ট্রাফিক ও দলের সক্ষমতা দেখে বেছে নিতে হয়।

> **CONCEPT:** Rollback মানে "পুরোনো কোড ফিরিয়ে আনা" নয় — মানে "ব্যবহারকারীর কাছে আবার কাজ করা সংস্করণ পৌঁছে দেওয়া, যত দ্রুত সম্ভব।" কখনো সেটা পুরোনো image চালানো, কখনো feature flag বন্ধ করা, কখনো ট্রাফিক সরিয়ে নেওয়া।

২২.৫ আর্কিটেকচার / ধারণাগত চিত্র



প্রতিটি কৌশলে পুরোনো ও নতুন সংস্করণ কিছু সময় একসঙ্গে চলে — তাই ডেটাবেস ও API-কে দুই সংস্করণের সঙ্গেই সামঞ্জস্যপূর্ণ হতে হয়।

২২.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

কৌশলগুলোর তুলনা

কৌশল	কীভাবে কাজ করে	সুবিধা	ঝুঁকি / পূর্বশর্ত
Recreate	পুরোনো বন্ধ, তারপর নতুন চালু	সহজ	Downtime হয়; ছোট অ্যাপ ছাড়া অনুপযুক্ত
Rolling	একে একে instance বদলানো	Downtime নেই, সম্পদ কম লাগে	দুই সংস্করণ একসঙ্গে চলে — সামঞ্জস্য দরকার
Blue-Green	পুরো নতুন পরিবেশ, একবারে সুইচ	দ্রুত rollback, পূর্ণ পরীক্ষা	দ্বিগুণ সম্পদ; ডেটা সিঙ্ক জটিল
Canary	অল্প ট্রাফিকে নতুন সংস্করণ	ঝুঁকি সীমিত, আসল ট্রাফিকে পরীক্ষা	ট্রাফিক ভাগ ও পর্যবেক্ষণ লাগে
Feature Flag	কোডে সুইচ; ডেপ্লয় ও চালু আলাদা	তাৎক্ষণিক rollback, A/B পরীক্ষা	Flag ব্যবস্থাপনা, পুরোনো flag পরিষ্কার করা

Health Check ও Deployment Verification

নতুন instance চালু হলেই ট্রাফিক দেওয়া যাবে না — প্রথমে health check: `/health` endpoint কি 200 দেয়? ডেটাবেস সংযোগ আছে? তারপর smoke test: লগইন, পণ্য দেখা, চেকআউটের মূল পথ কি কাজ করে? তারপর পর্যবেক্ষণ: error rate, latency, CPU — নতুন সংস্করণে হঠাৎ বেড়ে গেলে স্বয়ংক্রিয়ভাবে থামান (automated rollback)। Verification ছাড়া ডেপ্লয় মানে অন্ধভাবে বিশ্বাস করা।

Rollback কৌশল

সবচেয়ে নির্ভরযোগ্য rollback: আগের image-র digest দিয়ে আবার ডেপ্লয় করা (redeploy previous version)। Blue-Green-এ load balancer সুইচ ফিরিয়ে নেওয়া। Feature flag-এ flag বন্ধ করা — সবচেয়ে দ্রুত। বিপজ্জনক rollback: ডেটাবেস মাইগ্রেশন উল্টে দেওয়া — ডেটা হারাতে পারে। তাই মাইগ্রেশন সবসময় backward-compatible রাখুন (নিচে দেখুন)। Rollback-ও পরীক্ষা করুন: যে পথ কখনো পরীক্ষা হয়নি, জরুরি সময়ে তা কাজ করবে — এই ভরসা করবেন না।

ডেটাবেস মাইগ্রেশন নিরাপত্তা

Expand–Contract পদ্ধতি: (১) Expand — নতুন কলাম/ফিল্ড যোগ করুন, পুরোনো কোড তাতে কিছু করে না; (২) নতুন কোড ডেপ্লয় করুন যা নতুন ফিল্ড ব্যবহার করে, কিন্তু পুরোনো ফিল্ডও বোঝে; (৩) Contract — পুরোনো ফিল্ড তখনই সরান যখন নিশ্চিত পুরোনো কোড আর চলছে না। কখনো একই ডেপ্লয়ে কোড ও ভাঙা মাইগ্রেশন একসঙ্গে চালাবেন না। মাইগ্রেশনের আগে ব্যাকআপ নিন, আর মাইগ্রেশন যেন পুনরায় চালানো যায় (idempotent) তা নিশ্চিত করুন।

Release Note ও যোগাযোগ

ডেপ্লয়ের আগে জানান: কী যাচ্ছে, কখন, ঝুঁকি কী, রোলব্যাক কে করবে, যোগাযোগের পথ কী। ডেপ্লয়ের পর জানান: সফল হয়েছে কি না, কোনো অস্বাভাবিকতা আছে কি না। নীরব ডেপ্লয় সবচেয়ে বিপজ্জনক — সমস্যা হলে কেউ জানেই না কী বদলেছে।

২২.৭ কনফিগারেশন উদাহরণ

```
# Rolling deployment — Docker Compose (সরল উদাহরণ)
# docker compose up -d --no-deps --scale api=3
# নতুন image দিয়ে একে একে বদলানো:
docker compose pull api
docker compose up -d --no-deps api
# Compose একে একে পুরোনো container বদলে নতুন চালায়;
# healthcheck পাস না করা পর্যন্ত ট্রাফিক নতুনটিতে যায় না।
```

```
// Feature flag — সরল উদাহরণ (Node.js)
const flags = { newCheckout: process.env.FLAG_NEW_CHECKOUT === 'true' };
app.post('/api/checkout', (req, res) => {
  if (flags.newCheckout) return newCheckout(req, res);
  return legacyCheckout(req, res);
});
// flag বন্ধ করলেই পুরোনো পথে ফেরা যায় — ডেপ্লয় ছাড়াই।
// সতর্কতা: flag-এর মান পরিবেশ ভেরিয়েবলে, কোডে hardcode নয়।
```

```
# Deployment verification — smoke test স্ক্রিপ্ট (Bash)
#!/bin/bash
set -e
BASE="https://staging.technova.example"
curl -fsS "$BASE/health" | grep -q '"status":"ok"'
curl -fsS "$BASE/api/products" | grep -q '"products"'
echo "Smoke test passed — deploy verified"
# নমুনা আউটপুট — চিত্রণমূলক:
# Smoke test passed — deploy verified
```

```
# Rollback — আগের digest দিয়ে আবার ডেপ্লয়
# রিলিজ রেকর্ড থেকে আগের digest নিন (যেমন v1.3.0-র digest):
docker pull ghcr.io/technova/api@sha256:PREVIOUS-DIGEST-HERE
docker compose up -d api
# তারপর smoke test চালিয়ে নিশ্চিত হোন পুরোনো সংস্করণ কাজ করছে।
```

২২.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র নতুন চেকআউট শুধু ৫% ব্যবহারকারীর কাছে (canary) ছাড়া হয়েছিল। এক ঘণ্টায় দেখা গেল canary-তে error rate ০.১% থেকে ২%-এ উঠেছে — একটি edge case-এ পেমেন্ট ব্যর্থ হচ্ছিল। দল সঙ্গে সঙ্গে canary বন্ধ করে পুরোনো সংস্করণে ফিরে গেল; ক্ষতি সীমিত রইল ৫% ব্যবহারকারীর এক ঘণ্টায়। একই বাগ big-bang ডেপ্লয়ে গেলে সব ব্যবহারকারী আক্রান্ত হতো। শিক্ষা: canary-র মূল্য পর্যবেক্ষণে — ট্রাফিক ভাগ করলেই হবে না, মেট্রিক দেখে সিদ্ধান্ত নিতে হবে।

২২.৯ ধাপে ধাপে বাস্তবায়ন

1. অ্যাপের জন্য কৌশল বেছে নিন (ছোট দলের জন্য rolling বা blue-green দিয়ে শুরু)।
2. `/health` ও `/ready` endpoint যোগ করুন (liveness বনাম readiness)।
3. Smoke test স্ক্রিপ্ট লিখুন ও ডেপ্লয়ের পর স্বয়ংক্রিয়ভাবে চালান।
4. Rollback রানবুক লিখুন: কে করবে, কোন কমান্ড, কত সময় লাগবে — তারপর অনুশীলন করুন।
5. ডেটাবেস পরিবর্তনে Expand–Contract মেনে চলুন।
6. Canary বা feature flag ব্যবহার করলে পর্যবেক্ষণের মেট্রিক ও থামার সীমা (threshold) আগেই ঠিক করুন।

২২.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "Rolling-এ কিছু অনুরোধ ব্যর্থ" — পুরোনো ও নতুন কোড একসঙ্গে চলায় API-র অসামঞ্জস্য; সমাধান: backward-compatible API রাখুন। (২) "Blue-Green-এ ডেটা মিলছে না" — দুই পরিবেশে ডেটাবেস ভাগ করা হয়নি বা মাইগ্রেশন একটিতে হয়নি; ডেটাবেসকে পরিবেশের বাইরে ভাগ করা সম্পদ হিসেবে দেখুন। (৩) "Rollback-এর পরও সমস্যা" — মাইগ্রেশন উল্টানো হয়নি বা ক্যাশে নতুন কোডের ডেটা রয়ে গেছে; ক্যাশ পরিষ্কারের ধাপ রানবুকে রাখুন। (৪) "Canary-তে কোনো ট্রাফিক যাচ্ছে না" — load balancer-এর ওজন (weight) বা রাউটিং নিয়ম ভুল; staging-এ ট্রাফিক ভাগ পরীক্ষা করুন।

২২.১১ নিরাপত্তা বিবেচনা

WARNING: Canary বা blue-green পরিবেশেও production ডেটা ও গোপন তথ্য একই নিয়মে সুরক্ষিত রাখুন — "অস্থায়ী" পরিবেশ বলে নিরাপত্তা ঢিলা করবেন না। Feature flag-এর নিয়ন্ত্রণ (কে flag বদলাতে পারে) কঠোরভাবে সীমিত রাখুন — একটি ভুল flag পুরো ফিচার বন্ধ করে দিতে পারে। Rollback-এর সময়ও অনুমোদন ও লগ বজায় রাখুন; জরুরি অবস্থা নিয়ন্ত্রণহীনতার অজুহাত নয়।

২২.১২ পারফরম্যান্স ও খরচ

Blue-Green দ্বিগুণ সম্পদ খরচ করে — ছোট দলের জন্য ব্যয়বহুল হতে পারে; rolling সস্তা। Canary-তে অতিরিক্ত পর্যবেক্ষণ ও ট্রাফিক-নিয়ন্ত্রণ লাগে। Feature flag ব্যবস্থাপনা-সরঞ্জাম (যেমন LaunchDarkly-জাতীয় সেবা বা নিজস্ব সরল ব্যবস্থা) খরচ ও জটিলতা যোগ করে। সিদ্ধান্ত নিন ঝুঁকি বনাম খরচের ভারসাম্যে: অর্থ লেনদেনের মতো সংবেদনশীল পথে canary/flag-এর খরচ যুক্তিসংগত; অভ্যন্তরীণ অ্যাডমিন প্যানেলে rolling-ই যথেষ্ট হতে পারে।

২২.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: ফেরার পথ আগে ঠিক করুন; health check ও smoke test বাধ্যতামূলক করুন; rollback অনুশীলন করুন; Expand–Contract মেনে চলুন; ডেপ্লয় যোগাযোগ করুন। **অ্যান্টি-প্যাটার্ন:** Big-bang ডেপ্লয়; rollback পরিকল্পনা ছাড়া ডেপ্লয়; ভাঙা মাইগ্রেশন একই ডেপ্লয়ে; flag তৈরি করে কখনো না মোছা (flag debt); পর্যবেক্ষণ ছাড়া canary।

২২.১৪ অধ্যায়ের সারাংশ

ডেপ্লয় কৌশল ঝুঁকি ভাগ করে: rolling ধীরে বদলায়, blue-green পুরো পরিবেশ বদলায়, canary অল্প ট্রাফিকে পরীক্ষা করে, feature flag চালু করা ও ডেপ্লয় আলাদা করে। প্রতিটির জন্য দরকার health check, smoke test, পর্যবেক্ষণ, backward-compatible মাইগ্রেশন ও অনুশীলন করা rollback — কারণ প্রশ্ন "ভুল হবে কি না" নয়, "ভুল হলে কত দ্রুত ফিরবেন" তা।

২২.১৫ মূল প্রযুক্তিগত পরিভাষা

Rolling Deployment: একে একে instance বদলে নতুন সংস্করণ চালু করা।

Blue-Green Deployment: নতুন পরিবেশ তৈরি করে একবারে ট্রাফিক সুইচ করা।

Canary Deployment: অল্প ট্রাফিকে নতুন সংস্করণ পরীক্ষা করে ধাপে ধাপে বাড়ানো।

Feature Flag: কোডে সুইচ — ডেপ্লয় ছাড়াই ফিচার চালু/বন্ধ করা।

Health Check: অ্যাপ সুস্থ আছে কি না তার স্বয়ংক্রিয় পরীক্ষা।

Smoke Test: ডেপ্লয়ের পর মূল পথগুলো কাজ করে কি না তার দ্রুত পরীক্ষা।

Rollback: সমস্যা হলে আগের কাজ করা সংস্করণে ফেরা।

Expand–Contract: মাইগ্রেশন নিরাপদে করার ধাপ — যোগ করুন, ব্যবহার করুন, তারপর সরান।

Liveness / Readiness: প্রসেস বেঁচে আছে কি না / ট্রাফিক নেওয়ার জন্য প্রস্তুত কি না।

Big-Bang Deployment: একবারে সবাইকে নতুন সংস্করণ দেওয়া — সবচেয়ে ঝুঁকিপূর্ণ।

২২.১৬ জ্ঞান যাচাই প্রশ্ন

1. Blue-Green ও Canary-র মধ্যে কখন কোনটি বেছে নেবেন?
2. Rolling deployment-এ পুরোনো ও নতুন কোড একসঙ্গে চললে API-তে কী সমস্যা হতে পারে?
3. ডেটাবেস মাইগ্রেশন উল্টে দেওয়া (down migration) কেন বিপজ্জনক?
4. Feature flag কীভাবে rollback-কে ডেপ্লয়ের চেয়ে দ্রুত করে?
5. Smoke test ছাড়া ডেপ্লয় "সফল" বলা যায় কেন নয়?

২২.১৭ ব্যবহারিক অনুশীলন

1. Docker Compose দিয়ে একটি অ্যাপ rolling পদ্ধতিতে আপডেট করুন ও healthcheck পাস হতে দেখুন।
2. একটি feature flag যোগ করে নতুন ফিচার চালু/বন্ধ করুন — কোড ডেপ্লয় ছাড়াই।
3. একটি rollback রানবুক লিখে অনুশীলন করুন: আগের digest-এ ফিরে smoke test চালান ও সময় মাপুন।

PART V • DEVOPS ENGINEERING MASTERY

Production CI/CD Engineering

MERN অ্যাপ্লিকেশনের এন্ড-টু-এন্ড পাইপলাইন, ট্রাবলশুটিং, গভর্ন্যান্স ও প্রোডাকশন সেরা অনুশীলন।

অধ্যায় ২৩ • PART V

Chapter 23 — Building End-to-End CI/CD Pipelines for MERN Applications

২৩.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) MERN অ্যাপের জন্য সম্পূর্ণ CI/CD পাইপলাইন নকশা করতে; (২) GitHub Actions-এ PR validation থেকে production ডেপ্লয় পর্যন্ত workflow লিখতে; (৩) MongoDB-সহ integration টেস্ট পাইপলাইনে চালাতে; (৪) Docker image build, scan ও registry publish করতে; (৫) Staging ডেপ্লয়, approval ও production ডেপ্লয় সংযুক্ত করতে; (৬) ক্লাউড-ভিত্তিক ডেপ্লয় লক্ষ্য (AWS/Azure/GCP/IBM Cloud) তুলনা করতে।

২৩.২ কেন এটি গুরুত্বপূর্ণ

আগের সব অধ্যায় ছিল অংশ; এই অধ্যায়ে অংশগুলো একসঙ্গে জোড়া লাগে। একটি MERN অ্যাপ — MongoDB, Express, React/Next.js, Node.js — এর পাইপলাইনে একসঙ্গে দরকার: ফ্রন্টএন্ড build, ব্যাকএন্ড টেস্ট, ডেটাবেস মাইগ্রেশন, Docker image, স্ক্যান, staging, approval, production। কোনো একটি অংশ বাদ পড়লেই পাইপলাইন অসম্পূর্ণ থাকে। TechNova-র এই পাইপলাইনই এই বইয়ের capstone-এর ভিত্তি — এখানে শেখা নকশাই আপনি নিজের প্রকল্পে প্রয়োগ করবেন।

২৩.৩ পূর্বশর্ত জ্ঞান

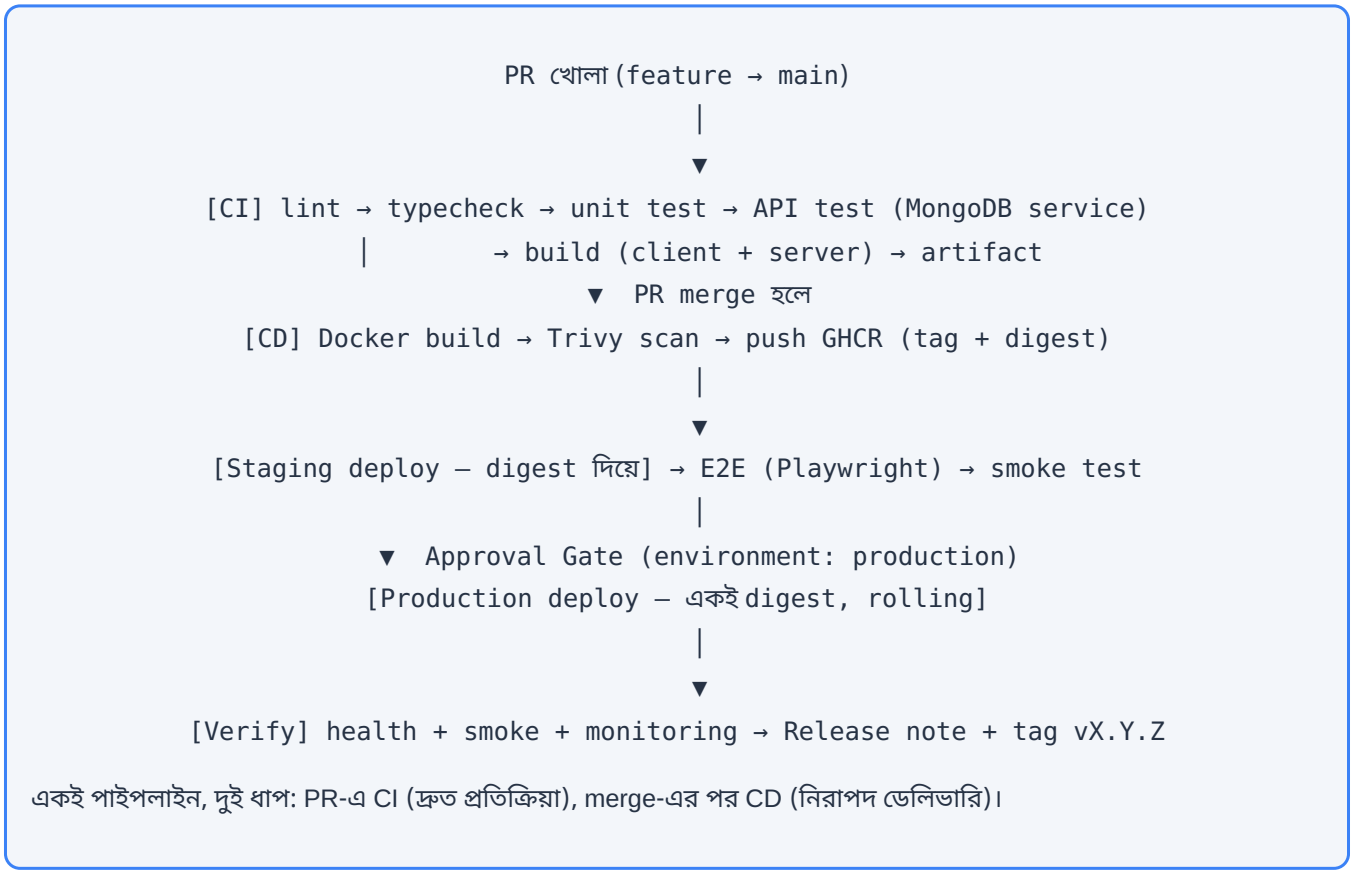
অধ্যায় ৬–২২-এর সব ধারণা: GitHub Actions, টেস্টিং, linting/SAST, Docker image, SemVer, promotion, deployment কৌশল। MERN অ্যাপের গঠন (client/server ভাগ) জানা থাকতে হবে।

২৩.৪ প্রথম নীতি থেকে ব্যাখ্যা

End-to-end পাইপলাইন মানে একটি মাত্র প্রশ্নের উত্তর দেওয়ার ব্যবস্থা: "এই কমিট কি production-এ যাওয়ার যোগ্য?" উত্তর পেতে পাইপলাইন ধাপে ধাপে প্রমাণ সংগ্রহ করে — কোড ঠিক আছে, টেস্ট পাস করেছে, নিরাপত্তা স্ক্যান পাস করেছে, image তৈরি হয়েছে, staging-এ চলেছে, মানুষ অনুমোদন দিয়েছে। প্রতিটি ধাপ একটি গেট; কোনো গেট ব্যর্থ হলে যাত্রা থামে। পাইপলাইনকে ভাবুন একটি assembly line হিসেবে — প্রতিটি স্টেশনে যাচাই, শেষে ডেলিভারি। আর মনে রাখবেন: পাইপলাইন নিজেও কোড — তারও সংস্করণ, পর্যালোচনা ও টেস্ট দরকার।

> **CONCEPT:** পাইপলাইনের গতি গুরুত্বপূর্ণ, কিন্তু আত্মবিশ্বাস তার চেয়ে গুরুত্বপূর্ণ। ৫ মিনিটের পাইপলাইন যা ভুল ধরতে পারে না, তার চেয়ে ১৫ মিনিটের নির্ভরযোগ্য পাইপলাইন ভালো। প্রথমে নির্ভরযোগ্যতা, তারপর গতি অপ্টিমাইজ করুন।

২৩.৫ আর্কিটেকচার / ধারণাগত চিত্র



একই পাইপলাইন, দুই ধাপ: PR-এ CI (দ্রুত প্রতিক্রিয়া), merge-এর পর CD (নিরাপদ ডেলিভারি)।

২৩.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

মনোরেপো বনাম আলাদা রিপো

MERN অ্যাপে client ও server এক রিপোতে (monorepo) রাখলে পাইপলাইন সহজ হয় — এক কমিটে দুই অংশের পরিবর্তন একসঙ্গে যায়। তবে build সময় বাড়ে; path filter দিয়ে শুধু বদলানো অংশের job চালানো যায়। আলাদা রিপোতে প্রতিটি অংশ স্বাধীনভাবে ডেপ্লয় হয়, কিন্তু সংস্করণের সামঞ্জস্য রাখা কঠিন। এই অধ্যায়ে monorepo ধরা হয়েছে — TechNova-র মতো ছোট দলের জন্য এটাই সাধারণ।

টেস্ট ডেটাবেস পাইপলাইনে

GitHub Actions-এ MongoDB service container চালিয়ে integration টেস্ট করা যায়। প্রতিটি রানে নতুন, খালি ডেটাবেস — টেস্ট শেষে ফেলে দেওয়া হয়। প্রোডাকশন ডেটা কখনো টেস্টে ব্যবহার করবেন না।

আর্টিফ্যাক্ট ও ক্যাশ

Build আউটপুট (client-এর `dist`, server-এর `dist`) artifact হিসেবে সংরক্ষণ করুন — যাতে ডেপ্লয় job আবার build না করে। `node_modules` ক্যাশ (`actions/setup-node`-এর `cache: npm`) ইনস্টল সময় কমায়। Docker layer ক্যাশ build সময় কমায় (অধ্যায় ২০)।

ক্লাউড-প্রোভাইডার প্রেক্ষাপট — CI/CD সংযোগের তুলনা

নিচের তুলনা উচ্চ-স্তরের ও পরিচালন-ভিত্তিক; দাম, সীমা বা পরিষেবার প্রাপ্যতা এখানে বলা হয়নি — সেগুলো অঞ্চল ও পরিকল্পনা (plan) অনুযায়ী বদলায়, এবং অফিসিয়াল প্রোভাইডার ডকুমেন্টেশনই চূড়ান্ত উৎস।

বিষয়	AWS	Microsoft Azure	Google Cloud (GCP)	IBM Cloud
OIDC পরিচয় (CI থেকে)	IAM role — GitHub/ GitLab OIDC provider দিয়ে স্বল্পস্থায়ী credential	Microsoft Entra ID — federated credential, Azure login action	Workload Identity Federation — স্বল্পস্থায়ী টোকেন	IBM Cloud IAM — trusted profile / API key; OIDC সমর্থন পরিষেবা অনুযায়ী ভিন্ন
Container registry	Amazon ECR	Azure Container Registry (ACR)	Artifact Registry	IBM Cloud Container Registry
ডেপ্লয় লক্ষ্য	ECS / EKS / App Runner / EC2	Container Apps / AKS / App Service	Cloud Run / GKE / App Engine	Code Engine / Kubernetes Service / VPC
Secret ব্যবস্থাপনা	Secrets Manager / SSM Parameter Store	Azure Key Vault	Secret Manager	Secrets Manager
নিজস্ব CI/ CD	CodePipeline / CodeBuild	Azure DevOps / GitHub Actions	Cloud Build	Continuous Delivery (Toolchain)

সাধারণ নীতি সব প্রোভাইডারে এক: দীর্ঘস্থায়ী কি-র বদলে OIDC দিয়ে স্বল্পস্থায়ী credential নিন; registry ও secret-এর অনুমতি least-privilege রাখুন; ডেপ্লয় করুন digest দিয়ে। কোন প্রোভাইডার বেছে নেবেন তা নির্ভর করে দলের অভিজ্ঞতা, বিদ্যমান অবকাঠামো ও অঞ্চলে প্রাপ্যতার ওপর — এই বই ক্লাউড সার্টিফিকেশন গাইড নয়; প্রোভাইডার-নির্দিষ্ট গভীরতা ডেডিকেটেড ক্লাউড খণ্ডে আসবে।

২৩.৭ সম্পূর্ণ উদাহরণ — GitHub Actions CI/CD (MERN)

```
# .github/workflows/ci-cd.yml — TechNova MERN অ্যাপের সম্পূর্ণ পাইপলাইন
name: CI/CD
on:
  pull_request:
    branches: [main]
  push:
    branches: [main]
    tags: ['v*.*.*']

permissions:
  contents: read
  packages: write
  id-token: write
  security-events: write

env:
  REGISTRY: ghcr.io
  IMAGE_NAME: technova/mern-app

jobs:
  ci:
    runs-on: ubuntu-latest
    services:
      mongo:
```

```
        image: mongo:7
        ports: ['27017:27017']
        options: >-
          --health-cmd "mongosh --eval 'db.runCommand({ ping: 1 })'"
          --health-interval 10s --health-timeout 5s --health-retries 5
    env:
      MONGODB_URI: mongodb://localhost:27017/technova_test
      JWT_SECRET: CHANGE-ME-TEST-ONLY-SECRET
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: '20', cache: 'npm' }
      - run: npm ci
      - run: npm run lint
      - run: npm run typecheck
      - run: npm run test -- --coverage
      - run: npm run build
      - uses: actions/upload-artifact@v4
        with:
          name: build-output
          path: |
            client/dist
            server/dist
          retention-days: 7

  build-push:
    needs: ci
    if: github.event_name == 'push'
    runs-on: ubuntu-latest
    outputs:
      digest: ${{ steps.build.outputs.digest }}
    steps:
      - uses: actions/checkout@v4
      - uses: docker/login-action@v3
        with:
          registry: ghcr.io
          username: ${{ github.actor }}
          password: ${{ secrets.GITHUB_TOKEN }}
      - name: Build and push
        id: build
        uses: docker/build-push-action@v6
        with:
          context: .
          push: true
          tags: |
            ghcr.io/technova/mern-app:${{ github.ref_name }}
            ghcr.io/technova/mern-app:${{ github.sha }}
          cache-from: type=gha
          cache-to: type=gha,mode=max
      - name: Scan image
        uses: aquasecurity/trivy-action@0.24.0
        with:
          image-ref: ghcr.io/technova/mern-app:${{ github.sha }}
          severity: CRITICAL,HIGH
          exit-code: '1'

  deploy-staging:
    needs: build-push
    if: github.ref == 'refs/heads/main'
```

```
    runs-on: ubuntu-latest
    environment:
      name: staging
      url: https://staging.technova.example
    steps:
      - name: Deploy to staging by digest
        run: |
          echo "Deploying ${{ needs.build-push.outputs.digest }} to staging"
          # এখানে আপনার ডেপ্লয় স্ক্রিপ্ট (SSH / cloud CLI) digest দিয়ে pull করে
      - name: Smoke test
        run: |
          curl -fsS https://staging.technova.example/health
          curl -fsS https://staging.technova.example/api/products

  deploy-production:
    needs: [build-push, deploy-staging]
    if: startsWith(github.ref, 'refs/tags/v')
    runs-on: ubuntu-latest
    environment:
      name: production
      url: https://app.technova.example
    steps:
      - name: Deploy to production by digest
        run: |
          echo "Deploying ${{ needs.build-push.outputs.digest }} to production"
      - name: Smoke test
        run: |
          curl -fsS https://app.technova.example/health
          curl -fsS https://app.technova.example/api/products
```

ব্যাখ্যা: `ci` job প্রতিটি PR-এ চলে; `build-push` শুধু push-এ; `deploy-staging` main-এ; `deploy-production` শুধু `v*.*.*` ট্যাগে — অর্থাৎ রিলিজ ট্যাগই production ডেপ্লয়ের ট্রিগার। `environment: production`-এ required reviewers বসালে অনুমোদন ছাড়া ডেপ্লয় হবে না। স্ক্যান ব্যর্থ হলে push হওয়া image-ও ডেপ্লয় হবে না, কারণ `deploy-staging` `build-push`-এর ওপর নির্ভরশীল।

```
# docker-compose.yml — স্থানীয়ভাবে পুরো MERN স্ট্যাক চালানো (ল্যাব/পরীক্ষা)
services:
  mongo:
    image: mongo:7
    volumes: [mongo-data:/data/db]
    ports: ['27017:27017']
  api:
    build: ./server
    environment:
      MONGODB_URI: mongodb://mongo:27017/technova
      JWT_SECRET: CHANGE-ME-LOCAL-ONLY
    ports: ['3000:3000']
    depends_on: [mongo]
  web:
    build: ./client
    ports: ['5173:80']
    depends_on: [api]
volumes: { mongo-data: {} }
```

২৩.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-র MERN দোকানে (app.technova.example) আগে ডেপ্লয় হতো একজন ডেভেলপারের ল্যাপটপ থেকে — SSH করে `git pull` ও `npm install`। একদিন তার ল্যাপটপে ভিন্ন Node সংস্করণ থাকায় production ভেঙে যায়। উপরের পাইপলাইন চালুর পর: PR-এ টেস্ট, merge-এ image build, staging-এ E2E, ট্যাগে production — আর কোনো ডেপ্লয় কারো ল্যাপটপের ওপর নির্ভর করে না। প্রথম মাসেই ডেপ্লয়ের সংখ্যা সপ্তাহে ১ থেকে ১২-এ উঠল, আর production ঘটনা কমল — কারণ ছোট ছোট পরিবর্তন, প্রতিটি যাচাইসহ।

২৩.৯ ধাপে ধাপে বাস্তবায়ন

1. রিপো গঠন ঠিক করুন: `client/`, `server/`, `.github/workflows/`, `Dockerfile`।
2. CI job লিখুন: lint → typecheck → unit/API টেস্ট (MongoDB service) → build → artifact।
3. Docker build + Trivy scan + GHCR push যোগ করুন; digest output হিসেবে সংরক্ষণ করুন।
4. Staging environment তৈরি করুন; ডেপ্লয়ের পর smoke test চালান।
5. Production environment-এ required reviewers বসান; ট্যাগ-ট্রিগার ডেপ্লয় যোগ করুন।
6. প্রথম রিলিজ করুন: `v1.0.0` ট্যাগ → পাইপলাইন দেখুন → health পরীক্ষা করুন → release note লিখুন।

২৩.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "MongoDB service-এ সংযোগ হচ্ছে না" — service-র health check পাস হয়নি বা `MONGODB_URI`-তে হোস্ট `localhost` নয়; job-এর service-এ হোস্ট `localhost`, container-এর ভেতরে সার্ভিসের নাম। (২) "Build job-এ artifact পাওয়া যাচ্ছে না" — upload/download-এর নাম ও path মেলাতে হবে; `retention-days` যাচাই করুন। (৩) "Production job চলছে না" — ট্যাগের ধরন (`v*.*.*`) মেলেনি বা environment অনুমোদন অপেক্ষমাণ; Actions ট্যাবে job-এর অবস্থা দেখুন। (৪) "Staging-এ E2E ফেল, CI-তে পাস" — পরিবেশের পার্থক্য (env var, ডেটা); staging-এর কনফিগারেশন production-এর কাছাকাছি রাখুন।

২৩.১১ নিরাপত্তা বিবেচনা

WARNING: এই পাইপলাইনে `permissions` শীর্ষে সীমিত করা হয়েছে — প্রতিটি job-এর শুধু যা দরকার তাই। `GITHUB_TOKEN` কখনো লগে ছাপাবেন না; secret শুধু environment-এ রাখুন। PR থেকে আসা কোড (fork-এর PR) কখনো production secret পায় না — GitHub-এ এটাই ডিফল্ট, আর এটাই সঠিক। OIDC ব্যবহার করুন দীর্ঘস্থায়ী ক্লাউড কি-র বদলে।

২৩.১২ পারফরম্যান্স ও খরচ

এই পাইপলাইনের সময়ের সিংহভাগ যায় `npm ci` ও Docker build-এ — ক্যাশ (npm cache, Docker layer cache) দিয়ে কমান। E2E শুধু staging-এ চালানো হয়েছে, PR-এ নয় — এতে PR দ্রুত থাকে। MongoDB service প্রতিটি রানে চালু হয়;

টেস্ট সমান্তরাল করলে সময় কমে, কিন্তু runner খরচ বাড়ে। নিয়মিত পাইপলাইনের সময় মাপুন ও সবচেয়ে ধীর ধাপটি আগে অপ্টিমাইজ করুন।

২৩.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: এক workflow-তে CI ও CD, কিন্তু job আলাদা; ট্যাগ-ভিত্তিক production; digest দিয়ে ডেপ্লয়; least-privilege অনুমতি; পাইপলাইনও code review-এর আওতায়। **অ্যান্টি-প্যাটার্ন:** এক বিশাল job-এ সবকিছু; PR-এ production secret ব্যবহার; ল্যাপটপ থেকে হাতে ডেপ্লয়; পাইপলাইনে `latest` ট্যাগ ব্যবহার।

২৩.১৪ অধ্যায়ের সারাংশ

End-to-end MERN পাইপলাইন: PR-এ lint, typecheck, টেস্ট ও build; merge-এ Docker build, স্ক্যান ও registry push; staging-এ ডেপ্লয় ও E2E; ট্যাগ ও অনুমোদনে production — একই digest সর্বত্র। ক্লাউড লক্ষ্য যেটাই হোক (AWS, Azure, GCP বা IBM Cloud), নীতি এক: OIDC, least-privilege, digest দিয়ে ডেপ্লয়।

২৩.১৫ মূল প্রযুক্তিগত পরিভাষা

End-to-End Pipeline: কমিট থেকে production পর্যন্ত সম্পূর্ণ স্বয়ংক্রিয় যাত্রা।

Monorepo: একাধিক অংশ (client/server) একই রিপোতে রাখা।

Service Container: CI job-এর পাশে চলা সহায়ক সার্ভিস (যেমন টেস্ট MongoDB)।

OIDC (OpenID Connect): স্বল্পস্থায়ী credential পাওয়ার পরিচয়-ভিত্তিক পদ্ধতি।

GHCR: GitHub Container Registry — GitHub-এর image registry।

ECR / ACR / Artifact Registry: AWS, Azure ও GCP-র container registry।

Workload Identity Federation: GCP-তে বাইরের পরিচয় দিয়ে স্বল্পস্থায়ী টোকেন পাওয়া।

Path Filter: শুধু নির্দিষ্ট পথের পরিবর্তনে job চালানো।

Least Privilege: শুধু প্রয়োজনীয় অনুমতি দেওয়ার নীতি।

Smoke Test: ডেপ্লয়ের পর মূল পথের দ্রুত যাচাই।

২৩.১৬ জ্ঞান যাচাই প্রশ্ন

1. Production ডেপ্লয় ট্যাগ-ট্রিগার করলে কী সুবিধা হয়? সরাসরি main push-এ ডেপ্লয় করলে কী ঝুঁকি?
2. CI job-এ MongoDB service container কেন দরকার? স্থানীয় টেস্টের সঙ্গে পার্থক্য কী?
3. OIDC দীর্ঘস্থায়ী API কি-র চেয়ে কেন নিরাপদ?
4. Staging ও production-এ একই digest ব্যবহার না করলে কী ভুল হতে পারে?
5. AWS, Azure, GCP ও IBM Cloud-এর মধ্যে ডেপ্লয় লক্ষ্য বেছে নেওয়ার সময় কী কী দেখবেন (দাম ছাড়া)?

২৩.১৭ ব্যবহারিক অনুশীলন

1. উপরের workflow-টি আপনার MERN প্রকল্পে বসিয়ে একটি PR খুলুন; CI পাস হতে দেখুন।
2. `v0.1.0` ট্যাগ দিয়ে staging ডেপ্লয় ট্রিগার করুন ও smoke test চালান।
3. একটি ক্লাউড প্রোভাইডারের অফিসিয়াল ডকুমেন্টেশন পড়ে তার OIDC সেটআপের ধাপগুলো নোট করুন (দাম না লিখে)।

অধ্যায় ২৪ • PART V

Chapter 24 — Pipeline Troubleshooting, Governance and Production Best Practices

২৪.১ শেখার উদ্দেশ্য

এই অধ্যায় শেষে আপনি পারবেন: (১) পাইপলাইন ব্যর্থতার মূল কারণ পদ্ধতিগতভাবে খুঁজে বের করতে; (২) লগ, মেট্রিক ও অ্যালার্ট দিয়ে pipeline observability নকশা করতে; (৩) Idempotency ও repeatability ব্যাখ্যা ও প্রয়োগ করতে; (৪) Governance — নীতি, অনুমোদন, অডিট ও কোটা — বাস্তবায়ন করতে; (৫) Production CI/CD-র সেরা অনুশীলন ও অ্যান্টি-প্যাটার্নের তালিকা তৈরি করতে।

২৪.২ কেন এটি গুরুত্বপূর্ণ

পাইপলাইন তৈরি করা একদিনের কাজ; চালিয়ে রাখা প্রতিদিনের কাজ। পাইপলাইন ভাঙবে — dependency আপডেট হবে, runner বদলাবে, secret মেয়াদোত্তীর্ণ হবে, তৃতীয় পক্ষের action বদলে যাবে। তখন প্রশ্ন: আপনি কত দ্রুত কারণ খুঁজে পান, আর কত দ্রুত ঠিক করেন? আর governance ছাড়া পাইপলাইন ধীরে ধীরে অনিয়ন্ত্রিত হয়: কে কী ডেপ্লয় করল, কোন secret কোথায় ব্যবহৃত হলো, কোন নিয়ম কে ভাঙল — কিছুই জানা যায় না। এই অধ্যায় বইয়ের সমাপ্তি: এখানে শেখা অভ্যাসই আপনাকে জুনিয়র থেকে নির্ভরযোগ্য DevOps ইঞ্জিনিয়ারে পরিণত করে।

২৪.৩ পূর্বশর্ত জ্ঞান

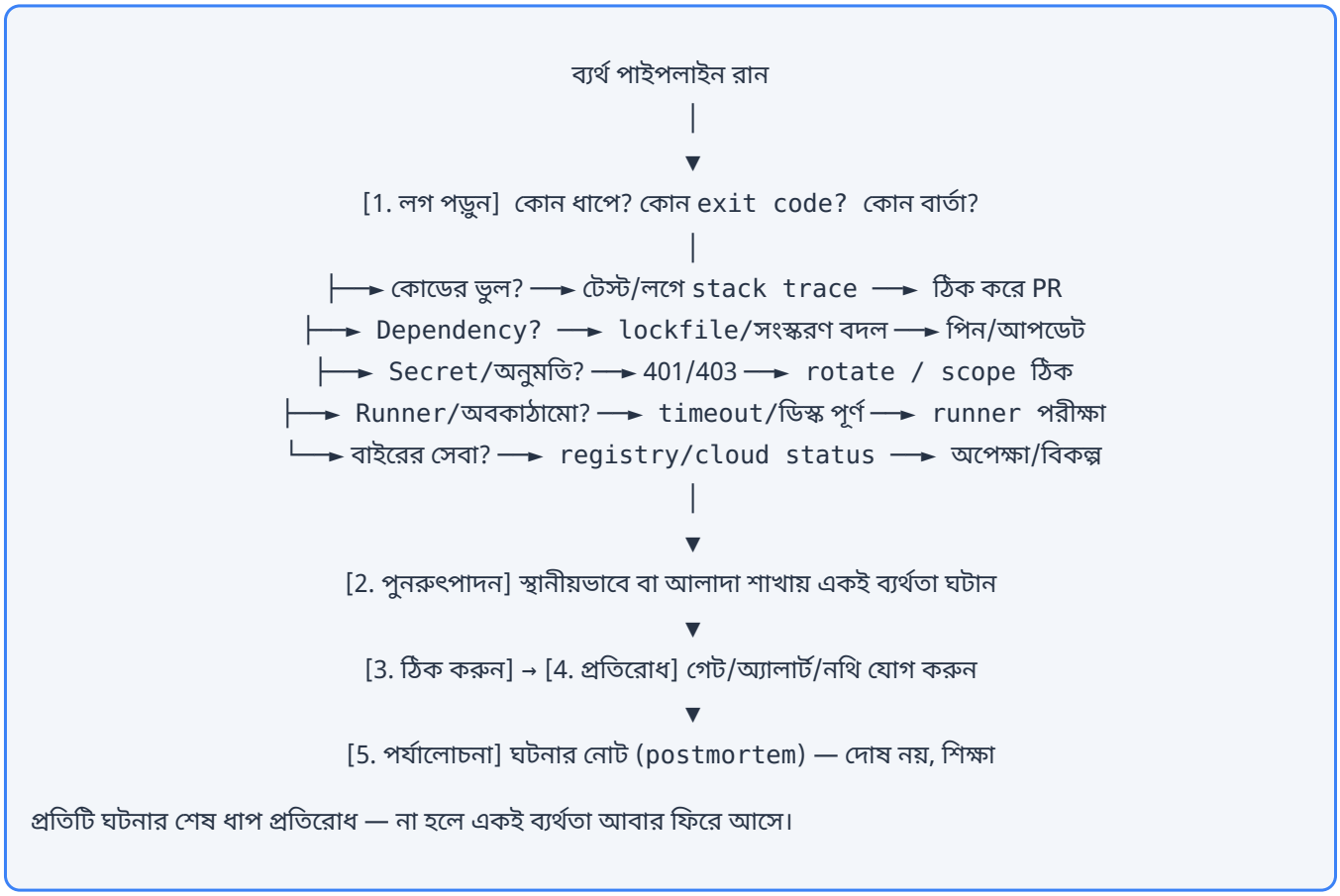
অধ্যায় ১–২৩-এর সব ধারণা, বিশেষ করে GitHub Actions/GitLab/Jenkins-এর লগ পড়া, secret ও অনুমতি ব্যবস্থাপনা, এবং অধ্যায় ২০–২২-এর artifact ও ডেপ্লয় কৌশল।

২৪.৪ প্রথম নীতি থেকে ব্যাখ্যা

Troubleshooting মানে অনুমান নয় — মানে প্রমাণের শৃঙ্খল। ব্যর্থতা দেখলে প্রশ্ন করুন: "শেষ কখন কাজ করেছিল? তারপর কী বদলেছে?" — কোড, dependency, secret, runner, বাইরের সেবা — পাঁচটির একটিতে বদল থাকবেই। Observability মানে সেই প্রশ্নের উত্তর দেওয়ার মতো তথ্য আগে থেকে সংগ্রহ করা: লগ (কী হলো), মেট্রিক (কতটা, কত দ্রুত) ও ট্রেস (কোথায় আটকে গেল)। Governance মানে নিয়মের কাঠামো: কে কী করতে পারে, কী করতে পারে না, আর তা কীভাবে যাচাই হয়। Idempotency মানে একই কাজ বারবার করলেও ফল একই — যেমন একই স্ক্রিপ্ট দুবার চালালে দুটি ডেপ্লয় নয়, একটি নিশ্চিত অবস্থা। Repeatability মানে একই ইনপুটে একই আউটপুট — পিন করা সংস্করণ, lockfile ও নির্ধারক (deterministic) ধাপ ছাড়া তা সম্ভব নয়।

> **CONCEPT:** "It worked yesterday" — এই বাক্যটিই তদন্তের শুরু। গতকাল থেকে আজকের মধ্যে কী বদলেছে, তার তালিকাই মূল কারণের তালিকা। বদল ছাড়া ব্যর্থতা হয় না — বদলটি শুধু চোখে পড়েনি।

২৪.৫ আর্কিটেকচার / ধারণাগত চিত্র



প্রতিটি ঘটনার শেষ ধাপ প্রতিরোধ — না হলে একই ব্যর্থতা আবার ফিরে আসে।

২৪.৬ বিস্তারিত প্রযুক্তিগত ব্যাখ্যা

Troubleshooting সিদ্ধান্ত-গাছ

লক্ষণ	প্রথম পরীক্ষা	সম্ভাব্য কারণ
Job হঠাৎ ফেল	শেষ সফল রানের সঙ্গে diff	কোড/dependency/secret বদল
401 / 403	টোকেনের মেয়াদ ও scope	মেয়াদোত্তীর্ণ secret, ভুল অনুমতি
Timeout	ধাপের সময় ও runner লোড	ধীর টেস্ট, নেটওয়ার্ক, ডিস্ক পূর্ণ
Flaky ফেল	একই রান বারবার চালানো	সময়-নির্ভরতা, ভাগ করা সম্পদ
Cache miss	ক্যাশ কি-র গঠন	lockfile বদল, কি ভুল
Image pull ব্যর্থ	ট্যাগ/digest ও registry status	ভুল ট্যাগ, registry বিভ্রাট
ডেপ্লয় সফল, অ্যাপ ভাঙা	health/smoke ও মেট্রিক	কনফিগারেশন, মাইগ্রেশন, ভুল digest

Pipeline Observability

প্রতিটি পাইপলাইনে তিনটি জিনিস মাপুন: (১) **সময়কাল** — প্রতিটি job ও ধাপের সময়; হঠাৎ বাড়লে কারণ খুঁজুন। (২) **সফলতার হার** — কত শতাংশ রান পাস করে; flaky-র হার আলাদা করে দেখুন। (৩) **ডেপ্লয় ফ্রিকোয়েন্সি ও ব্যর্থতা** —

কতবার ডেপ্লয় হয়, কতবার rollback লাগে (DORA মেট্রিকের সরল রূপ)। লগ সংরক্ষণ করুন (retention নীতি অনুযায়ী), আর ব্যর্থতায় স্বয়ংক্রিয় অ্যালার্ট (Slack/ইমেইল) বসান — মানুষ যেন নিজে থেকে খুঁজতে না হয়।

Idempotency ও Repeatability

Idempotent স্ক্রিপ্টের উদাহরণ: "ব্যবহারকারী থাকলে তৈরি করো না, না থাকলে তৈরি করো" — দুবার চালালেও একটি ব্যবহারকারী। বিপরীতে, প্রতিবার চালালে নতুন রেকর্ড তৈরি করা non-idempotent — ভুল করে আবার চালালে দ্বিগুণ ডেটা। Repeatability-র জন্য: Node ও action-এর সংস্করণ পিন করুন (`actions/checkout@v4`, `@v4.2.1` আরও ভালো), lockfile কমিট করুন, সময়-নির্ভর মান (যেমন `Date.now()`) build-এ ব্যবহার করবেন না।

Governance — নীতি, অনুমোদন, অডিট

- **Branch protection:** main-এ সরাসরি push নিষেধ; PR, পর্যালোচনা ও status check বাধ্যতামূলক।
- **CODEOWNERS:** পাইপলাইন ফাইল (`.github/workflows/`) বদলাতে নির্দিষ্ট দলের অনুমোদন লাগে।
- **Environment protection:** production-এ required reviewers ও অপেক্ষার সময় (wait timer)।
- **Secret governance:** secret-র মালিক, মেয়াদ ও rotation সময়সূচি নথিভুক্ত; নিয়মিত অডিট।
- **তৃতীয় পক্ষের action নীতি:** শুধু যাচাইকৃত/পিন করা action; নতুন action-এর পর্যালোচনা।
- **অডিট লগ:** কে, কখন, কী ডেপ্লয় করল — তার রেকর্ড সংরক্ষণ করুন।
- **কোটা ও খরচ:** runner মিনিট, স্টোরেজ ও registry খরচ পর্যবেক্ষণ করুন; অপ্রয়োজনীয় workflow বন্ধ করুন।

ক্লাউড-প্রোভাইডার প্রেক্ষাপট — Governance-এ প্রোভাইডার-ভিত্তিক পার্থক্য

Governance-এর নীতিগুলো (least-privilege, অডিট, secret rotation) সব প্রোভাইডারে এক, তবে সরঞ্জাম ভিন্ন: AWS-তে IAM ও CloudTrail, Azure-তে Microsoft Entra ID ও Azure Policy, GCP-তে IAM ও Cloud Audit Logs, IBM Cloud-এ IAM ও Activity Tracker। প্রতিটি প্রোভাইডারে OIDC দিয়ে স্বল্পস্থায়ী credential, secret manager-এ কেন্দ্রীয় গোপন-তথ্য ব্যবস্থাপনা, আর registry-তে image স্ক্যানিং চালু করা যায় — বিস্তারিত সেটআপ অফিসিয়াল ডকুমেন্টেশন অনুযায়ী, অঞ্চল ও পরিকল্পনা অনুযায়ী পার্থক্য থাকতে পারে। দাম বা সীমা এখানে বলা হয়নি।

Production সেরা অনুশীলনের চেকলিস্ট

- প্রতিটি PR-এ lint, typecheck, টেস্ট ও স্ক্যান — কোনো ব্যতিক্রম ছাড়া।
- Production ডেপ্লয় শুধু ট্যাগ + অনুমোদন + একই digest দিয়ে।
- Rollback রানবুক লেখা ও নিয়মিত অনুশীলন করা।
- Secret-র rotation সময়সূচি ও মালিক ঠিক করা।
- পাইপলাইনের সময় ও সফলতার হার সাপ্তাহিক পর্যালোচনা।
- ঘটনার পর blameless postmortem লেখা।
- পাইপলাইন ফাইলেও code review — workflow বদল মানেই production বদল।

২৪.৭ কনফিগারেশন উদাহরণ

```
# CODEOWNERS — পাইপলাইন ফাইলের মালিকানা (.github/CODEOWNERS)
/.github/workflows/ @technova/devops-team
Dockerfile @technova/devops-team
# এই ফাইলগুলো বদলাতে devops-team-এর অনুমোদন লাগবে।
```

```
# Idempotent ডেপ্লয় স্ক্রিপ্টের নমুনা (Bash)
#!/bin/bash
set -euo pipefail
IMAGE="ghcr.io/technova/mern-app@sha256:EXAMPLE-DIGEST"
# একই image আগেই চলছে কি না দেখুন — থাকলে আবার pull নয়:
if docker ps --format '{{.Image}}' | grep -q "$IMAGE"; then
  echo "Already running $IMAGE — nothing to do"
  exit 0
fi
docker pull "$IMAGE"
docker compose up -d api
# set -euo pipefail: কোনো কমান্ড ব্যর্থ হলে স্ক্রিপ্ট থামে,
# অনির্ধারিত ভেরিয়েবল ব্যবহারে ত্রুটি দেয়।
```

```
# পাইপলাইন ডিবাগ — GitHub Actions-এ ধাপভিত্তিক লগ
# 1. Actions ট্যাবে ব্যর্থ job খুলুন → ব্যর্থ ধাপের লগ দেখুন।
# 2. প্রয়োজনে debug logging চালু করুন (repo secret নয়, variable):
#    ACTIONS_STEP_DEBUG = true   (repository variable হিসেবে)
# 3. স্থানীয়ভাবে পুনরুৎপাদন: একই Node সংস্করণে npm ci && npm test
# সতর্কতা: debug লগে secret ছাপা হতে পারে — debug বন্ধ করতে ভুলবেন না।
```

```
# Blameless postmortem — ঘটনার নোটের কাঠামো (Markdown)
## ঘটনা: 2026-11-07 — staging ডেপ্লয় ব্যর্থ
## প্রভাব: staging 2 ঘণ্টা অনুপলব্ধ; production অপ্রভাবিত
## সময়রেখা: 10:02 ডেপ্লয় শুরু → 10:05 health check ব্যর্থ → 10:20 rollback
## মূল কারণ: নতুন env var staging-এ বসানো হয়নি
## শিক্ষা: env var-ও code review-এর আওতায়; ডেপ্লয়-পূর্ব চেকলিস্টে যোগ
## প্রতিরোধ: staging কনফিগারেশন সংস্করণ-নিয়ন্ত্রণে নেওয়া (মালিক: ___)
```

২৪.৮ বাস্তব পরিস্থিতি — TechNova

REAL WORLD: TechNova-তে এক শুক্রবার রাতে production ডেপ্লয় ব্যর্থ হলো — 401 ত্রুটি। কারণ খুঁজতে গিয়ে দেখা গেল registry-র টোকেনের মেয়াদ শেষ হয়েছে, আর তা rotate করার দায়িত্ব কারো ছিল না। ঘটনার পর দল তিনটি জিনিস বদলাল: (১) দীর্ঘস্থায়ী টোকেনের বদলে OIDC; (২) মেয়াদোত্তীর্ণের ৭ দিন আগে অ্যালার্ট; (৩) প্রতিটি secret-এর মালিক ও rotation তারিখ নথিভুক্ত। এরপর থেকে এমন ব্যর্থতা আর হয়নি। শিক্ষা: বেশিরভাগ production ঘটনার মূল কারণ কোড নয় — পরিচালনার ফাঁক।

২৪.৯ ধাপে ধাপে বাস্তবায়ন

1. Branch protection ও CODEOWNERS চালু করুন; পাইপলাইন ফাইলের মালিক ঠিক করুন।
2. Production environment-এ required reviewers ও wait timer বসান।
3. পাইপলাইনের মেট্রিক (সময়, সফলতার হার, rollback সংখ্যা) সাপ্তাহিক দেখার অভ্যাস করুন।
4. Secret-র তালিকা তৈরি করুন: নাম, মালিক, মেয়াদ, rotation তারিখ।
5. একটি ব্যর্থতার অনুশীলন করুন: ইচ্ছাকৃতভাবে staging ডেপ্লয় ভেঙে rollback রানবুক অনুসরণ করুন।
6. প্রতিটি ঘটনার postmortem লিখুন ও প্রতিরোধের কাজটি পরের sprint-এ নিন।

২৪.১০ সাধারণ ভুল ও সমস্যা সমাধান

TROUBLESHOOTING: (১) "পাইপলাইন ধীরে ধীরে ধীর হচ্ছে" — ক্যাশ বাড়ছে, টেস্ট বাড়ছে, কিন্তু কেউ পরিমাপ করছে না। সমাধান: প্রতিটি ধাপের সময় ট্র্যাক করুন, সবচেয়ে ধীর ধাপটি ঠিক করুন। (২) "কেউ জানে না secret কোথায় ব্যবহৃত হয়" — secret-র তালিকা ও মালিক নেই; অডিট করে নথিভুক্ত করুন। (৩) "একই ব্যর্থতা বারবার" — postmortem লেখা হয়নি বা প্রতিরোধের কাজ হয়নি; প্রতিটি ঘটনার একটি প্রতিরোধ-কাজ বাধ্যতামূলক করুন। (৪) "Debug লগে secret ফাঁস" — `ACTIONS_STEP_DEBUG` চালু রেখে ভুলে যাওয়া; debug শেষে সঙ্গে সঙ্গে বন্ধ করুন ও লগ পর্যালোচনা করুন।

২৪.১১ নিরাপত্তা বিবেচনা

WARNING: Governance-এর সবচেয়ে বড় ঝুঁকি "নিয়ম আছে, মানা হয় না"। Self-hosted runner-এ অপরিচিত PR-এর কোড চালাবেন না — এতে runner-এর secret চুরি হতে পারে (অধ্যায় ১০)। তৃতীয় পক্ষের action পিন করুন (SHA দিয়ে) — ট্যাগ বদলে গেলে আক্রমণকারী কোড ঢোকাতে পারে। Debug লগে secret ছাপাবেন না। আর মনে রাখবেন: অনুমোদন প্রক্রিয়া যেন দলকে এড়িয়ে যেতে বাধ্য না করে — কঠোর কিন্তু ব্যবহারযোগ্য রাখুন।

২৪.১২ পারফরম্যান্স ও খরচ

Runner মিনিট, স্টোরেজ (artifact, cache, image) ও ডেটা ট্রান্সফার — তিনটি খরচের উৎস। অপ্রয়োজনীয় workflow (যেমন প্রতিটি push-এ পূর্ণ E2E) বন্ধ করুন; artifact retention কমান; ক্যাশের আকার পর্যবেক্ষণ করুন। Governance-এর খরচও আছে — পর্যালোচনা ও অনুমোদনে সময় যায় — তবে production ঘটনার খরচের তুলনায় তা কম। মাসে একবার খরচ ও সময়ের পর্যালোচনা করুন।

২৪.১৩ সেরা অনুশীলন ও অ্যান্টি-প্যাটার্ন

সেরা: বদল-ভিত্তিক তদন্ত; observability; idempotent স্ক্রিপ্ট; পিন করা সংস্করণ; blameless postmortem; নিয়মিত secret অডিট; rollback অনুশীলন। **অ্যান্টি-প্যাটার্ন:** অনুমানে ঠিক করা; "retry দিয়ে পাস করানো"; debug লগ চালু রেখে দেওয়া; নিয়ম লিখে মানা না করা; দোষারোপের সংস্কৃতি; পাইপলাইনকে "একবার বানিয়ে ভুলে যাওয়া"।

২৪.১৪ অধ্যায়ের সারাংশ

Production CI/CD মানে শুধু পাইপলাইন বানানো নয় — মানে তাকে চালিয়ে রাখা: বদল খুঁজে ব্যর্থতার কারণ বের করা, লগ ও মেট্রিক দিয়ে পর্যবেক্ষণ করা, idempotent ও পুনরুৎপাদনযোগ্য রাখা, আর governance দিয়ে নিয়ন্ত্রণে রাখা। এই বইয়ের শেষ শিক্ষা: ভালো পাইপলাইন দ্রুত নয়, নির্ভরযোগ্য — আর নির্ভরযোগ্যতা আসে অভ্যাস থেকে, সরঞ্জাম থেকে নয়।

২৪.১৫ মূল প্রযুক্তিগত পরিভাষা

Observability: লগ, মেট্রিক ও ট্রেস দিয়ে ব্যবস্থার ভেতরের অবস্থা বোঝার ক্ষমতা।

Idempotency: একই কাজ বারবার করলেও ফল একই থাকা।

Repeatability: একই ইনপুটে একই আউটপুট পাওয়া।

Governance: কে কী করতে পারে, কীভাবে যাচাই হয় — তার নিয়মের কাঠামো।

Blameless Postmortem: দোষ না খুঁজে শিক্ষা ও প্রতিরোধের জন্য ঘটনার পর্যালোচনা।

Branch Protection: main শাখায় সরাসরি push আটকে PR ও পর্যালোচনা বাধ্যতামূলক করা।

CODEOWNERS: নির্দিষ্ট ফাইলের পর্যালোচনার মালিক ঠিক করা ফাইল।

DORA Metrics: ডেপ্লয় ফ্রিকোয়েন্সি, lead time, ব্যর্থতার হার ও পুনরুদ্ধারের সময়ের পরিমাপ।

Runbook: সমস্যা হলে ধাপে ধাপে কী করতে হবে তার লিখিত নির্দেশ।

Flaky Test: কখনো পাস কখনো ফেল করা অনির্ধারক টেস্ট।

২৪.১৬ জ্ঞান যাচাই প্রশ্ন

1. পাইপলাইন ব্যর্থ হলে প্রথম প্রশ্ন কী হওয়া উচিত, আর কেন?
2. Idempotency ও repeatability-র মধ্যে পার্থক্য কী? উদাহরণ দিন।
3. Blameless postmortem দোষ খোঁজার চেয়ে কেন বেশি কার্যকর?
4. CODEOWNERS পাইপলাইনের নিরাপত্তায় কীভাবে সাহায্য করে?
5. পাইপলাইনের কোন তিনটি মেট্রিক নিয়মিত দেখা উচিত, আর কেন?

২৪.১৭ ব্যবহারিক অনুশীলন

1. আপনার একটি ব্যর্থ পাইপলাইন রান নিয়ে সিদ্ধান্ত-গাছ অনুসরণ করে মূল কারণ লিখুন (অনুমান নয়, প্রমাণসহ)।
2. একটি ডেপ্লয় স্ক্রিপ্টকে idempotent করুন — দুবার চালিয়ে একই ফল হতে দেখুন।
3. একটি কাল্পনিক ঘটনার blameless postmortem লিখুন: সময়রেখা, মূল কারণ, প্রতিরোধের কাজ।